CB100

Algoritmos y Estructuras de Datos

Universidad de Buenos Aires · Facultad de Ingeniería

.UBAfiuba
FACULTAD DE INGENIERÍA

APUNTE DEL CURSO

Teoría y práctica

con TDA, POO y herramientas actuales

Un recorrido completo, desde el lenguaje Java y las herramientas profesionales hasta las estructuras de datos y los algoritmos.

Prof. Gustavo Schmidt

Cátedra Schmidt

Java 25 · IntelliJ IDEA · Gradle · Git · IA

Índice

1A Presentación de la materia 7
1A.1 Datos útiles de la cátedra . . . 7
1A.2 El mapa del curso . . . 7
1A.3 Cómo se aprueba . . . 8
1A.4 Uso de inteligencia artificial en la materia . . . 8
1A.5 Bibliografía y material de consulta . . . 9
1A.6 Cómo usar este apunte . . . 9

1B Introducción a Java y el entorno de trabajo 10
1B.1 Un poco de historia: por qué Java . . . 10
1B.2 ¿Cómo se ejecuta un programa Java? . . . 11
1B.3 Las herramientas del curso . . . 12
1B.4 Tu primer programa . . . 14
1B.5 Comentarios y constantes . . . 15
1B.6 Tipos de datos y variables . . . 15
1B.7 Rangos, conversiones y wrappers . . . 16
1B.8 Cadenas de texto: `String` y `StringBuilder` . . . 17
1B.9 Operadores . . . 19
1B.10 Matemática y azar: `Math` y `Random` . . . 19
1B.11 Estructuras de control . . . 20
1B.12 Métodos . . . 24
1B.13 Una clase propia (mínima) . . . 24
1B.14 Enumerados: `enum` . . . 25
1B.15 Modelar datos con `record` . . . 25
1B.16 Expresiones lambda . . . 27
1B.17 Manejo de excepciones . . . 28
1B.18 Fechas y horas . . . 32
1B.19 Entrada y salida por consola . . . 33
1B.20 Archivos . . . 33
1B.21 Validaciones . . . 34
1B.22 Probar el código: JUnit 5 . . . 34
1B.23 El repositorio y el esquema de paquetes . . . 35
1B.24 Usar una librería externa (Maven) . . . 35
1B.25 Ejemplo integrador . . . 36
1B.26 Errores comunes . . . 37
1B.27 Repaso . . . 37

Guía de ejercicios — Unidad 1B 38
1B.28 Preguntas teóricas . . . 38
1B.29 Nivel fácil . . . 38
1B.30 Nivel medio . . . 38
1B.31 Nivel difícil . . . 39

2 De la memoria a las referencias 40
2.1 El modelo de memoria: stack y heap . . . 40
2.2 Valores y referencias . . . 41
2.3 Crear objetos con `new` . . . 42
2.4 Referencias, no punteros . . . 42
2.5 `null` y la `NullPointerException` . . . 42
2.6 Identidad vs igualdad: `==` y `equals` . . . 43
2.7 Aliasing: dos referencias, un objeto . . . 44

2.13 Errores comunes . . . 48
2.14 Repaso . . . 48

Guía de ejercicios — Unidad 2 **49**
2.15 Preguntas teóricas . . . 49
2.16 Nivel fácil . . . 49
2.17 Nivel medio . . . 49
2.18 Nivel difícil . . . 50

3 Abstracción, TDA y Programación Orientada a Objetos **51**
3.1 Abstracción: la gran idea . . . 51
3.2 Tipos de Datos Abstractos (TDA) . . . 51
3.3 Clases y objetos . . . 52
3.4 Los pilares de la POO . . . 53
3.5 Composición: la alternativa a heredar . . . 55
3.6 Interfaces: el contrato puro . . . 56
3.7 Clases y métodos abstractos . . . 58
3.8 Los métodos estándar . . . 59
3.9 Genéricos . . . 60
3.10 Diagramas UML de clases . . . 60
3.11 Diseño profesional: elegancia, robustez y mantenimiento . . . 61
3.12 El camino profesional: integración continua . . . 62
3.13 Ejemplo integrador . . . 63
3.14 El checklist del ejercicio de TDA . . . 63
3.15 Errores comunes . . . 65
3.16 Repaso . . . 65

Guía de ejercicios — Unidad 3 **67**
3.17 Preguntas teóricas . . . 67
3.18 Nivel fácil . . . 67
3.19 Nivel medio . . . 67
3.20 Nivel difícil . . . 68

3B Stack y heap paso a paso **69**
3B.1 Cómo leer los diagramas . . . 69
3B.2 El debugger: el stack en vivo . . . 69
3B.3 Escenario 0 — Primitivos puros: el heap ni aparece . . . 70
3B.4 Escenario 1 — Un new simple: del stack al heap y al atributo . . . 71
3B.5 Escenario 2 — Llamadas anidadas: el puntero viaja por parámetro . . . 71
3B.6 Escenario 3 — El puntero this . . . 73
3B.7 Escenario 4 — Swap de punteros en un método: al salir se pierde . . . 74
3B.8 Escenario 5 — Swap de valores: se mantiene . . . 76
3B.9 Escenario 6 — Dos news: atributos vs punteros, cara a cara . . . 77
3B.10 Escenario 7 — Varios news reasignados: varios bloques colgados . . . 79
3B.11 Escenario 8 — El heap apunta al heap . . . 80
3B.12 Escenario 9 — `==` vs `equals` (y la trampa de String) . . . 82
3B.13 Escenario 10 — Dos news al mismo puntero: UN bloque colgado . . . 83
3B.14 Las siete morales del apartado . . . 83
3B.15 Repaso (en modo cuestionario del TP1) . . . 84

4 Complejidad computacional **85**
4.1 Un poco de historia . . . 85
4.2 ¿Por qué no medir en segundos? . . . 85
4.3 Órdenes de crecimiento . . . 86
4.4 Las tres cotas: O, Ω y Θ . . . 86
4.5 Calcular la complejidad de un algoritmo iterativo . . . 89
4.6 Calcular la de un algoritmo recursivo: la ecuación de recurrencia . . . 91

4.7 Resolver recurrencias: el método de expansión . . . 91
4.8 El Teorema Maestro por sustracción . . . 93
4.9 El Teorema Maestro por división . . . 94
4.10 Complejidad de algoritmos estándar . . . 95
4.11 Aplicar la complejidad a lo que ya hicimos . . . 96
4.12 Complejidad amortizada: el caso del vector dinámico . . . 96
4.13 Medición empírica vs. teórica . . . 98
4.14 Ejemplo integrador . . . 98
4.15 Errores comunes . . . 98
4.16 Repaso . . . 98

Guía de ejercicios — Unidad 4 **100**
4.17 Preguntas teóricas . . . 100
4.18 Nivel fácil . . . 100
4.19 Nivel medio . . . 100
4.20 Nivel difícil . . . 101

5 Estructuras de datos lineales **102**
5.1 El TDA Vector (arreglo dinámico) . . . 102
5.2 El TDA Conjunto (Set) . . . 110
5.3 Listas enlazadas . . . 113
5.4 Recorrer listas enlazadas: iteradores . . . 119
5.5 Pila (LIFO) . . . 122
5.6 Cola (FIFO) . . . 123
5.7 Comparación de complejidades . . . 125
5.8 Cuadro resumen: fortalezas y debilidades . . . 126
5.9 De lo nuestro a la API de Java . . . 126
5.10 Ejemplo integrador: un sistema de turnos . . . 127
5.11 El checklist del ejercicio de listas . . . 128
5.12 Errores comunes . . . 129
5.13 Repaso . . . 130

Guía de ejercicios — Unidad 5 **131**
5.14 Preguntas teóricas . . . 131
5.15 Nivel fácil (con la API de Java) . . . 131
5.16 Nivel medio (con la API de Java) . . . 131
5.17 Nivel difícil (implementadas a mano) . . . 132

6 Recursividad y División y Conquista **133**
6.1 Un poco de historia . . . 133
6.2 Recursividad . . . 133
6.3 Tipos de recursividad . . . 136
6.4 Cuidado con los subproblemas repetidos: Fibonacci . . . 137
6.5 División y conquista . . . 138
6.6 Mergesort . . . 139
6.7 Quicksort . . . 141
6.8 Multiplicación rápida: Karatsuba . . . 142
6.9 Ordenar sin comparar . . . 143
6.10 Ejemplo integrador: las Torres de Hanói . . . 148
6.11 Errores comunes . . . 149
6.12 Repaso . . . 150

Guía de ejercicios — Unidad 6 **151**
6.13 Preguntas teóricas . . . 151
6.14 Nivel fácil (recursión directa) . . . 151
6.15 Nivel medio (división y conquista y patrones recursivos) . . . 151
6.16 Nivel difícil (recursión avanzada y ordenamiento no comparativo) . . . 152

7 Diccionarios y tablas de hash 153
7.1 Un poco de historia . . . 153
7.2 El problema: buscar rápido . . . 153
7.3 El TDA Diccionario . . . 153
7.4 La función de hash . . . 154
7.5 Las colisiones son inevitables . . . 155
7.6 Resolución 1: encadenamiento . . . 156
7.7 Resolución 2: direccionamiento abierto . . . 157
7.8 Factor de carga y rehashing . . . 158
7.9 La complejidad, con precisión . . . 159
7.10 Las claves y el contrato equals/hashCode . . . 159
7.11 De lo nuestro a la API de Java . . . 160
7.12 Ejemplo integrador: contar palabras . . . 161
7.13 Errores comunes . . . 161
7.14 Repaso . . . 161

Guía de ejercicios — Unidad 7 163
7.15 Preguntas teóricas . . . 163
7.16 Nivel fácil (con la API de Java) . . . 163
7.17 Nivel medio (con la API de Java) . . . 163
7.18 Nivel difícil (implementadas a mano) . . . 164

8 Árboles 165
8.1 Un poco de historia . . . 165
8.2 Terminología: el vocabulario de los árboles . . . 165
8.3 El árbol binario de búsqueda (ABB) . . . 166
8.4 El árbol AVL: balanceo automático . . . 169
8.5 El árbol rojo-negro (la idea) . . . 173
8.6 Árboles B y B+ (la idea) . . . 173
8.7 De lo nuestro a la API de Java . . . 174
8.8 Ejemplo integrador: autocompletar . . . 175
8.9 Errores comunes . . . 175
8.10 Repaso . . . 175

Guía de ejercicios — Unidad 8 177
8.11 Preguntas teóricas . . . 177
8.12 El ejercicio de simulación: insertar, buscar y eliminar en cada árbol . . . 177
8.13 Nivel fácil (con la API de Java) . . . 178
8.14 Nivel medio (nodos propios y API) . . . 178
8.15 Nivel difícil (implementados a mano) . . . 178

9 Colas de prioridad y heaps 180
9.1 Un poco de historia . . . 180
9.2 El TDA Cola de Prioridad . . . 180
9.3 El heap binario: un árbol completo en un arreglo . . . 180
9.4 Encolar: entrar al final y flotar . . . 181
9.5 Desencolar el mínimo: la última hoja a la raíz, y hundir . . . 182
9.6 Heapify: construir un heap en O(n) . . . 183
9.7 Heapsort: ordenar con el heap . . . 184
9.8 La complejidad, junta . . . 185
9.9 De lo nuestro a la API de Java . . . 185
9.10 Ejemplo integrador: el triage de una guardia . . . 186
9.11 Errores comunes . . . 186
9.12 Repaso . . . 186

Guía de ejercicios — Unidad 9 188
9.13 Preguntas teóricas . . . 188

9.14 Nivel fácil (con `PriorityQueue`) . . . 188
9.15 Nivel medio (patrones clásicos con `PriorityQueue`) . . . 188
9.16 Nivel difícil (heap implementado a mano) . . . 189

10 Técnicas de diseño de algoritmos 190
10.1 Un poco de historia . . . 190
10.2 Backtracking: explorar con vuelta atrás . . . 190
10.3 Greedy: el atajo goloso . . . 192
10.4 Programación dinámica: no calcular nada dos veces . . . 194
10.5 ¿Qué técnica uso? La guía de decisión . . . 196
10.6 Ejemplo integrador: el vuelto por triplicado . . . 197
10.7 Errores comunes . . . 197
10.8 Repaso . . . 197

Guía de ejercicios — Unidad 10 198
10.9 Preguntas teóricas . . . 198
10.10 Nivel fácil . . . 198
10.11 Nivel medio . . . 199
10.12 Nivel difícil . . . 199

11 Grafos 200
11.1 Un poco de historia . . . 200
11.2 Qué es un grafo . . . 200
11.3 Cómo se representa un grafo . . . 201
11.4 Recorrer un grafo: BFS y DFS . . . 203
11.5 Orden topológico: el plan de correlatividades . . . 206
11.6 Caminos mínimos con pesos: Dijkstra . . . 207
11.7 Conectar todo al menor costo: árboles de tendido mínimo . . . 210
11.8 Ejemplo integrador: el planificador de viajes . . . 213
11.9 Errores comunes . . . 214
11.10 Repaso . . . 214

Guía de ejercicios — Unidad 11 216
11.11 Preguntas teóricas . . . 216
11.12 Nivel fácil . . . 216
11.13 Nivel medio . . . 217
11.14 Nivel difícil . . . 217

12 Colecciones, Streams y el trabajo profesional 219
12.1 Un poco de historia . . . 219
12.2 El mapa completo de las colecciones . . . 219
12.3 Streams: de decir CÓMO a decir QUÉ . . . 221
12.4 Cómo se trabaja hoy en una empresa . . . 223
12.5 Ejemplo integrador: el informe de ventas . . . 226
12.6 Errores comunes . . . 227
12.7 Repaso . . . 227
12.8 Cierre del curso . . . 228

Guía de ejercicios — Unidad 12 229
12.9 Preguntas teóricas . . . 229
12.10 Nivel fácil . . . 229
12.11 Nivel medio . . . 230
12.12 Nivel difícil . . . 230

13 Programación agéntica 232
13.1 Un poco de historia (muy reciente) . . . 232
13.2 Chat vs. agente: la diferencia que importa . . . 232

13.3 El bucle agéntico 233
13.4 Las herramientas: las manos del agente 234
13.5 La nueva forma de trabajo: de escribir a dirigir 235
13.6 Lo bueno y lo “malo” de esta era 236
13.7 El espectro de autonomía 236
13.8 Nomenclatura: el idioma de la era agéntica 237
13.9 ¿Y el programador en diez años? 240
13.10 Errores comunes 241
13.11 Repaso 241

Unidad 1A

Presentación de la materia

Objetivos de la unidad

Esta unidad te da el mapa completo: **qué** vas a aprender en CB100, **cómo** está organizada la cursada, **cómo se aprueba** y **cómo aprovechar este apunte**. Es corta; leela antes de arrancar.

Bienvenida y bienvenido a **Algoritmos y Estructuras de Datos (CB100)**. Venís de una materia de programación, así que ya sabés pensar en variables, condiciones, ciclos y funciones. Acá vamos a dar un salto: aprender a **diseñar** las estructuras con las que se organizan los datos, a **analizar** cuán eficientes son nuestros algoritmos, y a hacerlo con **Java** y con las **herramientas que se usan en la industria**. El objetivo es que te lleves conocimiento y práctica que sirvan en tu vida profesional.

1A.1 Datos útiles de la cátedra

Todo lo que necesitás tener a mano durante la cursada, en un solo lugar:

Teórica	miércoles de 19:00 a 22:00
Práctica	jueves de 19:00 a 22:00
Meet de todas las clases	https://meet.google.com/yta-tksg-zzr
Canal de YouTube	https://www.youtube.com/channel/UCoX2PLLQxmPR7Z8OFB9ti5g/videos
GitHub de la cátedra	https://github.com/gschmidt-cb100/cb100
Contacto	Gustavo Schmidt — gschmid@fi.uba.ar

Importante

Todas las clases son virtuales, por el mismo link de Meet de la tabla (teóricas y prácticas), y **arrancan puntual** — conectate unos minutos antes. Las clases quedan grabadas en el canal de YouTube. Las únicas instancias **presenciales** son los **exámenes** (parcial, recuperatorios y final).

1A.2 El mapa del curso

Estos son los grandes temas que vamos a recorrer. Cada uno tiene su unidad en el apunte, con teoría, diagramas, código en el repositorio y ejercicios.

- **Introducción a Java y el entorno.** El lenguaje desde cero y las herramientas: IntelliJ, Gradle, Git e IA.
- **Memoria y referencias.** Cómo se guardan los datos, referencias vs. punteros, y el recolector de basura.
- **Abstracción, TDA y POO.** El corazón del curso: separar *qué hace* una estructura de *cómo está hecha*, usando interfaces, genéricos y objetos.
- **Complejidad computacional.** Medir cuánto cuesta un algoritmo: notación O, Ω y Θ, algoritmos iterativos y recursivos, y el Teorema Maestro.
- **División y conquista.** Resolver un problema partiéndolo en subproblemas: Mergesort, Quicksort, multiplicación rápida.

- **Ordenamiento no comparativo.** Ordenar sin comparar elementos: Counting, Radix y Bucket sort.
- **Estructuras lineales.** Vector dinámico, listas enlazadas, pilas y colas, con su análisis amortizado.
- **Diccionarios y hashing.** Búsqueda en tiempo constante y cómo resolver colisiones.
- **Árboles.** Árboles binarios de búsqueda, autobalanceados (AVL, rojo-negro) y B/B+.
- **Colas de prioridad y heaps.** La estructura detrás de muchos algoritmos eficientes.
- **Técnicas de diseño.** Programación dinámica, algoritmos golosos (*greedy*) y *backtracking*.
- **Grafos.** Representaciones, recorridos (BFS/DFS), orden topológico, caminos mínimos y árboles de tendido mínimo.
- **Programación agéntica.** El cierre mirando adelante: dirigir agentes de IA — el bucle agéntico, las herramientas, la nueva forma de trabajo y su vocabulario.

Idea clave

Hay una idea que atraviesa toda la materia: **primero pensás el TDA (qué operaciones ofrece una estructura y qué garantiza), después elegís la implementación (cómo lo hacés) y siempre analizás su complejidad (cuánto cuesta)**. Ese triángulo —contrato, implementación, costo— se repite en cada unidad.

1A.3 Cómo se aprueba

La materia se aprueba cumpliendo con las siguientes instancias:

En orden cronológico durante la cursada:

1. **Un Trabajo Práctico individual**, donde resolvés y entregás tu propio código. Es la primera entrega de la cursada.
2. **Un examen parcial**, con **dos recuperatorios** (tenés dos oportunidades adicionales para recuperarlo).
3. **Un Trabajo Práctico grupal**, para practicar el trabajo en equipo con Git, como en la industria.
4. **Un examen final obligatorio**, que integra todos los contenidos y es requisito para aprobar la materia.

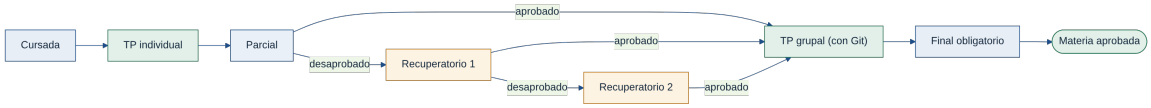


Figura 1A.1: El camino de aprobación, en orden: primero el TP individual, después el parcial (con sus dos recuperatorios si hace falta), después el TP grupal, y al final el examen final obligatorio.

Idea clave

El TP individual y el grupal no son sólo una nota: son la práctica donde el código del apunte se vuelve tuyo. Usá el repositorio de la materia, hacé commits seguido y corré los tests antes de entregar.

1A.4 Uso de inteligencia artificial en la materia

La IA es parte de las herramientas del curso (vas a usar el plugin de Claude en IntelliJ desde la Unidad 1B), así que las reglas son explícitas:

- **Para estudiar: usala todo lo que quieras.** Pedirle que te explique un concepto de otra forma, que te proponga ejercicios extra o que te ayude a entender un error es un uso excelente.
- **En los TP: permitida, con dos condiciones.** (1) Tenés que **entender y poder defender** cada línea que entregás — en la evaluación se pregunta sobre tu propio código, y no saber

explicarlo equivale a no haberlo hecho. (2) Declará el uso: un comentario o una nota en el README indicando en qué partes te asistió.
- **En parciales y finales: no está permitida.** Son individuales y sin asistencia, justamente para verificar lo que quedó en vos.

Con IA (plugin de Claude)

La trampa silenciosa no es "que te descubran": es llegar al parcial sin haber pensado nunca por tu cuenta. Usá la IA como un **profesor particular** (que te explica y te corrige), no como un **compañero que te pasa la tarea**. La diferencia se nota exactamente el día del parcial.

1A.5 Bibliografía y material de consulta

Este apunte es **autocontenido**: cubre todo lo que se evalúa. Para profundizar, estos son los libros de referencia de la materia (todos disponibles en biblioteca):

- **Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms*** (4.ª ed., MIT Press): la referencia clásica de algoritmos y complejidad; más formal que el curso.
- **Weiss — *Data Structures and Algorithm Analysis in Java*** (3.ª ed.): estructuras de datos en Java, muy alineado con nuestro temario.
- **Sedgewick, Wayne — *Algorithms*** (4.ª ed.): excelente equilibrio entre teoría y código Java; su sitio web tiene visualizaciones.
- **Goodrich, Tamassia, Goldwasser — *Data Structures and Algorithms in Java*** (6.ª ed.): enfoque TDA muy compatible con la Unidad 3.
- **Bloch — *Effective Java*** (3.ª ed.): para escribir Java **bien**; lectura recomendada después de la Unidad 3.
- **Documentación oficial:** docs.oracle.com/en/java — el Javadoc de la API es la fuente de verdad sobre cada clase.

Y los canales de la cátedra: las **consultas** se responden en el campus de la materia y en los horarios de consulta presenciales que se anuncian al inicio del cuatrimestre; el **repositorio Git** con todo el código del apunte se comparte la primera semana.

1A.6 Cómo usar este apunte

Cada unidad sigue la misma estructura para que te orientes fácil: **objetivos**, la **teoría** con diagramas, el **código en Java** (que vive en el repositorio de la materia y podés ejecutar), el **análisis de complejidad**, ejercicios **resueltos** y **propuestos**, los **errores comunes** y un **repaso** final.

A lo largo del texto vas a encontrar cajas de colores que cumplen siempre el mismo rol:

- **Idea clave:** lo más importante para retener.
- **Error común:** trampas típicas que conviene evitar.
- **En IntelliJ:** cómo hacerlo con el IDE.
- **Con Git / Gradle:** el comando o flujo de trabajo.
- **Con IA:** cómo apoyarte en el asistente de IA con criterio.

Todo el código que aparece está en el repositorio Git de la materia: se clona una vez y se actualiza durante la cursada. En la Unidad 1B lo ponemos en marcha.

Unidad 1B

Introducción a Java y el entorno de trabajo

Objetivos de la unidad

Esta unidad es tu **guía completa del lenguaje**. Al terminarla vas a poder: (1) contar qué es Java, de dónde viene y por qué sigue tan vigente; (2) explicar cómo la JVM ejecuta un programa; (3) usar IntelliJ, Gradle, Git e IA; (4) escribir programas con **toda la sintaxis de Java** (tipos, rangos y conversiones, `String`/`StringBuilder`, operadores, control de flujo, métodos, clases, `enum`, `record`, lambdas, fechas, entrada/salida y archivos); (5) **validar** datos y manejar errores con excepciones; y (6) probar tu código con JUnit 5.

Ya sabés programar: entendés qué es una variable, un `if`, un `for` o una función. Lo que todavía no conocés es **cómo se dicen esas cosas en Java** y **con qué herramientas** se trabaja hoy. Esta unidad cubre exactamente eso, desde cero. No hace falta saber nada de Java para empezar.

1B.1 Un poco de historia: por qué Java

Para entender Java conviene saber de dónde viene. A principios de los años 90, un equipo de **Sun Microsystems** liderado por **James Gosling** trabajaba en un lenguaje para electrodomésticos inteligentes (el “Green Project”). Ese lenguaje se llamó primero **Oak**. En **1995** se presentó públicamente como **Java**, y su gran promesa cambió las reglas: **“Write Once, Run Anywhere”** (escribí una vez, ejecutá en cualquier lado). En una época en la que cada programa había que recompilarlo para cada sistema, poder escribir un programa y correrlo igual en Windows, Mac o Unix fue una pequeña revolución. Los *applets* de Java, que corrían dentro del navegador, popularizaron el lenguaje casi de inmediato.

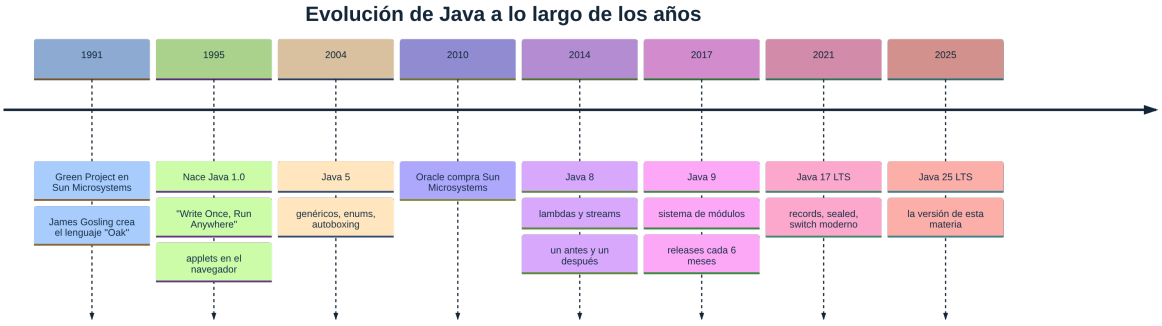


Figura 1B.1: La evolución de Java a lo largo de los años, de “Oak” a la versión que usamos hoy.

Con los años Java maduró: en 2004 (Java 5) llegaron los genéricos y los `enum`; en 2014 (Java 8), las *lambdas* y los *streams*, otro antes y después. En 2010 **Oracle** compró Sun, y desde 2017 hay una versión nueva cada seis meses, con versiones **LTS** (*Long-Term Support*, soporte de largo plazo) cada dos años. Nosotros usamos **Java 25**, la LTS publicada en septiembre de 2025.[1]

[1] Java 25 es la versión LTS lanzada el 16 de septiembre de 2025 (Oracle / OpenJDK, proyecto JDK 25).

¿Y hoy? Java sigue siendo uno de los lenguajes más elegidos del mundo. En el índice **TIOBE**, que mide la popularidad de los lenguajes, Java se mantiene entre los primeros puestos: en la edición de **agosto de 2026** está **4.º con 8,25 %**, a sólo 0,37 puntos de C++ — peleando el podio con Python, C y C++.[2] Está en el corazón de miles de sistemas bancarios, de gobierno y de empresas, y es la base histórica del desarrollo **Android**. Aprender Java no es aprender una curiosidad académica: es una herramienta que vas a poder usar profesionalmente durante muchos años.

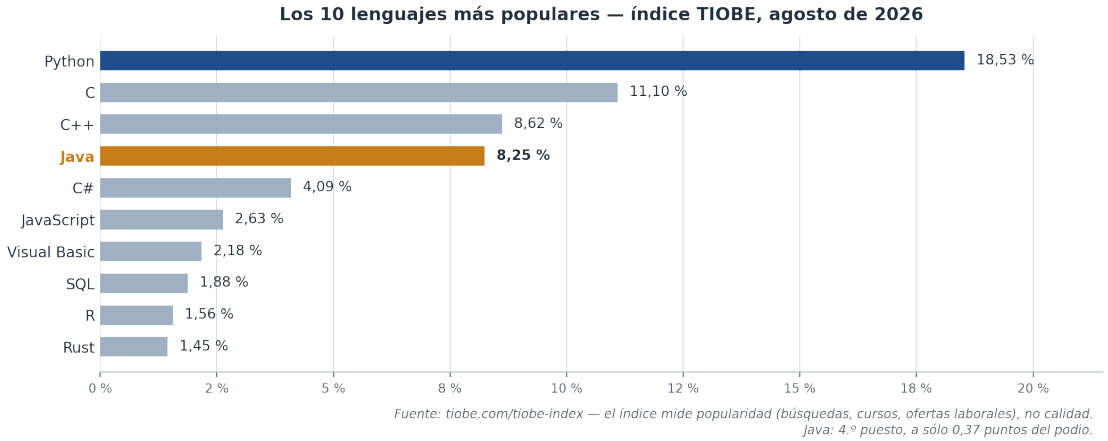


Figura 1B.2: El top 10 del índice TIOBE en la edición de agosto de 2026. Python lidera cómodo, y Java (4.º, 8,25 %) pelea el podio con C++ a menos de medio punto. Cuando leas esto el número exacto habrá cambiado — lo que no cambia hace 25 años es que Java está en los primeros puestos.

Idea clave

Java combina tres cosas que explican su longevidad: es **portable** (corre en cualquier máquina con una JVM), es **robusto y tipado** (muchos errores se detectan al compilar, no en producción) y tiene un **ecosistema gigante** de librerías y herramientas. Por eso sigue vigente 30 años después.

1B.2 ¿Cómo se ejecuta un programa Java?

Java es un lenguaje **compilado a un formato intermedio** que luego ejecuta una máquina virtual. Esto lo diferencia de lenguajes como C, que compilan directamente a instrucciones del procesador.

Cuando escribís un archivo `Hola.java` y lo compilás con `javac`, no obtenés un ejecutable para tu procesador, sino **bytecode**: un archivo `Hola.class` con instrucciones para una computadora "imaginaria" llamada **JVM** (*Java Virtual Machine*). Al correr el programa, la JVM lee ese bytecode y lo traduce a instrucciones reales de tu CPU, optimizando sobre la marcha con el compilador **JIT** (*Just-In-Time*).

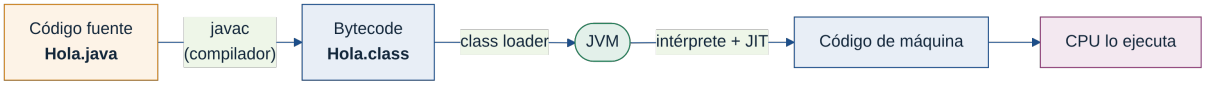


Figura 1B.3: Del código fuente a la CPU: `javac` produce bytecode, y la JVM lo ejecuta traduciéndolo a código de máquina.

Acá está la clave de "Write Once, Run Anywhere": el mismo `.class` corre igual en cualquier sistema, porque lo que cambia de una máquina a otra es **la JVM**, no tu programa.

[2] Índice TIOBE, edición de agosto de 2026 (tiobe.com/tiobe-index). El índice mide *popularidad* — búsquedas, cursos, avisos laborales —, no calidad; sirve como termómetro de qué se usa en la industria. Dato de color: el "lenguaje del año 2025" fue C#.

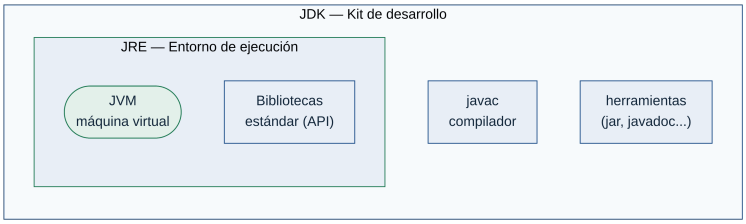


Figura 1B.4: El JDK contiene todo lo necesario para desarrollar; adentro, el JRE contiene lo necesario para ejecutar.

Para escribir un `.java` alcanza con un editor de texto, pero para **compilarlo y ejecutarlo** necesitás el **JDK** (*Java Development Kit*): incluye el compilador `javac`, herramientas varias y, adentro, el entorno de ejecución (**JRE**) con la **JVM** y las bibliotecas estándar. Cuando instales el JDK 25, verificá la versión en una terminal:

```
java -version      # muestra la versión de la JVM
javac -version     # muestra la versión del compilador
```

1B.3 Las herramientas del curso

No vas a trabajar con el Bloc de notas: vas a usar el mismo stack que se usa en la industria. Son cuatro piezas, cada una con un rol claro.

1B.3.1 IntelliJ IDEA — el entorno de desarrollo

IntelliJ IDEA es el *IDE* (entorno de desarrollo integrado): el programa donde escribís, ejecutás y depurás tu código. Integra en un solo lugar el editor inteligente, el compilador, el depurador y el control de versiones. Existe en dos ediciones: **Community** (gratuita y de código abierto) y **Ultimate** (de pago, con más funciones).

Idea clave

IntelliJ IDEA Ultimate es gratis para estudiantes. Con tu correo universitario podés obtener el *JetBrains Student Pack*, que incluye Ultimate y todos los demás IDEs de JetBrains sin costo mientras seas estudiante.[a] Para la materia, tanto Community como Ultimate sirven perfectamente.

[a] JetBrains ofrece licencias educativas gratuitas para estudiantes y docentes (JetBrains Student Pack), con verificación por correo universitario u otros medios. No pueden usarse con fines comerciales.

Así se ve la ventana con la que vas a convivir todo el cuatrimestre. Son tres zonas que se usan en este orden: el **árbol del proyecto** (izquierda) para moverte entre carpetas y clases; el **editor** (centro) donde escribís —con el triangulito verde del margen para ejecutar la clase—; y la **consola Run** (abajo), donde aparece la salida del programa:

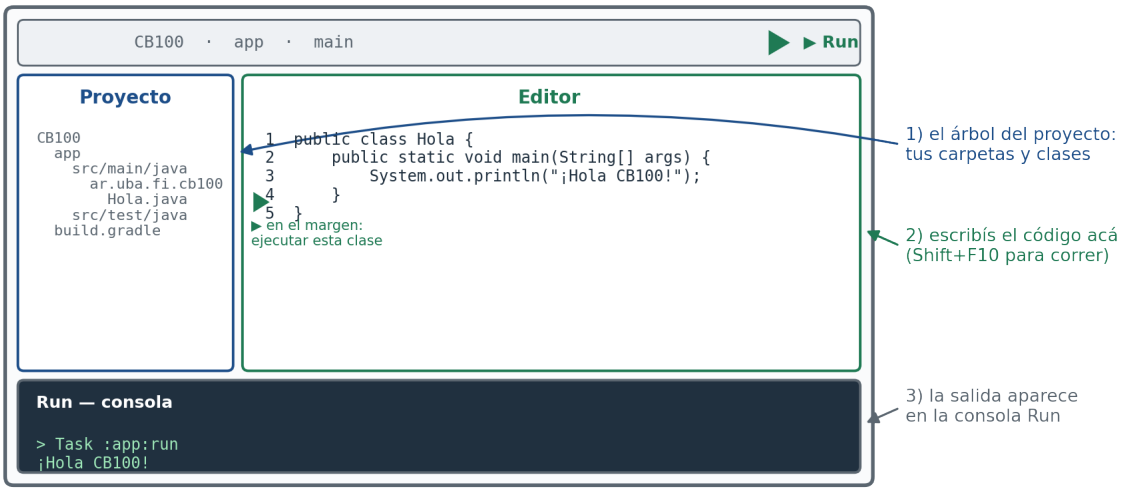


Figura 1B.5: Esquema de la ventana de IntelliJ: árbol del proyecto, editor con el botón de ejecución en el margen, y consola Run con la salida.

El flujo mínimo para tu primera vez: abrís el proyecto de la materia (`File → Open` sobre la carpeta clonada), esperás que Gradle lo importe (barra de progreso abajo), buscás la clase en el árbol, y la ejecutás con el **▶ del margen** (o **Shift + F10**). La salida aparece en la consola Run. Si algo no compila, IntelliJ lo subraya en rojo **antes** de ejecutar: leé el mensaje, que casi siempre dice exactamente qué falta.

En IntelliJ

Atajos que vas a usar todos los días: **Shift + F10** ejecuta el programa, **Shift + F9** lo ejecuta en modo depuración, **Ctrl + Alt + L** ordena el formato del código, y escribir `psvm` + Tab genera un método `main` completo. Con **Alt + Insert** IntelliJ te genera constructores, getters/setters y `toString`.

1B.3.2 Gradle — la herramienta de construcción

Gradle automatiza compilar, bajar librerías, correr los tests y empaquetar el programa. En vez de invocar `javac` a mano sobre decenas de archivos, le pedís a Gradle una tarea y se ocupa de todo, siguiendo las instrucciones del archivo `build.gradle`. No necesitás instalarlo: el proyecto trae el *Gradle Wrapper* (`gradlew`), que baja la versión correcta automáticamente.

1B.3.3 Git — el control de versiones

Git guarda la historia de tu código, te deja volver atrás y es la base para trabajar en equipo. Todo el código de la materia vive en un repositorio Git.

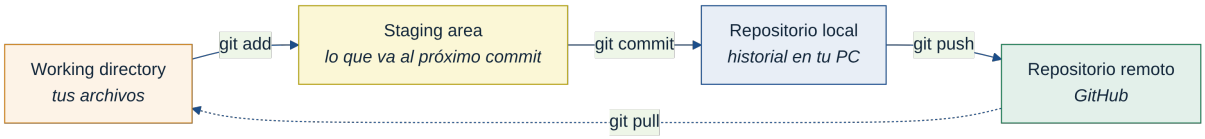


Figura 1B.6: El flujo básico de Git: tus cambios pasan del *working directory* al *staging* y de ahí a un *commit* en el repositorio.

Con Git / Gradle

Los cuatro comandos que resuelven el 90 % del uso diario:

```
git status                # ¿qué cambió?
git add .                 # preparar todos los cambios
git commit -m "mensaje"   # guardar una versión con una descripción
git push                  # subirla al repositorio remoto
```

1B.3.4 IA — un asistente para aprender

Vas a poder usar **IA** como asistente a través del *plugin de Claude* integrado en IntelliJ. Sirve para explicarte un error, sugerir una implementación o proponer tests. Sobre el acceso: **Claude tiene un plan gratuito** que podés usar desde ya, y además existe **Claude for Education**, un programa por el cual los estudiantes de instituciones asociadas acceden sin costo a funciones pensadas para estudiar (como el *Learning Mode*, que en vez de darte la respuesta te guía con preguntas).[3] Vamos a dedicarle una unidad entera a trabajar bien con IA.

Con IA (plugin de Claude)

La IA acelera el aprendizaje sólo si la usás para **comprender**, no para saltearte el pensar. Buenas preguntas: *"explicame este método línea por línea"*, *"¿por qué esto lanza `NullPointerException`?"*, *"escribime tests para esta clase"*. Mala idea: pegar un enunciado y copiar la respuesta sin entenderla.

1B.4 Tu primer programa

Todo programa Java vive dentro de una **clase**, y la ejecución arranca siempre en un método especial llamado `main`. Este es el "Hola mundo" de la materia:

```java
package ar.uba.fi.cb100.material.i01_intro;

public class HolaCB100 {

    // El método main es SIEMPRE el punto de arranque del programa.
    public static void main(String[] args) {
        String materia = "CB100 — Algoritmos y Estructuras de Datos";
        System.out.println("¡Hola! Bienvenidas y bienvenidos a " + materia);
    }
}
```

Leámoslo. La línea `package` indica en qué "carpeta lógica" vive la clase. `public class HolaCB100` declara la clase, que **debe** llamarse igual que el archivo (`HolaCB100.java`). Adentro, la firma `public static void main(String[] args)` es la fórmula exacta que la JVM busca para empezar. Y la línea `String materia = ...` declara una **variable**: primero el tipo (`String`, texto), después el nombre, después el valor — la anatomía de toda declaración en Java, que vemos en detalle enseguida.

Idea clave

Descomponé la firma de `main`: `public` (accesible desde afuera), `static` (pertenece a la clase, no a un objeto), `void` (no devuelve nada), `main` (el nombre que la JVM busca) y `String[] args` (los argumentos de la línea de comandos). La vas a escribir cientos de veces.

Para ejecutarlo con Gradle: `./gradlew run`.

[3] Claude for Education brinda acceso gratuito a estudiantes de universidades asociadas mediante licencia institucional; además existe el plan gratuito de Claude. Las cátedras e instituciones pueden solicitar el programa en claude.com/solutions/education.

1B.5 Comentarios y constantes

Los **comentarios** explican el código sin afectar la ejecución. Hay tres formas:

```
// comentario de una línea

/* comentario
   de varias líneas */

/**
 * Comentario Javadoc: describe una clase o método y sirve para generar
 * documentación automáticamente. IntelliJ lo muestra al pasar el mouse.
 */
```

Una **constante** es una variable que no puede cambiar una vez asignada: se marca con `final`. Por convención, las constantes se escriben en `MAYUSCULAS`.

```
final double PI = 3.14159;
static final int CANTIDAD_MAXIMA = 100;   // final + static: constante de clase
```

1B.6 Tipos de datos y variables

Java es **fuertemente tipado**: cada variable tiene un tipo que no cambia. Hay dos grandes familias. Los **tipos primitivos** guardan el valor directamente. Los **tipos por referencia** (todo lo demás: `String`, arreglos, tus clases) guardan una *referencia* a un objeto.

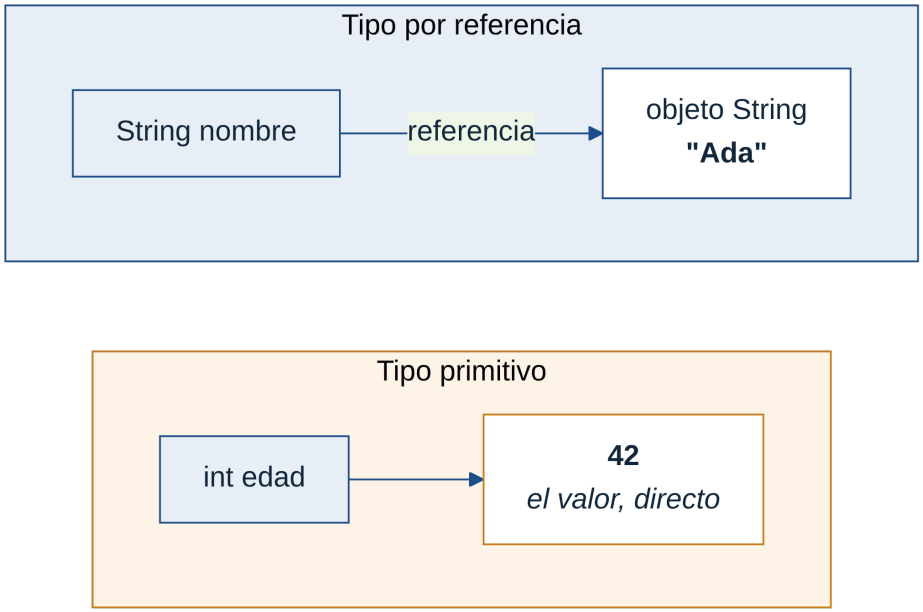


Figura 1B.7: Un primitivo guarda el valor; una referencia “apunta” a un objeto en memoria.

Estos son los tipos primitivos que más vas a usar:

Tipo	Guarda	Ejemplo
`int`	entero de 32 bits	`int n = 42;`
`long`	entero de 64 bits	`long g = 9_000L;`
`double`	número real (punto flotante)	`double x = 3.14;`
`boolean`	valor lógico	`boolean ok = true;`
`char`	un carácter Unicode[4]	`char c = 'A';`

```java
int entero = 42;                // enteros de 32 bits
double real = 3.14159;          // punto flotante de 64 bits
boolean verdadero = true;       // true / false
char letra = 'A';               // un carácter Unicode
String mensaje = "Hola";        // referencia: texto
```

1B.6.1 La palabra `var`: inferencia de tipos (con criterio)

Desde Java 10 existe `var`: le deja al compilador el trabajo de **inferir** el tipo a partir del valor de la derecha, sin dejar de ser tipado:

```java
var mensaje = "Hola";           // el compilador infiere String
var pi = 3.14;                  // ... infiere double
var texto = new StringBuilder(); // ... infiere StringBuilder
```

Idea clave

`var` no convierte a Java en un lenguaje dinámico: el tipo se fija **al compilar** y no cambia nunca — `var mensaje = "Hola"; mensaje = 42;` no compila. Es exactamente lo mismo que escribir el tipo, ahorrándote repetirlo cuando ya es evidente.

Buenas prácticas con `var` (las mismas que vas a encontrar en la industria):

- **Usalo cuando el tipo se ve a simple vista** del lado derecho: `var texto = new StringBuilder()`, `var archivo = Path.of("datos.txt")`. Ahí repetir el tipo es puro ruido.
- **Evitalo cuando el tipo NO se ve**: en `var resultado = procesar(datos);` el lector no sabe qué es `resultado` sin ir a mirar la firma de `procesar`. Con `double resultado = procesar(datos);` el renglón se explica solo.
- **Sólo funciona en variables locales** (adentro de un método): no existe para atributos, parámetros ni tipos de retorno — ahí el tipo explícito es obligatorio, porque es parte del contrato.
- **Mientras aprendés, preferí el tipo explícito**: escribir `String`, `int` o `double[]` te obliga a saber qué tenés entre manos. En este apunte vas a ver tipos explícitos en los ejemplos que enseñan un concepto, y `var` sólo donde el tipo es obvio y repetirlo distrae (p. ej. `try (var sc = new Scanner(...))`).
- La regla de oro: `var` está para **quitar ruido**, nunca para ocultar información. Si al leer el renglón dudás del tipo, escribilo.

1B.7 Rangos, conversiones y wrappers

Cada tipo primitivo tiene un **rango** de valores. Si te pasás del máximo, el valor "da la vuelta" (*overflow*): no da error, pero el resultado es incorrecto. Las clases envoltorio exponen esos límites (`Integer.MAX_VALUE`, etc.).

```java
System.out.println(Integer.MIN_VALUE + " .. " + Integer.MAX_VALUE);
int max = Integer.MAX_VALUE;
System.out.println(max + 1);    // desborda a -2147483648 (¡negativo!)
```

Conversiones (casting). Pasar de un tipo más chico a uno más grande es automático (*widening*); al revés hay que pedirlo explícitamente (*narrowing*) y puede perder información:

```java
double d = 9.99;
int truncado = (int) d;         // 9  (se pierde el decimal)
long l = truncado;              // automático (int cabe en long)
```

[4] Con precisión: un `char` es una unidad UTF-16 de 16 bits. Para los caracteres habituales (letras, dígitos, símbolos comunes) equivale a "un carácter"; algunos símbolos fuera de ese rango (emojis, escrituras históricas) ocupan **dos** `char`. Para esta materia, la idea "un `char` = un carácter" alcanza.

Wrappers y autoboxing. Cada primitivo tiene una clase envoltorio: `int` → `Integer`, `double` → `Double`, `boolean` → `Boolean`, etc. Java convierte entre ambos automáticamente (*autoboxing* / *unboxing*). Los wrappers también sirven para convertir desde y hacia texto.

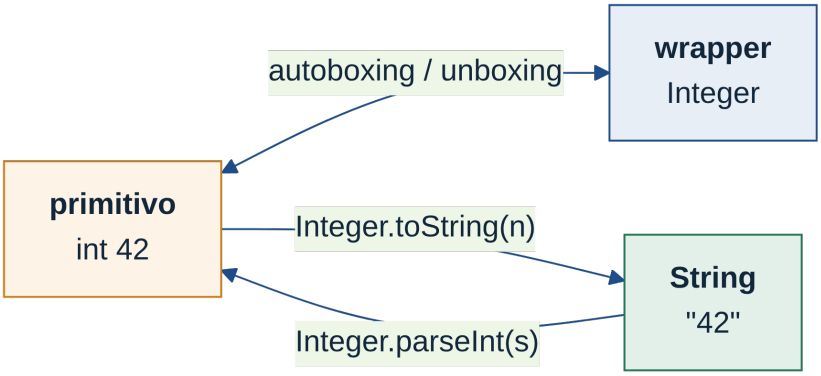


Figura 1B.8: El "triángulo" de conversiones: primitivo, wrapper y texto.

```
Integer boxed = 42;                  // autoboxing: int -> Integer
int back = boxed;                    // unboxing: Integer -> int

int n = Integer.parseInt("123");     // texto -> número
double x = Double.parseDouble("3.14");
String s = Integer.toString(255);    // número -> texto
```

Cuando necesitás **números muy grandes o cálculos con decimales exactos** (por ejemplo, dinero), están `BigInteger` y `BigDecimal`, que no tienen los límites ni los errores de redondeo de los primitivos:

```
BigInteger factorial = BigInteger.ONE;
for (int i = 1; i <= 30; i++) {
    factorial = factorial.multiply(BigInteger.valueOf(i));
}
BigDecimal precio = new BigDecimal("19.99").multiply(new BigDecimal("3")); // 59.97
```

Error común

Nunca uses `double` para representar **dinero**: `0.1 + 0.2` no da exactamente `0.3` en punto flotante. Para valores monetarios se usa `BigDecimal`.

1B.8 Cadenas de texto: `String` y `StringBuilder`

El `String` es uno de los tipos más usados. Ofrece muchos métodos, y todos **devuelven un nuevo texto sin modificar el original**, porque los `String` son **inmutables**.

```
String saludo = "Hola, CB100";
saludo.length();              // 11
saludo.toUpperCase();         // "HOLA, CB100"  (NO cambia saludo)
saludo.startsWith("Hola");    // true
saludo.indexOf(',');          // 4
saludo.substring(6);          // "CB100"
saludo.replace("Hola", "Chau"); // "Chau, CB100"
```

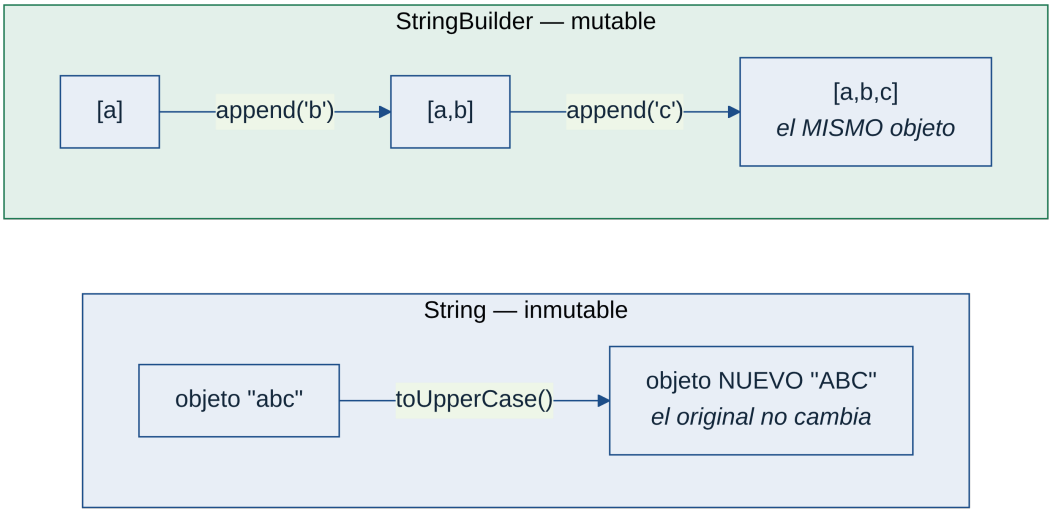


Figura 1B.9: El `String` es inmutable (cada método crea uno nuevo); el `StringBuilder` es mutable (modifica el mismo objeto).

Error común

Para comparar textos usá **`.equals(...)`** (o `.equalsIgnoreCase(...)`), nunca `==`. El `==` compara **referencias** (si son el mismo objeto), no el contenido.

Cuando tenés que **construir** un texto de a pedazos (por ejemplo, dentro de un ciclo), concatenar `String` con `+` crea muchos objetos intermedios y es ineficiente. Para eso está `StringBuilder`, que modifica un único objeto:

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    sb.append(i).append(' ');
}
String resultado = sb.toString().trim();   // "1 2 3 4 5"
```

Para dar formato a la salida se usan `printf` y `String.format`, con *especificadores* como `%d` (entero), `%f` (real), `%s` (texto):

```
System.out.printf("%d por ciento%n", 80);              // 80 por ciento
String linea = String.format("%-8s | %5.2f", "total", 3.14159); // "total    |  3.14"
```

Estos son los métodos más frecuentes del `String` (recordá: todos devuelven un valor nuevo, sin tocar el original):

Método	Qué hace
`length()`	cantidad de caracteres
`charAt(i)`	el carácter en la posición `i`
`substring(i, j)`	subcadena entre `i` (incl.) y `j` (excl.)
`indexOf(x)` / `lastIndexOf(x)`	primera / última posición de `x`
`contains(x)`	si contiene `x`
`startsWith(x)` / `endsWith(x)`	si empieza / termina con `x`
`toUpperCase()` / `toLowerCase()`	cambia mayúsculas/minúsculas
`strip()` / `trim()`	quita espacios de los extremos
`isBlank()` / `isEmpty()`	si está en blanco / vacío
`replace(a, b)`	reemplaza `a` por `b`
`split(sep)`	parte el texto en un arreglo
`String.join(sep, …)`	une varias partes con un separador

Método	Qué hace
`repeat(n)`	repite el texto `n` veces
`equals` / `equalsIgnoreCase`	compara contenido
`compareTo`	orden alfabético (negativo/cero/positivo)
`matches(regex)`	si cumple un patrón
`String.format(…)`	arma un texto con formato

Y estos son los de `StringBuilder` (que sí modifica el mismo objeto):

Método	Qué hace
`append(x)`	agrega `x` al final
`insert(i, x)`	inserta `x` en la posición `i`
`replace(i, j, s)`	reemplaza el tramo `i..j` por `s`
`delete(i, j)` / `deleteCharAt(i)`	borra un tramo / un carácter
`setCharAt(i, c)`	cambia el carácter en `i`
`charAt(i)` / `length()`	consulta
`indexOf(x)`	busca `x`
`reverse()`	invierte el contenido
`toString()`	lo convierte en un `String`

En el repositorio, las clases `MetodosDeString` y `MetodosDeStringBuilder` ejecutan todos estos métodos para que veas su resultado.

1B.9 Operadores

Los operadores de Java te van a resultar familiares. **Aritméticos:** `+`, `-`, `*`, `/`, `%` (resto). **De comparación:** `==`, `!=`, `<`, `>`, `<=`, `>=` (devuelven `boolean`). **Lógicos:** `&&` (y), `||` (o), `!` (no). **De asignación:** `=`, y las formas abreviadas `+=`, `-=`, `*=`, `++`, `--`.

Existe también el **operador ternario** `condición ? valorSiVerdadero : valorSiFalso`, una forma compacta de un `if`/`else` que devuelve un valor:

```
int edad = 20;
String tipo = (edad >= 18) ? "mayor" : "menor";
```

Un detalle importante: la división entre dos enteros da un entero (`7 / 2` es `3`); para obtener `3.5` al menos uno debe ser `double` (`7.0 / 2`).

1B.10 Matemática y azar: `Math` y `Random`

La clase `Math` reúne funciones matemáticas de uso común (son todos métodos `static`, no hace falta crear ningún objeto):

```
Math.abs(-5);        // 5
Math.max(3, 7);      // 7      Math.min(3, 7);  // 3
Math.pow(2, 10);     // 1024.0
Math.sqrt(144);      // 12.0
Math.ceil(2.1);      // 3.0    Math.floor(2.9); // 2.0
Math.round(2.5);     // 3
Math.PI;             // 3.141592653589793
```

Para generar valores al azar se usa `Random`. Si le pasás una **semilla** fija, siempre produce la misma secuencia, lo que es muy útil para tests reproducibles:

```java
Random r = new Random(42);          // semilla fija -> secuencia reproducible
int dado = r.nextInt(6) + 1;        // entero en 1..6
boolean cara = r.nextBoolean();
double x = r.nextDouble();           // real en [0, 1)
```

En el repo, `MatematicaYAzar` muestra estos métodos funcionando.

1B.11 Estructuras de control

Ya conocés estas ideas; acá está su sintaxis en Java.

1B.11.1 Condicionales: `if` y `switch`

El `if` / `else if` / `else` funciona como esperás:

```java
int nota = 7;
if (nota >= 8) {
    System.out.println("distinguido");
} else if (nota >= 4) {
    System.out.println("aprobado");
} else {
    System.out.println("insuficiente");
}
```

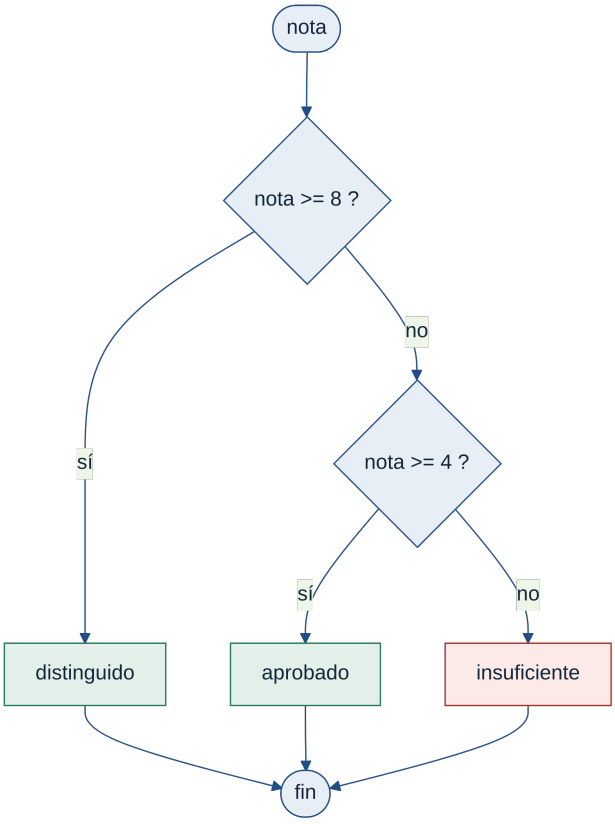


Figura 1B.10: El `if/else if/else` como flujo de decisiones.

Idea clave

Regla de estilo de la cátedra: las llaves van SIEMPRE, aunque el cuerpo sea una sola línea. `if (nota >= 4) { aprobado = true; }` y nunca `if (nota >= 4) aprobado = true;`. Las llaves marcan sin ambigüedad dónde empieza y dónde termina cada bloque, y evitan el clásico

> error de “agregué una segunda línea al if y quedó afuera”. Todo el código del apunte y del repositorio sigue esta regla — y se espera en TPs, parciales y finales.

Cuando hay que elegir entre muchos valores, el **switch expression** (moderno) es más claro: devuelve un valor, usa flechas `->`, no necesita `break` y el compilador te obliga a cubrir todos los casos.

```
String dia = "L";
String tipo = switch (dia) {
    case "L", "M", "X", "J", "V" -> "día hábil";
    case "S", "D" -> "fin de semana";
    default -> "desconocido";
};
```

1B.11.2 Ciclos: `for`, `while`, `do-while`

El `for` clásico tiene tres partes: inicialización, condición y actualización.

```
int suma = 0;
for (int i = 1; i <= 5; i++) {
    suma += i;                      // 1 + 2 + 3 + 4 + 5 = 15
}
```

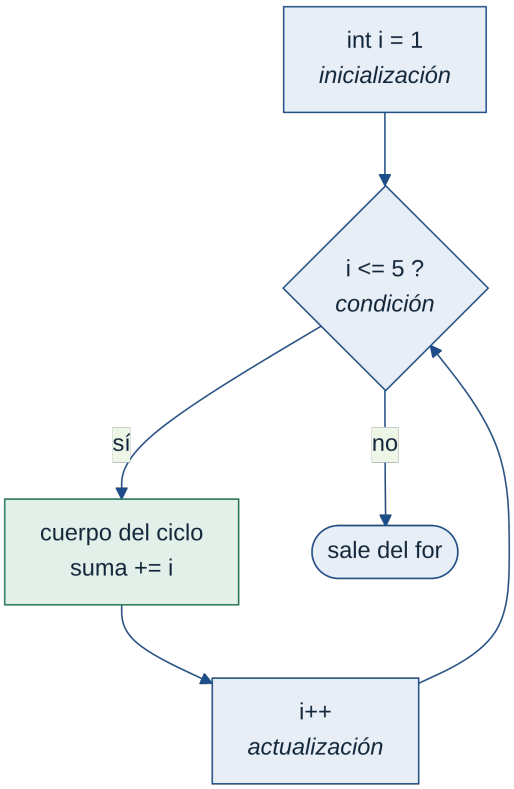


Figura 1B.11: Las tres partes del `for` y cómo se repite el ciclo.

El `while` repite mientras la condición sea verdadera:

```
int cuenta = 3;
while (cuenta > 0) {
    System.out.print(cuenta + " ");   // imprime: 3 2 1
    cuenta--;
}
```

El `do-while` es su hermano menos usado pero con una diferencia importante: ejecuta el cuerpo **al menos una vez**, porque chequea la condición **después**. Es la elección natural cuando algo debe ocurrir sí o sí la primera vez, como mostrar un menú y pedir una opción:

```
int opcion;
do {
    System.out.println("1) Jugar   2) Puntajes   3) Salir");
    opcion = scanner.nextInt();       // se pide AL MENOS una vez
} while (opcion != 3);                // repite hasta que elija salir
```

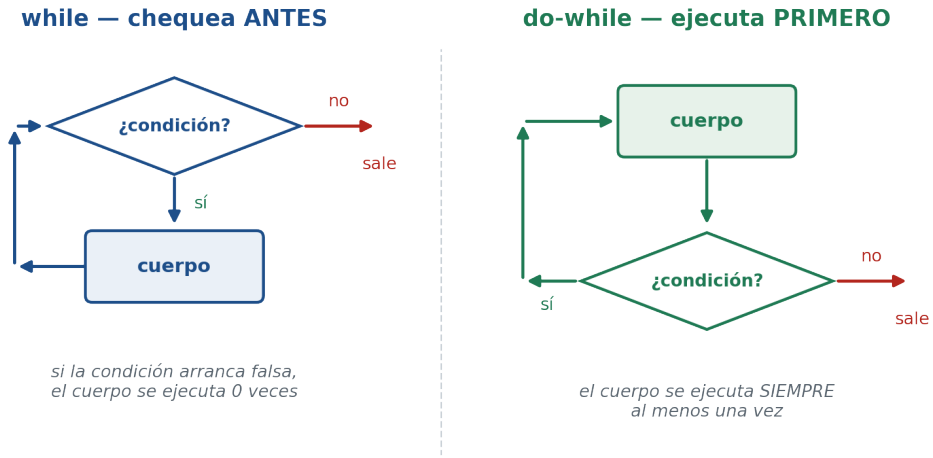


Figura 1B.12: `while` chequea la condición antes de entrar (puede ejecutar 0 veces); `do-while` ejecuta primero y chequea después (ejecuta al menos 1 vez).

Dentro de cualquier ciclo hay dos "controles de salto": **`break`** corta el ciclo por completo, y **`continue`** abandona la vuelta actual y pasa a la siguiente:

```
int[] datos = {7, -3, 12, 0, 5};
for (int d : datos) {
    if (d == 0) {
        break;   // un 0 corta la búsqueda: no mira el 5
    }
    if (d < 0) {
        continue;   // saltea los negativos: no imprime el -3
    }
    System.out.print(d + " ");         // imprime: 7 12
}
```

Usalos con moderación: un `break` claro (encontré lo que buscaba, corto) mejora el código; muchos saltos enredados lo vuelven ilegible.

1B.11.3 Arreglos y `for-each`

Un **arreglo** guarda muchos valores del mismo tipo en celdas contiguas y numeradas. Es la estructura más básica de todas —y la base de varias que vamos a construir—, así que vale la pena mirarla de cerca:

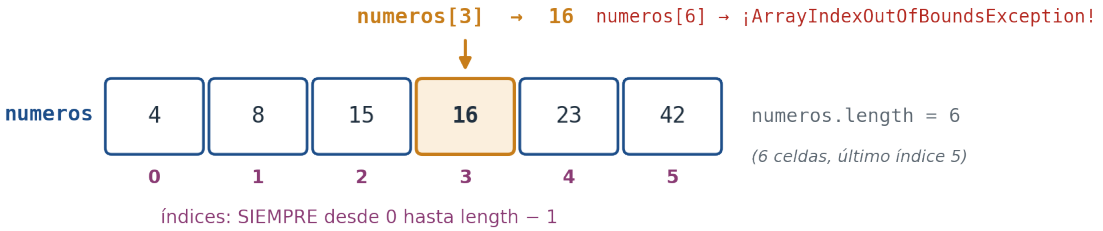


Figura 1B.13: Un arreglo de 6 enteros: las celdas se numeran con índices desde 0, `length` da la cantidad, y `numeros[3]` accede directo a la cuarta celda. Pasarse del rango lanza una excepción.

Tres reglas para no tropezar nunca: (1) los índices arrancan **en 0**, así que la primera celda es `numeros[0]` y la última `numeros[length - 1]`; (2) el tamaño es **fijo**: se decide al crear el arreglo y no cambia (para "crecer" hay que crear otro y copiar — exactamente lo que hará el vector dinámico de la Unidad 5); y (3) acceder fuera del rango (`numeros[6]` en un arreglo de 6) lanza `ArrayIndexOutOfBoundsException` al ejecutar.

```
int[] numeros = {4, 8, 15, 16, 23, 42};
numeros[0];                          // 4   (la primera celda)
numeros[numeros.length - 1];         // 42  (la última)
numeros[3] = 99;                     // también se puede escribir

int[] ceros = new int[5];            // {0,0,0,0,0}: se inicializa con "ceros"
```

Para recorrerlo sin manejar índices, se usa el **for-each**, que visita cada celda en orden:

```
int total = 0;
for (int n : numeros) {              // "para cada n en numeros"
    total += n;
}
System.out.println(total + " (largo " + numeros.length + ")");
```

Un arreglo también puede contener **arreglos**: es la forma de representar una **matriz** (tabla de filas y columnas). Se accede con dos índices, **primero la fila y después la columna**:

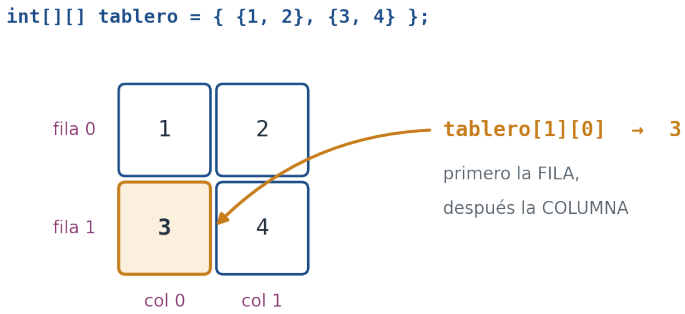


Figura 1B.14: Una matriz es un arreglo de arreglos: `tablero[1][0]` es el elemento de la fila 1, columna 0.

```
int[][] tablero = { {1, 2}, {3, 4} };   // 2 filas x 2 columnas
tablero[1][0];                           // 3  (fila 1, columna 0)

for (int f = 0; f < tablero.length; f++)          // recorrer una matriz:
    for (int c = 0; c < tablero[f].length; c++)   // un for por dimensión
        System.out.print(tablero[f][c] + " ");     // 1 2 3 4
```

1B.12 Métodos

Un **método** (función) agrupa un cálculo bajo un nombre. Se declara con un tipo de retorno, un nombre y sus parámetros. Java permite **sobrecarga** (mismo nombre, distintos parámetros) y **varargs** (cantidad variable de argumentos).

```
static int cuadrado(int x) {          // recibe un int, devuelve un int
    return x * x;
}

static double area(double lado) { return lado * lado; }           // sobrecarga
static double area(double base, double altura) { return base * altura; }

static int sumar(int... valores) {    // varargs: sumar(1,2,3,4)
    int acumulado = 0;
    for (int v : valores) {
        acumulado += v;
    }
    return acumulado;
}
```

1B.13 Una clase propia (mínima)

Hasta acá usamos clases con métodos `static`. Pero la idea central de Java es crear **objetos**: unidades que combinan **estado** (datos, en *campos*) y **comportamiento** (métodos). Veamos la sintaxis mínima de una clase con estado. Para no adelantarnos, la dejamos con todo público y sin constructor; en la **Unidad 3** vamos a ver por qué eso no es lo ideal y cómo se modela bien.

```
public class Rectangulo {

    public double base;      // estado: los datos del objeto
    public double altura;

    public double area() {           // comportamiento: qué sabe hacer
        return base * altura;
    }

    public double perimetro() {
        return 2 * (base + altura);
    }
}
```

Se usa creando un objeto con `new` y accediendo a sus campos y métodos con el punto:

```
Rectangulo r = new Rectangulo();     // se crea el objeto
r.base = 3;                          // se asignan sus campos
r.altura = 4;
System.out.println(r.area());        // 12.0
```

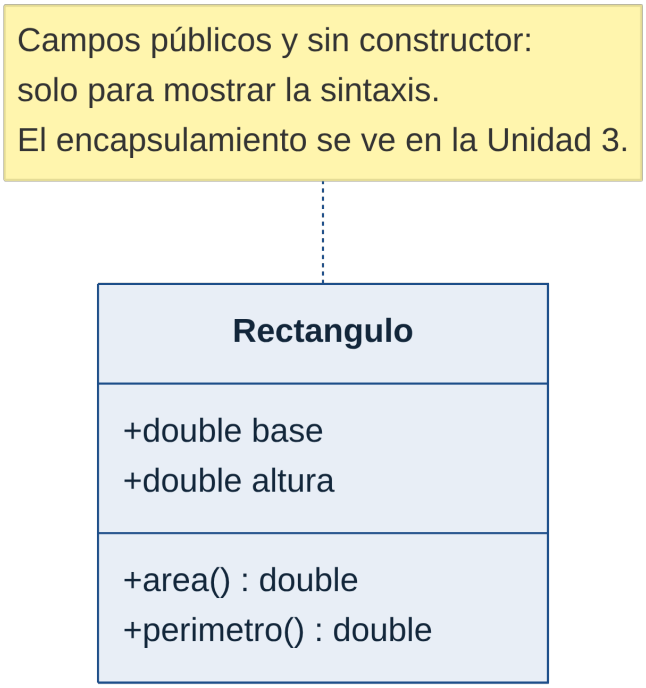


Figura 1B.15: Diagrama UML de la clase `Rectangulo`: sus campos y sus métodos.

1B.14 Enumerados: `enum`

Un **`enum`** define un tipo con un conjunto fijo y conocido de valores. Es más seguro que usar enteros o cadenas “mágicas”, porque el compilador conoce todos los casos posibles.

```
public enum DiaDeSemana {
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO;

    public boolean esFinDeSemana() {
        return this == SABADO || this == DOMINGO;
    }
}
```

Y así se usa: los valores se nombran como constantes, se pueden comparar con `==` (son únicos), recorrer con `values()` y usar en un `switch` que el compilador verifica completo:

```
DiaDeSemana hoy = DiaDeSemana.SABADO;
System.out.println(hoy);                    // SABADO
System.out.println(hoy.esFinDeSemana());    // true

for (DiaDeSemana d : DiaDeSemana.values()) {         // todos los valores, en orden
    System.out.print(d + ":" + (d.esFinDeSemana() ? "finde " : "hábil "));
}

String plan = switch (hoy) {                // switch sobre enum: cubre los casos
    case SABADO, DOMINGO -> "descansar";
    default              -> "cursar CB100";
};
```

1B.15 Modelar datos con `record`

Muchas veces necesitás un objeto que sólo **agrupa datos** y no cambia una vez creado (inmutable). Hoy se hace con un **`record`**: en una línea, Java genera el constructor, los accesores y los métodos `equals`, `hashCode` y `toString`.

```java
public record Temperatura(double celsius) {

    public Temperatura {                        // constructor compacto: valida
        if (celsius < -273.15) {
            throw new IllegalArgumentException("Bajo el cero absoluto: " + celsius);
        }
    }

    public String clasificar() {
        return switch (franja()) {
            case 0 -> "helado";  case 1 -> "frío";  case 2 -> "templado";
            case 3 -> "cálido";  default -> "caluroso";
        };
    }

    private int franja() {
        if (celsius < 0) {
            return 0;
        }
        if (celsius < 12) {
            return 1;
        }
        if (celsius < 22) {
            return 2;
        }
        if (celsius < 30) {
            return 3;
        }
        return 4;
    }
}
```

1B.15.1 Varios atributos y sus accesores

Un record puede tener todos los componentes que necesites, y por CADA uno Java genera un **accesor**: un método sin parámetros con el mismo nombre del componente. Mirá `FichaDeAlumno` (del repo), con tres atributos:

```java
public record FichaDeAlumno(String nombre, int padron, double promedio) {

    public FichaDeAlumno {                      // constructor compacto: valida TODO
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("el nombre no puede estar vacío");
        }
        if (padron <= 0) {
            throw new IllegalArgumentException("el padrón debe ser positivo: " + padron);
        }
        if (promedio < 0 || promedio > 10) {
            throw new IllegalArgumentException("el promedio va de 0 a 10: " + promedio);
        }
    }

    public boolean promociona() {               // los métodos combinan los atributos
        return promedio >= 7.0;
    }
}
```

Y así se **leen** los atributos — creando fichas y llamando a los accesores:

```java
FichaDeAlumno ana = new FichaDeAlumno("Ana Gómez", 111234, 8.5);
FichaDeAlumno beto = new FichaDeAlumno("Beto Díaz", 109876, 6.1);

System.out.println(ana.nombre());       // Ana Gómez
System.out.println(ana.padron());       // 111234
System.out.println(ana.promedio());     // 8.5
```

```java
// los accesores se usan como cualquier expresión:
System.out.println(ana.nombre() + " (" + ana.padron() + ")");   // Ana Gómez (111234)
double diferencia = ana.promedio() - beto.promedio();           // 2.4000000000000004

// toString, equals y hashCode vienen gratis:
System.out.println(beto);         // FichaDeAlumno[nombre=Beto Díaz, padron=109876, promedio=6.1]
FichaDeAlumno copia = new FichaDeAlumno("Ana Gómez", 111234, 8.5);
System.out.println(ana.equals(copia));   // true: igualdad por CONTENIDO, no por referencia
```

Idea clave

Tres cosas para fijar de los records: (1) el accesor se llama **igual que el componente** — `promedio()`, sin el prefijo `get` de las clases tradicionales; (2) `equals` compara **por contenido**: dos fichas con los mismos datos son iguales aunque sean objetos distintos; (3) **no hay setters** — un record es inmutable, y "modificarlo" significa crear otro (`ana.conPromedio(9.0)` devuelve una ficha NUEVA; `ana` queda intacta). Corré el `main` de `FichaDeAlumno` y verificá las tres.

Representamos su diseño con un **diagrama UML de clases**, la notación estándar para describir sistemas orientados a objetos, que vamos a usar toda la materia.

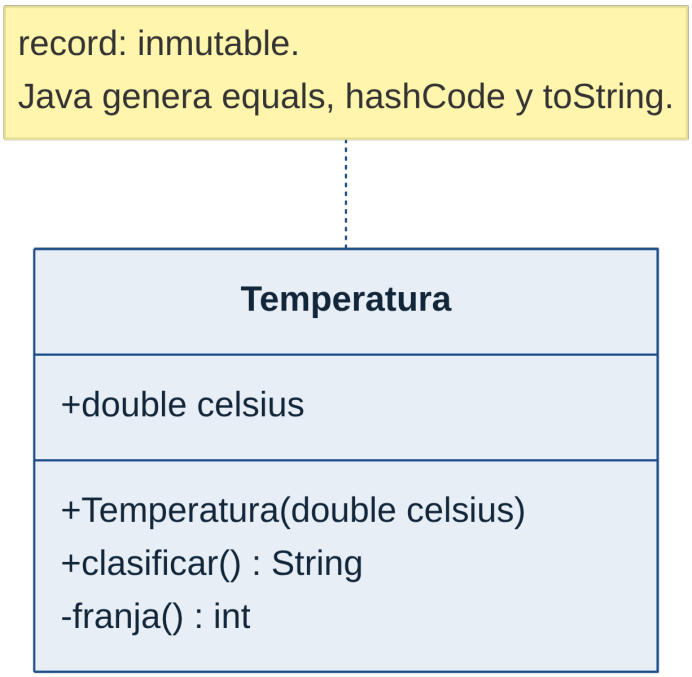


Figura 1B.16: Diagrama UML de la clase `Temperatura`. El `+` indica visibilidad pública; el `-`, privada.

1B.16 Expresiones lambda

Una **lambda** es una función anónima y breve. Sirve para pasar "un comportamiento" como si fuera un dato. Una lambda implementa una **interfaz funcional** (una interfaz con un solo método). La flecha `->` separa los parámetros del cuerpo.

```java
interface Operacion { int aplicar(int a, int b); }

Operacion suma = (a, b) -> a + b;
Operacion producto = (a, b) -> a * b;
System.out.println(suma.aplicar(3, 4));       // 7
System.out.println(producto.aplicar(3, 4));   // 12
```

La biblioteca estándar trae interfaces funcionales listas para usar (`Predicate`, `Function`,

`Comparator`…): las vas a cruzar a partir de la Unidad 5 y de lleno en la Unidad 12.

Idea clave

Por ahora nos quedamos con la idea: una lambda es una función corta que podemos guardar y pasar. Su uso más potente, junto con los *streams*, lo vamos a ver en detalle en la unidad de **estructuras lineales**.

1B.17 Manejo de excepciones

Un programa robusto reacciona ante situaciones excepcionales: un archivo que no existe, una división por cero, un dato inválido. Java modela esto con **excepciones**: cuando algo excepcional ocurre, se **lanza** (`throw`) un objeto que describe el problema; en algún lugar del programa, alguien lo **captura** (`try`/`catch`) y decide qué hacer. Lo interesante — y lo que vamos a mirar con lupa — es lo que pasa **en el medio**: entre el `throw` y el `catch` hay un viaje.

1B.17.1 El viaje de una excepción por la pila de llamadas

Cada vez que un método llama a otro, la JVM **apila un marco** con sus datos; cuando el método termina, su marco se desapila y la ejecución vuelve a donde estaba. Ahora mirá esta cadena de tres métodos — `main` llama a `procesarPedido`, que llama a `validarCantidad`, que valida y lanza — con `println` numerados para espiar qué se ejecuta:

```java
public class ViajeDeUnaExcepcion {

    public static void main(String[] args) {
        System.out.println("(1) main: por llamar a procesarPedido(-3)");
        try {
            procesarPedido(-3);
            System.out.println("(X) main: NO se ejecuta (la llamada terminó en excepción)");
        } catch (IllegalArgumentException e) {
            System.out.println("(5) main: ¡capturada! mensaje = " + e.getMessage());
        }
        System.out.println("(6) main: el programa SIGUE con normalidad");
    }

    static void procesarPedido(int cantidad) {
        System.out.println("(2) procesarPedido: por llamar a validarCantidad");
        validarCantidad(cantidad);
        System.out.println("(X) procesarPedido: NO se ejecuta (la excepción pasó de largo)");
    }

    static void validarCantidad(int cantidad) {
        System.out.println("(3) validarCantidad: verificando " + cantidad);
        if (cantidad <= 0) {
            System.out.println("(4) validarCantidad: cantidad inválida, LANZO");
            throw new IllegalArgumentException("la cantidad debe ser positiva y es " + cantidad);
        }
        System.out.println("(X) validarCantidad: NO se ejecuta (está después del throw)");
    }
}
```

La salida real (corré `ViajeDeUnaExcepcion` en el repo):

```
(1) main: por llamar a procesarPedido(-3)
(2) procesarPedido: por llamar a validarCantidad
(3) validarCantidad: verificando -3
(4) validarCantidad: cantidad inválida, LANZO
(5) main: ¡capturada! mensaje = la cantidad debe ser positiva y es -3
(6) main: el programa SIGUE con normalidad
```

Fijate qué **no** está en la salida: ninguna de las tres líneas marcadas `(X)`. El `throw` interrumpe `validarCantidad` en el acto (lo que está después del `throw` no corre), la excepción vuelve a

`procesarPedido` — que tampoco termina: la línea que seguía a la llamada se saltea — y sigue bajando hasta que encuentra el `catch` de `main`. Ahí el viaje termina y el programa continúa.

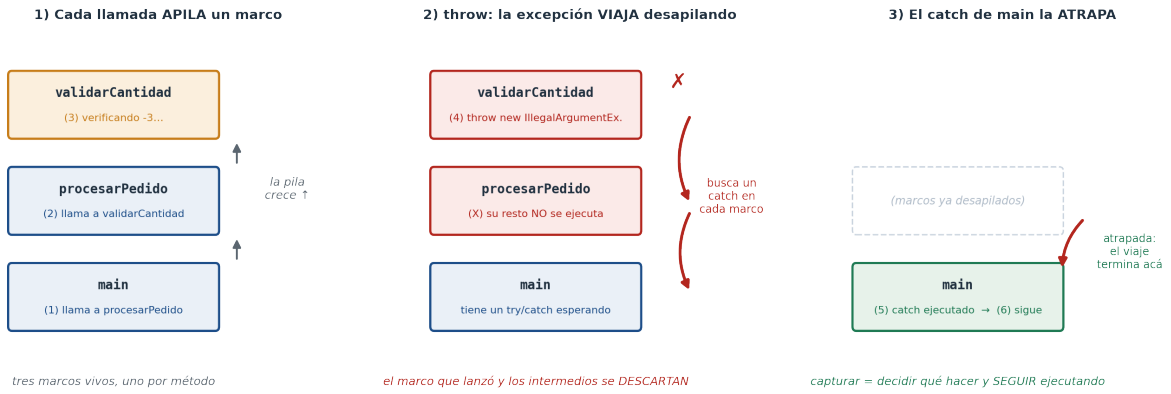


Figura 1B.17: El viaje completo: (1) las llamadas apilan marcos; (2) el throw descarta el marco actual y los intermedios, buscando un catch en cada uno; (3) el catch de main la atrapa y la ejecución continúa.

Idea clave

Una excepción **desapila la pila de llamadas** buscando el primer `catch` compatible. En ese viaje NO se ejecuta: (a) lo que está **después del `throw`** en el método que lanzó, y (b) lo que está **después de la llamada** en cada método intermedio que no la captura. Por eso una validación que lanza al principio de un método garantiza que el resto del método sólo corre con datos sanos — es el patrón que usamos en todos los TDA del curso.

1B.17.2 ¿Y si nadie la captura? El stack trace

En el repo está `ExcepcionSinCapturar`: la misma cadena, sin `try`. La excepción atraviesa los tres marcos, llega al fondo de la pila y la JVM corta el programa imprimiendo esto:

```
(1) main: por llamar (sin ninguna red)
(2) procesarPedido: por validar
(3) validarCantidad: verificando -3
Exception in thread "main" java.lang.IllegalArgumentException: la cantidad debe ser positiva y es -3
    at ...ExcepcionSinCapturar.validarCantidad(ExcepcionSinCapturar.java:30)
    at ...ExcepcionSinCapturar.procesarPedido(ExcepcionSinCapturar.java:23)
    at ...ExcepcionSinCapturar.main(ExcepcionSinCapturar.java:17)
```

Eso es un **stack trace**: la foto de la pila en el momento del lanzamiento. Se lee de arriba hacia abajo: primera línea, el tipo y el mensaje; después la cadena de llamadas, con **lo más profundo arriba** (`validarCantidad` lanzó, `procesarPedido` lo había llamado, `main` había llamado a aquél). Cada línea dice archivo y número de línea: hacé clic en IntelliJ y te lleva justo ahí.

En IntelliJ

Cuando un programa tuyo explote — y va a pasar seguido —, no entres en pánico: andá al stack trace en la consola Run y buscá **la primera línea que menciona un archivo TUYO**. Ésa es, casi siempre, la pista correcta. Las líneas de `java.base` u otras bibliotecas son el camino interno, no tu bug.

1B.17.3 `try`, varios `catch`, `finally`

La forma completa de la estructura tiene tres piezas: el `try` (el bloque vigilado), uno o más `catch` (qué hacer con cada tipo de problema) y un `finally` opcional (lo que debe pasar **siempre**, haya o no excepción: cerrar, devolver, limpiar).

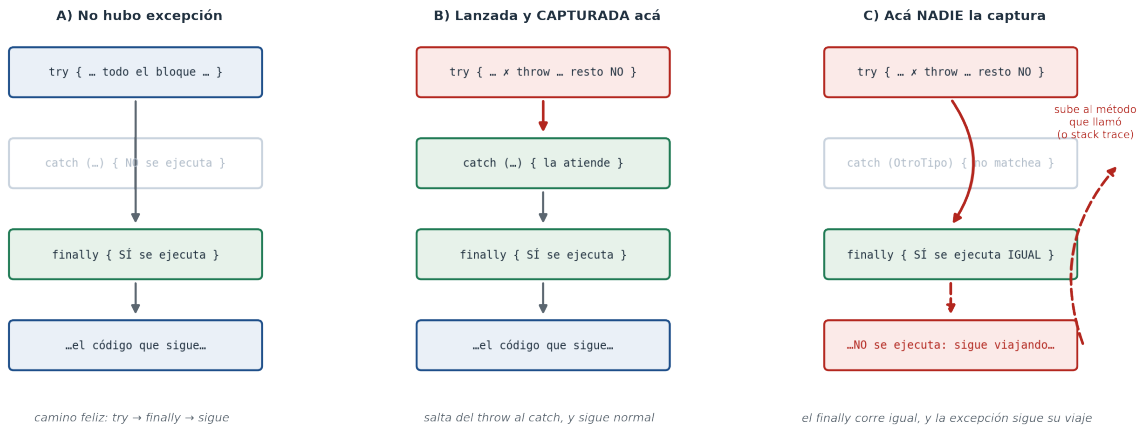


Figura 1B.18: Los tres destinos posibles de un try: sin excepción (A), excepción capturada acá (B), y excepción que este try no captura (C) — el finally se ejecuta en LOS TRES casos.

Error común

Cuando encadenás varios `catch`, ordenálos **del más específico al más general**. Si ponés `catch (Exception e)` primero, captura todo y los `catch` siguientes se vuelven inalcanzables: **el código no compila**. Una excepción hija siempre va antes que su clase padre — en el integrador de abajo, `NumberFormatException` (hija) va antes que `IllegalArgumentException` (padre).

1B.17.4 Excepciones del sistema, chequeadas y propias

La biblioteca estándar trae una jerarquía de excepciones listas para usar. Las que más vas a cruzar este cuatrimestre:

Excepción	La lanza…
`IllegalArgumentException`	un método al que le pasaron un argumento inválido
`IllegalStateException`	un objeto usado en un momento inválido (sacar de una pila vacía)
`ArithmeticException`	la división entera por cero: `7 / 0`
`NumberFormatException`	`Integer.parseInt("mil")`: el texto no es un número
`ArrayIndexOutOfBoundsException`	`arreglo[i]` con un índice fuera de rango
`NullPointerException`	usar una referencia que es `null`

Todas las de la tabla descienden de `RuntimeException` y son **no chequeadas**: se lanzan y capturan libremente, sin que el compilador exija nada. Las que descienden de `Exception` (pero no de `RuntimeException`) son **chequeadas**: si un método puede lanzarlas, debe declararlo con **`throws`** en su firma, y quien lo llame está OBLIGADO por el compilador a capturarla o a declararla a su vez. Es un contrato visible: “llamarme implica hacerse cargo de esto”.

Y podés definir **excepciones propias** de tu dominio: una clase que extiende `RuntimeException` (no chequeada) o `Exception` (chequeada), con un buen mensaje y, como cualquier objeto, los datos que le quieras cargar:

```
/** Propia y CHEQUEADA: quien llame a extraer debe hacerse cargo. */
public class SaldoInsuficienteException extends Exception {

    private final int faltante;

    public SaldoInsuficienteException(int montoPedido, int saldoDisponible) {
```

```
        super("pediste $" + montoPedido + " pero el saldo es $" + saldoDisponible);
        this.faltante = montoPedido - saldoDisponible;
    }

    public int faltante() {
        return faltante;
    }
}
```

1B.17.5 Ejemplo integrador: el cajero automático

Todos los casos juntos, en `CajeroAutomatico` (corrélo del repo): dos `try` anidados, excepciones del sistema y una propia chequeada, un `throw` dentro del `try`, varios `catch` bien ordenados y un `finally` inquebrantable.

```
/** Puede fallar de dos maneras: monto absurdo o saldo que no alcanza. */
public void extraer(int monto) throws SaldoInsuficienteException {
    System.out.println("  extraer: piden $" + monto + " (saldo $" + saldo + ")");
    if (monto <= 0) {
        throw new IllegalArgumentException("el monto debe ser positivo y es " + monto);
    }
    if (monto > saldo) {
        throw new SaldoInsuficienteException(monto, saldo);
    }
    saldo -= monto;
    System.out.println("  extraer: entregado, saldo restante $" + saldo);
}

/** Una operación completa del cajero. */
public void atender(String textoDelMonto) {
    System.out.println("atender(\"" + textoDelMonto + "\")");
    try {                                                   // ---- try EXTERNO
        int monto = interpretarMonto(textoDelMonto);        // parseInt puede explotar
        try {                                               // ---- try INTERNO
            extraer(monto);                                 // throw DENTRO del try
            System.out.println("  operación exitosa");
        } catch (SaldoInsuficienteException e) {
            // lo esperable del negocio se resuelve acá, cerca
            System.out.println("  no alcanza: " + e.getMessage()
                    + " (faltan $" + e.faltante() + ")");
        }
        System.out.println("  fin del camino con monto numérico");
    } catch (NumberFormatException e) {
        // PRIMERO la específica (es hija de IllegalArgumentException)
        System.out.println("  eso no es un número (" + e.getMessage() + ")");
    } catch (IllegalArgumentException e) {
        // el try interno NO la capturaba: siguió viajando hasta acá
        System.out.println("  monto inválido: " + e.getMessage());
    } finally {
        System.out.println("  finally: devolver la tarjeta (SIEMPRE)");
    }
}
```

El `main` atiende cuatro operaciones — una por caso — y esta es la salida real, para leer con el dedo en el código:

```
atender("300")                          <- CASO FELIZ
  interpretarMonto: leyendo "300"
  interpretarMonto: entendí 300
  extraer: piden $300 (saldo $1000)
  extraer: entregado, saldo restante $700
  operación exitosa
  fin del camino con monto numérico
  finally: devolver la tarjeta (SIEMPRE)
```

```
atender("5000")                          <- PROPIA CHEQUEADA, capturada en el try INTERNO
  interpretarMonto: leyendo "5000"
  interpretarMonto: entendí 5000
  extraer: piden $5000 (saldo $700)
  no alcanza: pediste $5000 pero el saldo es $700 (faltan $4300)
  fin del camino con monto numérico       <- el interno la resolvió: el externo ni se entera
  finally: devolver la tarjeta (SIEMPRE)

atender("-50")                           <- DEL SISTEMA: el try interno NO la captura...
  interpretarMonto: leyendo "-50"
  interpretarMonto: entendí -50
  extraer: piden $-50 (saldo $700)
  monto inválido: el monto debe ser positivo y es -50   <- ...la capturó el EXTERNO
  finally: devolver la tarjeta (SIEMPRE)      <- ojo: "fin del camino..." NO se imprimió

atender("mil")                           <- EXPLOTA EL PARSEO, antes del try interno
  interpretarMonto: leyendo "mil"
  eso no es un número (For input string: "mil")
  finally: devolver la tarjeta (SIEMPRE)

main: el cajero sigue en pie, saldo final $700
```

Releé el tercer caso, que es el fino: `extraer` lanza `IllegalArgumentException` **dentro del try interno**, pero ese `try` sólo sabe capturar `SaldoInsuficienteException` — así que la excepción sale del `try` interno (salteando `"operación exitosa"` Y `"fin del camino..."`) y recién la atrapa el `catch` del externo. Dos redes: la de adentro para lo esperable del negocio, la de afuera para el resto. Y el `finally` imprimió las cuatro veces: con éxito, con captura interna, con captura externa y con explosión temprana.

Error común

Errores clásicos con excepciones:

- **El catch vacío** (`catch (Exception e) { }`): la excepción desaparece sin dejar rastro y el bug se vuelve invisible. Como mínimo, mostrala.
- **Capturar `Exception` "por las dudas"**: atrapa también lo que no sabés manejar y esconde errores de programación. Capturá los tipos que SÍ sabés resolver.
- **Usar excepciones para el flujo normal**: si el caso pasa siempre (fin de un ciclo, opción del menú), es un `if`, no un `throw` — las excepciones son para lo excepcional.
- **Olvidar el orden específico → general** en los `catch` (no compila).
- Poner `return` adentro de un `finally`: pisa silenciosamente el resultado (o la excepción) del `try`. No lo hagas nunca.

1B.18 Fechas y horas

Para trabajar con fechas se usa la API moderna `java.time` (Java 8+). Sus objetos son **inmutables**: cada operación devuelve uno nuevo.

```java
LocalDate hoy = LocalDate.of(2026, 3, 15);
LocalDate manana = hoy.plusDays(1);
DayOfWeek dia = hoy.getDayOfWeek();                // SUNDAY

LocalDate fin = LocalDate.of(2026, 7, 4);
long dias = ChronoUnit.DAYS.between(hoy, fin);     // 111

LocalDateTime ahora = LocalDateTime.of(2026, 3, 15, 18, 30);
var fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
System.out.println(ahora.format(fmt));             // 15/03/2026 18:30
```

1B.19 Entrada y salida por consola

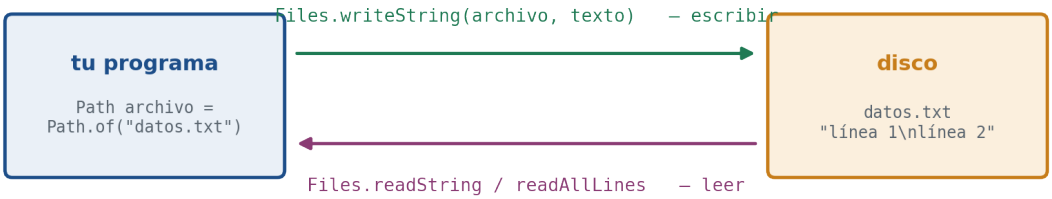
Ya usamos `System.out.println` para escribir. Para **leer** desde el teclado se usa `Scanner`, envolviendo la entrada estándar `System.in`:

```
Scanner sc = new Scanner(System.in);
System.out.print("¿Cómo te llamás? ");
String nombre = sc.nextLine();
System.out.print("¿Cuántos años tenés? ");
int edad = sc.nextInt();
System.out.printf("Hola %s, el año que viene vas a tener %d.%n", nombre, edad + 1);
```

Los métodos más usados son `nextLine()` (una línea completa), `nextInt()` (un entero) y `nextDouble()` (un real).

1B.20 Archivos

Hasta acá, todos los datos de tus programas viven en la memoria y **desaparecen** al terminar. Para que sobrevivan hay que llevarlos al **disco**: eso es trabajar con archivos. El ciclo completo es simple: tu programa nombra el archivo con un `Path`, **escribe** con `Files.writeString`, y más tarde (incluso en otra ejecución) **lee** con `Files.readString` o `readAllLines`:



si el recurso queda abierto (Scanner, streams), try-with-resources lo cierra SOLO al salir

Figura 1B.19: El ciclo de trabajo con archivos: el programa escribe al disco y lee de él; los recursos que quedan abiertos se cierran solos con try-with-resources.

Para leer y escribir archivos de texto, la forma más simple es la clase utilitaria `Files`:

```
Path archivo = Path.of("datos.txt");
Files.writeString(archivo, "línea 1\nlínea 2\n");    // escribir
String contenido = Files.readString(archivo);        // leer todo
String[] lineas = contenido.split("\n");             // separar por líneas
```

(Existe también `Files.readAllLines`, que devuelve una `List` — la vas a preferir cuando conozcas las listas, en la Unidad 5.)

Cuando un recurso hay que cerrarlo (como un archivo abierto), se usa **try-with-resources**, que lo cierra automáticamente al terminar, incluso si hubo un error:

```
try (var sc = new Scanner(Path.of("datos.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
}   // el Scanner se cierra solo al salir del try
```

Otras operaciones frecuentes con `Files`:

```
Files.exists(archivo);                      // ¿existe?
Files.size(archivo);                        // tamaño en bytes
Files.writeString(archivo, "más\n", StandardOpenOption.APPEND);  // anexar al final
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
Files.move(origen, destino);                // mover / renombrar
Files.createDirectories(Path.of("datos/2026"));   // crear carpetas
Files.deleteIfExists(archivo);
```

En el repo, `ArchivosDemo` y `ArchivosAvanzado` corren estos ejemplos usando archivos temporales (así no ensucian tu disco).

1B.21 Validaciones

Programar "a la defensiva" significa **verificar las precondiciones** antes de operar y rechazar los datos inválidos con un error claro. Dos herramientas básicas: `Objects.requireNonNull` (falla si algo es `null`) y lanzar `IllegalArgumentException` ante un argumento inválido.

```
static double promedio(int[] notas) {
    Objects.requireNonNull(notas, "el arreglo no puede ser null");
    if (notas.length == 0) {
        throw new IllegalArgumentException("no hay notas para promediar");
    }
    int suma = 0;
    for (int n : notas) {
        if (n < 0 || n > 10) {
            throw new IllegalArgumentException("nota fuera de rango [0..10]: " + n);
        }
        suma += n;
    }
    return (double) suma / notas.length;
}
```

Idea clave

Validar temprano y fallar con un mensaje claro es una de las mejores costumbres que podés tomar: convierte errores difíciles de rastrear en mensajes que dicen exactamente qué pasó y dónde.

1B.22 Probar el código: JUnit 5

En esta materia el código no se da por bueno porque "anduvo una vez": se **prueba** con tests automáticos usando **JUnit 5**. Un test es un método anotado con `@Test` que ejecuta tu código y verifica el resultado con *aserciones*.

```
class TemperaturaTest {

    @Test
    @DisplayName("Clasifica correctamente cada franja")
    void clasificaSegunLaFranja() {
        assertEquals("templado", new Temperatura(21.5).clasificar());
        assertEquals("caluroso", new Temperatura(35).clasificar());
    }

    @Test
    @DisplayName("Rechaza temperaturas por debajo del cero absoluto")
    void rechazaCeroAbsoluto() {
        assertThrows(IllegalArgumentException.class, () -> new Temperatura(-300));
    }
}
```

`assertEquals(esperado, obtenido)` falla si difieren; `assertThrows(...)` verifica que cierto código lance la excepción esperada.

Con Git / Gradle

Para correr todos los tests: `./gradlew test`. Acostumbrate a correrlos **antes de cada commit**: es la red de seguridad que te avisa si rompiste algo.

1B.23 El repositorio y el esquema de paquetes

Todo el código vive en **un único repositorio Git** para toda la materia — https://github.com/gschmidt-cb100/cb100 —, organizado en cuatro grandes áreas:

- **`material`** — la teoría del apunte. El código de esta unidad está en `ar.uba.fi.cb100.material.i01_intro`.
- **`guia`** — los ejercicios de cada tema: la teoría resuelta en `i01_teorico` y los niveles `i02_facil`, `i03_medio`, `i04_dificil`, con **un paquete por ejercicio** (por ejemplo `guia.i01_intro.i04_dificil.e05`).
- **`evaluaciones`** — `parciales`, `finales` y `tps`, acumulativos para todos los cuatrimestres.
- **`cuatrimestres`** — el código hecho en clase, por año, cuatrimestre, semana y tipo: por ejemplo `cuatrimestres.c2026.c02.s01.teorica` (o `…practica`).

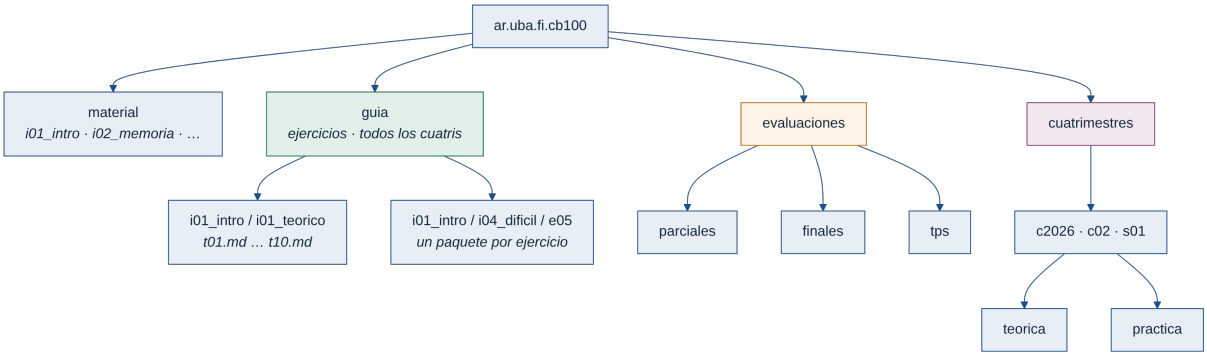


Figura 1B.20: Esquema de paquetes del repositorio único de la materia.

Un solo repositorio acumula el trabajo de todos los años, ordenado y fácil de encontrar.

1B.24 Usar una librería externa (Maven)

Rara vez programás todo desde cero: la mayoría de las funcionalidades comunes ya están resueltas en **librerías** publicadas en el **repositorio central de Maven** (*Maven Central*). Agregar una a tu proyecto con Gradle son cuatro pasos.

1. Buscar la librería. Entrá a `search.maven.org` o a `mvnrepository.com` y buscá lo que necesitás (por ejemplo, *"commons lang"*). Cada librería se identifica con tres **coordenadas**: **grupo**, **artefacto** y **versión**, en el formato `grupo:artefacto:versión`.

2. Declarar la dependencia en `build.gradle`:

```
dependencies {
    implementation 'org.apache.commons:commons-lang3:3.17.0'
}
```

3. Refrescar Gradle (IntelliJ lo ofrece con un botón, o corrés `./gradlew build`). Gradle descarga el `.jar` y lo deja disponible en el proyecto.

4. Usarla con un `import`:

```
import org.apache.commons.lang3.StringUtils;

StringUtils.capitalize("ada");     // "Ada"
StringUtils.reverse("Java");       // "avaJ"
```

```
StringUtils.isNumeric("12345");    // true
```

Con Git / Gradle

Elegí librerías **mantenidas y populares**: mirá la fecha de la última versión y cuántos proyectos la usan. Anotá siempre la **versión** exacta: eso hace que el build sea reproducible en cualquier máquina.

1B.25 Ejemplo integrador

Cerramos la unidad con un programa que usa **varias cosas juntas**: un `record` con validaciones, arreglos, un ciclo, `Math`, `StringBuilder`, `String.format` y escritura de un archivo. Genera el boletín de notas de un curso.

```
record Alumno(String nombre, int[] notas) {
    Alumno {
        Objects.requireNonNull(nombre, "el nombre no puede ser null");
        if (notas.length == 0) {
            throw new IllegalArgumentException("sin notas: " + nombre);
        }
        for (int n : notas)
            if (n < 0 || n > 10) {
                throw new IllegalArgumentException("nota inválida: " + n);
            }
    }
    double promedio() {
        int suma = 0;
        for (int n : notas) {
            suma += n;
        }
        return (double) suma / notas.length;
    }
    String condicion() {
        double p = promedio();
        return p >= 8 ? "Promociona" : p >= 4 ? "Aprueba" : "Recursa";
    }
}

Alumno[] curso = {
    new Alumno("Ada Lovelace", new int[]{10, 9, 8}),
    new Alumno("Alan Turing",  new int[]{7, 6, 5}),
    new Alumno("Grace Hopper", new int[]{4, 3, 5}),
};

StringBuilder reporte = new StringBuilder();
reporte.append(String.format("%-16s | %-8s | %s%n", "Alumno", "Promedio", "Condición"));
reporte.append("-".repeat(42)).append('\n');
for (Alumno a : curso) {
    double prom = Math.round(a.promedio() * 100.0) / 100.0;
    reporte.append(String.format("%-16s | %8.2f | %s%n", a.nombre(), prom, a.condicion()));
}
Files.writeString(Files.createTempFile("boletin", ".txt"), reporte.toString());
```

Y su salida:

```
Alumno           | Promedio | Condición
-------------------------------------------
Ada Lovelace     |     9.00 | Promociona
Alan Turing      |     6.00 | Aprueba
Grace Hopper     |     4.00 | Aprueba
```

El código completo está en el repositorio, en la clase `BoletinDeNotas`.

1B.26 Errores comunes

Error común

- Nombrar la clase distinto del archivo (`class Hola` en `Saludo.java`): no compila.
- Olvidar que Java distingue mayúsculas de minúsculas: `String` ≠ `string`.
- Usar `==` para comparar `String`: compara referencias, no contenido; usá `.equals(...)`.
- Dividir dos enteros esperando un decimal: `7 / 2` es `3`, no `3.5`.
- Usar `double` para dinero: acumula errores de redondeo; usá `BigDecimal`.
- Creer que `var` es un tipo dinámico: no lo es; el tipo se fija al compilar.
- Confundir el `switch` clásico (con `:` y `break`) con el switch expression (con `->`).

1B.27 Repaso

1. ¿Qué diferencia hay entre `javac` y la JVM? ¿Qué es el bytecode?
2. ¿Por qué el mismo `.class` corre en cualquier sistema operativo?
3. ¿Qué rol cumple cada herramienta: IntelliJ, Gradle y Git?
4. ¿Cuál es la firma exacta del método `main` y qué significa cada parte?
5. ¿Qué diferencia hay entre un tipo primitivo y uno por referencia?
6. ¿Por qué un `String` es inmutable y cuándo conviene un `StringBuilder`?
7. ¿Qué es el *overflow* y por qué `BigDecimal` es mejor que `double` para dinero?
8. ¿En qué se diferencia un switch expression de un `switch` clásico?
9. ¿Qué te genera automáticamente un `record`? ¿Y para qué sirve un `enum`?
10. ¿Qué es una lambda y qué es una interfaz funcional?
11. ¿Cómo validás las precondiciones de un método y cómo verificás, con un test, que lanza una excepción?
12. ¿Qué ofrece la clase `Math`? ¿Para qué sirve pasarle una semilla a `Random`?
13. ¿Cuáles son las tres coordenadas de una librería en Maven y cómo la agregás a tu proyecto con Gradle?

Guía de ejercicios — Unidad 1B

Objetivos de la unidad

Practicá la sintaxis de Java y el flujo de trabajo con el repositorio. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i01_intro` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio, p. ej. `guia.i01_intro.i04_dificil.e05`), con su **test JUnit**, y hacé un commit por ejercicio. Las preguntas teóricas se responden en `guia/i01_intro/i01_teorico`. Usá la IA para *entender*, no para copiar.

1B.28 Preguntas teóricas

1. Explicá con tus palabras qué es el *bytecode* y por qué permite "Write Once, Run Anywhere".
2. ¿Qué diferencia hay entre el JDK, el JRE y la JVM?
3. ¿Qué hace el compilador JIT y en qué momento actúa?
4. Diferenciá tipo primitivo y tipo por referencia. Dá un ejemplo de cada uno.
5. ¿Por qué `7 / 2` da `3` en Java? ¿Cómo obtendrías `3.5`?
6. ¿Qué genera automáticamente un `record`? ¿Cuándo conviene usarlo?
7. ¿Para qué sirve un `enum`? ¿Qué ventaja tiene sobre usar cadenas de texto?
8. ¿En qué se diferencia una excepción propia que extiende `RuntimeException` de una que extiende `Exception`?
9. ¿Por qué el orden de los bloques `catch` importa?
10. ¿Qué ventaja aporta escribir tests con JUnit en lugar de probar "a mano" con `System.out.println`?

1B.29 Nivel fácil

1. **Hola con tu nombre.** Escribí un programa que imprima un saludo con tu nombre y tu legajo.
2. **Suma de dos números.** Leé dos enteros desde `args` y mostrá su suma, resta, producto y división.
3. **Par o impar.** Dado un entero, indicá si es par o impar usando el operador `%`.
4. **Mayor de dos.** Escribí un método `int mayor(int a, int b)` que devuelva el mayor.
5. **Conversión de temperatura.** Escribí un método que convierta grados Celsius a Fahrenheit (`F = C * 9/5 + 32`). Cuidado con la división entera.
6. **Tabla de multiplicar.** Con un `for`, imprimí la tabla del número que se pase por `args`.
7. **Cuenta regresiva.** Con un `while`, imprimí desde `n` hasta 1 y luego "¡fin!".
8. **Promedio de un arreglo.** Dado un `double[]`, calculá el promedio con un for-each.
9. **Día hábil.** Usando un switch expression, dado un `char` (`'L'`, `'M'`, …) devolvé "hábil" o "fin de semana".
10. **Enum de estaciones.** Definí un `enum Estacion` con las cuatro estaciones y un método que imprima todas con `values()`.

1B.30 Nivel medio

1. **Máximo de un arreglo.** Escribí `int maximo(int[] v)` y testealo con JUnit (incluí el caso de un solo elemento).
2. **Contar vocales.** Dado un `String`, contá cuántas vocales tiene (recorré con un for y `charAt`).
3. **Invertir un arreglo.** Escribí un método que devuelva un nuevo arreglo con los elementos en orden inverso.
4. **Record `Punto`.** Creá `record Punto(double x, double y)` con `distanciaA(Punto o)`. Escribí dos tests.

5. **Clasificador de notas.** Método que reciba una nota entera y devuelva “insuficiente”/“aprobado”/“distinguido con un switch expression; lanzá una excepción propia si está fuera de 0–10.
6. **Factorial iterativo.** Escribí `long factorial(int n)` con un `for`. Testealo para 0, 1 y 5.
7. **Es primo.** Escribí `boolean esPrimo(int n)` y testeá varios casos, incluidos bordes (0, 1, 2).
8. **Buscar en arreglo.** Método que devuelva el índice de un valor en un `int[]`, o `-1` si no está.
9. **Enum con datos.** Definí `enum Moneda` con un símbolo asociado a cada valor (constructor de enum) y un método que lo devuelva.
10. **Validación con excepción.** Escribí una clase `CuentaBancaria` con `depositar` y `extraer`; que `extraer` lance una excepción propia si no hay saldo suficiente. Testeala con `assertThrows`.

1B.31 Nivel difícil

1. **FizzBuzz.** Del 1 al `n`: “Fizz” si es múltiplo de 3, “Buzz” si de 5, “FizzBuzz” si de ambos, y el número si no. Resolvelo con un switch expression sobre un `boolean` combinado.
2. **Palíndromo.** Método `boolean esPalindromo(String s)` que ignore mayúsculas y espacios. Testeá casos afirmativos y negativos.
3. **Estadísticas.** Dado un `int[]`, devolvé un `record Estadisticas(int min, int max, double promedio)` recorriéndolo una sola vez.
4. **Ordenar tres números.** Sin usar librerías, ordená tres enteros de menor a mayor y devolvelos en un arreglo.
5. **Conteo de palabras.** Dado un `String` con espacios, contá cuántas palabras tiene (cuidando espacios múltiples).
6. **Serie de Fibonacci.** Escribí `long fibonacci(int n)` iterativo (sin recursión) y testeá los primeros valores.
7. **Matriz transpuesta.** Dada una matriz `int[][]`, devolvé su transpuesta.
8. **Calculadora con excepciones.** Método `double operar(double a, double b, char op)` que soporte `+ - * /`, lanzando excepciones propias ante división por cero u operador inválido. Cubrí todo con tests.
9. **Histograma de caracteres.** Dado un `String`, devolvé cuántas veces aparece cada letra (usá un `int[26]` para las letras a–z).
10. **Validador de contraseña.** Escribí un método que valide una contraseña (mínimo 8 caracteres, al menos un dígito y una mayúscula) y devuelva, mediante una excepción propia con mensaje claro, qué regla falló. Testeá cada regla.

Unidad 2

De la memoria a las referencias

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) explicar cómo se organiza la memoria de un programa Java (**stack** y **heap**); (2) distinguir **valores** de **referencias** y entender qué es (y qué no es) una referencia; (3) manejar `null` y `Optional` sin miedo; (4) diferenciar **identidad** (`==`) de **igualdad** (`equals`); (5) razonar sobre **aliasing**, **paso de parámetros por valor** y objetos **mutables/inmutables**; y (6) entender, en lo esencial, cómo la **recolección de basura** libera la memoria por vos.

En la Unidad 1B vimos que hay dos familias de tipos: primitivos y referencias. Esta unidad se mete de lleno en esa distinción, porque es la base para entender todo lo que viene: cuando armemos listas, árboles o grafos, vamos a estar conectando objetos con **referencias**. Entender bien cómo funcionan ahora te va a ahorrar muchísimos errores más adelante.

2.1 El modelo de memoria: stack y heap

Cuando un programa Java se ejecuta, la JVM organiza la memoria en dos zonas que nos interesan. El **stack** (pila) guarda las **variables locales** y las llamadas a métodos; es rápido y se organiza solo (cada método, al terminar, libera su parte). El **heap** (montículo) guarda los **objetos** que creás con `new`; es la zona de memoria dinámica.

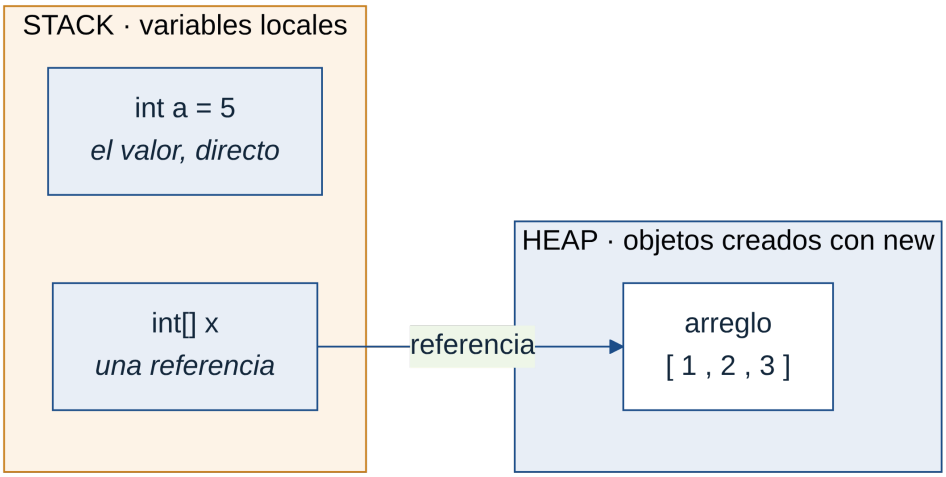


Figura 2.1: Los primitivos locales viven en el stack; los objetos, en el heap. Una variable de objeto guarda una referencia al heap.

Idea clave

Regla mental simple: **lo que hacés con `new` vive en el heap**; las variables locales viven en el stack. Una variable de objeto no “contiene” el objeto: contiene una **referencia** que lleva hasta él.

2.2 Valores y referencias

Un tipo **primitivo** guarda el valor directamente. Al asignarlo o pasarlo, se copia **el valor**. Un tipo por **referencia** guarda una referencia al objeto; al asignarlo, se copia **la referencia**, no el objeto: las dos variables quedan apuntando al mismo objeto.

Esta diferencia es tan importante que vale la pena verla en cámara lenta, con el mismo guion de tres pasos para cada familia:

```
int a = 5;
int b = a;          // copia el valor
b = 99;             // a sigue valiendo 5

int[] x = {1, 2, 3};
int[] y = x;        // copia la referencia: x e y son el MISMO arreglo
y[0] = 99;          // x[0] también es 99 ahora
```

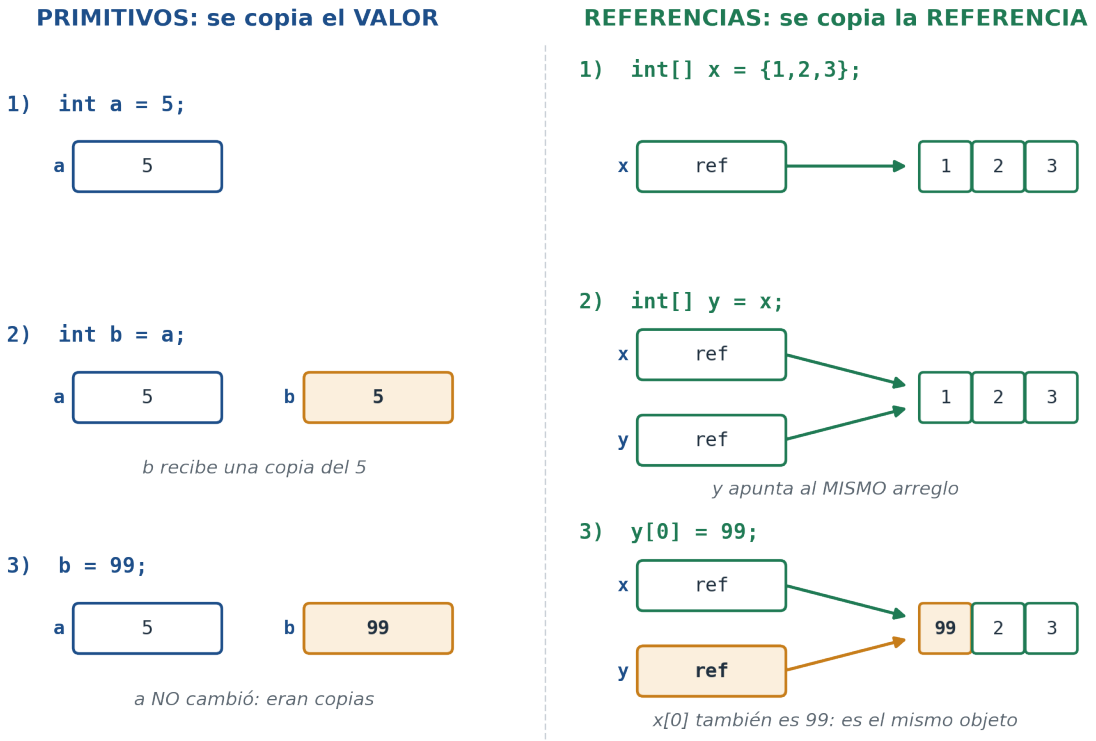


Figura 2.2: La misma secuencia de tres pasos con primitivos (izquierda) y con referencias (derecha). Con primitivos, `b = a` crea una copia independiente. Con referencias, `y = x` copia la flecha: hay un solo arreglo, y mutarlo por `y` se ve desde `x`.

Leé la columna izquierda: en el paso 2, `b` recibe **una copia del 5**; son dos cajas separadas, y por eso en el paso 3 cambiar `b` no toca a `a`. Ahora la derecha: en el paso 2, `y = x` **no copia el arreglo** —copia la flecha—, así que `x` e `y` llevan al **mismo** objeto del heap. En el paso 3, `y[0] = 99` modifica ese único arreglo… y `x[0]` "cambió solo". No hubo magia: siempre hubo **un solo arreglo**.

Idea clave

La asignación en Java copia **lo que hay en la caja de la variable**: si la caja tiene un valor, copia el valor; si tiene una referencia (una flecha), copia la flecha. **Nunca** copia el objeto apuntado.

2.3 Crear objetos con `new`

¿Qué pasa exactamente cuando escribís esta línea?

```
var cuenta = new Cuenta(1000);   // new -> objeto en el heap, 'cuenta' guarda la referencia
```

Aunque es una sola línea, adentro suceden **cuatro pasos**, en orden:

1. Se **declara la variable** `cuenta` en el stack. Todavía no lleva a ningún lado.
2. `new` **reserva memoria** en el heap para un objeto `Cuenta`, con sus campos aún sin inicializar.
3. Se **ejecuta el constructor** `Cuenta(1000)`, que puebla los campos (`saldo = 1000`).
4. `new` **devuelve la referencia** al objeto recién creado, y la asignación la guarda en `cuenta`. Recién ahora la variable "lleva" al objeto.

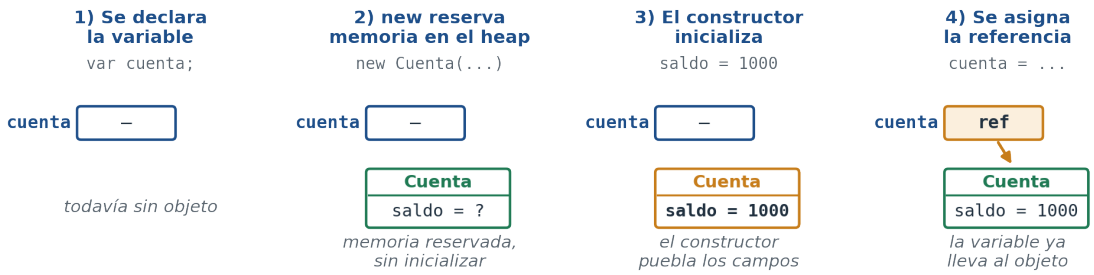


Figura 2.3: Los cuatro pasos de `var cuenta = new Cuenta(1000)`: declarar, reservar en el heap, inicializar con el constructor, y asignar la referencia.

Si el heap se queda sin memoria en el paso 2, la JVM lanza un `OutOfMemoryError` (raro en la práctica de la materia).

2.4 Referencias, no punteros

Si venís de C o C++, cuidado: **Java no tiene punteros**. Una referencia se parece a un puntero en que "lleva hasta" un objeto, pero con diferencias importantes: **no** podés ver ni manipular la dirección de memoria, **no** hay aritmética de punteros, y **no** podés convertir un número en una referencia. Una referencia sólo sirve para una cosa: acceder al objeto al que apunta (o ser `null`). Esto elimina de un plumazo toda una familia de errores clásicos de C.

Error común

Imprimir un objeto sin `toString` propio muestra algo como `Cuenta@65ae6ba4`: **no** es "la dirección de memoria", es el nombre de la clase y un código de hash en hexadecimal. No lo uses como si fuera una dirección.

2.5 `null` y la `NullPointerException`

`null` es una referencia especial que **no apunta a ningún objeto**. Una variable de objeto puede valer `null` (los primitivos, nunca). Si intentás usar un método o campo sobre una referencia `null`, la JVM lanza la famosa **`NullPointerException`** (NPE).

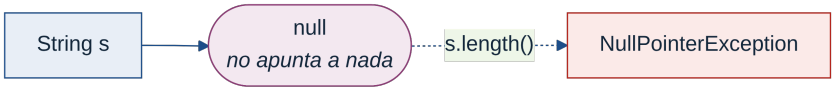


Figura 2.4: `null` no apunta a ningún objeto; usarlo lanza `NullPointerException`.

```
String s = null;
s.length();          // ¡NullPointerException!
```

Podés chequear con `if (s != null)`, pero para expresar "puede no haber valor" de forma más clara existe **`Optional`**, que te obliga a pensar el caso vacío:

```
Optional<String> nombre = buscarNombre(id);
String texto = nombre.orElse("desconocido");     // valor por defecto si está vacío
nombre.ifPresent(n -> System.out.println(n));    // hace algo sólo si hay valor
```

Idea clave

Recordá de la Unidad 1B: los wrappers (`Integer`, `Boolean`…) son objetos, así que **pueden ser `null`**; los primitivos (`int`, `boolean`…) no. Mezclar ambos sin cuidado es una fuente típica de NPE por *unboxing* de un `null`.

2.6 Identidad vs igualdad: == y `equals`

Esta distinción es de las más importantes de la materia. El operador == compara **identidad**: ¿las dos referencias apuntan al **mismo objeto**? El método `equals` compara **igualdad de contenido**: ¿los dos objetos **valen lo mismo**?

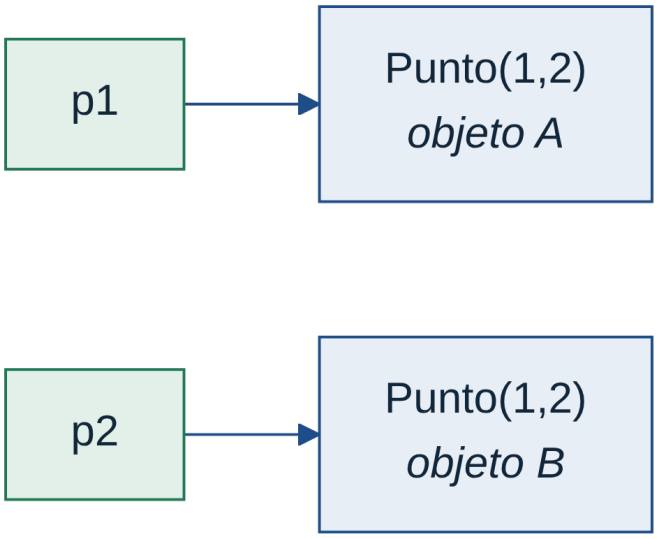


Figura 2.5: `p1` y `p2` son objetos distintos con el mismo contenido.

```
String s1 = new String("java");
String s2 = new String("java");
s1 == s2;          // false: son dos objetos distintos
s1.equals(s2);     // true:  tienen el mismo contenido

Punto p1 = new Punto(1, 2);
Punto p2 = new Punto(1, 2);
p1 == p2;          // false
p1.equals(p2);     // true (un record compara por valor)
```

Error común

Para comparar el **contenido** de objetos (textos, puntos, etc.) usá siempre `equals`, no ==. El == entre objetos casi nunca es lo que querés (salvo para comparar contra `null`, donde sí se usa ==).

Cuando definas tus propias clases y quieras compararlas por contenido, vas a tener que escribir `equals` (y su compañero `hashCode`). Un `record` ya te los da gratis; lo veremos en detalle al llegar a **Hashing**.

2.7 Aliasing: dos referencias, un objeto

Cuando dos variables apuntan al mismo objeto, decimos que hay **aliasing** (de *alias*: dos nombres para la misma cosa). Como comparten el objeto, un cambio hecho a través de una lo ve también la otra. Veámoslo en tres pasos con una `Cuenta`:

```
Cuenta c = new Cuenta(1000);   // 1) un objeto, una referencia
Cuenta alias = c;              // 2) segunda referencia AL MISMO objeto
alias.depositar(500);          // 3) muto por el alias...
c.saldo();                     //    ...y c "ve" 1500
```

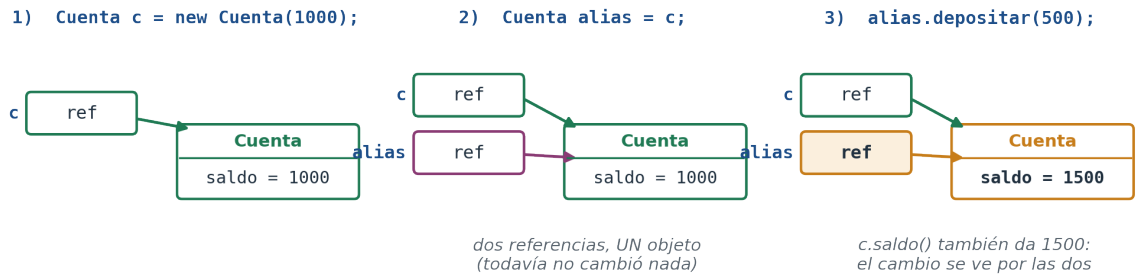


Figura 2.6: Aliasing en tres pasos: al asignar `alias = c` no se crea otra cuenta, sólo otra flecha. Depositar por `alias` cambia el único objeto, y `c` ve el nuevo saldo.

El paso 2 es el que engaña: `Cuenta alias = c;` **parece** crear otra cuenta, pero no hay ningún `new` —sin `new` no hay objeto nuevo—. Sólo se copió la flecha. Por eso en el paso 3 el depósito hecho "por el alias" le cambia el saldo "a `c`": son **el mismo objeto** con dos nombres. Esto es potente (podés compartir estructuras sin copiarlas), pero es una fuente clásica de errores si no te das cuenta de que estás compartiendo.

2.8 Paso de parámetros: siempre por valor

En Java, los argumentos se pasan **siempre por valor**: el método recibe una **copia** de la variable. Para un primitivo, copia el valor. Para un objeto, copia **la referencia** (no el objeto). De esa única regla salen las dos consecuencias que más confunden, así que vamos a mirarlas por separado, con la pila de llamadas a la vista.

```
static void mutar(int[] arr)     { arr[0] = 99; }            // se ve afuera
static void reasignar(int[] arr) { arr = new int[]{9,9,9}; } // NO se ve afuera
```

Caso 1: mutar el objeto — SÍ se ve afuera. Cuando `main` llama a `mutar(datos)`, se crea un frame nuevo en el stack con el parámetro `arr`, que recibe una **copia de la referencia**. Dos flechas, un solo arreglo. Entonces `arr[0] = 99` modifica **el único arreglo que existe**, y cuando el frame de `mutar` desaparece, `main` sigue apuntando a ese arreglo… ya modificado.

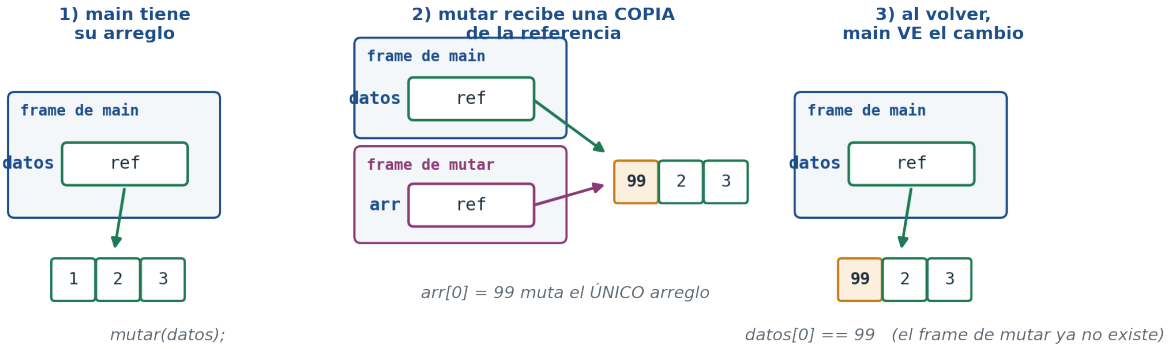


Figura 2.7: Mutar a través del parámetro: `arr` es una copia de la referencia de `datos`, pero ambas llevan al mismo arreglo. La mutación sobrevive al método.

Caso 2: reasignar el parámetro — NO se ve afuera. Ahora el método hace `arr = new int[]{9,9,9}`. Eso cambia **la copia local** de la referencia: la flecha de `arr` pasa a apuntar a un arreglo nuevo, pero la flecha de `datos` en `main` **no se enteró de nada**. Al volver, el frame de `reasignar` muere, el arreglo nuevo queda inalcanzable (lo recolectará el GC) y `main` sigue viendo `{1,2,3}`.

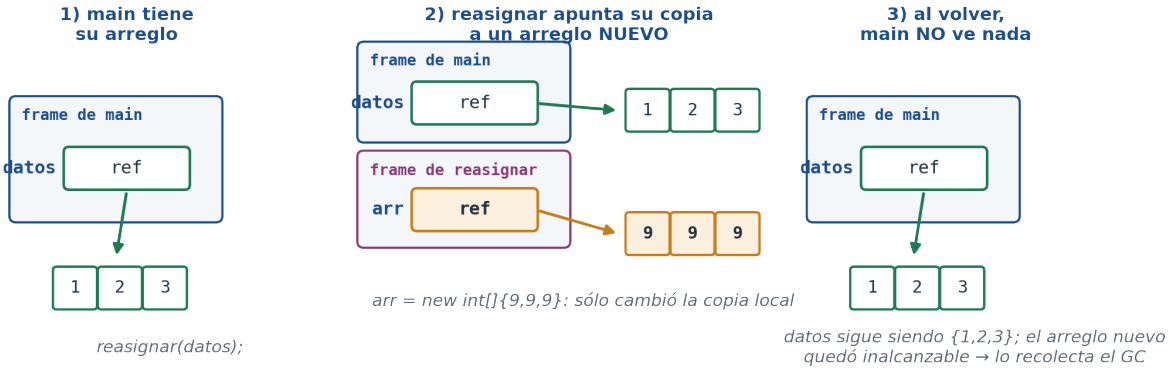


Figura 2.8: Reasignar el parámetro: la copia local apunta a otro arreglo, pero la referencia original de `main` no cambia. El arreglo nuevo muere con el método.

Idea clave

Si querés que un método le "cambie el objeto" al llamador, tenés dos opciones honestas: **mutar** el objeto recibido, o **devolver** el objeto nuevo con `return` para que el llamador lo asigne. Reasignar el parámetro adentro no hace nada visible afuera.

2.9 Objetos mutables e inmutables

Un objeto es **inmutable** si su estado no cambia después de creado: `String`, los wrappers y los `record` lo son. Es **mutable** si su estado puede cambiar: los arreglos y las clases con *setters* lo son. "Modificar" un inmutable en realidad significa **crear uno nuevo**.

```
String s = "abc";
s.toUpperCase();       // devuelve "ABC" pero NO cambia s; s sigue siendo "abc"
s = s.toUpperCase();   // así sí: guardo la referencia al String NUEVO
```

Fijate lo que pasa en la primera línea: `toUpperCase()` **crea un `String` nuevo** en el heap con el contenido `"ABC"` y devuelve su referencia… que nadie guarda. El original `"abc"` no se tocó (es

inmutable), así que `s` sigue viendo `"abc"`. La solución es la segunda forma: **reasignar** `s` para que apunte al nuevo. Este es uno de los errores más comunes al empezar con Java.

Idea clave

La inmutabilidad simplifica la vida: un objeto inmutable se puede compartir sin miedo (no hay aliasing peligroso, porque nadie lo puede cambiar). Por eso, cuando puedas, preferí datos inmutables (`record`).

2.10 Copias: superficial y profunda

A veces querés una **copia independiente** de un objeto para que los cambios no se "contagien". El problema aparece con estructuras **anidadas**: objetos que contienen referencias a otros objetos, como una matriz (un arreglo de arreglos). Ahí hay dos niveles de copia. Una copia **superficial** (*shallow*) copia la estructura de más afuera, pero **comparte** los objetos internos. Una copia **profunda** (*deep*) copia también los internos, quedando totalmente independiente.

```
int[][] matriz = {{1, 2}, {3, 4}};
int[][] superficial = matriz.clone();     // comparte las filas internas
superficial[0][0] = 99;                    // ¡también cambia matriz[0][0]!

int[][] profunda = new int[matriz.length][];
for (int i = 0; i < matriz.length; i++) {
    profunda[i] = Arrays.copyOf(matriz[i], matriz[i].length);   // independiente
}
```

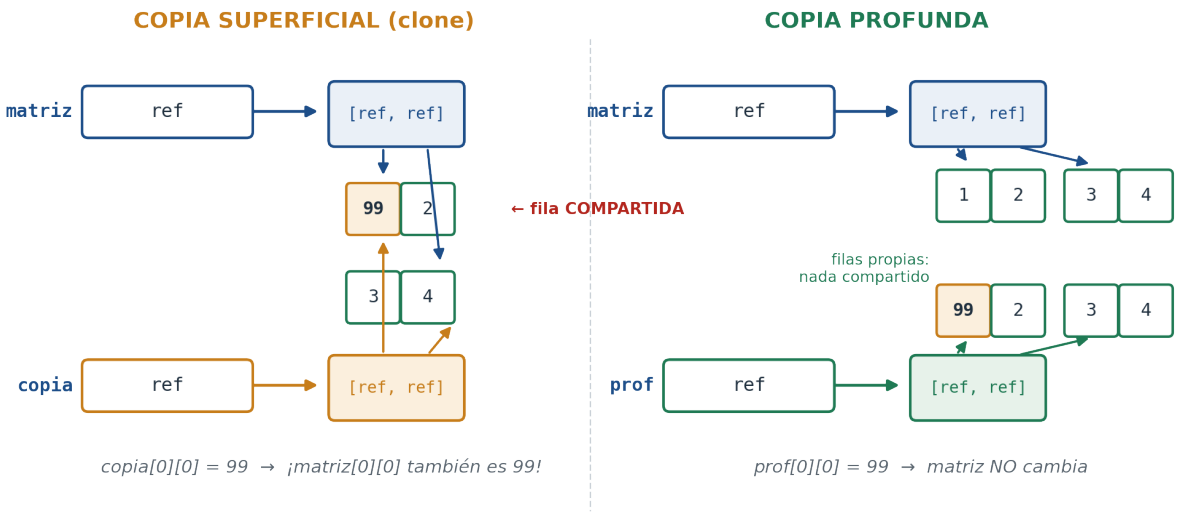


Figura 2.9: Copia superficial vs profunda de una matriz. Izquierda: `clone()` copia sólo el arreglo exterior; las filas quedan compartidas, y mutar `copia[0][0]` cambia también `matriz[0][0]`. Derecha: la copia profunda duplica también las filas; nada queda compartido.

Seguí las flechas de la izquierda: `matriz` y `copia` tienen **arreglos exteriores distintos** (eso copió `clone()`), pero las flechas de adentro de ambos llevan a **las mismas filas**. Por eso `copia[0][0] = 99` "le cambió" la matriz al original: la fila `[99, 2]` es compartida. A la derecha, la copia profunda duplicó también las filas: dos árboles completamente separados, y la mutación queda contenida.

Error común

`clone()` sobre arreglos y la mayoría de las copias "de un nivel" (como `Arrays.copyOf`) son **superficiales**. Si la estructura tiene objetos mutables adentro y necesitás independencia real, tenés que copiar **cada nivel** vos.

2.11 La recolección de basura

En C o C++ el programador reserva y **libera** la memoria a mano, lo que provoca errores difíciles (fugas de memoria, liberar dos veces, usar memoria liberada). En Java eso lo hace la JVM automáticamente: el **recolector de basura** (*garbage collector*, GC).

La idea es simple: un objeto se puede **alcanzar** si existe alguna cadena de referencias que llega hasta él desde una variable "viva". Cuando un objeto ya **no es alcanzable** por ninguna referencia, el GC lo detecta y libera su memoria, sin que vos hagas nada.

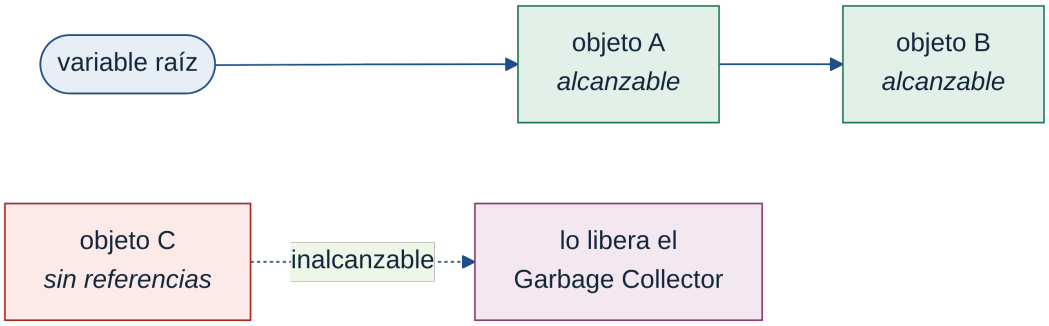


Figura 2.10: El GC libera los objetos que ya no son alcanzables desde ninguna referencia viva.

```
var c = new Cuenta(1000);
c = new Cuenta(2000);   // la primera cuenta ya no es alcanzable -> el GC la recolectará
```

Seguí la secuencia: al ejecutar la segunda línea, la flecha de `c` pasa a apuntar a la cuenta de saldo 2000. La cuenta de saldo 1000 sigue **existiendo** en el heap un rato más, pero ya **nadie llega hasta ella**: ninguna variable viva la apunta. Ése es exactamente el criterio del GC: en algún momento la detecta inalcanzable y recicla su memoria. Notá que vos no hiciste nada para "borrarla" —soltar la última referencia **es** soltarla—.

No necesitás (ni podés) liberar memoria a mano: no hay `delete`. Las JVM modernas usan por defecto un recolector llamado **G1**, y para aplicaciones de muy baja latencia existe **ZGC**; como programador casi nunca tenés que elegir ni configurar nada.[1]

Error común

El GC libera lo **inalcanzable**, no lo "inútil". Si dejás una referencia apuntando a algo que ya no usás (por ejemplo, en una estructura que crece y nunca limpiás), ese objeto sigue siendo alcanzable y **no** se libera: es una *fuga de memoria lógica*. La solución es soltar la referencia (por ejemplo, ponerla en `null` o quitarla de la estructura).

2.12 Ejemplo integrador

Este programa junta varias ideas: un objeto **mutable** compartido por dos referencias (**aliasing**), la diferencia entre **identidad** y hacer una **copia**, y por qué a veces conviene copiar explícitamente.

```
Cuenta c = new Cuenta(1000);
Cuenta mismaCuenta = c;          // ALIAS: no es una copia, es la misma cuenta
mismaCuenta.depositar(500);
c.saldo();                       // 1500  (¡cambió a través del alias!)
c == mismaCuenta;                // true  (mismo objeto)

Cuenta copia = new Cuenta(c.saldo());   // copia independiente del estado
copia.depositar(9999);
```

[1] G1 (*Garbage First*) es el recolector por defecto desde Java 9; ZGC es una opción para pausas muy bajas. Detalles como generaciones o algoritmos internos exceden esta materia.

```
c.saldo();                    // 1500  (no se afecta)
```

El código completo está en el repositorio, en `CuentaCompartida`.

2.13 Errores comunes

Error común

- Comparar objetos con == esperando comparar contenido: usá `equals`.
- Olvidar chequear `null` antes de usar una referencia que puede no tener valor.
- Creer que pasar un objeto a un método lo "protege": si lo mutás adentro, cambia afuera.
- Hacer `clone()` de una estructura anidada creyendo que es una copia profunda: es superficial.
- Pensar que poner una variable en `null` "borra" el objeto: sólo suelta esa referencia; el GC actúa si nadie más lo alcanza.

2.14 Repaso

1. ¿Qué se guarda en el stack y qué en el heap?
2. ¿Qué copia una asignación entre variables primitivas? ¿Y entre variables de objeto?
3. ¿En qué se diferencia una referencia de Java de un puntero de C?
4. ¿Qué es `null` y cuándo se produce una `NullPointerException`? ¿Qué aporta `Optional`?
5. ¿Cuál es la diferencia entre == y `equals`? Dá un ejemplo donde difieran.
6. ¿Qué es el aliasing y qué riesgo trae con objetos mutables?
7. Si un método muta un objeto que recibió por parámetro, ¿se ve el cambio afuera? ¿Y si reasigna el parámetro?
8. ¿Qué diferencia hay entre una copia superficial y una profunda?
9. ¿Cuándo el recolector de basura libera un objeto? ¿Qué es una fuga de memoria lógica?

Guía de ejercicios — Unidad 2

Objetivos de la unidad

Practicá el manejo de memoria y referencias. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i02_memoria` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio, p. ej. `guia.i02_memoria.i04_dificil.e05`), con su **test JUnit**. Las preguntas teóricas se responden en `guia/i02_memoria/i01_teorico`. Usá la IA para *entender*, no para copiar.

2.15 Preguntas teóricas

1. ¿Qué se guarda en el stack y qué en el heap? Dá un ejemplo de cada uno.
2. ¿Qué copia una asignación entre variables primitivas? ¿Y entre variables de objeto?
3. ¿En qué se diferencia una referencia de Java de un puntero de C?
4. ¿Qué es `null`? ¿Cuándo se produce una `NullPointerException` y qué aporta `Optional`?
5. Explicá la diferencia entre `==` y `equals` con un ejemplo donde den distinto.
6. ¿Qué es el aliasing y qué riesgo trae cuando el objeto es mutable?
7. Si un método muta un objeto recibido por parámetro, ¿se ve afuera? ¿Y si reasigna el parámetro? ¿Por qué?
8. ¿Qué diferencia hay entre un objeto mutable y uno inmutable? ¿Qué ventaja tiene la inmutabilidad?
9. ¿Qué diferencia hay entre una copia superficial y una profunda?
10. ¿Cuándo el recolector de basura libera un objeto? ¿Qué es una fuga de memoria lógica?

2.16 Nivel fácil

1. **Copia de primitivo.** Escribí un método que muestre que asignar `int b = a; b = 99;` no cambia `a`. Devolvé ambos valores y testealo.
2. **Aliasing de arreglo.** Método que reciba un `int[]`, cree un alias y cambie el primer elemento; verificá con un test que el arreglo original también cambió.
3. **`==` vs `equals`.** Método que reciba dos `String` y devuelva un `boolean[]` con `{s1 == s2, s1.equals(s2)}`. Testealo con dos `new String("x")`.
4. **Seguro ante null.** `String seguro(String s)` que devuelva `s`, o `""` si `s` es `null`. Testealo con `null` y con un texto.
5. **Optional con default.** `String orDefault(Optional<String> o)` que devuelva el valor o `"N/A"`. Testealo con `Optional.of` y `Optional.empty`.
6. **Reasignar no afecta.** Método que reciba un `int[]` y lo reasigne a otro adentro; verificá con un test que el original no cambió.
7. **Record inmutable.** `record Punto(int x, int y)` con `conX(int)` que devuelva un nuevo `Punto`; test que el original no cambia.
8. **Copiar arreglo.** Copiá un `int[]` con `Arrays.copyOf`, modificá la copia y verificá con un test que el original quedó intacto.
9. **Igualdad de records.** Test que dos `Punto(1,2)` cumplen `equals` pero no `==`.
10. **¿Mismo objeto?** `boolean mismoObjeto(Object a, Object b)` usando `==`; testealo con un alias y con dos objetos distintos de igual contenido.

2.17 Nivel medio

1. **Swap que no anda.** Mostrá que un `swap(int a, int b)` no intercambia nada afuera, y escribí `swap(int[] v, int i, int j)` que sí lo haga. Testealo.

2. **Mutar en un método.** Método que duplique cada elemento de un `int[]` recibido (in place); test que el arreglo de afuera quedó duplicado.
3. **`equals`/`hashCode` propios.** Clase `Punto` (no record) con `equals` y `hashCode` bien hechos; test del contrato (dos iguales → `equals` true y mismo `hashCode`).
4. **Copia defensiva.** Clase `Equipo` que guarde un `int[]` copiándolo en el constructor y devolviéndolo copiado en el getter; test que mutar el arreglo externo no la afecta.
5. **Copia profunda de matriz.** `int[][] copiaProfunda(int[][] m)`; test que modificar la copia no toca el original.
6. **Optional encadenado.** `int largoOrCero(Optional<String> o)` con `map` y `orElse`; test con presente y vacío.
7. **Identidad vs igualdad.** `boolean sonElMismo(Object,Object)` (`==`) y `boolean sonIguales(Object,Object)` (`equals`); test que distingan ambos casos.
8. **Envoltura inmutable.** Clase `ArregloInmutable` que reciba un `int[]`, lo copie, y sólo permita leer (getter que devuelve copia); test de inmutabilidad.
9. **Contar inalcanzables.** Dado un `boolean[]` que indica qué objetos son alcanzables, devolvé cuántos serían recolectados por el GC; testealo.
10. **NPE controlada.** Método que lance `NullPointerException` ante un `null`, y una versión segura que devuelva un valor por defecto; test con `assertThrows` y con la versión segura.

2.18 Nivel difícil

1. **Lista enlazada simple.** `Nodo(int valor, Nodo siguiente)` y una `ListaEnlazada` con `agregar`, `tamanio` y `aArreglo`; test.
2. **Detectar ciclo.** Dada una lista de nodos, detectá si hay un ciclo (por aliasing) con el algoritmo de la liebre y la tortuga; test con y sin ciclo.
3. **Copia profunda anidada.** `record Poligono(Punto[] puntos)` y un método que haga su copia profunda; test que mutar la copia no afecta el original.
4. **Caja mutable vs inmutable.** Implementá `CajaMutable` (con setter) y `CajaInmutable` (con "cambio" que devuelve una nueva); test que muestre la diferencia de comportamiento.
5. **Alcanzabilidad (mini-GC).** Dado un grafo por matriz de adyacencia y un nodo raíz, marcá los alcanzables (recorrido) y devolvé cuáles quedarían sin marcar; test.
6. **Contrato de `equals`.** Clase con varios campos: `equals`/`hashCode` correctos; test reflexivo, simétrico, consistente y contra `null`.
7. **Copy-on-write.** `TextoCOW` cuyo "agregar" devuelve una **nueva** instancia sin tocar la anterior; test que la original no cambia.
8. **Indicar la salida.** Reproducí en un método un escenario de aliasing y paso de parámetros, devolvé el estado final y verificalo con un test (homenaje al ejercicio del PowerPoint).
9. **Copia profunda de objetos.** `Cuenta[] copiaProfunda(Cuenta[] cuentas)` que copie cada cuenta; test que depositar en una copia no afecta al original.
10. **Evitar la fuga.** Implementá un `AlbumDeFotos` sobre arreglo donde `quitarUltima()` ponga en `null` la posición liberada (para no retener el objeto y dejar que el recolector lo levante); test que la posición quedó en `null`.

Unidad 3

Abstracción, TDA y Programación Orientada a Objetos

Objetivos de la unidad

Esta es una de las unidades más importantes del curso. Al terminarla vas a poder: (1) explicar qué es la **abstracción** y por qué es la idea central; (2) definir un **Tipo de Dato Abstracto (TDA)** separando el contrato de la implementación; (3) programar con los pilares de la **POO** —encapsulamiento, herencia y polimorfismo—; (4) usar **interfaces**, **clases y métodos abstractos** y **genéricos**; (5) implementar los **métodos estándar** (`toString`, `equals`, `hashCode`, `compareTo`, `clone`); (6) leer y escribir **diagramas UML**; y (7) reconocer las prácticas que hacen un código **elegante, robusto y mantenible**.

Hasta acá aprendiste el lenguaje (Unidad 1B) y cómo se comportan los objetos en memoria (Unidad 2). Ahora damos el salto conceptual del curso: **diseñar**. Vamos a dejar de pensar "qué instrucciones escribo" para empezar a pensar "qué piezas tiene mi sistema, qué hace cada una y cómo se relacionan". Esa es la esencia de la Programación Orientada a Objetos.

3.1 Abstracción: la gran idea

Abstraer es quedarse con lo esencial y ocultar el detalle. Cuando manejás un auto usás el volante, el acelerador y el freno: no te importa cómo funciona el motor por dentro. Esa separación entre **qué hace** algo (su interfaz) y **cómo lo hace** (su implementación) es la herramienta más poderosa que tenemos para domar la complejidad. Todo lo que sigue en esta unidad son formas concretas de aplicar abstracción.

3.2 Tipos de Datos Abstractos (TDA)

Un **Tipo de Dato Abstracto** es la especificación de un tipo por sus **operaciones** y sus **propiedades**, sin comprometerse con una implementación. Un TDA se define con: un **nombre**, un conjunto de **operaciones** (con sus precondiciones y postcondiciones) y sus **invariantes** (lo que siempre debe cumplirse). Eso es el **contrato**.

La clave es que **el contrato es independiente de la implementación**. Un Tablero de ta-te-ti (operaciones `poner`, `fichaEn`, `hayGanador`, con sus invariantes: es de 3×3, una casilla ocupada no se pisa, los turnos se alternan) es un TDA; podés implementarlo con una **matriz** `Ficha[3][3]` o con un **arreglo plano** `Ficha[9]` (donde la casilla (fila, col) vive en la posición `fila * 3 + col`), y quien lo usa no debería notar la diferencia. Vas a ver este principio en cada estructura de la materia.

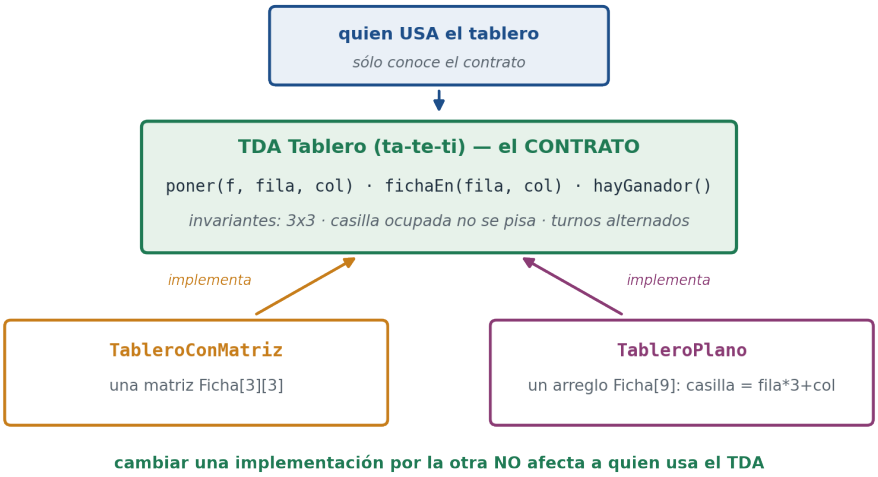


Figura 3.1: El TDA como contrato: quien usa el Tablero conoce sólo sus operaciones y sus invariantes. Abajo, dos implementaciones intercambiables — matriz o arreglo plano —: cambiar una por otra no afecta al usuario.

Veamos el contrato en acción. Sin saber **nada** de cómo está implementado el tablero, el contrato alcanza para predecir qué pasa:

```
Tablero t = /* cualquiera de las dos implementaciones */;
t.poner(Ficha.X, 1, 1);   // X al centro
t.poner(Ficha.O, 0, 0);   // O a la esquina
t.fichaEn(1, 1);          // X — lo que se puso se puede consultar
t.fichaEn(2, 2);          // null: casilla todavía vacía
t.poner(Ficha.X, 0, 0);   // ¡EXCEPCIÓN! el invariante dice: ocupada no se pisa
t.hayGanador();           // false: nadie tiene tres en línea todavía
```

Cada línea se dedujo **del contrato** (las operaciones y los invariantes), no del código interno. Ése es el poder del TDA: podés razonar, testear y hasta escribir programas enteros contra el contrato, y recién después decidir (o cambiar) la implementación — de la matriz al arreglo plano, sin que el resto del programa se entere.

Idea clave

Una aclaración importante: **la POO es el paradigma; el TDA es una herramienta de diseño**. Se trabajan juntos —usamos objetos para implementar TDAs— pero no son lo mismo. Hablar de “paradigma TDA” es un error frecuente que conviene evitar.

3.3 Clases y objetos

En POO, una **clase** es un molde que define un **estado** (los datos, en campos) y un **comportamiento** (los métodos). Un **objeto** es una instancia concreta de esa clase, creada con `new`, con sus propios valores.

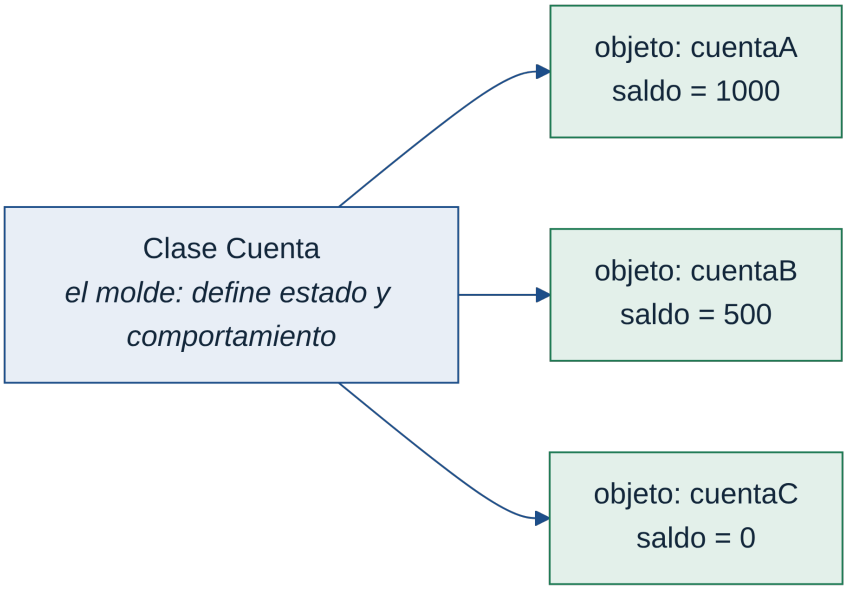


Figura 3.2: Una clase es el molde; los objetos son las instancias, cada uno con su estado.

```java
public class CuentaBancaria {
    private double saldo;                       // estado (privado)

    public CuentaBancaria(double saldoInicial) { // constructor: inicializa el objeto
        this.saldo = saldoInicial;
    }
    public double saldo() { return saldo; }      // comportamiento
}
```

La palabra `this` se refiere al objeto actual; el **constructor** es el método especial que prepara un objeto recién creado.

3.4 Los pilares de la POO

Sobre la abstracción se apoyan tres pilares que vas a usar constantemente.

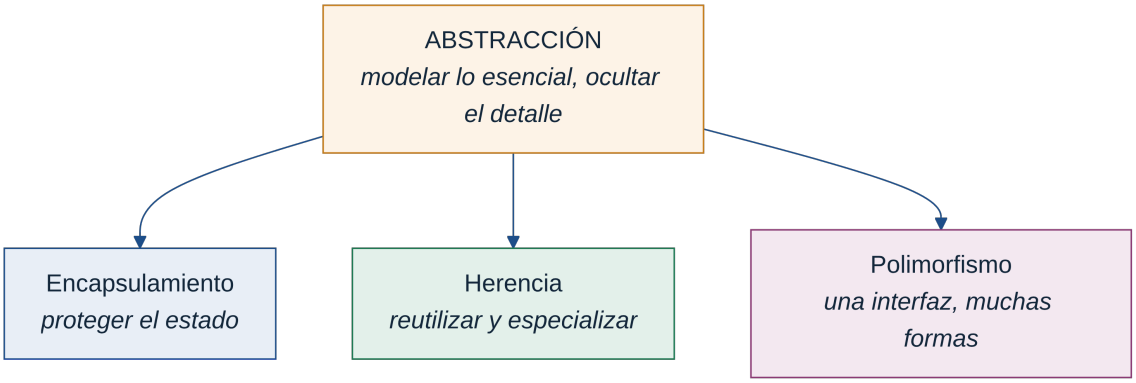


Figura 3.3: Sobre la abstracción se apoyan el encapsulamiento, la herencia y el polimorfismo.

3.4.1 Encapsulamiento

Encapsular es **ocultar el estado** detrás de una interfaz que lo protege. Los campos se declaran `private` y sólo se tocan mediante métodos que garantizan los **invariantes**. Así nadie puede dejar el objeto en un estado inválido.

```java
public class CuentaBancaria {
    private double saldo;                       // nadie lo cambia directamente

    public void extraer(double monto) {
        if (monto <= 0) {
            throw new IllegalArgumentException("monto inválido");
        }
        if (monto > saldo) {
            throw new SaldoInsuficienteException(saldo, monto);
        }
        saldo -= monto;                         // el saldo nunca queda negativo
    }
}
```

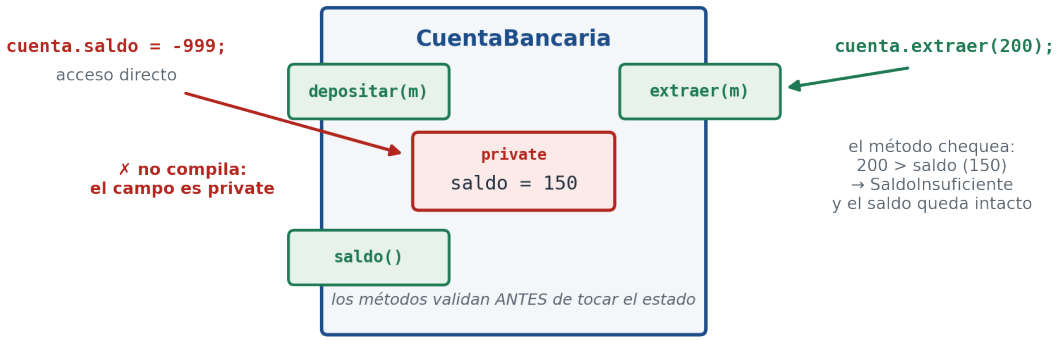


Figura 3.4: Encapsulamiento: el campo `saldo` es privado, así que el acceso directo ni siquiera compila. La única forma de tocar el estado son los métodos, que validan antes de modificar.

Seguí las dos flechas de la figura. La roja intenta el camino directo (`cuenta.saldo = -999`) y **ni siquiera compila**: el campo es `private`, la puerta no existe. La verde entra por el método: `cuenta.extraer(200)` con saldo 150 hace que el propio método detecte el problema (`200 > 150`), lance `SaldoInsuficienteException` y deje el saldo **intacto**. El invariante "el saldo nunca es negativo" está garantizado no porque todos los que usan la clase sean cuidadosos, sino porque **es imposible violarlo desde afuera**.

Error común

Exponer los campos como `public` (como hicimos a propósito en la clase mínima de la Unidad 1B) rompe el encapsulamiento: cualquiera podría poner un saldo negativo. La regla es **campos privados, acceso controlado**.

3.4.2 Herencia

La **herencia** permite que una clase (hija) **reutilice y especialice** a otra (padre) con `extends`. Modela una relación "es un/una": un `Perro` **es un** `Animal`. La hija hereda el comportamiento del padre y puede agregar o redefinir lo suyo; con `super` accede al padre.

```java
public class Animal {
    private final String nombre;
    public Animal(String nombre) { this.nombre = nombre; }
    public String nombre()  { return nombre; }
    public String sonido()  { return "..."; }
    public String presentarse() { return nombre + " dice " + sonido(); }
}

public class Perro extends Animal {             // Perro ES UN Animal
    public Perro(String nombre) { super(nombre); }        // llama al constructor del padre
```

```
    @Override public String sonido() { return "Guau"; }   // redefine SOLO lo suyo
}
```

Mirá lo que `Perro` **no** tiene que escribir: ni `nombre()`, ni `presentarse()` —los hereda—. Sólo llama al constructor del padre con `super(nombre)` y redefine `sonido()` con la anotación `@Override` (que le pide al compilador verificar que efectivamente estás redefiniendo algo del padre; si te equivocás en el nombre, te avisa).

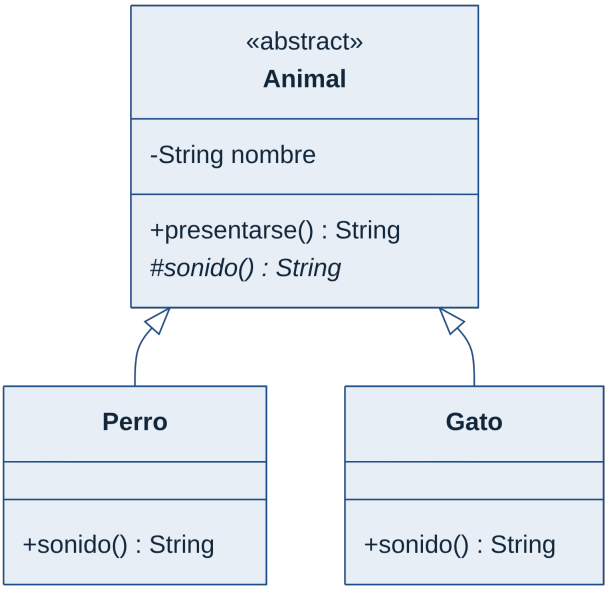


Figura 3.5: Herencia: `Perro` y `Gato` son `Animal`; heredan `presentarse()` y definen su `sonido()`.

3.4.3 Polimorfismo

Polimorfismo significa “muchas formas”: una misma referencia del tipo padre puede apuntar a objetos de distintas subclases, y al llamar un método se ejecuta **el del objeto real** (despacho dinámico), no el del tipo de la variable.

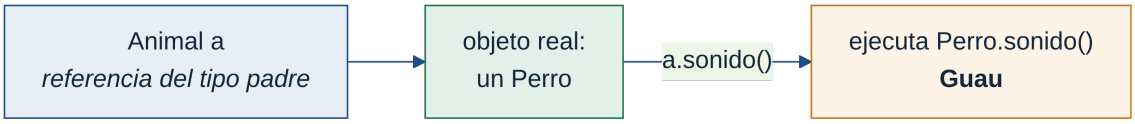


Figura 3.6: Polimorfismo: la referencia es `Animal`, pero se ejecuta el `sonido()` del objeto real.

```
Animal[] animales = { new Perro("Fido"), new Gato("Milo") };
for (Animal a : animales) {
    System.out.println(a.presentarse());   // "Fido dice Guau" / "Milo dice Miau"
}
```

Esto es lo que hace al código **extensible**: podés agregar una clase `Vaca` sin tocar el bucle que recorre los animales.

3.5 Composición: la alternativa a heredar

La herencia es potente, pero se **abusa** de ella. Antes de escribir `extends`, hacete la pregunta clave: ¿“X **es un** Y”… o en realidad “X **tiene un** Y”? Cuando la respuesta es “tiene un”, la herramienta correcta es la **composición**: guardar el otro objeto como **campo** y delegarle trabajo.

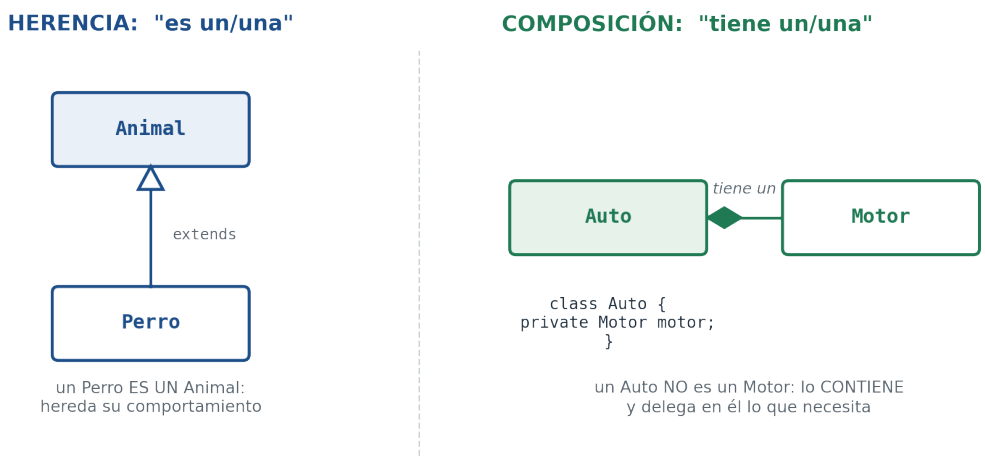


Figura 3.7: Herencia vs composición: `Perro extends Animal` porque un perro ES un animal; `Auto` tiene un campo `Motor` porque un auto TIENE un motor (en UML, el rombo lleno del lado del todo).

```
public class Auto {
    private final Motor motor;                    // el Auto TIENE un Motor
    public Auto(Motor motor) { this.motor = motor; }
    public void arrancar() { motor.encender(); }  // delega en su motor
}
```

¿Por qué importa tanto? Un `Auto extends Motor` "funcionaría" (heredaría `encender()`), pero el diseño queda **roto**: el auto expondría operaciones que no le corresponden, no podría cambiar de motor, y quedaría atado para siempre a una jerarquía sin sentido. La composición es más **flexible** (podés cambiar el motor por otro, incluso en tiempo de ejecución) y más **honesta** (el auto muestra sólo sus propias operaciones). En UML, la composición se dibuja con un **rombo lleno** del lado del "todo" (mirá la leyenda de la sección de UML).

Idea clave

Regla práctica: **herencia para "es un", composición para "tiene un"** — y ante la duda, empezá por composición: es más fácil pasar de composición a herencia que deshacer una jerarquía equivocada.

3.6 Interfaces: el contrato puro

Una **interfaz** declara un conjunto de operaciones **sin implementarlas**: es el contrato de un TDA llevado a código. Una clase se compromete a cumplirlo con `implements`. Una interfaz puede tener muchas implementaciones distintas, y ese es su gran valor.

```
public interface Figura {          // el CONTRATO
    double area();
    double perimetro();
}

public class Circulo implements Figura {
    private final double radio;
    public Circulo(double radio) { this.radio = radio; }

    @Override public double area()      { return Math.PI * radio * radio; }
    @Override public double perimetro() { return 2 * Math.PI * radio; }
}
```

Notá que la interfaz no le impone a `Circulo` **cómo** calcular: sólo lo obliga a tener `area()` y `perimetro()`. Un `Rectangulo` cumple el mismo contrato con fórmulas totalmente distintas:

```
public class Rectangulo implements Figura {
    private final double base;
    private final double altura;

    public Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override public double area()      { return base * altura; }
    @Override public double perimetro() { return 2 * (base + altura); }
}
```

3.6.1 Usar la interfaz: un solo código para todas las implementaciones

Declarar el contrato es la mitad de la historia; el valor aparece al **usarlo**. Tres formas de uso, de menor a mayor potencia:

1. Variables del tipo de la interfaz. Una variable `Figura` puede apuntar a CUALQUIER implementación — y se puede cambiar sin tocar el resto del código:

```
Figura figura = new Circulo(2);            // hoy es un círculo
System.out.println(figura.area());         // 12.566370614359172
figura = new Rectangulo(3, 4);             // mañana, un rectángulo: nada más cambia
System.out.println(figura.area());         // 12.0
```

2. Parámetros del tipo de la interfaz. Un método que recibe `Figura` funciona con todas las figuras que existen… y con las que se inventen después. Se escribe UNA vez:

```
static void describir(Figura figura) {     // sirve para CUALQUIER figura
    System.out.println("área " + figura.area() + ", perímetro " + figura.perimetro());
}

describir(new Circulo(1));                 // área 3.14..., perímetro 6.28...
describir(new Rectangulo(2, 5));           // área 10.0, perímetro 14.0
```

3. Arreglos (y después colecciones) de la interfaz. Implementaciones distintas conviven en la misma estructura, y el **polimorfismo** hace que cada una responda a su manera:

```
Figura[] figuras = { new Circulo(2), new Rectangulo(3, 4), new Circulo(1) };
double areaTotal = 0;
for (Figura f : figuras) {
    areaTotal += f.area();                 // cada figura calcula con SU fórmula
}
System.out.println(areaTotal);             // 27.707963267948966
```

3.6.2 Otro dominio, mismo poder: intercambiar implementaciones

Para que no quede pegado a la geometría, un contrato de la vida real: “algo que sabe avisar”. Y una clase que lo USA dependiendo de la interfaz — nunca de una implementación concreta:

```
public interface Notificador {
    void notificar(String mensaje);

    default void notificarUrgente(String mensaje) {   // método default: cómo común
        notificar("¡URGENTE! " + mensaje);
    }
}

public class RecordatorioDeEntrega {
    private final Notificador notificador;    // depende del CONTRATO, no del cómo
    private final String materia;
```

```
    public RecordatorioDeEntrega(Notificador notificador, String materia) {
        this.notificador = notificador;
        this.materia = materia;
    }

    public void avisarEntrega(String tp, int diasQueFaltan) {
        if (diasQueFaltan == 0) {
            notificador.notificarUrgente(materia + ": ¡" + tp + " vence HOY!");
        } else {
            notificador.notificar(materia + ": " + tp + " vence en " + diasQueFaltan + " días");
        }
    }
}
```

En el repo hay dos implementaciones: `NotificadorPorConsola` (imprime) y `NotificadorEnMemoria` (no imprime nada: **acumula** los mensajes para poder inspeccionarlos). La misma `RecordatorioDeEntrega` funciona con ambas, sin tocarle una línea:

```
// con una implementación...
RecordatorioDeEntrega porConsola =
        new RecordatorioDeEntrega(new NotificadorPorConsola(), "CB100");
porConsola.avisarEntrega("TP individual", 3);   // [consola] CB100: TP individual vence en 3 días

// ...y con OTRA: RecordatorioDeEntrega ni se enteró del cambio
NotificadorEnMemoria memoria = new NotificadorEnMemoria();
RecordatorioDeEntrega paraTest = new RecordatorioDeEntrega(memoria, "CB100");
paraTest.avisarEntrega("TP grupal", 7);
paraTest.avisarEntrega("TP grupal", 0);
System.out.println(memoria.cantidadDeAvisos());  // 2
```

¿Y para qué querría alguien un notificador que no notifica? **Para los tests**: enchufás la implementación en memoria, ejercitás tu clase, y verificás qué avisos salieron — sin llenar la consola ni mandar nada de verdad. Este truco (una implementación "de mentira" que cumple el contrato) es moneda corriente en la industria, y funciona únicamente porque `RecordatorioDeEntrega` depende de la interfaz.

Idea clave

Programá contra el contrato, no contra la implementación. Atributos, parámetros y retornos declarados por la interfaz (`Figura`, `Notificador`) — la implementación concreta aparece SOLO en el `new`. Beneficio doble: podés cambiar el cómo sin tocar a los usuarios, y podés testear con implementaciones falsas. Vas a ver esta idea a escala en la U5 (la interfaz `Lista` con dos implementaciones) y en cada TDA del curso.

Las interfaces admiten métodos `default` (con implementación heredable, como `notificarUrgente`) para dar comportamiento común, pero su rol principal es **especificar qué**, dejando el **cómo** a cada implementación.

3.7 Clases y métodos abstractos

A veces querés una clase base que **comparta código** entre subclases pero que no tenga sentido instanciar por sí sola: eso es una **clase abstracta** (`abstract class`). Puede tener métodos concretos y **métodos abstractos** (sin cuerpo) que cada subclase debe completar.

```
public abstract class FiguraBase implements Figura, Comparable<Figura> {
    private final String nombre;
    protected FiguraBase(String nombre) { this.nombre = nombre; }

    // area() y perimetro() quedan ABSTRACTOS: los define cada figura concreta.

    @Override public int compareTo(Figura otra) { return Double.compare(area(), otra.area()); }
    @Override public String toString() { return String.format("%s(área=%.2f)", nombre, area()); }
```

```
}
```

Idea clave

¿Interfaz o clase abstracta? Usá una **interfaz** para declarar un contrato que muchas clases (aun sin relación entre sí) pueden cumplir; usá una **clase abstracta** cuando además querés **compartir estado o código** entre subclases emparentadas. Una clase implementa muchas interfaces, pero hereda de una sola clase.

3.8 Los métodos estándar

Casi toda clase de "valor" implementa algunos métodos que Java y sus bibliotecas esperan encontrar:

- **`toString()`** — una representación legible del objeto (para imprimir y depurar).
- **`equals(Object)`** — igualdad por **contenido** (Unidad 2), no por identidad.
- **`hashCode()`** — un entero consistente con `equals`: si dos objetos son iguales, deben tener el mismo `hashCode` (esencial para Hashing, más adelante).
- **`compareTo(...)`** (de `Comparable`) — define el **orden natural**, para poder ordenar.
- **`clone()`** — una copia del objeto; en la práctica se prefiere un **constructor copia**, porque `clone` es delicado (copia superficial y diseño discutido).

```java
public final class Fraccion implements Comparable<Fraccion> {
    private final int numerador, denominador;

    public Fraccion(int numerador, int denominador) {
        if (denominador == 0) {
            throw new IllegalArgumentException("denominador 0");
        }
        int divisor = mcd(Math.abs(numerador), Math.abs(denominador));
        this.numerador   = numerador / divisor;    // guarda SIEMPRE simplificada:
        this.denominador = denominador / divisor;  // 2/4 se almacena como 1/2
    }
    private static int mcd(int a, int b) { return b == 0 ? a : mcd(b, a % b); }

    @Override public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Fraccion f)) {
            return false;
        }
        return numerador == f.numerador && denominador == f.denominador;
    }
    @Override public int hashCode()  { return Objects.hash(numerador, denominador); }
    @Override public int compareTo(Fraccion o) {
        return Long.compare((long) numerador * o.denominador, (long) o.numerador * denominador);
    }
    @Override public String toString() { return numerador + "/" + denominador; }
}
```

Un detalle de diseño: el constructor **simplifica** al crear la fracción (dividiendo por el máximo común divisor), así el objeto guarda siempre la forma canónica. Eso hace que el `equals` de arriba sea correcto por comparación directa: `new Fraccion(1, 2)` y `new Fraccion(2, 4)` almacenan ambos `1/2`, y por lo tanto son `equals`. Sin esa normalización, el `equals` campo a campo diría que $1/2 \neq 2/4$ — un bug silencioso.

Error común

`equals` y `hashCode` van **siempre juntos**: si redefinís uno, redefiní el otro de forma consistente. Romper esta regla causa errores muy difíciles de encontrar cuando uses tablas de hash.

¿Por qué es innegociable? La respuesta completa llega en la **Unidad 7 (Hashing)**: ahí vas a construir estructuras que ubican cada objeto en un lugar calculado **a partir de su `hashCode`**, y vas a ver — con diagrama y todo — cómo un objeto con `equals` sin `hashCode` literalmente "se pierde" adentro de ellas. Por ahora, la regla práctica: **si dos objetos son `equals`, deben tener el mismo `hashCode`** — y lo más simple es generar ambos con el IDE, o usar un `record`, que te los da consistentes gratis.

3.9 Genéricos

Los **genéricos** permiten escribir clases y métodos que funcionan con **cualquier tipo**, manteniendo la seguridad de tipos en tiempo de compilación. Se escriben con un parámetro de tipo entre `< >`.

```
public class Caja<T> {          // T es el tipo que se decide al usar la caja
    private T contenido;
    public Caja(T contenido) { this.contenido = contenido; }
    public T get() { return contenido; }
}

Caja<String> c = new Caja<>("hola");     // una caja de String
```

También podés **acotar** el tipo (`<E extends Comparable<E>>`) para exigir que soporte ciertas operaciones. Los genéricos son la base de las estructuras de datos reutilizables que vamos a construir en las próximas unidades.

3.10 Diagramas UML de clases

UML (*Unified Modeling Language*) es la notación estándar para diseñar y comunicar sistemas orientados a objetos. El diagrama de clases muestra las clases (con sus campos y métodos), su **visibilidad** y sus **relaciones**.

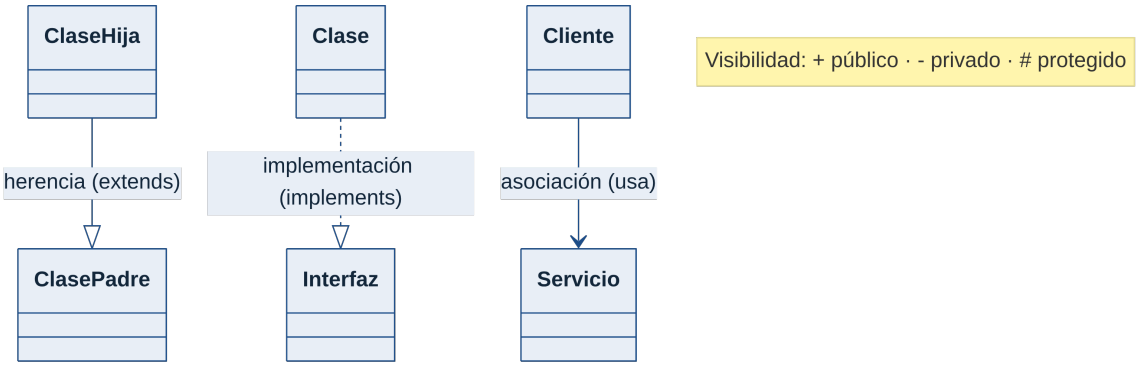


Figura 3.8: Notación UML: herencia (flecha de triángulo hueco), implementación (línea punteada), asociación, y composición (rombo lleno del lado del todo).

Así se ve el diseño de nuestro TDA `Figura` con su clase abstracta y sus figuras concretas:

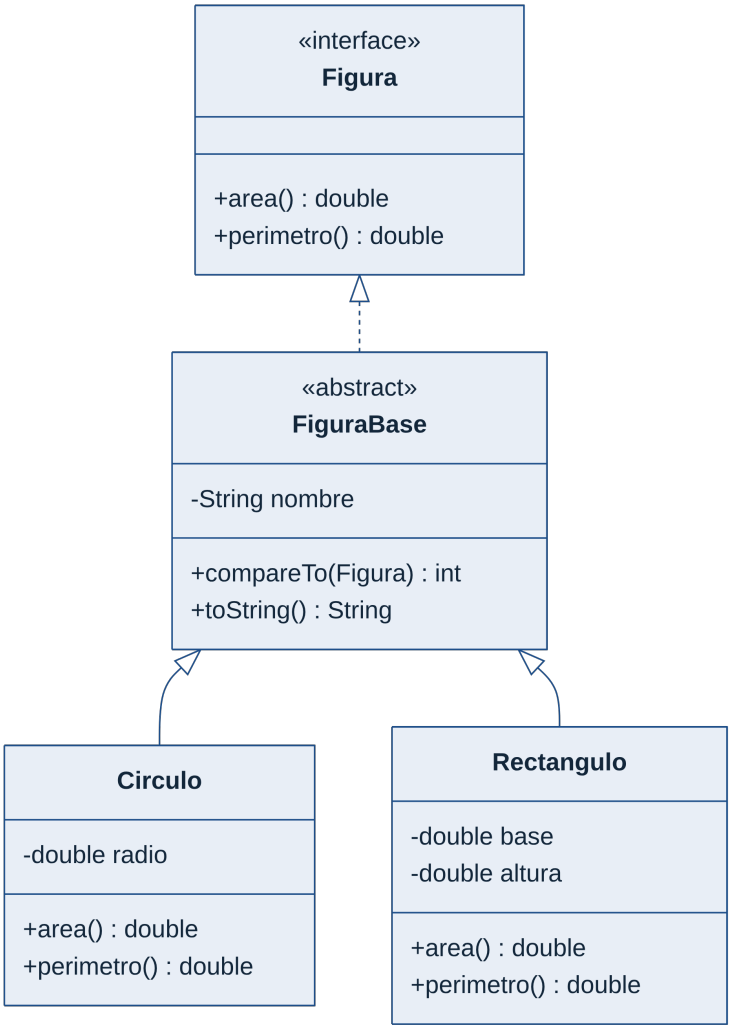


Figura 3.9: El TDA `Figura` (interfaz), la clase abstracta `FiguraBase` y las figuras concretas.

3.11 Diseño profesional: elegancia, robustez y mantenimiento

Programar bien no es sólo que "funcione": es escribir código que otra persona (¡o vos en seis meses!) pueda **entender y modificar** sin miedo. Algunas prácticas que perseguimos en la materia:

- **Nombres claros**: que digan qué hacen; el código se lee muchas más veces de las que se escribe.
- **Responsabilidad única**: cada clase y cada método hacen **una** cosa bien.
- **Encapsulamiento**: estado privado, invariantes protegidos.
- **Inmutabilidad cuando se pueda**: menos errores por estado compartido (Unidad 2).
- **Validar y fallar claro**: precondiciones explícitas y excepciones con buen mensaje (robustez).
- **Modularización**: separar en piezas con contratos claros (interfaces/TDAs) permite cambiar una sin romper las otras.
- **Tests**: cada pieza con sus pruebas, como red de seguridad ante cambios.

Idea clave

Un código **elegante** no es el más "corto" ni el más "ingenioso": es el que se entiende de un vistazo, es difícil de usar mal y fácil de cambiar. Ese es el estándar profesional al que apuntamos.

3.11.1 Validar con criterio

Validar está bien, pero **no todo se valida**. Una buena validación protege un invariante barato de chequear; una mala validación gasta tiempo comprobando algo que ya es parte del **contrato** de quien llama.

Validaciones que tienen sentido (baratas y evitan estados inválidos): un valor no nulo, o un número dentro de un rango.

```
public Persona(String nombre, int edad) {
    this.nombre = Objects.requireNonNull(nombre, "nombre no puede ser null");
    if (edad < 0 || edad > 150) {
        throw new IllegalArgumentException("edad inválida: " + edad);
    }
    this.edad = edad;
}
```

Validaciones que NO tienen sentido: si un método tiene como **precondición** que el arreglo venga **ordenado** (por ejemplo, la búsqueda binaria), **no** hay que verificar que lo esté: recorrerlo para chequearlo cuesta O(n) y destruiría la ventaja de la búsqueda binaria (O(log n), que veremos en la Unidad 4). Que el arreglo esté ordenado es responsabilidad de quien llama; se **documenta** en el contrato, no se comprueba.

```
/** PRECONDICIÓN: 'a' está ordenado de menor a mayor.
 *  NO se verifica acá: costaría O(n) y anularía la ventaja de la búsqueda binaria. */
static int busquedaBinaria(int[] a, int x) { /* ... O(log n) ... */ }
```

Unificar las validaciones en una clase utilitaria evita repetir código y mantiene los mensajes consistentes:

```
public final class Preconditions {
    private Preconditions() {}
    public static <T> T noNulo(T v, String nombre) {
        return Objects.requireNonNull(v, nombre + " no puede ser null");
    }
    public static int enRango(int v, int min, int max, String nombre) {
        if (v < min || v > max) {
            throw new IllegalArgumentException(nombre + " fuera de rango [" + min + ".." + max + "]: " + v);
        }
        return v;
    }
}

// uso:
this.edad = Preconditions.enRango(edad, 0, 150, "edad");
```

En el repositorio está en `material.i03_poo`, clase `Preconditions`.

3.12 El camino profesional: integración continua

Todo lo anterior se potencia con una práctica de la industria: la **Integración Continua** (*Continuous Integration*, CI). La idea: cada vez que subís cambios al repositorio, un servidor **compila el proyecto y corre todos los tests automáticamente**, y avisa si algo se rompió. Así los errores se detectan en minutos, no en semanas.

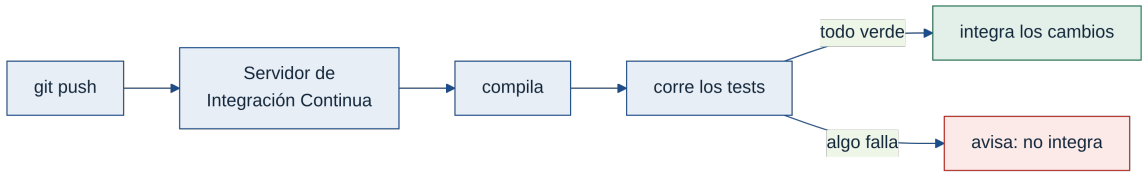


Figura 3.10: Integración continua: cada push dispara compilar y testear; si algo falla, avisa.

Con Git / Gradle

Este apunte es el primer paso. Ya tenés Git, un proyecto Gradle y tests JUnit: son exactamente los ingredientes que la CI automatiza. Como **objetivo a futuro**, la cátedra apunta a sumar CI (por ejemplo con *GitHub Actions*) para que los TP se integren y validen solos, más medición de cobertura y análisis de estilo. Ir tomando estas costumbres ahora es lo que te va a diferenciar como profesional.

3.13 Ejemplo integrador

El TDA `Figura` reúne casi todo lo de la unidad: una **interfaz** (contrato), una **clase abstracta** que comparte código, **herencia** y **polimorfismo** en las figuras concretas, **orden natural** con `compareTo`, y `toString`. Este código está completo en el repositorio (`material.i03_poo`):

```
Figura[] figuras = { new Circulo(2), new Rectangulo(3, 4), new Circulo(1) };
for (Figura f : figuras) {
    System.out.println(f);   // polimorfismo: cada una da su área
}

FiguraBase[] ordenables = { new Circulo(2), new Rectangulo(3, 4), new Circulo(1) };
Arrays.sort(ordenables);                          // usa compareTo (orden por área)
```

3.14 El checklist del ejercicio de TDA

Todo lo anterior, convertido en una lista para pasar **punto por punto antes de entregar** — en el TP y en el parcial, el ejercicio de TDA se corrige exactamente contra esto.

Idea clave

1. El objetivo principal es el DISEÑO ELEGANTE. No se evalúa “que ande”: se evalúa que el TDA esté bien diseñado — que se lea como el dominio, que se use sin sorpresas, y que se pueda cambiar por dentro sin romper a nadie. Los puntos que siguen son la definición operativa de “elegante”.

2. Conjunto completo de operaciones. ¿Cómo sabés que no te falta ni sobra ninguna? Con un **caso de uso**: una historia concreta y completa de uso del TDA, contada de punta a punta. Para un TDA `Cine`: “se crea la sala con 30 butacas, se venden tres entradas, alguien intenta comprar una butaca vendida y se lo rechaza, se pregunta la recaudación, se cancela una venta”. Cuatro formas de detectarlo, todas con el mismo espíritu:

- **Escribí el `main` del caso de uso**: si para contar la historia te faltó un método, el conjunto estaba incompleto; si un método no apareció nunca, probablemente sobre.
- **Escribí el test del caso de uso**: mismo efecto, con la ventaja de que queda verificado para siempre (JUnit, U1B).
- **Mirá el TDA desde el punto de vista del USUARIO** (el que lo usa sin ver la implementación): ¿puede hacer todo lo que el dominio pide, sólo con los métodos públicos?
- **Imprimí el contenido**: el TDA tiene que poder mostrar TODOS los atributos con los que opera (su `toString`, punto 7). Si algo no se puede ver, difícilmente esté bien pensado.

3. Robustez: validá los parámetros de CADA método. Todo método público verifica sus precondiciones y falla con una excepción clara (`IllegalArgumentException` y compañía, U1B). Un TDA que acepta basura promete cualquier cosa.

4. Encapsular: que nadie rompa tu TDA por acceso directo. Atributos `private`, siempre. El único camino a los datos son los métodos — y ellos defienden el invariante.

5. Pre y post: cada método tiene contrato. El javadoc dice qué exige (pre), qué garantiza (post), qué devuelve y qué lanza. Quien llama sabe a qué atenerse sin leer la implementación.

6. No implementes los métodos del TDA chico en el TDA que lo contiene. Si `Biblioteca` contiene `Libro`, la lógica de un libro (¿está vencido?, ¿es del autor X?) vive en `Libro` — la `Biblioteca` **le pregunta**, no lo resuelve por él. Cada TDA implementa lo suyo; el contenedor coordina.

7. `equals` y `toString`. `equals` por contenido (con su `hashCode`, "Los métodos estándar") y un `toString` que muestre el estado completo (punto 2). Sin ellos, el TDA no se puede comparar ni inspeccionar.

8. Nunca hardcoding. Ni números ni textos mágicos sueltos en el código: constantes con nombre (`CAPACIDAD_MAXIMA`) o parámetros. Si un valor puede cambiar, entra por el constructor.

9. Si hay un vector, la capacidad entra por el constructor (o el vector se redimensiona, U2). `new Pelicula[100]` con el 100 clavado en el código es el punto 8 y el 9 violados a la vez.

10. "Sin valor" se dice `null` — nunca −1 ni `MAX_VALUE`. Cuando un dato puede "todavía no estar" (la mejor nota, el primer turno, la fecha de devolución), usá el tipo wrapper (`Integer`, `Double`) inicializado en `null`. Un centinela mágico como −1 es un valor DISFRAZADO de ausencia: el día que −1 sea un dato válido, el TDA miente.

11. ¿Hace falta diagrama de estados? Y el estado es un ENUM, no un boolean. Si el TDA pasa por etapas (pendiente → confirmada → cancelada), dibujá el diagrama de estados y modelalo con un `enum` (U1B). Dos booleans (`confirmada`, `cancelada`) permiten el estado imposible `confirmada && cancelada`; el enum lo hace irrepresentable.

12. El tipo de dato no va en los nombres. `aplicaciones`, no `vectorDeAplicaciones`; `Agenda`, no `VectorDeTurnos`. El nombre habla del DOMINIO; la estructura interna es un secreto de implementación (que mañana puede cambiar — y el nombre quedaría mintiendo).

13. El `null` es ideal para máximos y mínimos. Es la otra cara del punto 10, y queda así de limpio — sin centinelas, sin casos especiales:

```java
public Pelicula laMasLarga() {
    Pelicula maxima = null;
    for (int i = 0; i < cantidad; i++) {
        if (maxima == null || peliculas[i].duracion() > maxima.duracion()) {
            maxima = peliculas[i];
        }
    }
    return maxima;              // null si no hay películas: ausencia honesta
}
```

14. En el ejercicio de TDA NO se usan listas, ni pilas, ni colas. Ni de `java.util` ni propias: **arreglos administrados por vos** (con su cantidad, su redimensión si hace falta). Administrar el arreglo ES parte de lo que el ejercicio evalúa. Las colecciones llegan en la Unidad 5, y desde ahí sí.

15. Lo calculable es un MÉTODO, no un atributo. La recaudación es la suma de las entradas vendidas: `recaudacion()` la calcula cuando se la piden. Un atributo `recaudacion` que hay que "mantener actualizado" es un bug en cámara lenta: tarde o temprano se desincroniza del dato real.

16. Recibí el TDA entero por parámetro, no sus atributos sueltos (la excepción: un `id` para buscar). El TDA viaja completo:

```java
// así NO: la película desarmada en pedacitos
public void agregar(String titulo, int duracion, String genero) { ... }
```

```java
// así SÍ: el TDA entero...
public void agregar(Pelicula pelicula) { ... }

// ...y la excepción razonable: un id para buscar
public Pelicula buscarPorId(int id) { ... }
```

17. Nada de `System.out` adentro del TDA (salvo que el TDA *sea* el manejo de pantalla). El TDA **devuelve** datos; imprimirlos es decisión de quien lo usa (el `main`, la pantalla). Un TDA que imprime no se puede usar en un test, en una ventana ni en un servidor — quedó soldado a la consola.

Y tres más que redondean la lista:

18. Copia defensiva al devolver lo interno. Si un método devuelve tu arreglo interno (o un objeto mutable tuyo), devolvé una **copia** (`Arrays.copyOf`): si entregás el original, el punto 4 era decorativo — cualquiera edita tu estado por afuera.

19. El constructor entrega un TDA válido. Los invariantes valen desde el primer instante: el constructor valida sus parámetros (punto 3) y deja todo listo. Lo que no cambia nunca, `final`.

20. El test también prueba lo INVÁLIDO. Por cada validación del punto 3, un `assertThrows` que demuestre que el TDA se defiende. El caso de uso (punto 2) prueba que hace lo que debe; estos prueban que NO hace lo que no debe.

Error común

Los clásicos que más puntos cuestan en el ejercicio de TDA, todos violaciones de esta lista: atributos públicos (4) · `-1` como "no hay" (10/13) · dos booleans en vez de un enum (11) · `vectorDeSocios` (12) · `ArrayList` en el ejercicio (14) · el atributo `promedio` desactualizado (15) · el método que recibe siete parámetros que eran un objeto (16) · el `System.out.println` enterrado en el TDA (17) · el getter que regala el arreglo interno (18).

3.15 Errores comunes

Error común

- Confundir el TDA (el contrato) con su implementación (el cómo). Programá contra el contrato.
- Exponer campos `public` en vez de encapsular.
- Redefinir `equals` y olvidar `hashCode` (o al revés).
- Usar herencia donde no hay una relación "es un/una" real (a veces conviene composición).
- Creer que "paradigma TDA" es lo mismo que POO: el paradigma es la POO.
- Pensar que un código más corto o ingenioso es mejor: priorizá que se entienda.

3.16 Repaso

1. ¿Qué es la abstracción y por qué es la idea central de la POO?
2. ¿Qué define un TDA y por qué su contrato es independiente de la implementación?
3. ¿Cuál es la diferencia entre una clase y un objeto?
4. Explicá los tres pilares: encapsulamiento, herencia y polimorfismo.
5. ¿"Un auto tiene ruedas" pide herencia o composición? ¿Y "un cuadrado es una figura"? Justificá con la regla "es un / tiene un".
6. ¿Cuándo usarías una interfaz y cuándo una clase abstracta?
7. ¿Por qué `equals` y `hashCode` deben ser consistentes? ¿Qué clase de estructura (que vas a ver en la Unidad 7) depende de esa consistencia para no "perder" objetos?
8. ¿Para qué sirven los genéricos? Dá un ejemplo.
9. ¿Qué relaciones se representan en un diagrama UML de clases?
10. Nombrá tres prácticas que hacen un código más mantenible.

11. ¿Qué es la integración continua y qué ingredientes de la materia la habilitan?

Guía de ejercicios — Unidad 3

Objetivos de la unidad

Practicá diseño con TDA y POO. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i03_poo` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio, p. ej. `guia.i03_poo.i04_dificil.e05`), con su **test JUnit**. Las preguntas teóricas se responden en `guia/i03_poo/i01_teorico`. Programá contra el **contrato**, encapsulá el estado y validá los invariantes. Dos reglas de oro: todo TDA se entrega pasado por el **checklist del ejercicio de TDA** (fin del capítulo), y acá se trabaja con el **vector nativo `[]`** — nada de listas, pilas ni colas, que recién llegan en la Unidad 5 (checklist, punto 14).

3.17 Preguntas teóricas

1. ¿Qué es la abstracción y por qué es la idea central de la POO?
2. ¿Qué define un TDA y por qué su contrato es independiente de la implementación?
3. ¿Cuál es la diferencia entre una clase y un objeto?
4. Explicá los tres pilares: encapsulamiento, herencia y polimorfismo.
5. ¿Cuándo usarías una interfaz y cuándo una clase abstracta?
6. ¿Por qué `equals` y `hashCode` deben ser consistentes?
7. ¿Para qué sirven los genéricos? Dá un ejemplo.
8. ¿Qué relaciones se representan en un diagrama UML de clases?
9. Nombrá tres prácticas que hacen un código más mantenible y explicá por qué.
10. ¿Qué es la integración continua y qué ingredientes de la materia la habilitan?

3.18 Nivel fácil

1. **Punto encapsulado.** Clase `Punto` con campos privados `x`, `y`, constructor y getters. Test.
2. **`toString` propio.** Clase `Persona(nombre, edad)` con un `toString` legible; test que incluye el nombre.
3. **`equals`/`hashCode`.** Agregá `equals` y `hashCode` a `Punto`; test que dos puntos iguales lo cumplen.
4. **Orden natural.** Clase `Persona` que implemente `Comparable` por edad; test del orden.
5. **Interfaz simple.** Interfaz `Saludador` con `saludar()`; dos implementaciones (`Formal`, `Informal`); test de polimorfismo.
6. **Herencia.** `Vehiculo(marca)` con `Auto` y `Moto` que redefinan `descripcion()`; test.
7. **Clase abstracta.** `Forma` abstracta con `area()` abstracto; `Cuadrado` la implementa; test.
8. **Genérico `Caja<T>`.** Con get/set; test usándola con `String` y con `Integer`.
9. **Constructor copia.** Clase con constructor copia; test que la copia es independiente.
10. **Invariante.** `Termometro` que rechace temperaturas bajo el cero absoluto con una excepción; test.

3.19 Nivel medio

1. **TDA Cadena.** Sobre `char[]` (sin `String` por dentro, salvo en `toString`): `largo()`, `caracterEn(i)`, `contiene(char)`, `enMayusculas()` e `invertida()` (devuelven una Cadena NUEVA: inmutable), `equals` y `toString`. Test.
2. **TDA Tablero (ta-te-ti).** El del apunte: enum `Ficha` (X, O), `poner(ficha, fila, col)` validando casilla libre y turno alternado, `fichaEn`, `hayGanador`, `hayEmpate`. Elegí matriz 3×3 o arreglo plano de 9 — y justificá en el javadoc. Test con una partida completa y con jugadas inválidas.
3. **Polimorfismo con figuras.** Interfaz `Figura` + `Circulo`/`Rectangulo`; método que sume las áreas de un `Figura[]`; test.

4. **Ordenar por criterio.** Clase `Producto(precio) Comparable`; ordená un arreglo con `Arrays.sort`; test.
5. **Contrato de `equals`.** Clase con dos campos: `equals`/`hashCode` correctos; test reflexivo, simétrico, consistente y contra `null`.
6. **Herencia con `super`.** `Empleado(sueldoBase)` con `sueldo()`; `Gerente` que agrega un bono redefiniendo `sueldo()` y usando `super`; test.
7. **Genérico acotado.** `static <T extends Comparable<T>> T maximo(T[] a)`; test con `Integer` y `String`.
8. **Método plantilla.** Clase abstracta con un método concreto que llama a pasos abstractos que definen las subclases; test.
9. **Método `default`.** Interfaz con un método `default`; una implementación; test.
10. **Copia profunda.** Clase `Equipo` con un `int[]`; constructor copia que copie el arreglo; test de independencia.

3.20 Nivel difícil

1. **TDA Zoologico.** `Animal` (nombre, especie, peso) + `Zoologico` sobre arreglo con capacidad por constructor: `ingresar(animal)`, `buscarPorNombre(nombre)`, `elMasPesado()` (con `null` si no hay animales — checklist 13), `pesoTotal()` CALCULADO (checklist 15), `toString` con todo el contenido. Pasale el checklist completo antes de entregar; el test incluye los casos inválidos (checklist 20).
2. **TDA Tren con vagones.** `Vagon` (capacidad en kilos, carga actual; su `cargar` valida y ES SU lógica — checklist 6: el tren delega) + `Tren`: `engancharVagon`, `desengancharUltimo`, `cargar(kilos)` que reparte al primer vagón con lugar (y lanza si no entra en ninguno), `cargaTotal()` calculada, `vagonMasCargado()` (con `null`). Test del reparto y de los bordes.
3. **TDA Restaurant con mesas.** `Mesa` (número, capacidad, estado ENUM `LIBRE/RESERVADA/OCUPADA` — checklist 11: dibujá el diagrama de estados en el javadoc) + `Restaurant`: `reservar(personas)` asigna la mesa LIBRE **más chica** en la que entren (ésa es la problemática — pensala bien), `ocupar(numero)`, `liberar(numero, gasto)`, `recaudacion()` calculada y `mesasLibres()`. Test de todas las transiciones de estado inválidas (ocupar sin reserva, liberar una libre, reservar sin lugar).
4. **Jerarquía completa.** Interfaz `Figura` + clase abstracta + tres figuras concretas, `Comparable` por área y `toString`; test de polimorfismo y orden.
5. **Par genérico.** `Par<A,B>` con `equals`, `hashCode` y `toString`; test.
6. **TDA Cadena PRO.** Ampliá la Cadena del nivel medio: `concatenar(otra)`, `subcadena(desde, hasta)`, `indiceDe(otraCadena)` y `reemplazar(unChar, otroChar)` — todo sobre `char[]`, siempre devolviendo Cadenas nuevas. Test usando `String` como oráculo (mismo resultado, otra implementación).
7. **Ordenar con `Comparator`.** Además del orden natural, ordená por otro criterio pasando un `Comparator` (lambda); test.
8. **Coherencia equals/compareTo.** Clase de valor donde `equals` y `compareTo` sean consistentes (`compareTo==0` sii `equals`); test.
9. **Repositorio genérico.** Interfaz `Repositorio<T>` con `guardar`/`buscarPorId`; una implementación en memoria sobre arreglo; test.
10. **Un TDA, dos implementaciones.** Interfaz `Tablero` con **dos** implementaciones — `TableroConMatriz` (`Ficha[3][3]`) y `TableroPlano` (`Ficha[9]`, casilla = `fila * 3 + col`), como en la figura del apunte; un MISMO juego de tests verifica que ambas se comportan igual (contrato vs. implementación).

Unidad 3B

Stack y heap paso a paso

Objetivos de la unidad

Este apartado es el microscopio de la materia: **once escenarios** de código, cada uno con la evolución de la memoria dibujada **paso a paso**, atando cada diagrama a la línea que lo produce. Al terminarlo vas a poder predecir, para cualquier programa de las Unidades 1 a 3, **qué hay en el stack, qué hay en el heap y quién apunta a quién** en cada instante — y verificarlo solo, con el debugger de IntelliJ. Es exactamente la habilidad que evalúa el cuestionario del TP1.

Ya tenés las piezas: la Unidad 2 te presentó las dos memorias y la Unidad 3 los objetos. Acá las juntamos y las miramos en cámara lenta. Todos los escenarios están en el repo, en `material.i02_memoria.stackheap`, listos para ejecutar y — más importante — para **recorrer con el debugger**.

3B.1 Cómo leer los diagramas

Cada figura es una foto de la memoria en un instante, con el formato del pizarrón de clase:

- La **línea punteada vertical** separa el **STACK** (izquierda) del **HEAP** (derecha).
- Cada **frame** del stack es una caja con el nombre del método y sus variables; los frames se apilan **hacia abajo** (main arriba, la llamada más profunda abajo).
- Cada **objeto** del heap es una caja con su dirección inventada (`@A`, `@B`… — en la máquina real es un número) y sus atributos.
- Las **flechas** son las referencias: nacen en una variable y cruzan hacia su bloque. En Java no las ves como números ni podés operarlas: sólo seguirlas.
- **Colores**: negro = la estructura; **rojo** = lo que **cambió** en este paso; **verde** = lo **recién creado**; gris = lo **inalcanzable**. Un frame desapilado queda gris con una **X verde** encima.

3B.2 El debugger: el stack en vivo

Todo lo que dibujamos acá, IntelliJ lo muestra en tiempo real. Es la herramienta central de este apartado (y del cuestionario del TP1):

1. **Breakpoint**: clic en el margen izquierdo del editor, junto al número de línea — aparece el círculo rojo. El programa va a frenar ANTES de ejecutar esa línea.
2. **Debug**: ejecutá con el bicho verde (Shift + F9) en vez del play.
3. **Panel Frames** (izquierda abajo): ES el stack, dibujado. Cada fila es un frame; el de arriba es donde está frenado el programa. Hacé clic en cualquier frame para inspeccionarlo.
4. **Panel Variables** (derecha abajo): las variables del frame seleccionado. Un primitivo muestra su valor (`a = 5`). Una referencia muestra `Rectangulo@519` — ¡ese `@519` es nuestra flecha! — y se despliega con el triangulito para ver los atributos del bloque.
5. **Los tres pasos**: **Step Over** (F8) ejecuta la línea completa y frena en la siguiente; **Step Into** (F7) se METE en el método llamado (vas a ver apilarse el frame en el panel Frames); **Step Out** (Shift + F8) termina el método actual y vuelve al llamador (vas a ver el frame desapilarse). **Resume** (F9) sigue hasta el próximo breakpoint.

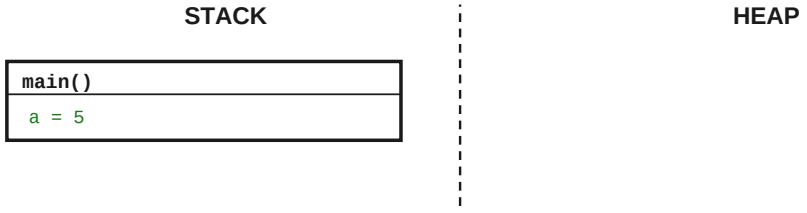
En IntelliJ

La comprobación reina, que vale para TODOS los escenarios: dos variables apuntan al mismo objeto **si y sólo si muestran el mismo `@`** en el panel Variables. Cuando dudes de un diagrama de este apartado, ponele un breakpoint al escenario y comparalo contra Frames + Variables: el debugger es el árbitro. El cuestionario del TP1 pregunta exactamente esto: qué muestra Frames, qué muestra Variables, y qué hace cada Step.

3B.3 Escenario 0 — Primitivos puros: el heap ni aparece

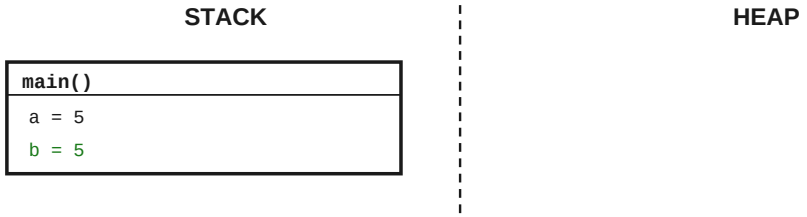
```java
int a = 5;
int b = a;
b = 7;
System.out.println(a);   // 5
System.out.println(b);   // 7
```

Paso 1 — `int a = 5;` La variable nace **adentro del frame** de `main`, con el valor adentro del casillero. No hay `new`, no hay heap, no hay flechas.



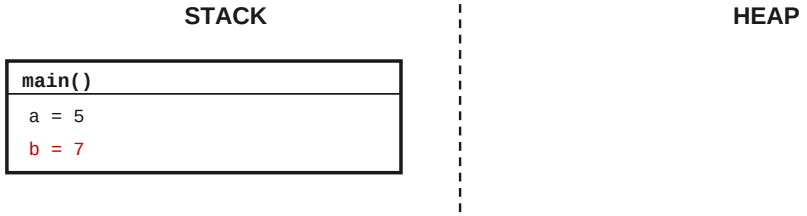
a nace en el frame de main con su valor adentro; el heap ni aparece

Paso 2 — `int b = a;` Asignar un primitivo **fotocopia el valor**: ahora hay dos casilleros con un 5 cada uno, sin ninguna conexión entre sí.



b COPIO el valor de a: dos casilleros independientes

Paso 3 — `b = 7;` Cambia el casillero de `b`. El de `a` ni se entera: nunca estuvieron conectados.



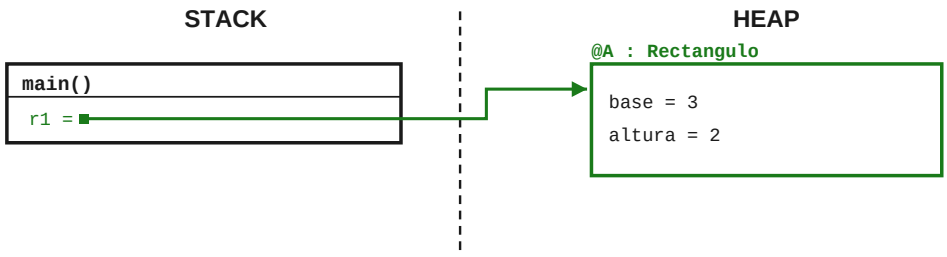
cambio b; a ni se entero. Con primitivos, asignar es fotocopiar

Debugger: breakpoint en el primer `println`; en Variables, `a` y `b` muestran sus valores directamente — sin `@`, sin triangulito. Ésa es la marca de un primitivo. (Repo: `E00PrimitivosPuros`.)

3B.4 Escenario 1 — Un new simple: del stack al heap y al atributo

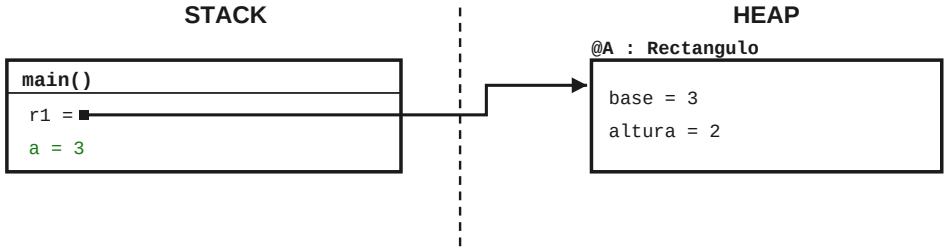
```
Rectangulo r1 = new Rectangulo(3, 2);
int a = r1.base;
```

Paso 1 — `Rectangulo r1 = new Rectangulo(3, 2);` El new hace DOS cosas: reserva un **bloque en el heap** (con los atributos adentro) y devuelve la **flecha** hacia él, que se guarda en `r1`. La variable `r1` vive en el stack; el Rectangulo vive en el heap; la flecha los une.



new reservo el bloque @A en el heap y guardo EN r1 la flecha hacia el

Paso 2 — `int a = r1.base;` El punto (`.`) significa "**seguí la flecha** y tocá ese atributo": se llega a `@A`, se lee `base` y ese 3 se copia al casillero primitivo `a`.



r1.base = seguir la flecha hasta @A y leer base: a copio ese 3

Debugger: Step Over (F8) sobre el `new` y mirá aparecer `r1 = Rectangulo@...` en Variables. Desplegalo: estás viendo el bloque del heap. (Repo: `E01NewSimple`.)

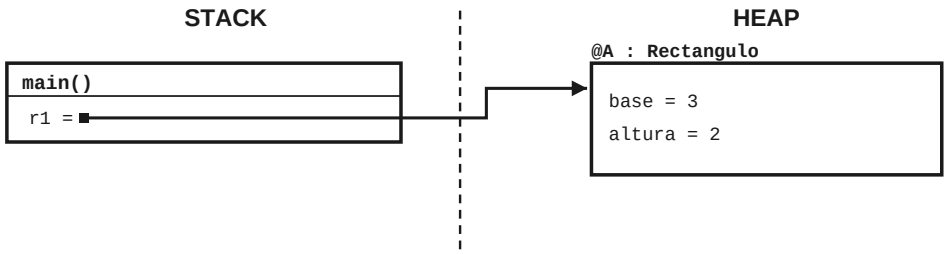
3B.5 Escenario 2 — Llamadas anidadas: el puntero viaja por parámetro

```
public static void main(String[] args) {
    Rectangulo r1 = new Rectangulo(3, 2);
    describir(r1);
    System.out.println("sigue main");
}

static void describir(Rectangulo figura) {
    int a = area(figura);
    System.out.println("área = " + a);
}

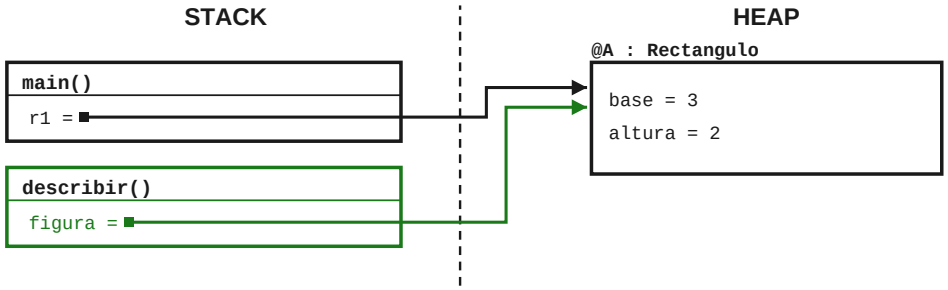
static int area(Rectangulo r) {
    return r.base * r.altura;
}
```

Paso 1 — el `new` en `main`. Un solo objeto; `main` tiene la única flecha.



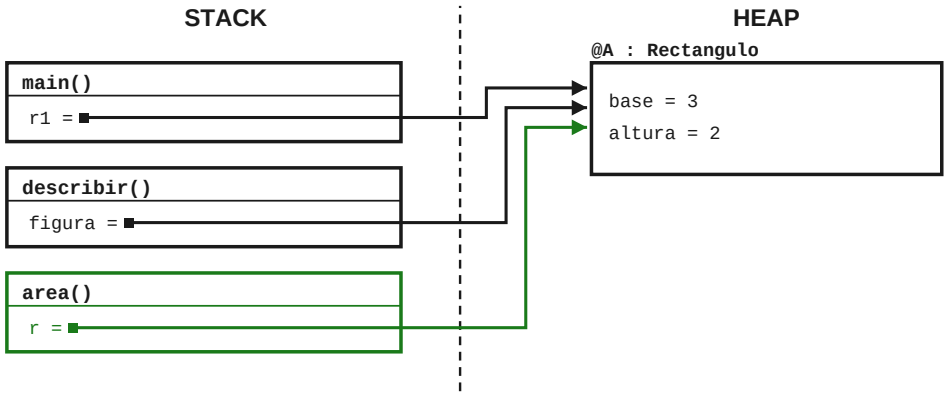
un solo objeto en el heap; main tiene la unica flecha

Paso 2 — `describir(r1);` La llamada **apila un frame**. Pasar `r1` por parámetro es una asignación disfrazada: `figura` recibe una **copia de la flecha** (no del objeto — el Rectangulo no se duplica).



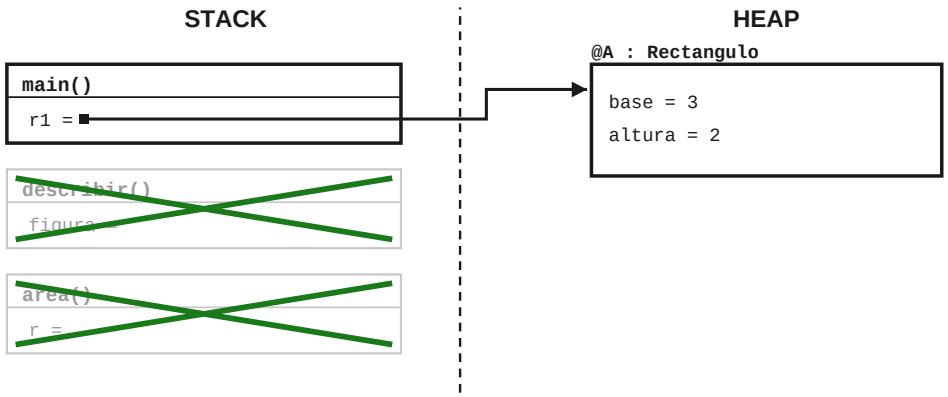
la llamada APILO el frame de describir; figura es una COPIA de la flecha

Paso 3 — `area(figura);` Otro frame más. Tres frames vivos, tres nombres distintos (`r1`, `figura`, `r`)... y **un solo bloque**: las tres flechas apuntan a `@A`.



tres frames vivos, tres nombres distintos, UN solo bloque

Paso 4 — los `return`. Cada método que termina **desapila su frame**, y sus variables mueren con él. El objeto del heap sigue ahí, con la flecha de `main` intacta.



al terminar, cada frame se desapila con sus variables; el objeto sigue

Debugger: breakpoint en la línea de `area`, y llegá hasta ahí con Step Into (F7) desde `main`. El panel

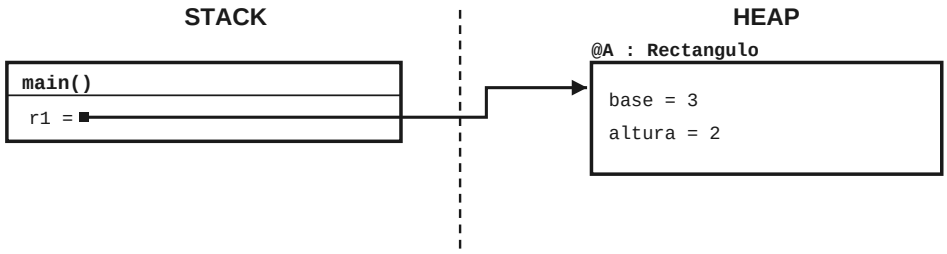
Frames muestra la pila entera (`area` ← `describir` ← `main`): hacé clic en cada frame y verificá que `r`, `figura` y `r1` tienen el **mismo** `@`. Después, Step Out (Shift + F8) dos veces y mirá los frames desaparecer. (Repo: `E02LlamadasAnidadas`.)

3B.6 Escenario 3 — El puntero this

```
Rectangulo r1 = new Rectangulo(3, 2);
r1.crecer(2);
System.out.println(r1);   // Rectangulo(5x4)

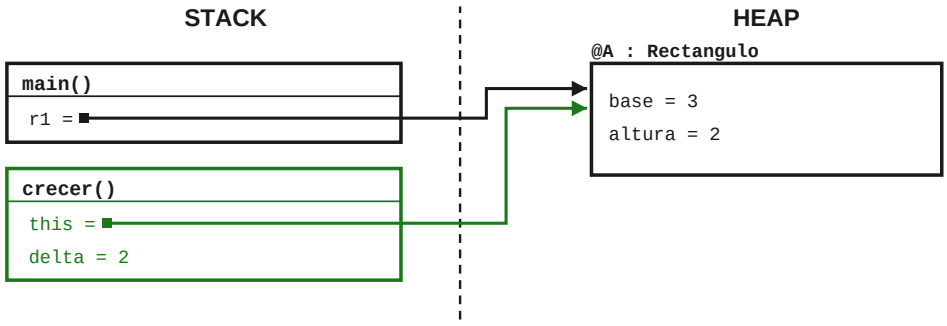
// y en Rectangulo.java:
public void crecer(int delta) {
    this.base = this.base + delta;
    this.altura = this.altura + delta;
}
```

Paso 1 — el `new`. Lo conocido: `r1` → `@A`.



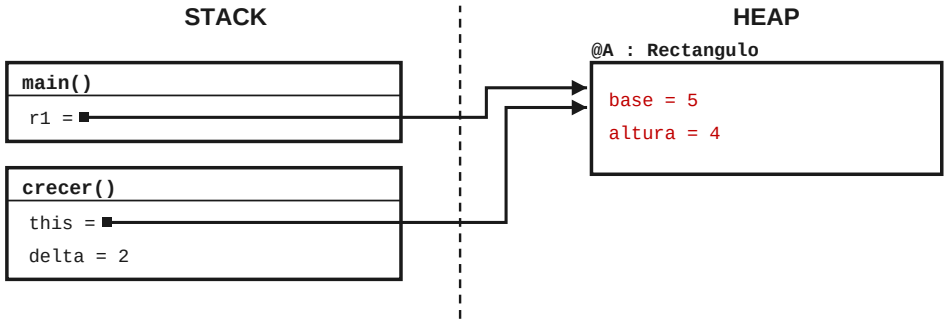
r1 apunta a @A, un Rectangulo de 3x2

Paso 2 — `r1.crecer(2);` Al llamar un método SOBRE un objeto, el frame recibe un parámetro oculto: **`this`, la copia de la flecha del objeto de la llamada**. "`r1.crecer(...)`" significa "ejecutá `crecer` con `this` = la flecha de `r1`".



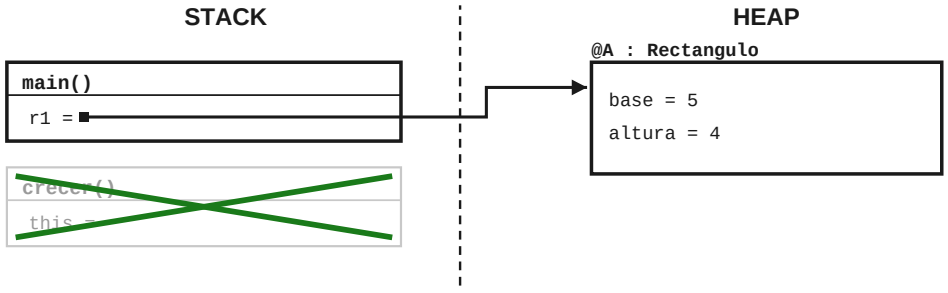
r1.crecer(2): el frame recibe el puntero OCULTO this = la flecha de r1

Paso 3 — `this.base = this.base + delta;` `this` se usa como cualquier puntero: seguir la flecha, escribir el atributo. Los cambios ocurren DENTRO de `@A`.



this.base = this.base + delta: se escribe DENTRO de @A

Paso 4 — vuelve a `main`. El frame de `crecer` muere, pero `r1` ve `Rectangulo(5x4)`: `this` era **su mismo bloque**.



crecer murio, pero r1 ve Rectangulo(5x4): this era SU mismo bloque

Debugger: breakpoint dentro de `crecer` (en `Rectangulo.java`). En Variables aparece `this = Rectangulo@...`: compará ese `@` con el de `r1` en el frame de `main` (clic en Frames). Idénticos. (Repo: `E03PunteroThis`.)

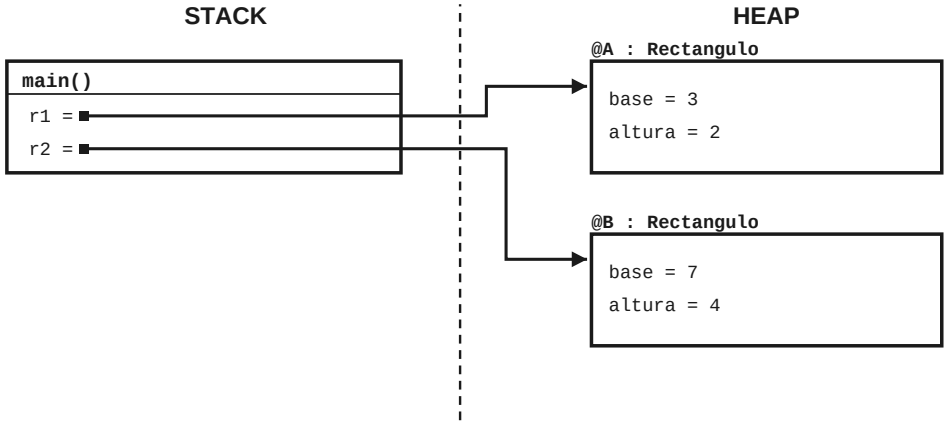
3B.7 Escenario 4 — Swap de punteros en un método: al salir se pierde

El clásico de parcial. ¿Intercambia `r1` y `r2`... o no?

```java
public static void main(String[] args) {
    Rectangulo r1 = new Rectangulo(3, 2);
    Rectangulo r2 = new Rectangulo(7, 4);
    intercambiar(r1, r2);
    System.out.println(r1);   // ¿?
    System.out.println(r2);   // ¿?
}

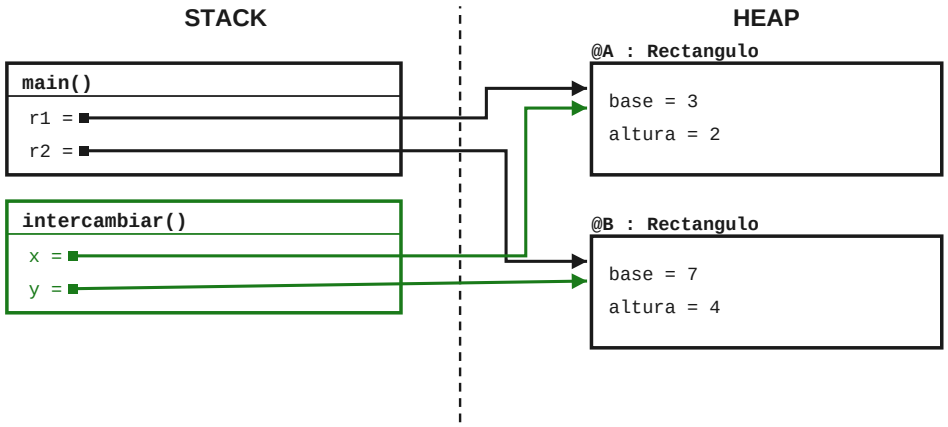
static void intercambiar(Rectangulo x, Rectangulo y) {
    Rectangulo tmp = x;
    x = y;
    y = tmp;
}
```

Paso 1 — los dos `new`. `r1` → `@A`, `r2` → `@B`.



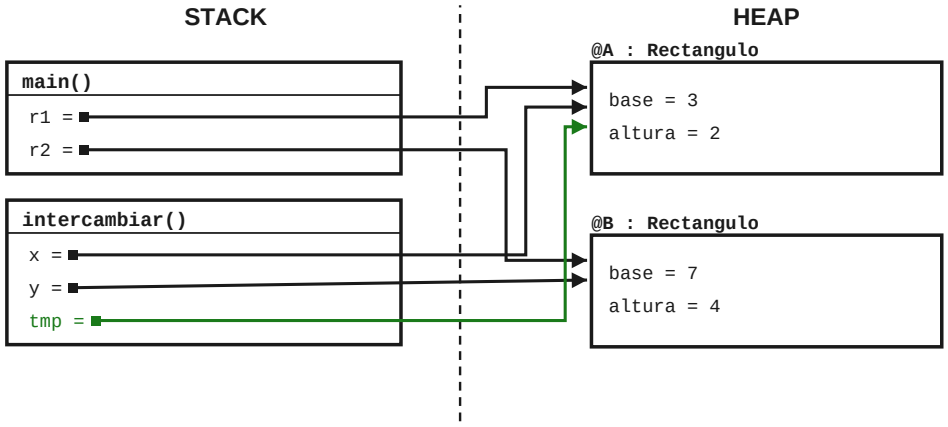
dos news: r1 -> @A y r2 -> @B

Paso 2 — `intercambiar(r1, r2);` Se apila el frame; `x` e `y` reciben **copias** de las flechas. Cuatro flechas, dos bloques.



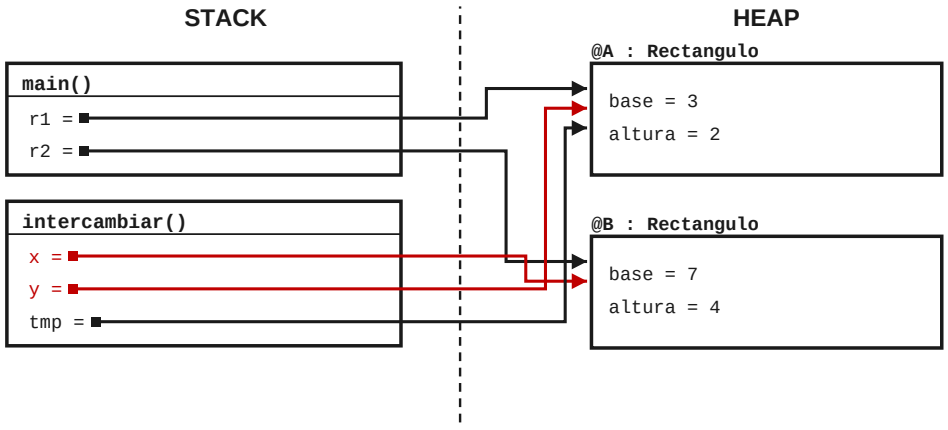
x e y son COPIAS de las flechas: cuatro flechas, dos bloques

Paso 3 — `Rectangulo tmp = x;` El backup del swap: `tmp` copia la flecha de `x`. (Tres flechas a `@A`.)



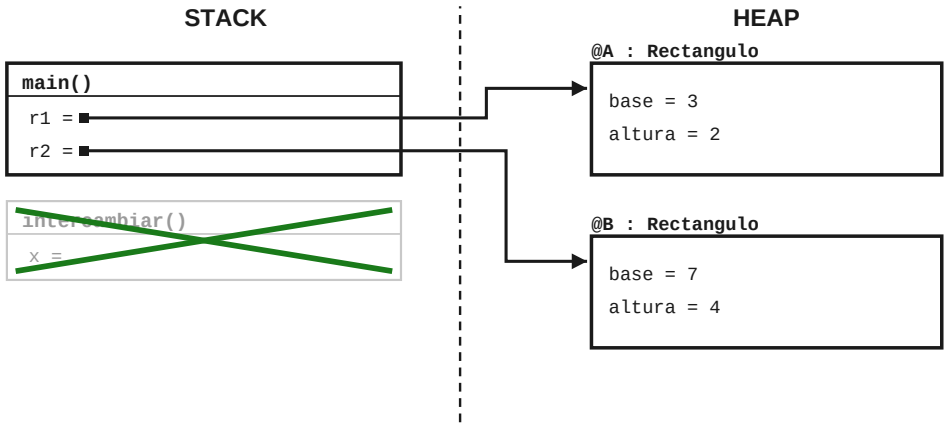
tmp guarda la flecha de x (el backup del swap)

Paso 4 — `x = y;` y `y = tmp;` Las copias se cruzan, con toda la prolijidad del mundo… pero son **copias**: `r1` y `r2`, en el frame de `main`, no se movieron un milímetro.



las COPIAS se cruzaron; r1 y r2 ni se enteraron

Paso 5 — el `return`. El frame se desapila y el cruce **muere con él**. En `main`: `r1` → `@A` y `r2` → `@B`, como si nada. El método intercambió sus propias copias, no tus variables.



el frame se desapilo y el cruce murio con el: r1 -> @A y r2 -> @B, intactos

Debugger: breakpoint en la primera línea de `intercambiar`. Con Step Over mirá los `@` de `x` e `y` cruzarse en Variables; con Step Out volvé a `main` y comprobá que los `@` de `r1` y `r2` siguen donde estaban. (Repo: `E04SwapDePunteros`.)

Idea clave

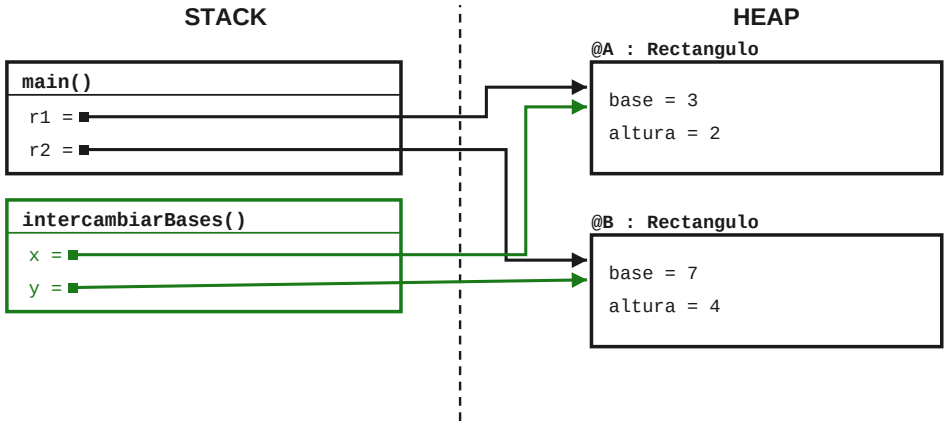
Java pasa TODO por valor. Un primitivo: se copia el valor. Una referencia: se copia **la flecha** (no el objeto). Por eso un método jamás puede REASIGNAR las variables del llamador (escenario 4)… pero sí puede modificar el CONTENIDO del bloque compartido al que la flecha apunta — que es exactamente el próximo escenario.

3B.8 Escenario 5 — Swap de valores: se mantiene

El mismo esqueleto, pero el método no toca sus flechas: las **usa** para escribir dentro de los bloques.

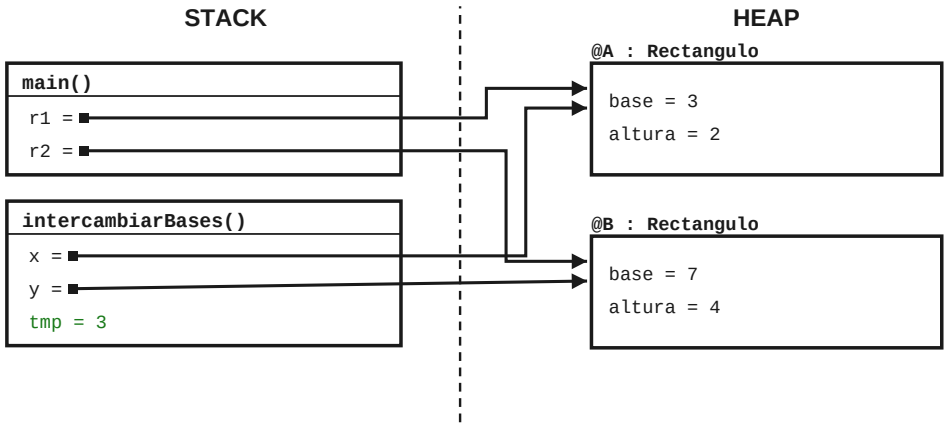
```
static void intercambiarBases(Rectangulo x, Rectangulo y) {
    int tmp = x.base;
    x.base = y.base;
    y.base = tmp;
}
// en main, después de llamar:
// r1 = Rectangulo(7x2)  <- ¡cambió!
// r2 = Rectangulo(3x4)  <- ¡cambió!
```

Paso 1 — la llamada. Igual que antes: `x` e `y` son copias de las flechas.



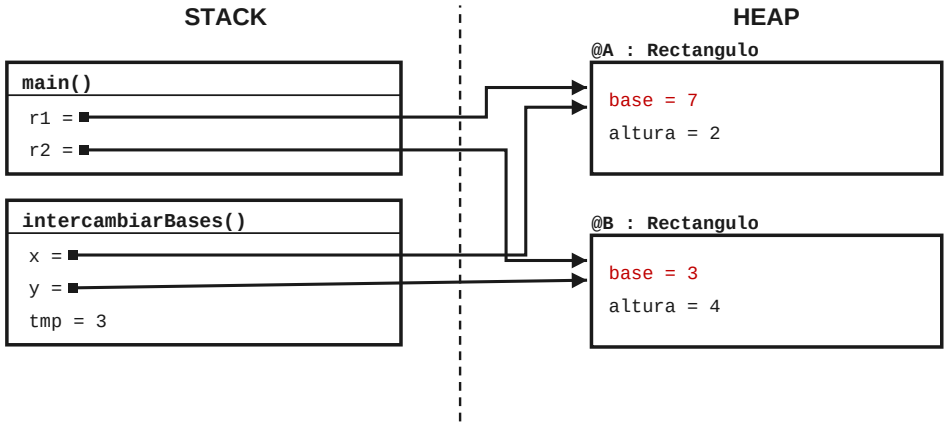
igual que el escenario 4: x e y son copias de las flechas

Paso 2 — `int tmp = x.base;` Seguir la flecha, leer el 3, copiarlo al primitivo `tmp` del frame.



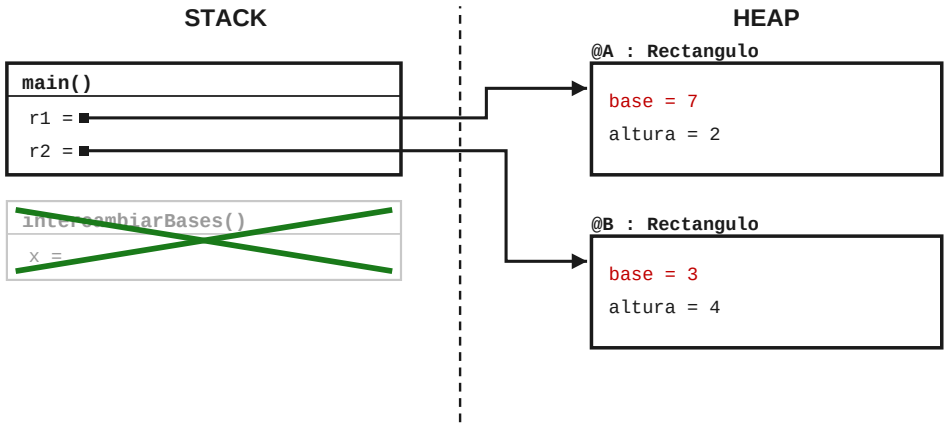
tmp = x.base: siguio la flecha y COPIO el 3 que estaba dentro de @A

Paso 3 — `x.base = y.base;` y `y.base = tmp;` Acá está TODA la diferencia con el escenario 4: no se reasignan flechas — se escribe **dentro de los bloques** `@A` y `@B`.



x.base = y.base; y.base = tmp: se escribe DENTRO de los bloques

Paso 4 — el `return`. El frame muere igual que antes... pero los cambios NO estaban en el frame: estaban en el heap, que es **compartido**. `main` ve las bases intercambiadas.



el frame murio, PERO el heap es compartido: main ve las bases cruzadas

Debugger: breakpoint en `main` DESPUÉS de la llamada; desplegá `r1` y `r2`: los `@` no cambiaron, los atributos sí. (Repo: `E05SwapDeValores`.)

3B.9 Escenario 6 — Dos news: atributos vs punteros, cara a cara

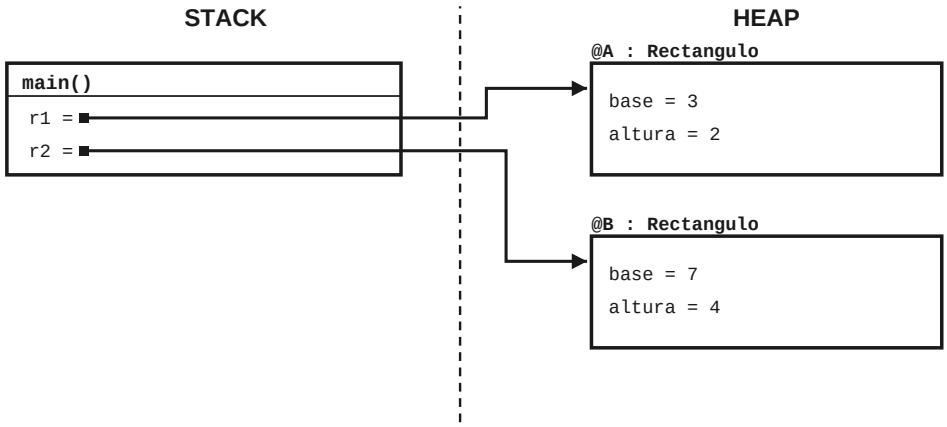
En `main`, donde las variables son tuyas, los dos intercambios funcionan — pero hacen cosas MUY distintas en la memoria:

```java
Rectangulo r1 = new Rectangulo(3, 2);
Rectangulo r2 = new Rectangulo(7, 4);

// (a) intercambiar ATRIBUTOS
int tmp = r1.base;  r1.base = r2.base;  r2.base = tmp;

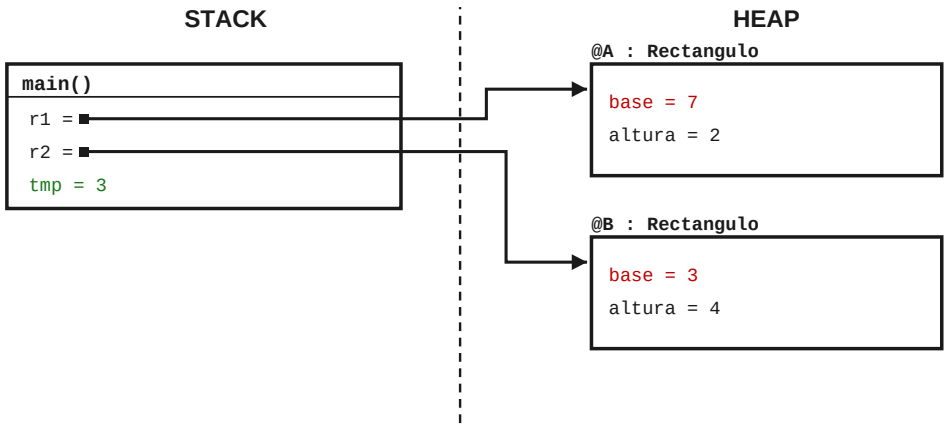
// (b) intercambiar PUNTEROS
Rectangulo aux = r1;  r1 = r2;  r2 = aux;
```

Paso 1 — los dos `new`.



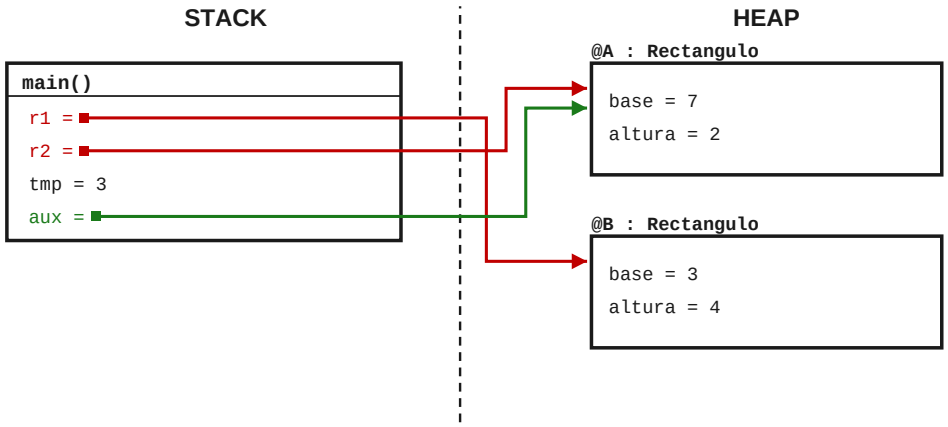
dos news, cada puntero con su bloque

Paso 2 — swap de atributos. Las flechas quedan quietas; los bloques cambian **por dentro**. Cualquier otra variable que apuntara a `@A` vería el cambio.



(a) swap de ATRIBUTOS: las flechas quedan; los bloques cambian por dentro

Paso 3 — swap de punteros. Los bloques ni se tocan; las **flechas se cruzan**. Nadie que mire los objetos nota nada — cambió sólo qué nombre apunta a qué bloque.



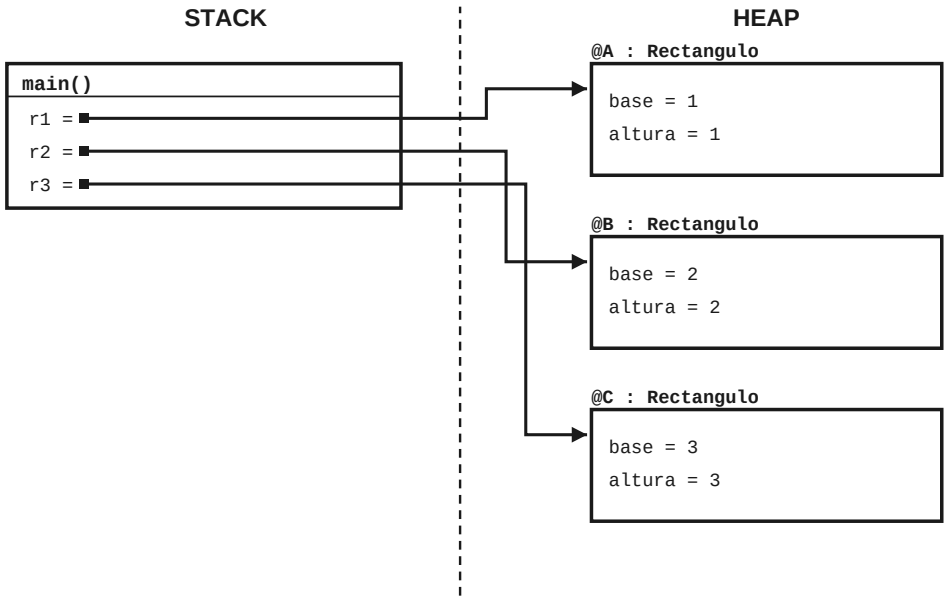
(b) swap de PUNTEROS: los bloques ni se tocan; las flechas se cruzan

Debugger: un breakpoint después de cada intercambio. En el primero, los `@` de `r1`/`r2` son los de siempre y las bases están cruzadas; en el segundo, los `@` se intercambiaron. (Repo: `E06DosNews`.)

3B.10 Escenario 7 — Varios news reasignados: varios bloques colgados

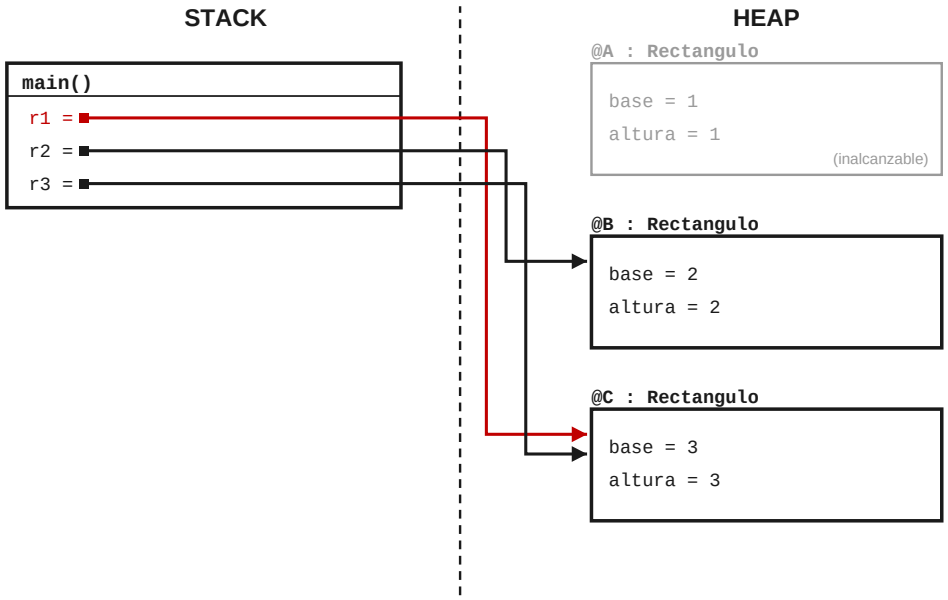
```
Rectangulo r1 = new Rectangulo(1, 1);
Rectangulo r2 = new Rectangulo(2, 2);
Rectangulo r3 = new Rectangulo(3, 3);
r1 = r3;
r2 = r3;
System.out.println(r1 == r2);   // true
```

Paso 1 — tres `new`. Cada puntero con su bloque.



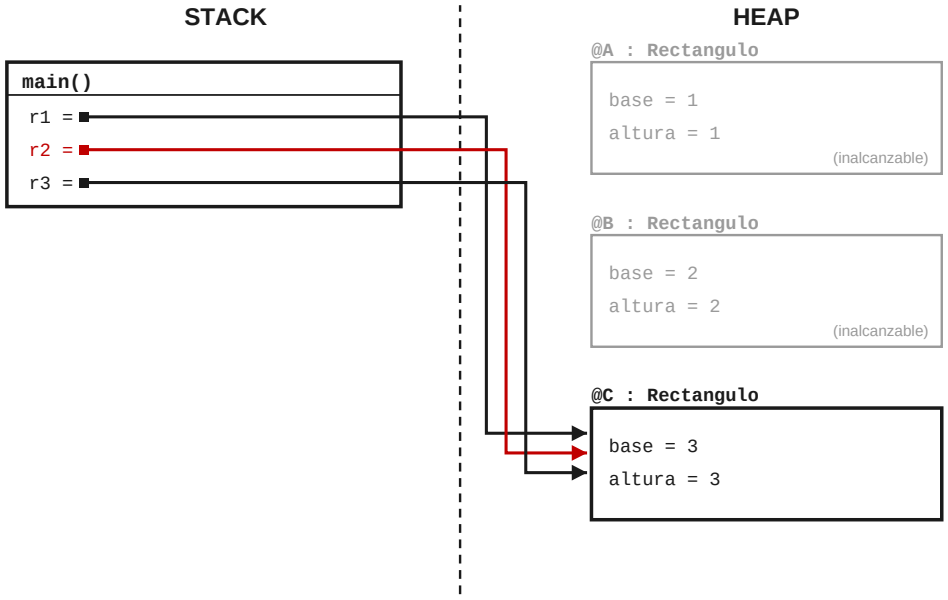
tres news, cada puntero con el suyo

Paso 2 — `r1 = r3;` `r1` copia la flecha de `r3`. `@A` pierde su ÚNICA flecha: sigue existiendo, pero **nadie puede llegar a él**. Está inalcanzable.



r1 = r3: @A perdio su UNICA flecha y quedo inalcanzable

Paso 3 — `r2 = r3;` Ídem `@B`. Resultado: los tres punteros miran a `@C`, y quedaron **DOS bloques huérfanos distintos**. El **recolector de basura** (garbage collector) los va a liberar cuando quiera — vos no hacés nada, y no hay forma de recuperarlos.



r2 = r3: DOS bloques huerfanos DISTINTOS; el GC los levantara cuando quiera

Debugger: breakpoint en el `println`: `r1`, `r2` y `r3` muestran el **mismo `@`**. A `@A` y `@B` no los encontrás desde ninguna variable — ésa es la definición práctica de “inalcanzable”. (Repo: `E07VariosNews`.)

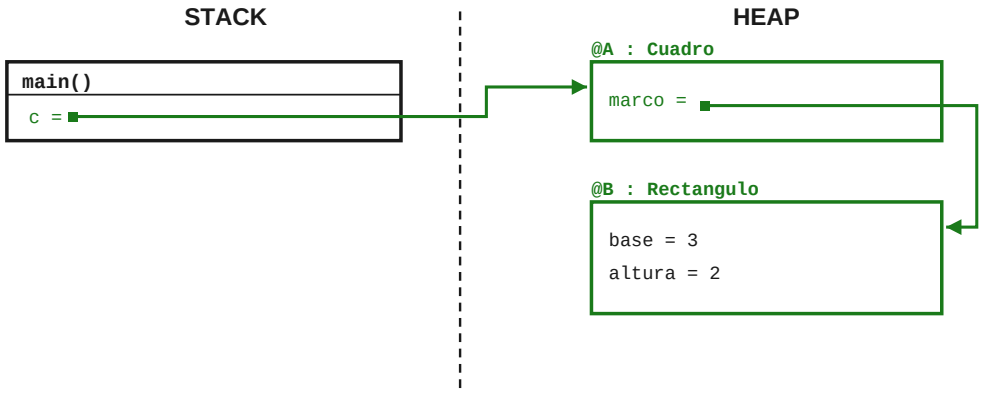
3B.11 Escenario 8 — El heap apunta al heap

Los punteros no son un privilegio del stack: un **atributo** puede ser una referencia, y entonces hay flechas que nacen y mueren dentro del heap.

```
public class Cuadro {
    public Rectangulo marco;      // un puntero DENTRO de un bloque del heap
    public Cuadro(Rectangulo marco) { this.marco = marco; }
}
```

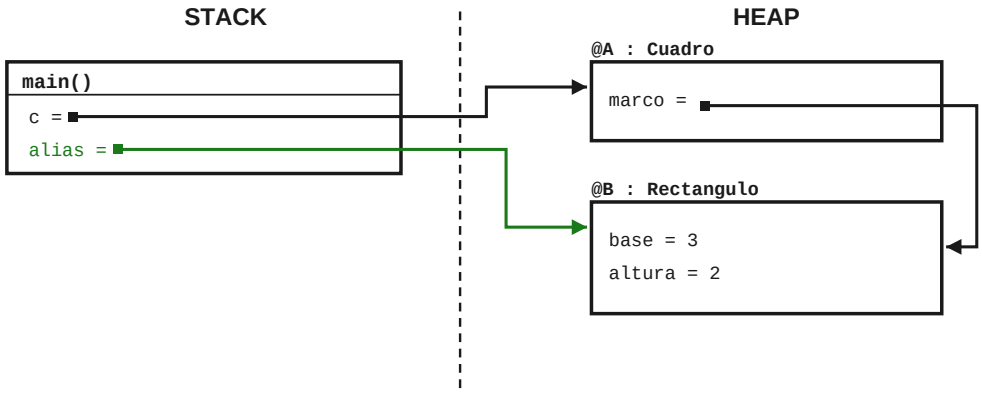
```
Cuadro c = new Cuadro(new Rectangulo(3, 2));
Rectangulo alias = c.marco;
alias.base = 9;
System.out.println(c.marco.base);   // 9
```

Paso 1 — los dos `new` anidados. Se crea el Rectangulo `@B`, se crea el Cuadro `@A`, y el campo `marco` — que vive EN `@A`, en el heap — guarda la flecha hacia `@B`.



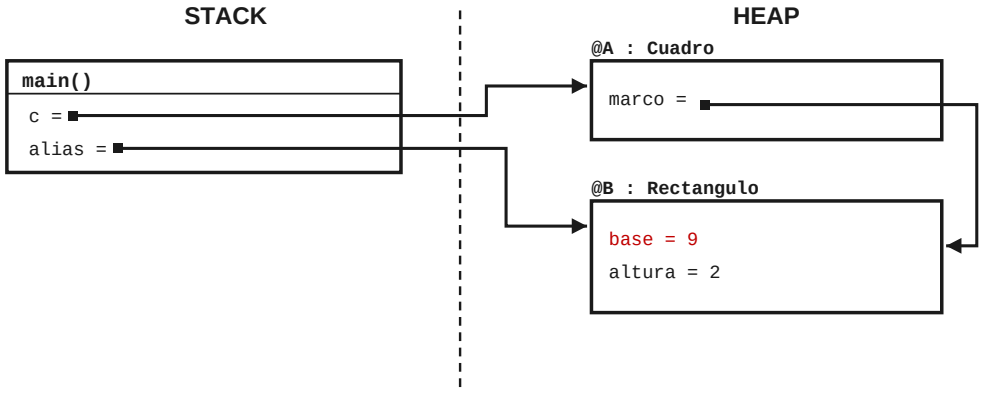
dos news: el campo marco vive EN el heap y apunta a otro bloque del heap

Paso 2 — `Rectangulo alias = c.marco;` Seguir la flecha de `c`, leer el campo `marco`, copiar ESA flecha a una variable del stack. Ahora `@B` tiene dos caminos de llegada: uno desde el heap y uno desde el stack.



alias = c.marco: el stack TAMBIEN puede apuntar a @B; dos caminos, un bloque

Paso 3 — `alias.base = 9;` Se escribe en `@B`... y `c.marco.base` también es 9, porque `c.marco` y `alias` son **la misma flecha al mismo bloque**.



alias.base = 9: c.marco.base tambien es 9 — es EL MISMO bloque

Debugger: breakpoint en el `println`; desplegá `c` en Variables: adentro está `marco = Rectangulo@...` — una flecha DENTRO del objeto. Compará ese `@` con el de `alias`. (Repo: `E08HeapApuntaAlHeap`.) En

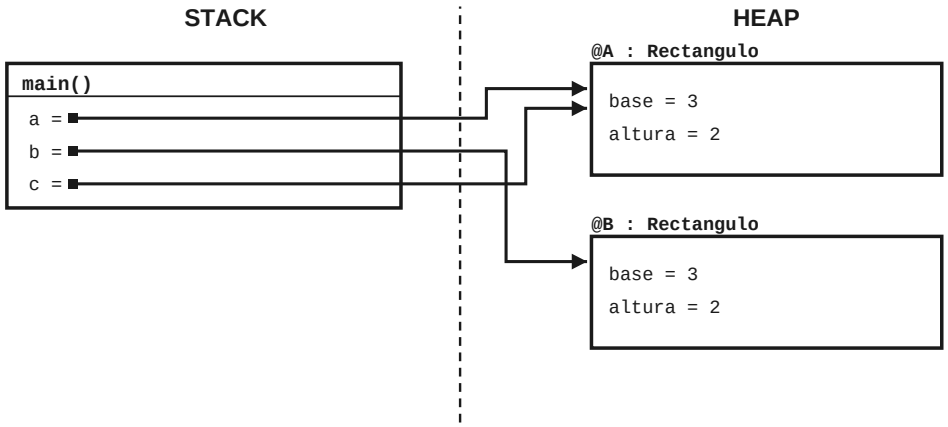
la Unidad 5 esta idea — bloques del heap apuntándose entre sí — se convierte en las estructuras enlazadas.

3B.12 Escenario 9 — == vs `equals` (y la trampa de String)

```
Rectangulo a = new Rectangulo(3, 2);
Rectangulo b = new Rectangulo(3, 2);
Rectangulo c = a;

a == b        // false: bloques distintos
a == c        // true:  la MISMA flecha
a.equals(b)   // true:  mismo CONTENIDO
```

La foto. `a` y `c` comparten flecha; `b` tiene su propio bloque con el mismo contenido. **`==` compara flechas** (¿es el mismo bloque?); **`equals` compara contenido** (si la clase lo define, como `Rectangulo`).

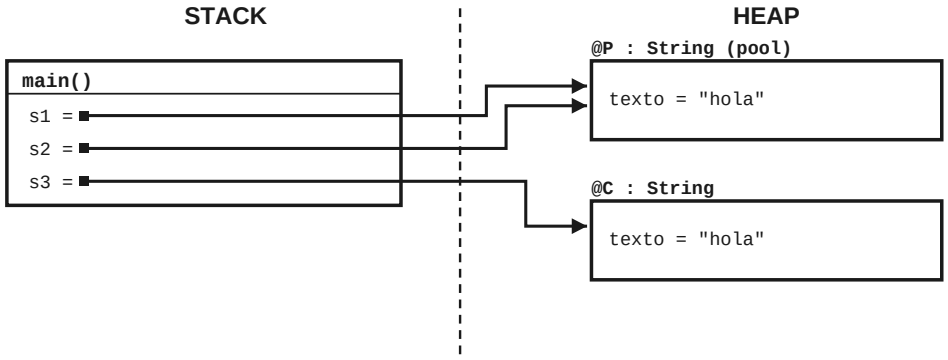


a == b: false (flechas distintas) · a == c: true · a.equals(b): true (contenido)

```
String s1 = "hola";
String s2 = "hola";
String s3 = new String("hola");

s1 == s2        // true  <- ¡sorpresa!
s1 == s3        // false
s1.equals(s3)   // true
```

El caso String. Java guarda los **literales** en un "pool": dos literales iguales comparten el MISMO bloque — por eso `s1 == s2` da `true` y parece que `==` "funciona". Es una ilusión: `new String(...)` obliga un bloque nuevo y la ilusión se rompe. **Los String SIEMPRE se comparan con `equals`.**



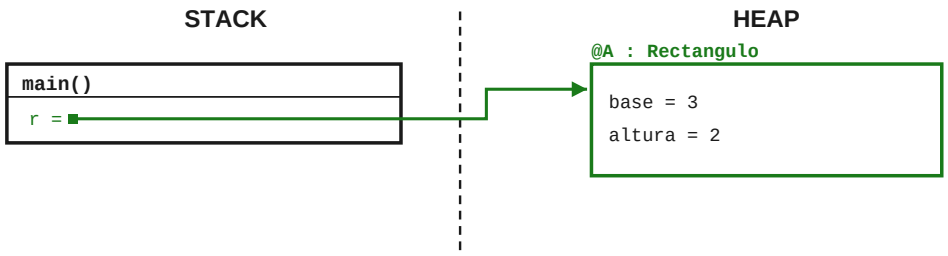
dos LITERALES iguales comparten el bloque del pool; new String obliga otro

Debugger: los `@` lo cuentan todo — `a` y `c` comparten, `b` no; `s1` y `s2` comparten (¡pool!), `s3` no. (Repo: `E09EqualsVsIgualIgual`.)

3B.13 Escenario 10 — Dos news al mismo puntero: UN bloque colgado

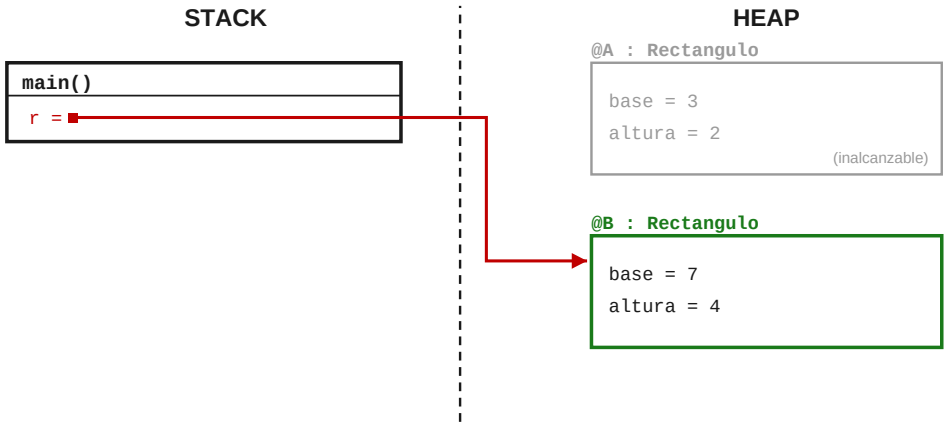
```
Rectangulo r = new Rectangulo(3, 2);
r = new Rectangulo(7, 4);
System.out.println(r);   // Rectangulo(7x4)
```

Paso 1 — el primer `new`. `r` → `@A`.



primer new: r -> @A

Paso 2 — el segundo `new`, sin backup. `r` pasa a apuntar al bloque nuevo `@B` y queda **perfectamente válido**. El que se pierde es el bloque VIEJO: `@A` se quedó sin flechas — huérfano, futuro trabajo del recolector. Como no guardamos backup (`Rectangulo aux = r;` ANTES del segundo `new`), no hay forma de volver a él.



segundo new SIN backup: r apunta a @B (100% valido); @A quedo huerfano — UNO solo

Error común

No confundir con el escenario 7. Allá: VARIOS punteros, cada uno con su bloque, reasignados → quedan VARIOS huérfanos distintos. Acá: UN puntero reasignado dos veces → queda UN huérfano, y el puntero sigue siendo totalmente usable. La pregunta que decide todo es siempre la misma: después de la asignación, **¿cuántos bloques quedaron sin ninguna flecha que los alcance?**

Debugger: breakpoint en cada `new`; anotá el `@` de `r` antes y después del segundo — cambia. El `@` viejo no aparece más en ninguna variable. (Repo: `E10DosNewsAlMismoPuntero`.)

3B.14 Las siete morales del apartado

1. **Primitivo**: el valor vive en el frame; asignar = fotocopiar (E0).
2. **`new`**: bloque en el heap + flecha en la variable (E1). El punto `.` es "seguí la flecha".
3. **Llamar a un método** apila un frame; **pasar un parámetro es asignar**: se copia el valor o la flecha, nunca el objeto (E2). `this` es la flecha oculta del objeto de la llamada (E3).
4. Un método **no puede reasignar** tus variables (copias, E4) pero **sí puede mutar** el contenido de los bloques compartidos (E5). Las dos caras de "pasar por valor".

5. **Swap de atributos** cambia los bloques; **swap de punteros** cambia las flechas (E6). Son operaciones distintas con efectos distintos.
6. Un bloque sin flechas es **huérfano**: uno solo si reasignaste un puntero (E10), varios si reasignaste varios (E7). El GC los libera; vos no podés volver.
7. **`==` compara flechas, `equals` compara contenido** (E9) — y el pool de String es la trampa clásica. Las flechas también viven en el heap: un atributo puede apuntar a otro bloque (E8).

3B.15 Repaso (en modo cuestionario del TP1)

1. Dibujá stack y heap tras: `int x = 3; int y = x; y++;`. ¿Qué muestra el panel Variables en cada línea?
2. ¿Qué DOS cosas hace un `new`? ¿Dónde queda cada una?
3. En el escenario 2, con el programa frenado dentro de `area`: ¿cuántos frames muestra el panel Frames y qué variable de cada uno apunta a `@A`?
4. ¿Qué es `this`? ¿Quién lo carga y con qué valor? ¿Dónde lo ves en el debugger?
5. Explicá con los diagramas del escenario 4 por qué `intercambiar(r1, r2)` no intercambia nada. ¿Qué UNA línea de main sí intercambia los punteros?
6. ¿Por qué `intercambiarBases` (escenario 5) sí deja efecto y `intercambiar` (escenario 4) no, si los dos reciben las mismas copias?
7. Con el programa frenado, ¿cómo verificás en el panel Variables si dos referencias apuntan al mismo objeto?
8. Tras `Rectangulo p = new Rectangulo(1,1); Rectangulo q = new Rectangulo(2,2); p = q;`: ¿cuántos bloques huérfanos hay? ¿Y tras `r = new Rectangulo(3,3); r = new Rectangulo(4,4);`? ¿Por qué el escenario 7 y el 10 no son el mismo caso?
9. ¿Qué imprime `s1 == s2` si ambos son el literal `"hola"`? ¿Y si uno es `new String("hola")`? ¿Cuál es la regla para comparar String?
10. ¿Qué hace cada uno: Step Over, Step Into, Step Out? Para cada botón, ¿qué le pasa al panel Frames?
11. Un atributo de un objeto, ¿puede ser una flecha? ¿Dónde vive esa flecha y cómo la ves en el debugger? (escenario 8)
12. Definí “inalcanzable” sin decir la palabra “gris”: ¿qué condición exacta cumple un bloque huérfano y quién lo libera?

Unidad 4

Complejidad computacional

Objetivos de la unidad

Esta unidad te da una herramienta que vas a usar en **toda** la materia: medir cuánto cuesta un algoritmo. Al terminarla vas a poder: (1) contar el costo de un algoritmo en función del tamaño de la entrada; (2) usar las **cotas** O, Ω y Θ; (3) calcular la complejidad de cualquier algoritmo **iterativo**; (4) plantear la **ecuación de recurrencia** de un algoritmo **recursivo** y resolverla con el **método de expansión**; (5) aplicar el **Teorema Maestro** completo (por sustracción y por división), entendiendo cada caso en su árbol de recursión; (6) analizar la **complejidad amortizada** del vector dinámico y el efecto del criterio de redimensión; y (7) conocer la complejidad de algoritmos estándar como Quicksort.

Hasta ahora nos preguntamos si un programa **anda**. Desde acá nos preguntamos algo más profundo: **¿cuánto cuesta?** Dos algoritmos pueden resolver el mismo problema y ser abismalmente distintos: uno tarda un segundo y el otro, siglos. Saber estimar ese costo —sin siquiera ejecutar el código— es lo que distingue a quien programa de quien **diseña** algoritmos.

4.1 Un poco de historia

La idea de analizar algoritmos es tan vieja como los algoritmos mismos: ya en 1843 **Ada Lovelace** escribió el primero pensado para una máquina. La notación que usamos hoy, la **"O grande"**, la introdujo el matemático **Paul Bachmann** en **1894** y la popularizó **Edmund Landau** (por eso se la llama *notación de Landau*). Recién en **1965**, **Juris Hartmanis** y **Richard Stearns** fundaron formalmente la **teoría de la complejidad computacional** (les valió el premio Turing). Y fue **Donald Knuth**, alrededor de **1976**, quien estandarizó el uso de O, Ω y Θ en la computación tal como los usamos hoy.

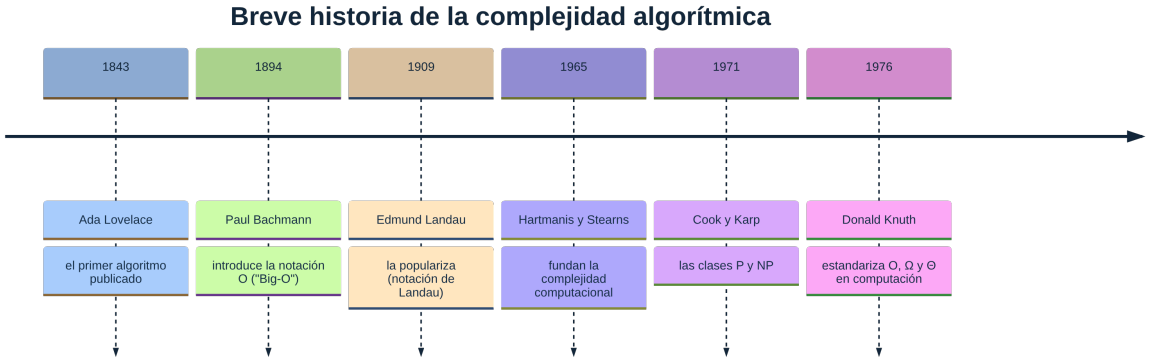


Figura 4.1: De Ada Lovelace a Knuth: cómo se fue construyendo la forma de medir algoritmos.

4.2 ¿Por qué no medir en segundos?

Lo intuitivo sería cronometrar. Pero el tiempo en segundos depende de la máquina, del lenguaje, de qué otra cosa esté corriendo… El mismo algoritmo tarda distinto en tu notebook que en un servidor. Necesitamos una medida **independiente de la máquina**.

La solución: contar cuántas **operaciones elementales** (una suma, una comparación, un acceso a memoria) hace el algoritmo, **en función del tamaño de la entrada**, que llamamos n. No nos interesa el número exacto, sino **cómo crece** ese número cuando n crece. Esa es la **complejidad temporal**.

4.3 Órdenes de crecimiento

Para n grande, lo único que importa es el **término que más crece**; las constantes y los términos menores se vuelven irrelevantes. Por eso decimos que recorrer un arreglo de n elementos "es $O(n)$", sin importar si hacemos una o tres operaciones por elemento. Estos son los órdenes más frecuentes, de mejor a peor:

Orden	Nombre	Ejemplo típico
$O(1)$	constante	acceder a `a[i]`
$O(\log n)$	logarítmico	búsqueda binaria
$O(n)$	lineal	recorrer un arreglo
$O(n \log n)$	lineal-logarítmico	Mergesort, Quicksort (promedio)
$O(n^2)$	cuadrático	dos bucles anidados
$O(2^n)$	exponencial	Fibonacci recursivo ingenuo

La diferencia no es un detalle: mirá cómo se disparan las curvas al crecer n.

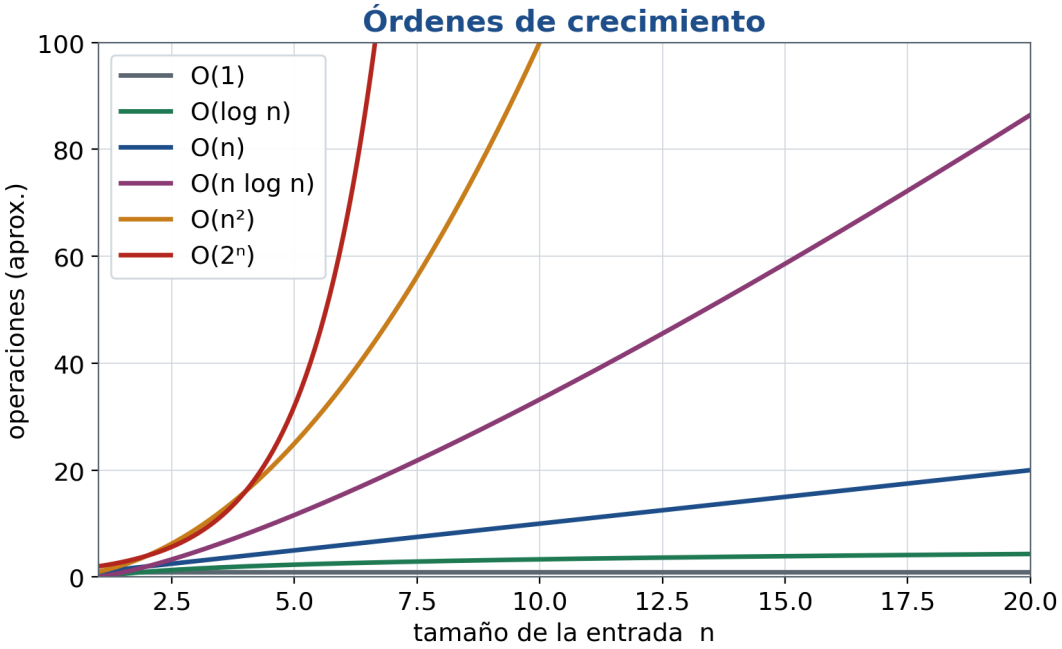


Figura 4.2: Cómo crecen los distintos órdenes. Un algoritmo $O(2^n)$ o $O(n^2)$ se vuelve impracticable enseguida.

Idea clave

Para $n = 1\,000\,000$, un algoritmo $O(n)$ hace un millón de pasos; uno $O(n^2)$ hace un billón (un millón de millones). El primero corre en un instante; el segundo puede tardar horas. **Elegir el algoritmo correcto importa más que cualquier micro-optimización.**

4.4 Las tres cotas: O, Ω y Θ

Para hablar con precisión del crecimiento usamos tres cotas. Veamos cada una por separado, con su gráfico. En los tres gráficos las curvas son las mismas —la $f(n)$ azul es el costo real del algoritmo,

con sus irregularidades— y en los tres aparece el mismo protagonista silencioso: el punto n_0.

La O grande es la **cota superior**: dice que la función no crece más rápido que $g(n)$ (salvo una constante), a partir de cierto punto. Formalmente:

$$f(n) = O(g(n)) \iff \exists\, c > 0,\ n_0 > 0 :\ 0 \leq f(n) \leq c\, g(n)\ \ \forall\, n \geq n_0$$

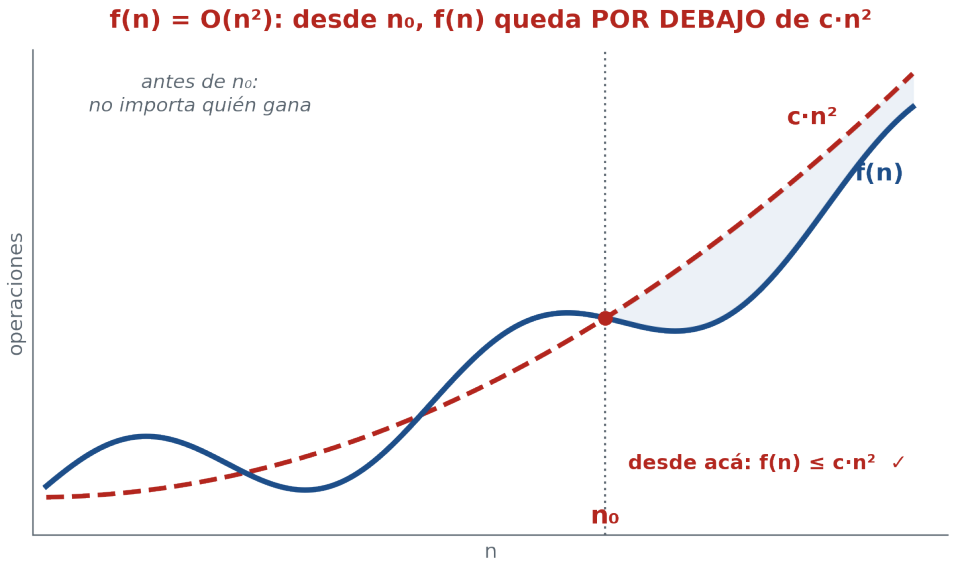


Figura 4.3: Cota superior: existe un c tal que, **desde n_0 en adelante**, $f(n)$ nunca vuelve a superar a $c \cdot n^2$.

Mirá el papel del n_0 en el gráfico: antes de él, $f(n)$ cruza la cota varias veces —a veces gana una, a veces la otra— y **no importa**. La definición sólo exige que **a partir de** n_0 la desigualdad valga **siempre**. Por eso la notación captura el comportamiento *asintótico* (para n grande) e ignora el arranque, donde mandan las constantes.

La Ω (omega) es la **cota inferior**: la función crece **al menos** tan rápido como $g(n)$, también a partir de un n_0:

$$f(n) = \Omega(g(n)) \iff \exists\, c > 0,\ n_0 > 0 :\ 0 \leq c\, g(n) \leq f(n)\ \ \forall\, n \geq n_0$$

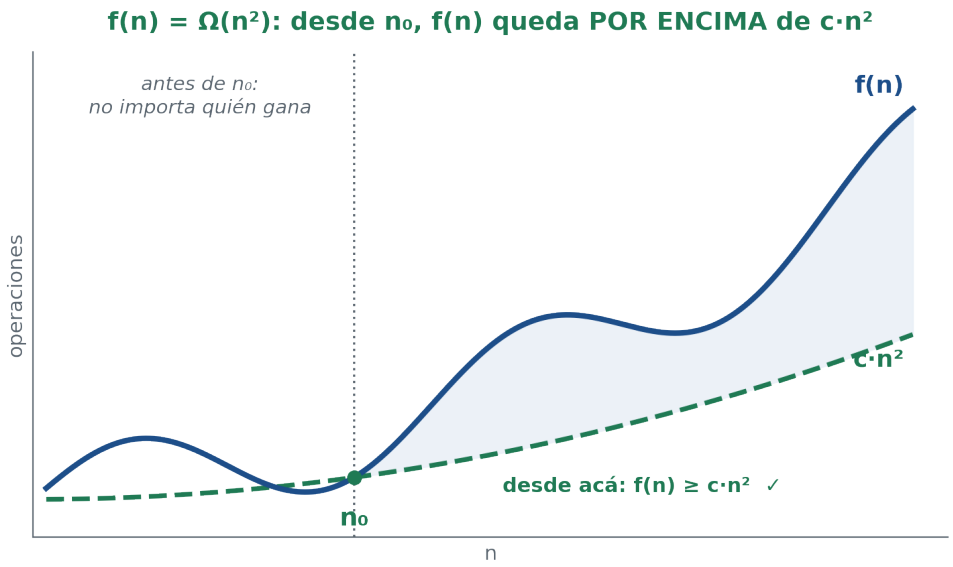


Figura 4.4: Cota inferior: desde su propio n_0 (fijate que **no es el mismo** que el de la figura anterior), $f(n)$ nunca vuelve a caer por debajo de $c \cdot n^2$.

La Θ (theta) es la **cota ajustada**: crece **exactamente** como $g(n)$, es decir, es $O(g(n))$ **y** $\Omega(g(n))$ a la vez. Ahora hacen falta **dos** constantes, y el n_0 es el mayor de los dos anteriores (desde ahí valen las dos desigualdades juntas):

$$f(n) = \Theta(g(n)) \iff \exists\, c_1, c_2 > 0,\ n_0 > 0 :\ 0 \le c_2\, g(n) \le f(n) \le c_1\, g(n)\ \ \forall\, n \ge n_0$$

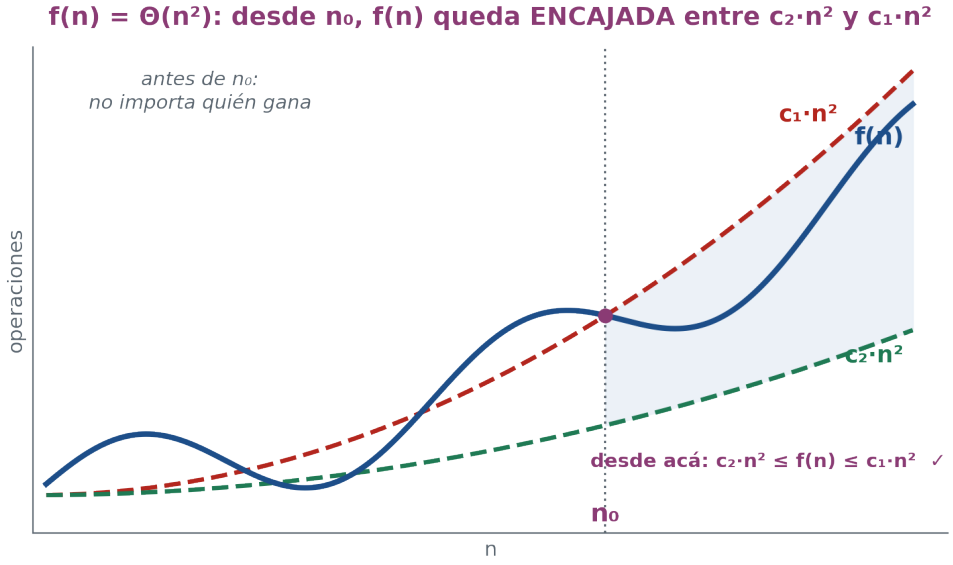


Figura 4.5: Cota ajustada: desde n_0, $f(n)$ queda "encajada" para siempre entre $c_2 \cdot n^2$ y $c_1 \cdot n^2$.

En la práctica, O suele referirse al **peor caso** (la garantía: "no tardará más que esto"), Ω al **mejor caso**, y Θ cuando ambos coinciden. Por ejemplo, la búsqueda lineal es $O(n)$ en el peor caso (el elemento no está o está último) y $\Omega(1)$ en el mejor (está primero).

4.5 Calcular la complejidad de un algoritmo iterativo

Para los algoritmos con bucles, alcanza con seguir unas pocas reglas:

1. **Operación simple** (asignación, comparación, acceso): $O(1)$.
2. **Secuencia** de instrucciones: se **suma**, y domina la mayor.
3. **Bucle**: (cantidad de iteraciones) $\times$ (costo del cuerpo).
4. **Bucles anidados**: se **multiplican** las iteraciones.
5. **Bucle que divide** el rango a la mitad (`i *= 2` o `i /= 2`): hace $\log_2 n$ vueltas, o sea $O(\log n)$.
6. Al final, **descartá constantes y términos menores**: $3n^2 + 5n + 8$ es $O(n^2)$.

Tres ejemplos rápidos:

```
// (a) O(n): un bucle de n vueltas
for (int i = 0; i < n; i++) {
    suma += a[i];
}

// (b) O(n^2): dos bucles anidados, cada uno n vueltas
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {
        contar++;
    }

// (c) O(log n): el rango se divide por 2 en cada vuelta
for (int i = 1; i < n; i *= 2) {
    paso++;
}
```

4.5.1 El conteo línea por línea

Antes de usar las reglas “de memoria”, conviene hacer **una vez** el conteo completo, línea por línea, para ver de dónde salen. Tomemos la suma de un arreglo (el código está en el repo, `SumaArreglo`):

```
static long sumar(int[] a) {
    long suma = 0;                      // L1
    for (int i = 0; i < a.length; i++) {  // L2
        suma += a[i];                   // L3
    }
    return suma;                        // L4
}
```

Contemos cuántas veces se ejecuta cada línea con un arreglo de n elementos:

Línea	¿Qué hace?	¿Cuántas veces?	Costo total
L1	asignación	1	$O(1)$
L2	inicialización, $n{+}1$ comparaciones, n incrementos	$2n + 2$	$O(n)$
L3	acceso + suma + asignación	n	$O(n)$
L4	return	1	$O(1)$
Total		$\approx 3n + 4$	$\mathbf{O(n)}$

Fijate el detalle de L2: la condición `i < a.length` se evalúa $n + 1$ veces (las n que da verdadera más la última que da falsa y corta). ¿Importa? **No**: $3n + 4$, $2n + 2$ o $5n$ son todos $O(n)$. Éste es justamente el punto de la notación: el conteo fino sirve para entender, pero al final sólo queda **el término dominante**. Con esa tranquilidad, de acá en adelante aplicamos las reglas directamente.

4.5.2 Un ejemplo con truco: la exponenciación rápida

¿Siempre que hay un solo bucle es $O(n)$? No: hay que mirar **cómo cambia** la variable de control. En el repo, `PotenciaRapida` calcula $base^{exp}$ así:

```java
public static long potencia(long base, int exp) {
    long resultado = 1;
    while (exp > 0) {                // el exponente se divide por 2 -> O(log n)
        if ((exp & 1) == 1) {
            resultado *= base;   // si exp es impar
        }
        base *= base;
        exp >>= 1;                   // exp = exp / 2
    }
    return resultado;
}
```

Antes de analizarlo, dos operadores nuevos que trabajan sobre los **bits** del número (su escritura en binario):

- **`exp & 1`** es el **AND bit a bit** con 1: compara el **último bit** de `exp` con el 1 y deja el resto en cero. El último bit de un número binario vale 1 si el número es **impar** y 0 si es **par** (igual que en decimal el último dígito te dice si es múltiplo de 10). O sea: `(exp & 1) == 1` es simplemente una forma rapidísima de preguntar **"¿`exp` es impar?"**, equivalente a `exp % 2 == 1`.
- **`exp >>= 1`** es el **desplazamiento a la derecha** (*shift*): corre todos los bits un lugar hacia la derecha y el último **se cae**. Tirar el último dígito en binario es **dividir por 2** descartando el resto (como tachar el último dígito decimal divide por 10): equivale a `exp = exp / 2` con división entera.

Miralo sobre el 13, que en binario es `1101`:

`exp` (decimal)	`exp` (binario)	`exp & 1` (¿impar?)	`exp >> 1`
13	`1101`	`1` → sí	`110` = 6
6	`110`	`0` → no	`11` = 3
3	`11`	`1` → sí	`1` = 1
1	`1`	`1` → sí	`0` = 0

(Los usamos acá porque es el código "de manual" de este algoritmo y te los vas a cruzar en código real; el análisis sería idéntico con `% 2` y `/ 2`.)

El `while` no recorre de a uno: en cada vuelta el exponente se **divide por 2** (`exp >>= 1`). Tracemos 3^{13} vuelta a vuelta:

Vuelta	`exp`	¿impar?	`resultado`	`base`
inicio	13	—	1	3
1	13	sí	$1 \cdot 3 = 3$	9
2	6	no	3	81
3	3	sí	$3 \cdot 81 = 243$	6561
4	1	sí	$243 \cdot 6561 = 1\,594\,323$	—

Cuatro vueltas para $exp = 13$ (y no trece), porque $13 \to 6 \to 3 \to 1 \to 0$: tantas vueltas como veces entra el 2 en 13, o sea $\lfloor \log_2 13 \rfloor + 1 = 4$. Verificá: $3^{13} = 1\,594\,323$. ✓ La regla 5 en acción: **bucle que divide, costo logarítmico**.

4.6 Calcular la de un algoritmo recursivo: la ecuación de recurrencia

Cuando un algoritmo se llama a sí mismo, su costo se describe con una **ecuación de recurrencia**: el costo $T(n)$ se expresa en términos del costo sobre entradas más chicas, más el trabajo hecho fuera de la recursión.

$$T(n) = (\text{llamadas recursivas}) + (\text{trabajo no recursivo})$$

4.6.1 La ecuación se lee del algoritmo

La ecuación **no es magia**: es la **foto de la recursividad**, y se lee directamente del código, renglón por renglón. Hagámoslo con el factorial ($n! = n \cdot (n-1)!$):

```
static long factorial(int n) {
    if (n <= 1) {                    // comparar: O(1)
        return 1;                    // caso base: costo fijo  ->  T(1) = O(1)
    }
    return n * factorial(n - 1);     // UNA llamada sobre n-1  ->  T(n-1)
                                     // más 1 multiplicación   ->  + O(1)
}
```

Ahora leamos la ecuación mirando el código:

- ¿Cuántas **llamadas recursivas** hace `factorial(n)`? **Una** sola: `factorial(n - 1)`. ¿Sobre qué tamaño? Sobre $n-1$. Eso aporta el término $T(n-1)$: "lo que cueste resolver el problema de tamaño $n-1$, sea lo que sea".
- ¿Qué **trabajo propio** hace, fuera de esa llamada? Una comparación y una multiplicación: costo constante, $O(1)$.
- ¿Y el **caso base**? Cuando $n \leq 1$ no hay recursión: $T(1) = O(1)$.

Juntando las dos observaciones:

$$T(n) = \underbrace{T(n-1)}_{\texttt{factorial(n-1)}} + \underbrace{O(1)}_{\texttt{if + n * ...}} \qquad T(1) = O(1)$$

Cada pedazo de la ecuación **señala un pedazo del código**. Ese es el método general: contá las llamadas recursivas (coeficiente), mirá sobre qué tamaño se hacen (el argumento de T), y sumá el costo de todo lo demás. Con la misma lectura:

- **Búsqueda binaria**: **una** llamada sobre **la mitad** que queda, más comparar con el del medio, $O(1)$: $T(n) = T(n/2) + O(1)$.
- **Mergesort**: **dos** llamadas, cada una sobre **una mitad**, más la fusión que recorre todo, $O(n)$: $T(n) = 2\,T(n/2) + O(n)$.

4.7 Resolver recurrencias: el método de expansión

El **método de expansión** (o sustitución iterativa) consiste en **desarrollar** la recurrencia reemplazándola en sí misma, varias veces, hasta ver el patrón y llegar al caso base. Tomemos $T(n) = 2\,T(n/2) + n$ (la de Mergesort) y hagamos la operatoria **reemplazo por reemplazo**, sin saltear pasos.

La clave del método: la ecuación es un **molde** que vale para *cualquier* tamaño, no sólo para n. Si en $T(n) = 2\,T(n/2) + n$ ponemos $n/2$ donde dice n (en **los tres** lugares), obtenemos qué vale $T(n/2)$:

$$T(n/2) = 2\,T\left(\tfrac{n/2}{2}\right) + \tfrac{n}{2} = 2\,T(n/4) + \tfrac{n}{2}$$

Primer reemplazo. Metemos esa expresión de $T(n/2)$ dentro de la original, y distribuimos el 2 que la multiplica:

$$T(n) = 2\underbrace{T(n/2)}_{\text{reemplazamos}} + n = 2\left(2\,T(n/4) + \tfrac{n}{2}\right) + n$$
$$= \underbrace{2\cdot 2}_{4}\,T(n/4) + \underbrace{2\cdot\tfrac{n}{2}}_{n} + n = 4\,T(n/4) + 2n$$

Fijate la cuenta fina: el 2 de afuera multiplica **a los dos términos** del paréntesis. Con el primero forma $4\,T(n/4)$; con el segundo, $2\cdot n/2 = n$, que sumado al n que ya estaba da $2n$.

Segundo reemplazo. Usamos el molde otra vez, ahora con $n/4$: $T(n/4) = 2\,T(n/8) + n/4$. Y lo metemos donde corresponde:

$$T(n) = 4\underbrace{T(n/4)}_{\text{reemplazamos}} + 2n = 4\left(2\,T(n/8) + \tfrac{n}{4}\right) + 2n$$
$$= \underbrace{4\cdot 2}_{8}\,T(n/8) + \underbrace{4\cdot\tfrac{n}{4}}_{n} + 2n = 8\,T(n/8) + 3n$$

Otra vez lo mismo: el coeficiente se **duplica** ($4 \to 8$) y aparece **un** n **más** en la suma ($2n \to 3n$).

El patrón. Después de k reemplazos, el coeficiente se duplicó k veces y juntamos k enes:

Reemplazos	Expresión
0	$2\,T(n/2) + n$
1	$4\,T(n/4) + 2n$
2	$8\,T(n/8) + 3n$
k	$2^k\,T(n/2^k) + k\,n$

El final. La expansión termina al llegar al caso base: cuando $n/2^k = 1$, o sea cuando $2^k = n$, o sea cuando $k = \log_2 n$. Sustituimos esos valores en la fila del patrón:

$$T(n) = \underbrace{2^k}_{n}\,\underbrace{T(1)}_{O(1)} + \underbrace{k}_{\log_2 n}\,n = n\,T(1) + n\log_2 n = \Theta(n\log n)$$

Se ve muy claro con el **árbol de recursión**: en cada nivel el trabajo total suma n, y hay $\log_2 n$ niveles, así que el total es $n\log_2 n$.

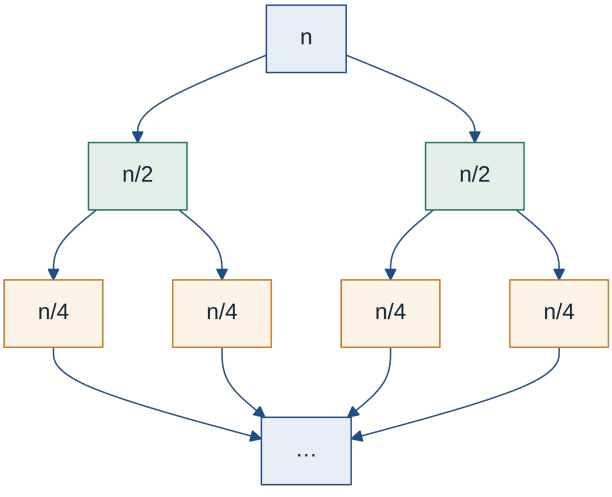


Figura 4.6: Árbol de recursión de $T(n) = 2T(n/2) + n$: cada nivel suma n; hay $\log_2 n$ niveles.

Con la misma técnica: $T(n) = T(n-1) + n$ da $n + (n-1) + \cdots + 1 = \Theta(n^2)$, y $T(n) = T(n/2) + 1$ da $\Theta(\log n)$.

4.8 El Teorema Maestro por sustracción

Expandir a mano funciona, pero es tedioso. Para las familias de recurrencias más comunes hay recetas listas: los **Teoremas Maestros**. Hay dos, según **cómo se achica** el problema en cada llamada: **restando** un poquito ($T(n-b)$, como el factorial) o **dividiendo** en partes ($T(n/b)$, como Mergesort). Empecemos por el de **sustracción**, que es el más fácil de los dos, y cubre las recursiones "más parecidas a un bucle":

$$T(n) = a\,T(n-b) + O(n^k), \qquad a > 0,\ b > 0$$

Su resultado depende de a, la **cantidad de llamadas recursivas**:

- Si $a < 1$: $T(n) = O(n^k)$.
- Si $a = 1$: $T(n) = O(n^{k+1})$ — el trabajo se repite $\sim n/b$ veces: **se suma un grado**.
- Si $a > 1$: $T(n) = O\big(n^k\, a^{n/b}\big)$ — **exponencial**.

Ejemplos: $T(n) = T(n-1) + O(1)$ (el factorial que acabamos de plantear): $a = 1$, $k = 0$, da $O(n^{0+1}) = O(n)$ — razonable: n llamadas de costo constante. $T(n) = T(n-1) + O(n)$ da $O(n^2)$. Y $T(n) = 2\,T(n-1) + O(1)$ da $O(2^n)$.

¿Por qué $a > 1$ explota? Porque restar achica el problema **muy de a poco**: el árbol de llamadas tiene n niveles (no $\log n$), y si además cada llamada abre $a = 2$ hijas, la cantidad de nodos se duplica en cada nivel:

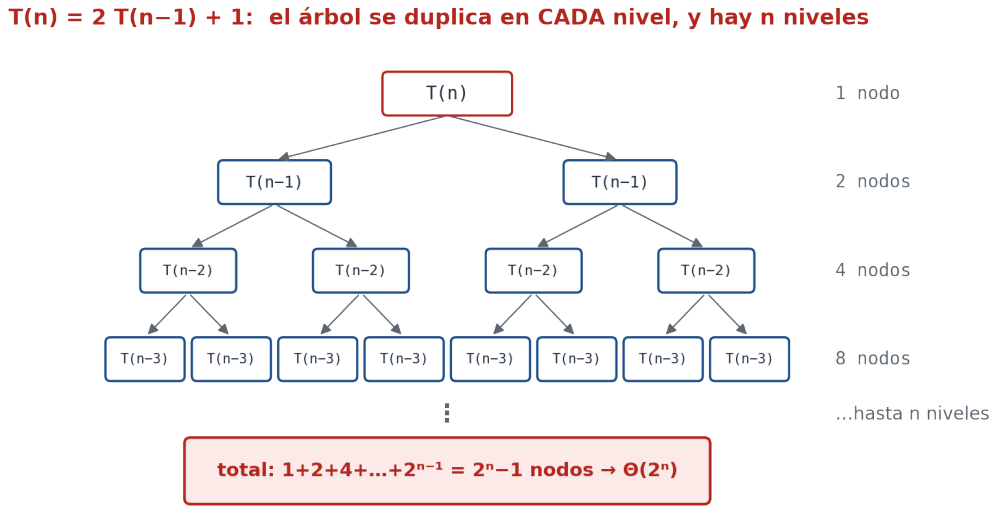


Figura 4.7: El árbol de $T(n) = 2T(n-1)+1$: con n niveles y duplicación por nivel, el total es $2^n - 1$ llamadas.

Éste es exactamente el drama del **Fibonacci recursivo ingenuo** de la Unidad 6. Su recurrencia real es $T(n) = T(n-1) + T(n-2) + O(1)$ —una rama baja un escalón y la otra, dos—, que no calza literalmente en el molde $a\,T(n-b)$; pero como $T(n-2) \le T(n-1)$, vale la cota $T(n) \le 2\,T(n-1) + O(1) = O(2^n)$. (El valor exacto crece como $\varphi^n \approx 1.618^n$: un poco menos que 2^n, pero exponencial al fin — igual de impracticable.)

4.9 El Teorema Maestro por división

El otro molde es el de las recurrencias **"divide y vencerás"**, donde cada llamada parte el problema en pedazos de tamaño n/b. Acá el árbol tiene $\log_b n$ niveles (mucho más bajo que el de sustracción) y el análisis es más fino: el resultado depende de una **comparación de fuerzas**:

$$T(n) = a\,T(n/b) + f(n), \qquad a \geq 1,\ b > 1$$

donde a es la cantidad de subproblemas y b cuánto se achica cada uno. La clave es comparar $f(n)$ (el trabajo de dividir y combinar) con $n^{\log_b a}$:

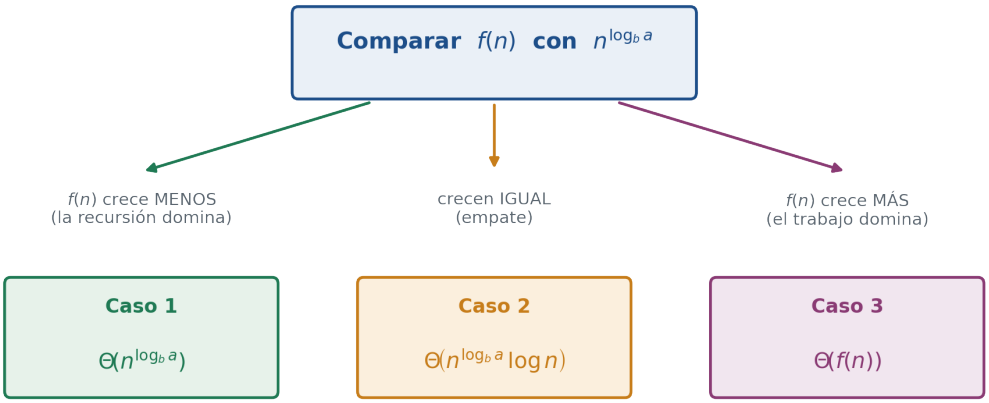


Figura 4.8: Los tres casos del Teorema Maestro según quién "gana" el crecimiento.

- **Caso 1** — si $f(n) = O(n^{\log_b a - \varepsilon})$ para algún $\varepsilon > 0$ (la recursión domina): $T(n) = \Theta(n^{\log_b a})$.
- **Caso 2** — si $f(n) = \Theta(n^{\log_b a})$ (empatan): $T(n) = \Theta(n^{\log_b a} \log n)$.
- **Caso 3** — si $f(n) = \Omega(n^{\log_b a + \varepsilon})$ (el trabajo domina) y se cumple la condición de regularidad: $T(n) = \Theta(f(n))$.

Un ejemplo de cada caso:

- **Caso 1:** $T(n) = 8\,T(n/2) + n^2$. Acá $\log_2 8 = 3$, y $f(n) = n^2$ crece menos que n^3. Resultado: $\Theta(n^3)$.
- **Caso 2:** $T(n) = 2\,T(n/2) + n$ (Mergesort). $\log_2 2 = 1$, y $f(n) = n$ empata con n^1. Resultado: $\Theta(n \log n)$.
- **Caso 3:** $T(n) = 2\,T(n/2) + n^2$. $\log_2 2 = 1$, y $f(n) = n^2$ crece más que n^1. Resultado: $\Theta(n^2)$.

4.9.1 Ver los tres casos en el árbol de recursión

El teorema puede parecer una receta mágica, pero detrás hay una intuición muy concreta: en el árbol de recursión, **¿el trabajo por nivel crece, se mantiene o se achica al bajar?** Dibujemos los tres ejemplos de arriba con sus números:

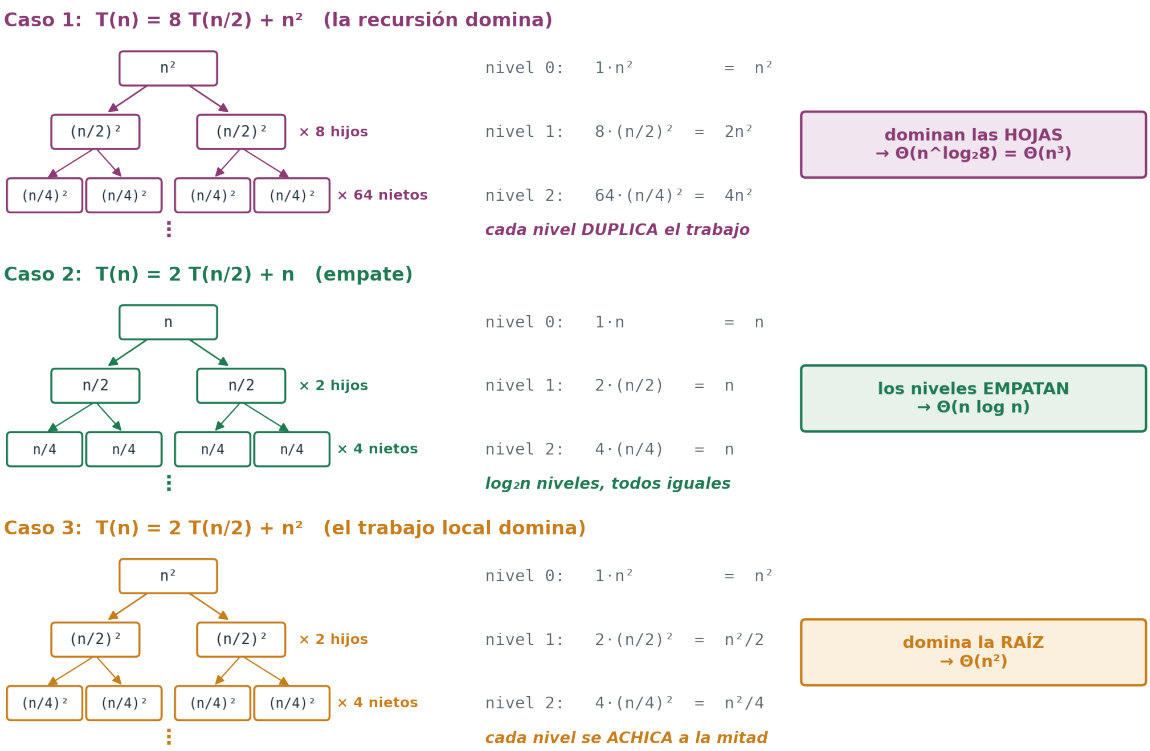


Figura 4.9: Los tres casos del Teorema Maestro vistos en el árbol de recursión. Caso 1: el trabajo por nivel se duplica al bajar, así que mandan los niveles de abajo (las hojas). Caso 2: todos los niveles trabajan lo mismo, y hay log n. Caso 3: el trabajo por nivel se achica, así que manda el nivel de arriba (la raíz).

Seguí las cuentas del **Caso 1**: la raíz trabaja n^2; sus 8 hijos trabajan $8 \cdot (n/2)^2 = 2n^2$ (¡el doble!); los 64 nietos, $4n^2$... Cada nivel duplica al anterior, así que la suma la dominan los **últimos** niveles: las hojas, que son $8^{\log_2 n} = n^3$. En el **Caso 2**, cada nivel suma exactamente n y hay $\log_2 n$ niveles: total $n \log n$ (es el árbol de Mergesort que ya expandimos a mano). En el **Caso 3**, cada nivel trabaja la **mitad** que el anterior, así que la suma $n^2(1 + \frac{1}{2} + \frac{1}{4} + \dots) \leq 2n^2$ la domina la **raíz**. El teorema no hace más que empaquetar estas tres situaciones.

4.10 Complejidad de algoritmos estándar

Vale la pena tener a mano las complejidades de los algoritmos que más vas a usar:

Algoritmo	Mejor	Promedio	Peor
Búsqueda lineal	$O(1)$	$O(n)$	$O(n)$
Búsqueda binaria	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble / Insertion / Selection sort	$O(n)$–$O(n^2)$	$O(n^2)$	$O(n^2)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

El caso de **Quicksort** es instructivo: en promedio el pivote parte el arreglo en dos mitades, dando $T(n) = 2\,T(n/2) + O(n) = O(n \log n)$; pero si el pivote es siempre el menor o el mayor, parte en 1 y $n-1$, dando $T(n) = T(n-1) + O(n) = O(n^2)$. Aun así, en la práctica suele ser el más rápido por sus constantes pequeñas.

4.11 Aplicar la complejidad a lo que ya hicimos

La complejidad no es un tema aparte: **atraviesa toda la materia**. Cada operación de un TDA tiene su costo, y elegir la implementación correcta es elegir la complejidad correcta. Un adelanto de lo que viene:

Operación	Arreglo	Lista enlazada
Acceder al elemento i	$O(1)$	$O(n)$
Insertar al final	$O(1)$*	$O(1)$
Insertar al principio	$O(n)$	$O(1)$
Buscar un valor	$O(n)$	$O(n)$

(*amortizado, si hay que redimensionar: es la sección que sigue.) A partir de acá, de cada estructura y algoritmo nos vamos a preguntar **siempre**: ¿cuánto cuesta cada operación?

4.12 Complejidad amortizada: el caso del vector dinámico

Hay operaciones cuyo costo es **casi siempre barato, pero de vez en cuando carísimo**. El ejemplo estrella —que vas a usar toda la materia— es **agregar al final de un arreglo que se redimensiona** (la técnica que viste en la Unidad 2; en la Unidad 5 la vas a ver empaquetada como el TDA `Vector` propio, y es exactamente lo que hace el `ArrayList` de Java por dentro). Mientras el arreglo interno tiene lugar, agregar cuesta $O(1)$: escribir en la próxima celda libre. Pero cuando el arreglo se **llena**, hay que **redimensionar**: crear un arreglo más grande y **copiar todos los elementos**, lo que cuesta $O(n)$.

¿Decimos entonces que agregar "es $O(n)$"? Sería técnicamente cierto como peor caso puntual, pero **injusto**: castiga a todas las operaciones por lo que hacen unas pocas. La medida correcta es la **complejidad amortizada**: el costo **total** de una secuencia de n operaciones, **repartido** entre las n operaciones.

$$\text{costo amortizado} = \frac{\text{costo total de } n \text{ operaciones}}{n}$$

4.12.1 El criterio de redimensión importa: duplicar

Supongamos que cuando el arreglo se llena lo **duplicamos** (capacidad 1, 2, 4, 8, 16…). Mirá el costo de cada `agregar` en una secuencia de 20 inserciones:

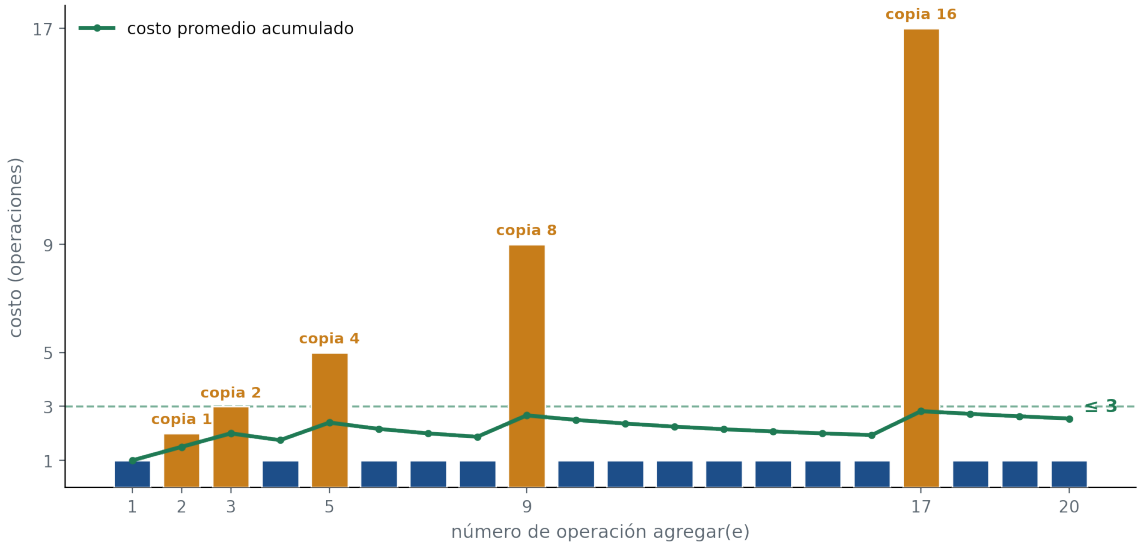


Figura 4.10: El costo de cada operación `agregar` con redimensión por duplicación. Casi todas cuestan 1 (azul); las que caen justo en una redimensión (naranja) pagan además la copia de todo lo existente. La línea verde —el promedio acumulado— nunca supera 3.

Los picos naranjas son las redimensiones: la operación 17, por ejemplo, paga la copia de los 16 elementos existentes. Pero fijate la línea verde: el **promedio** se mantiene siempre por debajo de 3. No es casualidad. Sumemos el costo total de n inserciones: cada una paga su escritura (n en total), y las copias suman

$$1 + 2 + 4 + 8 + \cdots + \tfrac{n}{2} + n \;<\; 2n$$

(una suma geométrica: cada término es el doble del anterior, y la suma entera es menos que el doble del último). Total: menos de $3n$ operaciones para n inserciones, o sea **menos de 3 por operación**. Por eso decimos que agregar al final es $O(1)$ **amortizado**: picos aislados de $O(n)$, pero promedio constante garantizado.

4.12.2 ¿Y si en vez de duplicar agrandamos de a 1?

El criterio de redimensión no es un detalle. Si al llenarse el arreglo lo agrandáramos **de a una celda**, cada inserción redimensionaría, y la inserción i copiaría $i - 1$ elementos. El total sería

$$1 + 2 + 3 + \cdots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2),$$

es decir $\Theta(n)$ amortizado **por operación**: mil veces más lento para $n = 1000$. Compará los dos criterios:

Criterio de redimensión	Costo total de n inserciones	Amortizado
Duplicar la capacidad	$< 3n$	$O(1)$
Multiplicar por 1.5	$O(n)$	$O(1)$
Sumar una constante ($+1$, $+10$)	$\Theta(n^2)$	$\Theta(n)$

Cualquier factor **multiplicativo** ($\times 2$, $\times 1.5$...) da amortizado constante, porque genera una suma geométrica; cualquier incremento **aditivo** da amortizado lineal, porque genera una suma aritmética. (Dato para la Unidad 5: el `ArrayList` de Java usa $\times 1.5$; el `Vector` propio que vas a construir ahí usa $\times 2$.)

Idea clave

Amortizado no es "promedio con suerte". Es una garantía sobre el **total**: cualquier secuencia de n inserciones cuesta $O(n)$, terminen donde terminen los picos. Distinto del "caso promedio" de Quicksort, que depende de la distribución de los datos: acá no hay azar, sólo contabilidad bien hecha.

4.13 Medición empírica vs. teórica

Además de razonar la complejidad, podés **medirla** con `System.nanoTime`. Es útil para confirmar la teoría, pero es engañosa: la afectan el compilador JIT (que acelera el código "en caliente"), el sistema operativo y la máquina. Por eso, para **comparar** algoritmos, la teoría es más confiable; la medición sirve para detectar sorpresas. En el repo, `Benchmark` mide la suma de un arreglo para n creciente.

4.14 Ejemplo integrador

Analicemos un algoritmo completo. Este cuenta cuántos pares de elementos iguales hay en un arreglo:

```
static int paresIguales(int[] a) {
    int pares = 0;
    for (int i = 0; i < a.length; i++)          // n vueltas
        for (int j = i + 1; j < a.length; j++)  // ~n/2 vueltas en promedio
            if (a[i] == a[j]) {
                pares++;   // O(1)
            }
    return pares;
}
```

Dos bucles anidados sobre n dan $\Theta(n^2)$. ¿Se puede mejor? Sí: usando una tabla de hash (Unidad de Hashing) se cuenta en $\Theta(n)$. Ese salto de n^2 a n —que parece magia— es exactamente lo que esta unidad te enseña a ver y a buscar.

4.15 Errores comunes

Error común

- Confundir "mejor/peor caso" con "O/Ω": son cosas distintas; podés dar O del mejor caso.
- Dejar constantes o términos menores en la respuesta final ($O(2n+3)$ es $O(n)$).
- Sumar cuando hay que multiplicar: dos bucles **anidados** son n^2, no $2n$.
- Plantear mal la recurrencia (olvidar el trabajo no recursivo, o la cantidad de llamadas).
- Aplicar el Teorema Maestro por división a una recurrencia que **resta** ($T(n-1)$).
- Creer que la medición en segundos "refuta" la teoría: mide otra cosa (incluye JIT, SO, máquina).

4.16 Repaso

1. ¿Por qué medimos operaciones en función de n y no en segundos?
2. Definí O, Ω y Θ. ¿Cuál se asocia al peor caso?
3. ¿Cómo se calcula la complejidad de dos bucles anidados? ¿Y de uno que hace `i *= 2`?
4. Planteá la ecuación de recurrencia de la búsqueda binaria y de Mergesort.
5. Resolvé $T(n) = 2T(n/2) + n$ con el método de expansión.
6. Enunciá los tres casos del Teorema Maestro por división y dá un ejemplo de cada uno. En el árbol de recursión, ¿qué distingue a cada caso? ¿Y qué dice el de sustracción cuando $a = 1$ y cuando $a > 1$?
7. Hacé el conteo línea por línea de un bucle que suma los elementos de una matriz de $n \times n$.

8. Tracé `potencia(2, 10)` como hicimos con 3^{13}. ¿Cuántas vueltas da y por qué?
9. ¿Qué es la complejidad amortizada? ¿Por qué agregar al final de un vector que **duplica** es $O(1)$ amortizado?
10. Si el vector agrandara su capacidad de a 10 celdas en vez de duplicar, ¿cuál sería el costo amortizado de agregar? Justificá con la suma.
11. ¿Qué complejidad tiene Quicksort en promedio y en el peor caso? ¿Por qué?
12. ¿Por qué la medición empírica del tiempo puede ser engañosa?

Guía de ejercicios — Unidad 4

Objetivos de la unidad

Practicá el análisis y la implementación de algoritmos con distinta complejidad. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i04_complejidad` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las preguntas teóricas se responden en `guia/i04_complejidad/i01_teorico`. **El objetivo de toda la guía es calcular el $O(n)$ de cada algoritmo: por Teorema Maestro, o por el método de expansión, o por ambos** (cuando aplican los dos, verificá que dan lo mismo). En cada solución, **anotá la complejidad** en un comentario y justificala.

4.17 Preguntas teóricas

1. ¿Por qué medimos operaciones en función de n y no en segundos?
2. Definí O, Ω y Θ. ¿Cuál se asocia al peor caso?
3. ¿Cómo se calcula la complejidad de dos bucles anidados? ¿Y de uno que hace `i *= 2`?
4. Planteá la ecuación de recurrencia de la búsqueda binaria y de Mergesort.
5. Resolvé $T(n) = 2T(n/2) + n$ con el método de expansión.
6. Enunciá los tres casos del Teorema Maestro y dá un ejemplo de cada uno.
7. ¿Qué complejidad tiene Quicksort en promedio y en el peor caso? ¿Por qué?
8. ¿Por qué la medición empírica del tiempo puede ser engañosa?
9. ¿Qué complejidad tiene el Fibonacci recursivo ingenuo y por qué? ¿Cómo mejorarla?
10. Ordená de menor a mayor crecimiento: $O(n^2)$, $O(\log n)$, $O(n \log n)$, $O(1)$, $O(2^n)$, $O(n)$.

4.18 Nivel fácil

1. **Suma O(n).** `long sumar(int[] a)`; comentá que es $O(n)$. Test.
2. **Pares iguales O(n²).** `int paresIguales(int[] a)` con dos bucles anidados; test.
3. **Máximo O(n).** `int maximo(int[] a)`; test (incluí un solo elemento).
4. **¿Hay duplicado? O(n²).** `boolean hayDuplicado(int[] a)` comparando todos con todos; test.
5. **Potencia lenta O(n).** `long potenciaLenta(long base, int exp)` multiplicando `exp` veces; test.
6. **Contar dígitos O(log n).** `int cantidadDigitos(int n)` dividiendo por 10; test.
7. **Búsqueda lineal O(n).** `int indiceDe(int[] a, int x)` (o -1); test.
8. **Invertir O(n).** `int[] invertir(int[] a)`; test con `assertArrayEquals`.
9. **¿Potencia de 2? O(log n).** `boolean esPotenciaDe2(int n)` dividiendo por 2; test.
10. **Contar iteraciones.** `int iteraciones(int n)` que cuente cuántas vueltas da un `for (i=1; i<n; i*=2)`; test que da $\lfloor \log_2 n \rfloor$ (aprox.).

4.19 Nivel medio

1. **Búsqueda binaria O(log n).** Sobre arreglo ordenado; test (encontrado y no).
2. **Potencia rápida O(log n).** `long potencia(long base, int exp)` por cuadrados; test.
3. **Bubble sort O(n²).** `void ordenar(int[] a)`; test que ordena.
4. **Insertion sort O(n²).** Test que ordena; comentá el mejor caso $O(n)$.
5. **Mergesort O(n log n).** Con su recurrencia comentada; test.
6. **Suma recursiva.** `long sumar(int[] a)` recursivo, con la recurrencia $T(n) = T(n-1) + O(1)$; test.
7. **Factorial recursivo.** `long factorial(int n)`; comentá $T(n) = T(n-1)+O(1)$; test y excepción si $n < 0$.

8. **Máximo divide y vencerás.** `int maximo(int[] a)` que parta el arreglo en dos ($T(n) = 2T(n/2) + O(1)$); test.
9. **Contar apariciones O(log n).** En un arreglo ordenado, contar cuántas veces aparece `x` con dos búsquedas binarias (primera y última posición); test.
10. **Ordenar y comentar.** Selection sort $O(n^2)$; test y comentario de por qué es siempre $O(n^2)$.

4.20 Nivel difícil

1. **Quicksort.** Con partición de Lomuto; comentá promedio $O(n \log n)$ y peor $O(n^2)$; test.
2. **Quickselect.** `int kesimoMenor(int[] a, int k)` (promedio $O(n)$); test.
3. **Mergesort iterativo (bottom-up).** Sin recursión; test que ordena.
4. **Contar inversiones O(n log n).** Con una variante de Mergesort; test contra el conteo ingenuo.
5. **Kadane O(n).** `int maxSubarreglo(int[] a)` (suma máxima de un subarreglo); test.
6. **Búsqueda en arreglo rotado O(log n).** Buscar `x` en un arreglo ordenado y rotado; test.
7. **Exponenciación modular.** `long potModular(long base, long exp, long mod)` en $O(\log \exp)$; test.
8. **Torres de Hanoi.** `long movimientos(int n)` (con $T(n) = 2T(n-1) + 1 \Rightarrow 2^n - 1$); test.
9. **Fibonacci rápido.** En $O(\log n)$ por exponenciación (de matriz 2×2 o *fast doubling*); test contra el iterativo.
10. **Merge de k arreglos ordenados.** Fusionar `int[][]` en un solo arreglo ordenado; test.

Unidad 5

Estructuras de datos lineales

Objetivos de la unidad

En esta unidad construimos, **a mano y desde cero**, las estructuras lineales más importantes, para entender cómo funcionan por dentro; y para cada una vemos su equivalente en la **API de Java**. Al terminarla vas a poder: (1) implementar el TDA **Vector** propio (arreglo dinámico, con redimensión y ordenamiento), compararlo con `ArrayList` y `Vector` de la API, y elegir con fundamento; (2) **recorrer, buscar y ordenar** un `ArrayList` de todas las formas (índice, for-each, iteradores); (3) el TDA **Conjunto** con sus operaciones (unión, intersección, diferencia); (4) las **listas enlazadas** (simple, doble, circular y circular doble) con sus **iteradores** (`Iterator` y `ListIterator`), más las **pilas** (LIFO) y **colas** (FIFO); (5) razonar el **costo** de cada operación y las **fortalezas y debilidades** de cada estructura; y (6) usar las estructuras nativas de Java, que son las que vas a emplear en parciales, TPs y finales.

Ya tenemos las herramientas: diseñar con TDA y POO (Unidad 3) y medir complejidad (Unidad 4). Ahora las juntamos para construir las **estructuras de datos lineales**: colecciones donde los elementos forman una secuencia. Para cada estructura vamos a ver **su forma en memoria, cada operación con un gráfico y con código de uso, sus casos de uso, y sus características positivas y negativas**. Recién al final conectamos todo con las clases que Java ya trae hechas.

Casi todas comparten el mismo contrato, el TDA **Lista**:

```
public interface Lista<T> {
    void agregar(T x);            // agrega al final
    void insertar(int i, T x);    // inserta en la posición i
    T    obtener(int i);          // devuelve el elemento en i
    T    eliminar(int i);         // quita y devuelve el elemento en i
    int  indiceDe(T x);           // primera posición de x, o -1
    boolean contiene(T x);        // ¿está x?
    void agregarTodos(Lista<T> otra);
    int  tamanio();
    boolean estaVacia();
}
```

Como vimos en la Unidad 3, esto es el **contrato**: la misma Lista se puede implementar de formas muy distintas, y quien la usa no debería notar la diferencia.

5.1 El TDA Vector (arreglo dinámico)

La primera implementación usa un **arreglo** por dentro. La gran ventaja del arreglo es el **acceso directo**: llegar a `datos[i]` es $O(1)$, instantáneo. El problema es que un arreglo tiene tamaño fijo, y nosotros queremos una estructura que crezca. La solución es el **vector dinámico**: guardar dos cosas, el **tamaño** (cuántos elementos hay) y la **capacidad** (cuántas celdas tiene el arreglo), y agrandar el arreglo cuando se llena.

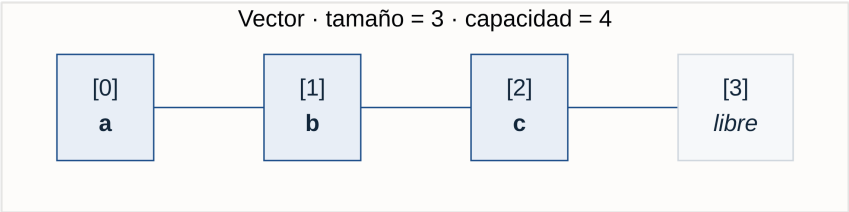


Figura 5.1: Un vector guarda `tamaño` elementos dentro de un arreglo de `capacidad` celdas.

5.1.1 La declaración de la clase: nuestra primera estructura

Empecemos como se empieza todo TDA (Unidad 3): por su **estado privado**. Nuestro `Vector<T>` guarda exactamente dos cosas:

```
public class Vector<T> {

    private T[] valores;    // el arreglo interno: capacidad = valores.length
    private int tamanio;    // cuántos elementos hay DE VERDAD

    @SuppressWarnings("unchecked")
    public Vector() {
        this.valores = (T[]) new Object[4];   // capacidad inicial
        this.tamanio = 0;
    }
}
```

Dos detalles de la parte privada. Primero, **no hay más estado que ése**: la capacidad no se guarda aparte porque *es* `valores.length` (checklist de TDA: no duplicar lo calculable). Segundo, el constructor tiene una rareza de Java: **no se puede escribir `new T[4]`** (los genéricos se borran al compilar y el arreglo no sabría su tipo), así que se crea un `Object[]` y se castea **una sola vez**, acá; el `@SuppressWarnings` documenta que es deliberado. De la clase para afuera, nada de esto se ve: sólo el contrato.

5.1.2 Agregar al final y crecer

Agregar al final es poner el elemento en la primera celda libre. Pero si el arreglo está **lleno**, primero hay que **agrandarlo**: se crea uno nuevo del **doble** de capacidad y se copian los elementos.

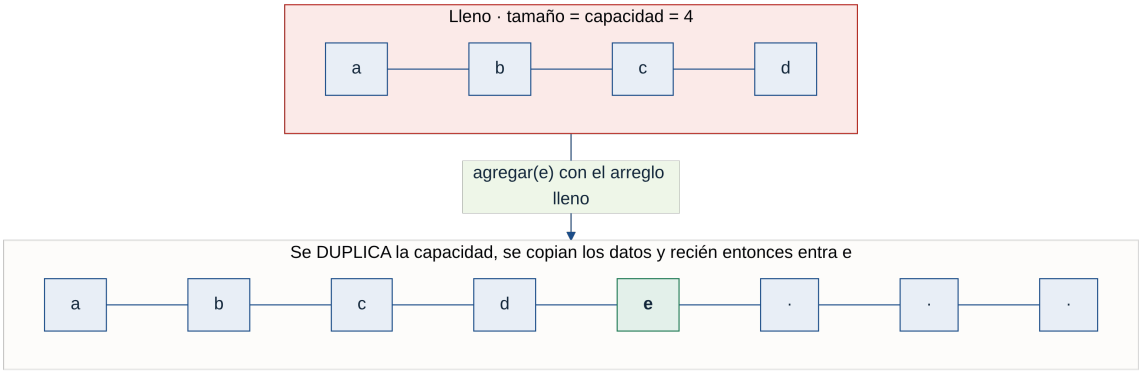


Figura 5.2: Cuando el arreglo se llena, se duplica la capacidad y se copian los datos.

```
public void agregar(T x) {
    if (tamanio == valores.length) {
        redimensionar(valores.length * 2);
    }
    valores[tamanio++] = x;
```

```
}
private void redimensionar(int nuevaCapacidad) {
    valores = Arrays.copyOf(valores, nuevaCapacidad);
}
```

Copiar cuesta $O(n)$... pero pasa **muy de vez en cuando**. Este es el famoso **análisis amortizado**: aunque una operación puntual sea cara, si se reparte su costo entre muchas operaciones baratas, el promedio es bajo.

Idea clave

¿Por qué agregar al final es $O(1)$ amortizado? Al duplicar, las copias a lo largo de n inserciones suman $1 + 2 + 4 + \cdots + n \approx 2n = O(n)$ en total; repartido entre las n inserciones, da $O(1)$ **por inserción en promedio**. Duplicar (en vez de agrandar de a uno, que costaría $O(n)$ por inserción) es lo que lo hace eficiente.

5.1.3 Insertar, eliminar, buscar

Insertar en el medio obliga a **correr a la derecha** todos los elementos posteriores; eliminar, a **correr a la izquierda**. Ambas son $O(n)$. Buscar también recorre ($O(n)$). Pero **obtener por índice es** $O(1)$.

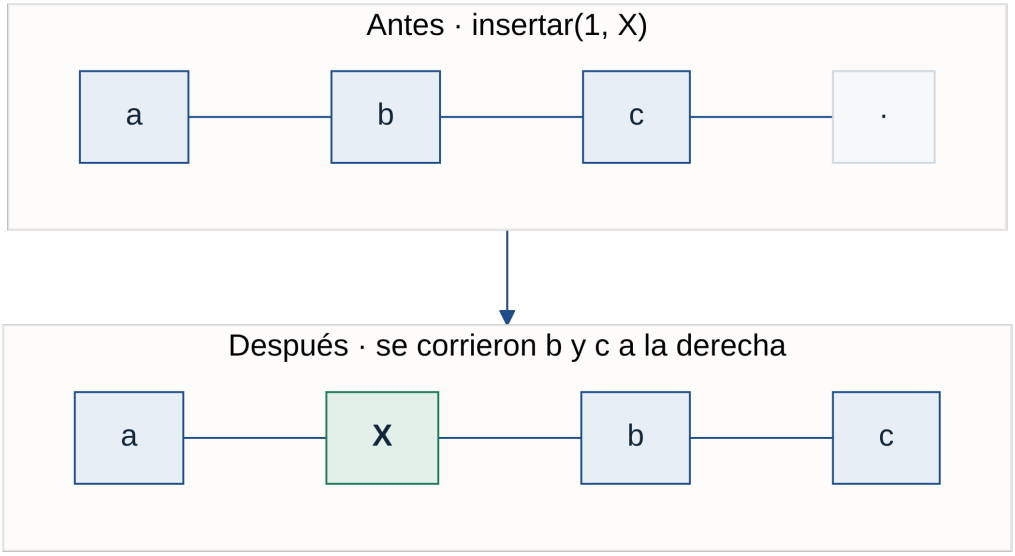


Figura 5.3: `insertar(1, X)`: hay que correr `b` y `c` una posición a la derecha.

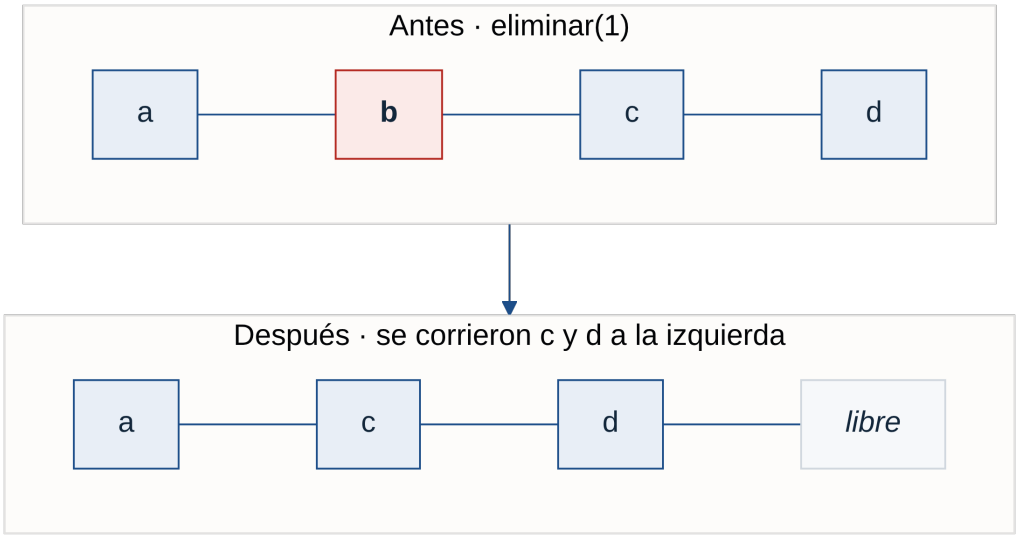


Figura 5.4: `eliminar(1)`: se corren `c` y `d` a la izquierda para tapar el hueco.

5.1.4 Buscar

Para saber **si un elemento está** (`contiene`) o **en qué posición** (`indiceDe`), el vector recorre celda por celda comparando con `equals`, desde la 0 en adelante:

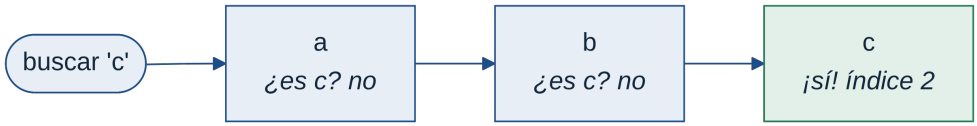


Figura 5.5: Buscar `"c"`: se recorre desde el principio comparando con `equals` hasta encontrarlo (o agotar el vector).

En el peor caso (el elemento está último o no está) se recorren las n celdas: es la **búsqueda lineal**, $O(n)$. Tener acceso directo por índice **no** ayuda a buscar **por valor** — para eso harían falta datos ordenados (búsqueda binaria) o una tabla de hash, que veremos más adelante.

5.1.5 Agregar todos

`agregarTodos(otra)` vuelca todos los elementos de otra lista al final, en orden. Es equivalente a un bucle de `agregar`, y su costo es $O(m)$ amortizado, con m el tamaño de la otra lista:

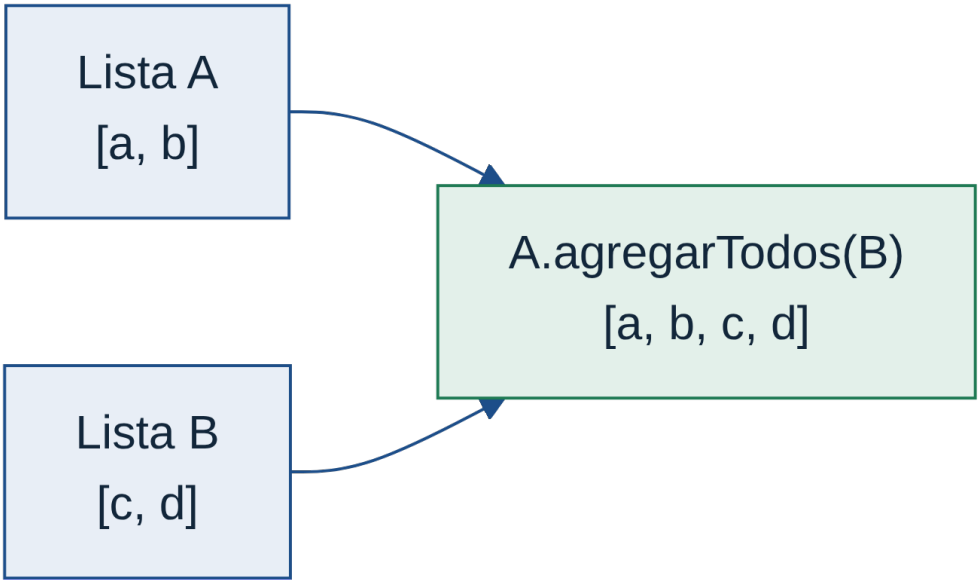


Figura 5.6: `agregarTodos`: los elementos de la otra lista se agregan al final, en orden.

5.1.6 Ordenar

Como el vector tiene acceso directo, se puede **ordenar** eficientemente ($O(n \log n)$) con un algoritmo como Mergesort o Quicksort (Unidad 4):

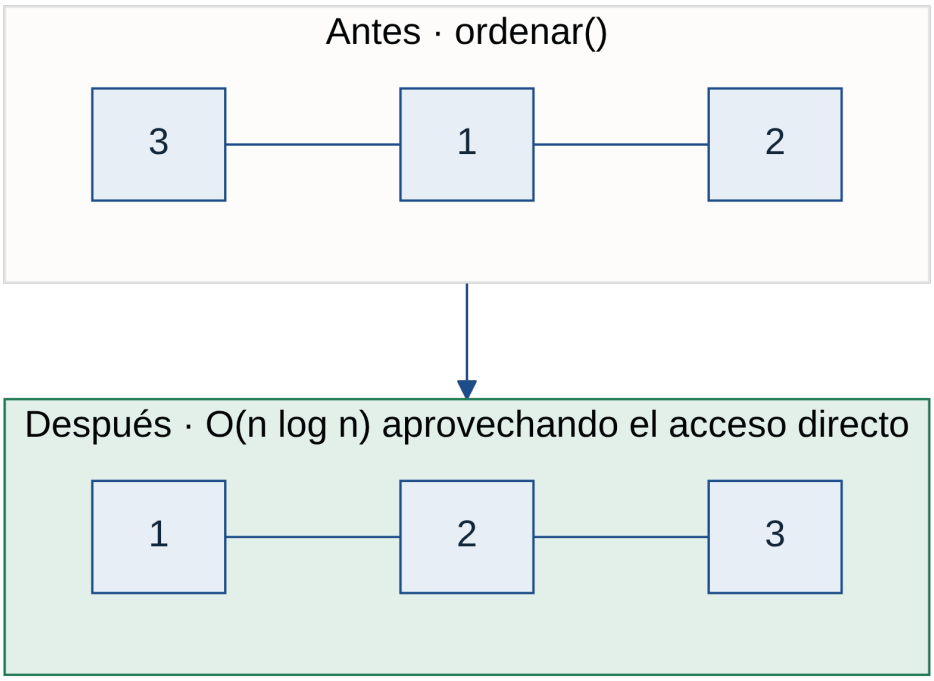


Figura 5.7: `ordenar()`: reacomoda los elementos de menor a mayor.

```java
public void ordenar(Comparator<? super T> comparador) {
    Arrays.sort(valores, 0, tamanio, comparador);   // sólo la parte con datos
}
```

5.1.7 Código de uso: todas las operaciones

```
Vector<String> v = new Vector<>();
v.agregar("a");                 // [a]
v.agregar("b");                 // [a, b]
v.insertar(1, "x");             // [a, x, b]
System.out.println(v.obtener(2));       // b   (acceso directo O(1))
System.out.println(v.indiceDe("x"));    // 1
System.out.println(v.contiene("b"));    // true
v.eliminar(0);                  // [x, b]
v.ordenar(Comparator.naturalOrder());   // [b, x]  (orden alfabético)

Vector<String> otro = new Vector<>();
otro.agregar("c"); otro.agregar("d");
v.agregarTodos(otro);           // [b, x, c, d]
System.out.println(v.tamanio() + " " + v.capacidad());
```

(Un detalle fino: el orden de `String` compara por el **valor Unicode** de cada carácter; con todo en minúsculas coincide con el alfabético, pero ojo que las mayúsculas ordenan **antes**: `"Z" < "a"`.)

5.1.8 Casos de uso y características

Idea clave

Cuándo conviene un vector. Cuando hacés mucho **acceso por índice** (leer/escribir `v[i]`), cuando agregás sobre todo **al final**, o cuando necesitás **ordenar** o recorrer rápido. Ejemplos: una tabla de puntajes, los píxeles de una fila, un buffer de lecturas. Es la estructura **por defecto** para una secuencia.

Error común

Limitaciones del vector. (1) Insertar o borrar **al principio o en el medio** es $O(n)$ (hay que correr todo). (2) Deja **espacio sin usar**: la capacidad suele ser mayor que el tamaño, así que hay celdas reservadas y "desperdiciadas". (3) La redimensión, aunque amortizada, implica una copia costosa puntual. Admite **valores duplicados** y mantiene el **orden de inserción**.

5.1.9 El mismo TDA en la API: `ArrayList` y `Vector`

Java trae **dos** clases con exactamente esta idea. Mirá sus declaraciones reales (del código fuente del JDK) — no son un misterio, son **nuestro mismo TDA** con otros nombres:

```
public class ArrayList<E> extends AbstractList<E>
        implements List<E>, RandomAccess, Cloneable, java.io.Serializable {

    transient Object[] elementData;   // el arreglo interno: nuestro "valores"
    private int size;                 // nuestro "tamanio"
```

```
public class Vector<E> extends AbstractList<E>
        implements List<E>, RandomAccess, Cloneable, java.io.Serializable {

    protected Object[] elementData;   // el mismo arreglo interno
    protected int elementCount;       // el mismo tamaño
```

La parte privada es **idéntica** a la nuestra: un arreglo y un contador. Hasta la redimensión es la misma técnica; sólo cambia el factor de crecimiento. Éste es el código real de `ArrayList.grow()`:

```
int newCapacity = oldCapacity + (oldCapacity >> 1);   // 1.5x: capacidad + su mitad
```

(`>> 1` es el desplazamiento de la Unidad 4: dividir por 2. `ArrayList` crece un 50%; `java.util.Vector` duplica, como el nuestro.) ¿Y en qué se diferencian entre sí? En una sola cosa importante — y en su historia:

	`Vector` (nuestro)	`java.util.Vector`	`java.util.ArrayList`
Existe desde	esta unidad	Java 1.0 (1996)	Java 1.2 (1998)
Crecimiento	$\times 2$	$\times 2$	$\times 1.5$
Sincronizado	no	**sí** (cada método)	no
Estado hoy	didáctico	**legado**: evitarlo	**la elección estándar**

`java.util.Vector` tiene cada método marcado `synchronized` (`public synchronized boolean add(E e)`): cada llamada paga un candado para poder usarse desde varios hilos a la vez. Suena bien, pero casi nunca es la sincronización que un programa concurrente de verdad necesita, y mientras tanto **todos** pagan el costo. Por eso desde 1998 la recomendación es:

Importante

En la práctica, la clase elegida es `ArrayList`. `java.util.Vector` se mantiene sólo por compatibilidad con código viejo; verlo en código nuevo es una *red flag*. Y nuestro `Vector<T>` propio es para **entender** — por eso su implementación completa vive en la **guía** (difícil e01) y no en `material`: en los TP y parciales se usa `ArrayList`.

5.1.10 ¿Por qué implementan la interfaz `List`?

Fijate que las dos declaraciones dicen `implements List<E>`. `List` es, ni más ni menos, **el contrato del TDA Lista** — la misma idea de la interfaz `Lista<T>` con la que abrimos la unidad, pero de la API. Declara *qué* hace una lista (`add`, `get`, `remove`, `contains`, `size`…) sin decir *cómo*; `ArrayList`, `java.util.Vector` y `LinkedList` son tres **implementaciones** del mismo contrato, cada una con sus costos.

Por eso el código profesional declara las variables **por la interfaz**:

```
List<String> lista = new ArrayList<>();   // el TIPO es el contrato...
// List<String> lista = new LinkedList<>();  // ...y la implementación se
                                             // puede cambiar en UNA línea
```

Quien usa `lista` sólo conoce el contrato, así que cambiar la implementación no rompe nada — es exactamente el argumento de la Unidad 3, ahora en escala industrial. (El otro nombre de la declaración, `RandomAccess`, es una interfaz **marcadora**: no tiene métodos, sólo "etiqueta" a las listas cuyo acceso por índice es $O(1)$, para que los algoritmos genéricos sepan si les conviene recorrer por índice o por iterador.)

5.1.11 Recorrer, buscar y ordenar un `ArrayList`

Ya elegida la herramienta, hay que saber manejarla. Éstas son las tres formas de **recorrer** una lista — las tres visitan los mismos elementos, cambian la comodidad y lo que te dejan hacer:

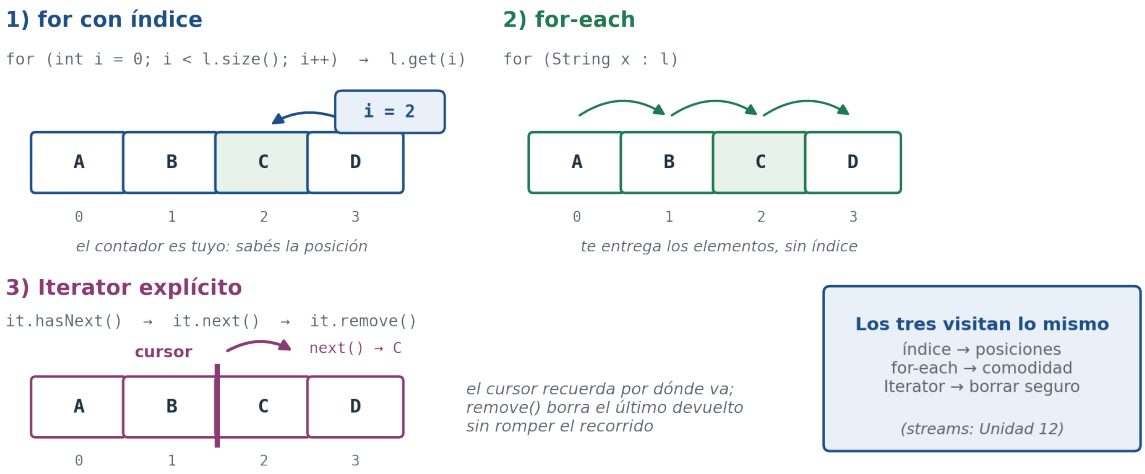


Figura 5.8: Las tres formas de recorrer: el `for` con índice lleva un contador y accede con `get(i)`; el for-each entrega los elementos de a uno; el `Iterator` explícito es un cursor que avanza con `next()` mientras `hasNext()`.

```java
List<String> materias = new ArrayList<>();
materias.add("Álgebra"); materias.add("AlgoritmosI"); materias.add("Física");

// 1) for clásico con índice: cuando NECESITÁS la posición i
for (int i = 0; i < materias.size(); i++) {
    System.out.println(i + ": " + materias.get(i));
}

// 2) for-each: la forma habitual, cómoda y sin índice
for (String m : materias) {
    System.out.println(m);
}

// 3) Iterator explícito: un "cursor" que avanza a pedido
Iterator<String> it = materias.iterator();
while (it.hasNext()) {
    String m = it.next();
    if (m.startsWith("Á")) {
        it.remove();   // la ÚNICA forma segura de borrar mientras recorrés
    }
}
```

El tercero parece el más incómodo, pero tiene un superpoder: **`it.remove()` borra el elemento actual sin romper el recorrido**. Si en cambio hacés `materias.remove(...)` adentro de un for-each, la lista te corta el recorrido con una `ConcurrentModificationException` — es uno de los errores más comunes de Java. (El for-each, por dentro, *usa* un `Iterator`: es azúcar sintáctico.) La cuarta forma de recorrer —los *streams*— la dejamos para la Unidad 12.

Buscar tiene dos niveles, según cómo estén los datos:

```java
materias.contains("Física");        // ¿está?           -> búsqueda lineal O(n)
materias.indexOf("Física");         // ¿en qué posición? -> búsqueda lineal O(n)

Collections.sort(materias);                          // primero ordenar...
int pos = Collections.binarySearch(materias, "Física");  // ...y buscar en O(log n)
```

`contains`/`indexOf` recorren comparando con `equals` (la búsqueda lineal de esta unidad). `binarySearch` es la búsqueda binaria de la Unidad 4 — pero **sólo vale sobre una lista ordenada**: sobre una desordenada devuelve cualquier cosa, sin avisar. Si es negativo, el resultado codifica dónde insertarlo: `(-pos - 1)`.

Ordenar se puede de dos maneras equivalentes, y el `Comparator` decide el criterio:

```java
materias.sort(Comparator.naturalOrder());     // la forma moderna (Java 8+)
Collections.sort(materias);                   // la clásica: hace lo mismo

// Criterios a medida, componibles:
materias.sort(Comparator.comparing(String::length));           // por longitud
materias.sort(Comparator.comparing(String::length).reversed()); // al revés
materias.sort(String.CASE_INSENSITIVE_ORDER);  // ignorando mayúsculas
```

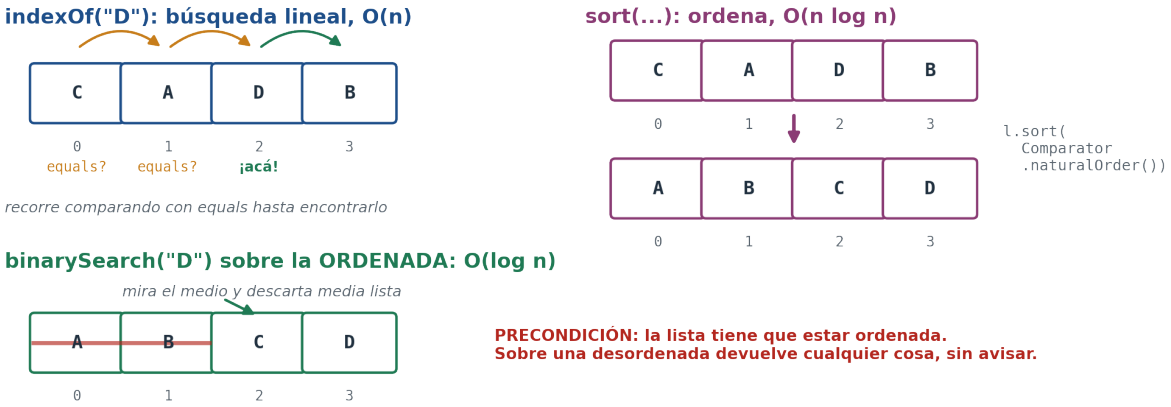


Figura 5.9: Buscar y ordenar sobre la misma lista: `indexOf` recorre celda por celda; ordenar reacomoda; `binarySearch` sobre la lista ya ordenada descarta mitades.

Por dentro, `sort` usa un algoritmo $O(n \log n)$ (Timsort, un Mergesort mejorado que aprovecha tramos ya ordenados — Unidad 6). El detalle fino de comparar `String`: el orden natural compara por **valor Unicode**, así que todas las mayúsculas ordenan antes que las minúsculas (`"Z" < "a"`); para el orden "de diccionario" usá `String.CASE_INSENSITIVE_ORDER`.

5.2 El TDA Conjunto (Set)

Un **Conjunto** es una colección **sin duplicados** y sin posición: sólo importa si un elemento **está o no está**. Su operación distintiva es que `agregar` **ignora** lo que ya está.

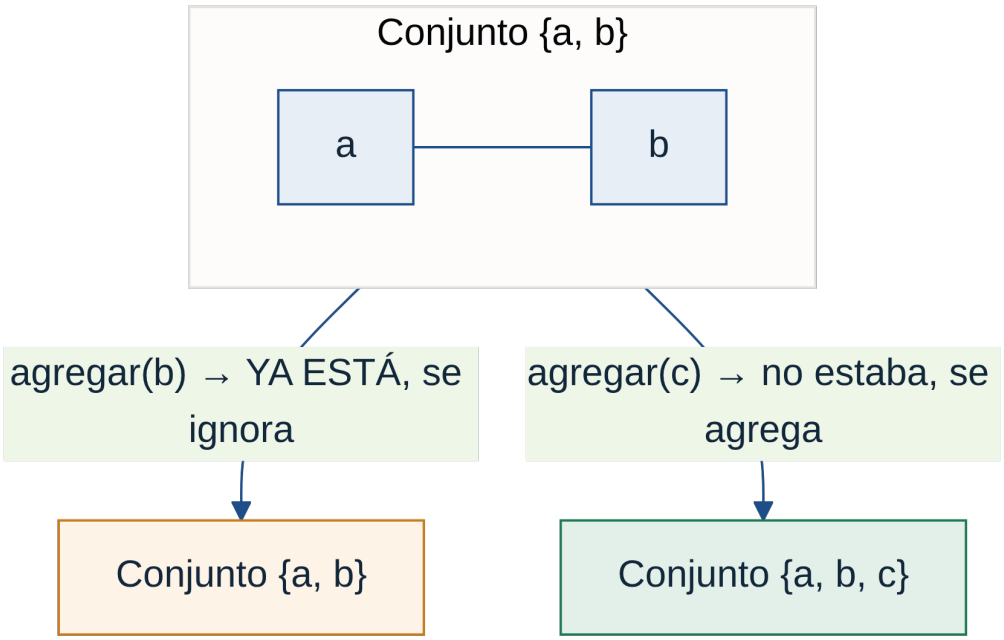


Figura 5.10: `agregar` sólo suma el elemento si no estaba; los repetidos se ignoran.

5.2.1 Implementación manual y operaciones de conjuntos

Como todo TDA, el Conjunto arranca por su **contrato** (la interfaz) y su **estado privado**. El contrato declara las operaciones —fijate que `agregar` devuelve `boolean`: avisa si el elemento entró o ya estaba—:

```
public interface Conjunto<T> {
    boolean agregar(T x);        // false si ya estaba (los repetidos se ignoran)
    boolean contiene(T x);
    boolean eliminar(T x);
    void agregarTodos(Conjunto<T> otro);
    Conjunto<T> union(Conjunto<T> otro);
    Conjunto<T> interseccion(Conjunto<T> otro);
    Conjunto<T> diferencia(Conjunto<T> otro);
    int tamanio();
    Object[] aArreglo();         // los elementos, para poder recorrerlos
}
```

Y nuestra implementación se apoya en una lista — su parte privada es **una sola línea**, porque delega el almacenamiento:

```
public class ConjuntoLista<T> implements Conjunto<T> {

    private final ListaSimplementeEnlazada<T> elementos =
            new ListaSimplementeEnlazada<>();
}
```

El trabajo del conjunto no es guardar (eso lo hace la lista): es **hacer cumplir la regla de no duplicar**. Antes de agregar, chequea si el elemento ya está. Además de agregar/quitar/contiene, un conjunto ofrece las operaciones clásicas: **unión**, **intersección** y **diferencia**.

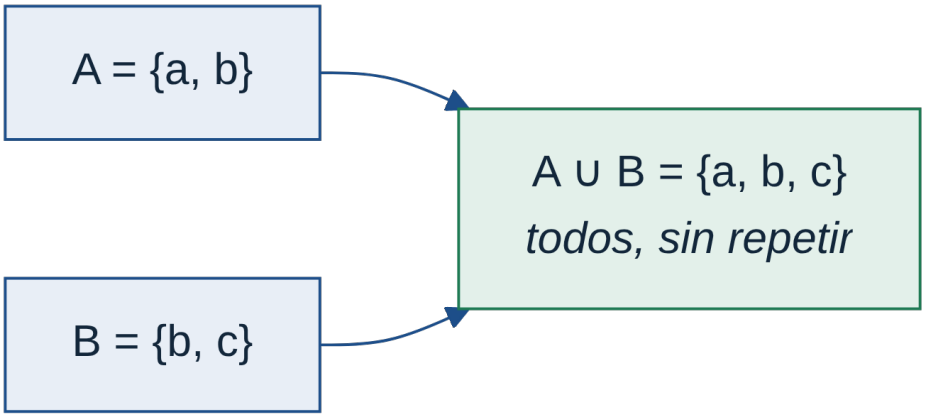


Figura 5.11: Unión: todos los elementos de ambos, sin repetir.

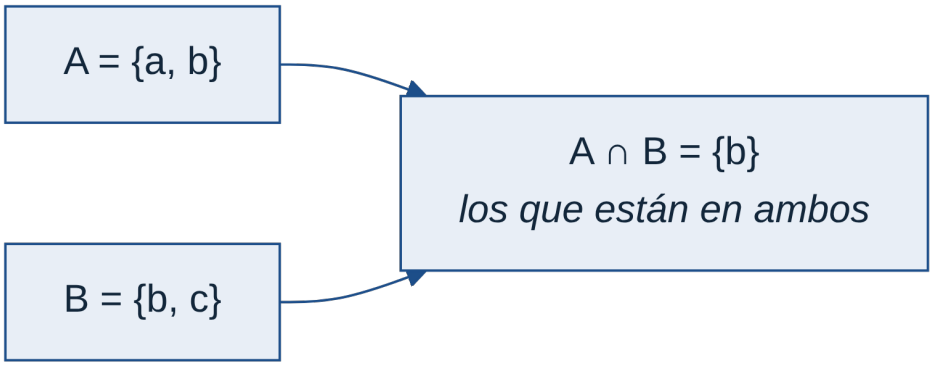


Figura 5.12: Intersección: sólo los que están en los dos.

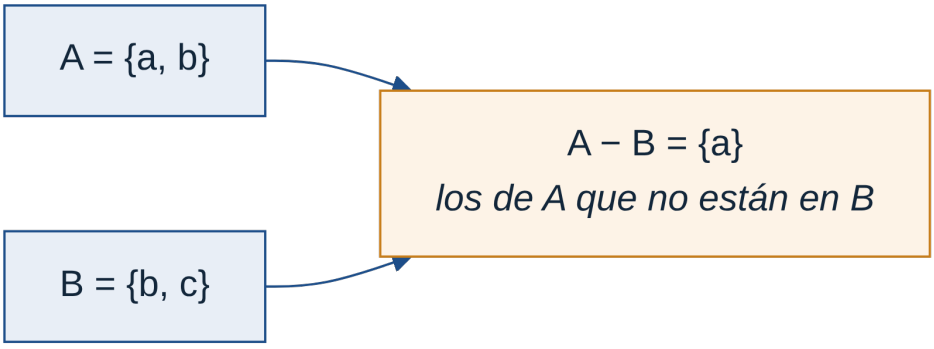


Figura 5.13: Diferencia A − B: los de A que no están en B.

```java
public Conjunto<T> interseccion(Conjunto<T> otro) {
    ConjuntoLista<T> r = new ConjuntoLista<>();
    for (Object o : aArreglo()) {
        if (otro.contiene((T) o)) {
            r.agregar((T) o);   // sólo los que están en ambos
        }
    }
    return r;
}
```

(`aArreglo()` es un método auxiliar del conjunto que devuelve sus elementos en un `Object[]` para poder recorrerlos; como los arreglos de Java no pueden ser genéricos, al leer cada elemento hace falta el *cast* `(T) o` — está completo en el repo, en `ConjuntoLista`.)

5.2.2 Código de uso

```
ConjuntoLista<String> a = new ConjuntoLista<>();
a.agregar("a"); a.agregar("b"); a.agregar("a");   // el segundo "a" se ignora
System.out.println(a.contiene("a"));               // true
a.eliminar("b");                                    // {a}

ConjuntoLista<String> otro = new ConjuntoLista<>();
otro.agregar("a"); otro.agregar("c");
System.out.println(a.union(otro));          // {a, c}
System.out.println(a.interseccion(otro));   // {a}
System.out.println(a.diferencia(otro));     // {}
```

5.2.3 Casos de uso y características

Idea clave

Cuándo conviene un conjunto. Cuando necesitás **pertenencia sin duplicados**: "¿ya vimos este usuario?", "palabras únicas de un texto", "etiquetas de un elemento". También para operaciones de conjuntos (unión de dos grupos, intersección de intereses, etc.).

Error común

Limitaciones del conjunto. (1) **No admite duplicados** (si necesitás contar repetidos, no es la estructura). (2) **No tiene posición ni orden** (no existe "el elemento i"). (3) En nuestra implementación sobre lista, `agregar`/`contiene` son $O(n)$; con una **tabla de hash** (próxima unidad) serán $O(1)$ promedio.

5.2.4 En la API de Java

`HashSet` (sin orden, $O(1)$ promedio), `LinkedHashSet` (mantiene el orden de inserción) y `TreeSet` (mantiene los elementos ordenados, $O(\log n)$). Todos implementan `Set`.

```
Set<String> s = new HashSet<>();
s.add("a"); s.add("a");          // el segundo se ignora
s.retainAll(otro);               // intersección
s.addAll(otro);                  // unión
s.removeAll(otro);               // diferencia
```

5.3 Listas enlazadas

Las listas enlazadas no usan un arreglo, sino **nodos** encadenados por referencias (Unidad 2). Su gran ventaja frente al vector: insertar y borrar en los **extremos** es $O(1)$ y **no desperdician espacio** (usan exactamente los nodos que necesitan). Su desventaja: **no tienen acceso directo** (llegar a la posición i es $O(n)$).

5.3.1 Lista simplemente enlazada

Cada **nodo** guarda su valor y una referencia al **siguiente**. La lista conoce el `primero` (y, para agregar al final rápido, el `ultimo`); el último apunta a `null`. Ésa es, literalmente, toda la parte privada:

```
public class ListaSimplementeEnlazada<T> implements Lista<T> {

    private static final class Nodo<T> {
```

```
        T valor;
        Nodo<T> siguiente;          // null si es el último
        Nodo(T valor) { this.valor = valor; }
    }

    private Nodo<T> primero;        // null si la lista está vacía
    private Nodo<T> ultimo;         // para agregar al final en O(1)
    private int tamanio;            // para no contar recorriendo
}
```

Tres decisiones de diseño para leer con atención. `Nodo` es una **clase privada y anidada**: es un detalle de implementación que nadie de afuera debe ver ni tocar (si fuera pública, cualquiera podría romper los enganches). `ultimo` existe **sólo** para que `agregar` al final no recorra toda la lista. Y `tamanio` se mantiene actualizado en cada operación para que `tamanio()` sea $O(1)$ — el clásico compromiso de guardar un dato redundante pero barato de mantener, en vez de recalcularlo caro.

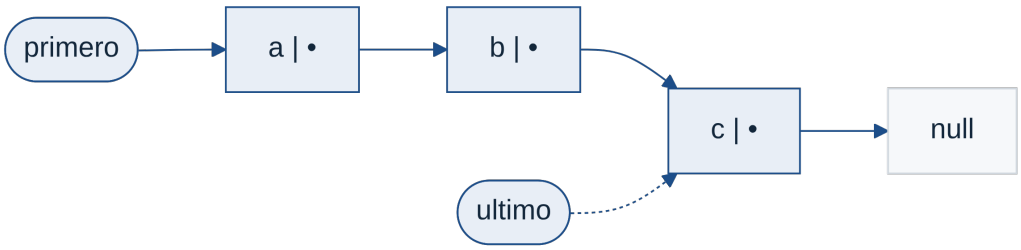


Figura 5.14: Cada nodo apunta al siguiente; el último a `null`.

Insertar no requiere correr nada, sólo **re-enganchar** referencias:

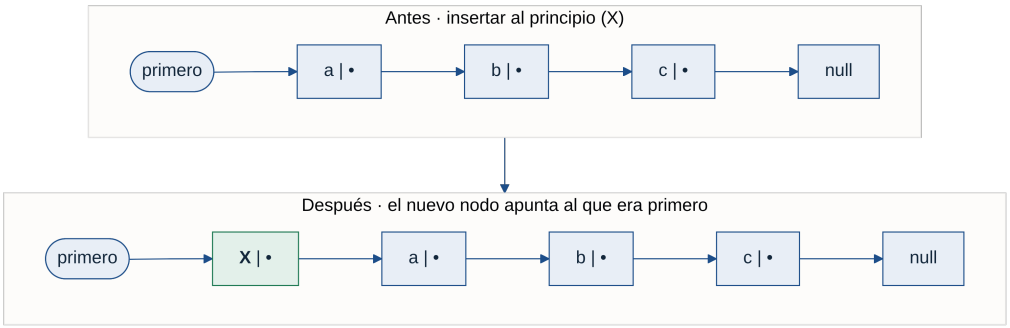


Figura 5.15: Insertar al principio: el nuevo nodo apunta al viejo primero, sin mover el resto.

```
public void insertar(int i, T x) {
    if (i < 0 || i > tamanio) {
        throw new IndexOutOfBoundsException("índice " + i);
    }
    if (i == tamanio) { agregar(x); return; }
    Nodo<T> n = new Nodo<>(x);
    if (i == 0) { n.siguiente = primero; primero = n; }
    else {
        Nodo<T> anterior = nodoEn(i - 1);
        n.siguiente = anterior.siguiente;
        anterior.siguiente = n;
    }
    tamanio++;
}
```

Eliminar es “saltear” el nodo; el eliminado queda inalcanzable y lo libera el GC (Unidad 2). **Buscar** recorre comparando con `equals`.

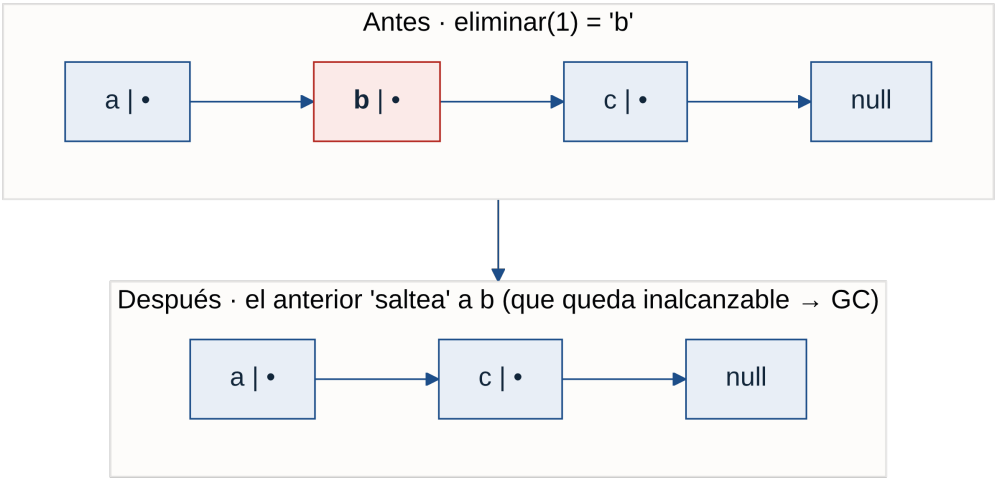


Figura 5.16: Eliminar: el nodo anterior saltea al que se elimina.

Código de uso:

```
ListaSimplementeEnlazada<String> l = new ListaSimplementeEnlazada<>();
l.agregar("a"); l.agregar("b"); l.agregar("c");
l.insertar(0, "X");            // [X -> a -> b -> c]
l.obtener(2);                  // b   (¡esto recorre, es O(n)!)
l.indiceDe("c");               // 3
l.eliminar(1);                 // saca "a"
```

Idea clave

Cuándo conviene una lista enlazada. Cuando hacés muchas **inserciones y borrados al principio** (o en posiciones que ya tenés ubicadas), o cuando **no sabés cuántos elementos** vas a tener y no querés desperdiciar espacio. Es la base de pilas, colas y muchas otras estructuras.

Error común

Limitaciones de la lista simple. (1) **No hay acceso directo**: `obtener(i)` es $O(n)$. (2) Cada nodo gasta memoria extra en la **referencia** al siguiente. (3) Sólo se recorre **hacia adelante**. Admite **duplicados** y mantiene el orden.

5.3.2 Lista doblemente enlazada

Si cada nodo guarda también una referencia al **anterior**, tenemos una lista **doblemente enlazada**. Permite recorrer en los **dos sentidos** y eliminar un nodo dado en $O(1)$ (no hace falta buscar al anterior).

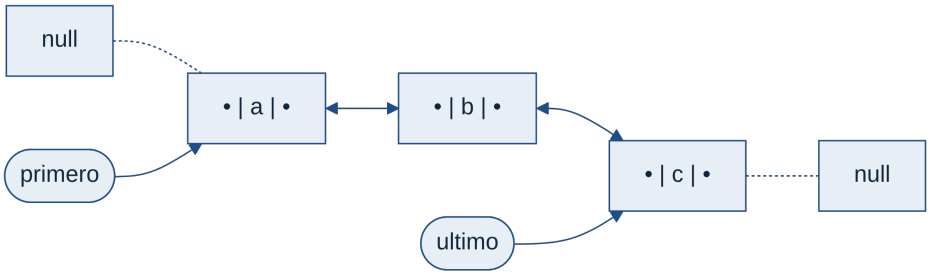


Figura 5.17: Cada nodo conoce a su anterior y a su siguiente.

Insertar y eliminar re-enganchan referencias en **ambos** sentidos. Insertar `X` entre `a` y `b` requiere **exactamente cuatro** asignaciones de referencias, y el **orden** en que las hagas importa:

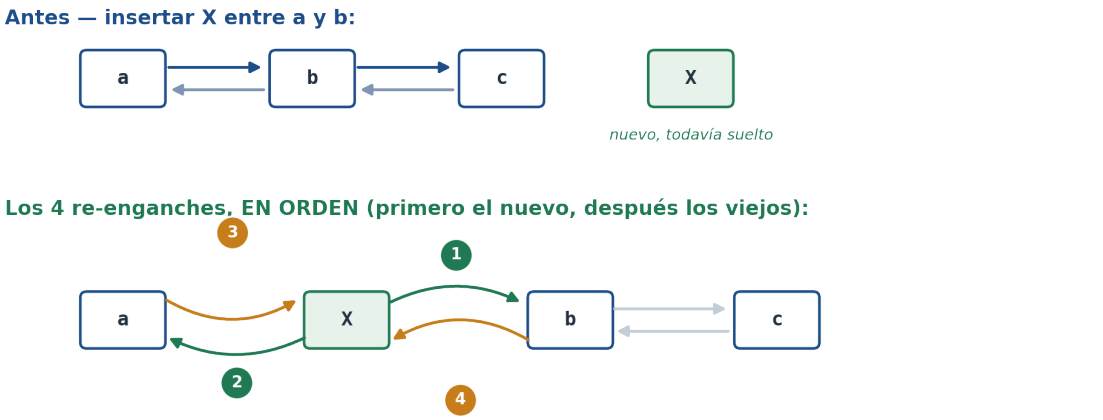


Figura 5.18: Insertar en una lista doble: los 4 re-enganches, numerados y en orden. Primero se conecta el nodo nuevo (1 y 2); recién después se re-apuntan los viejos (3 y 4).

Seguí los números: **(1)** `X.siguiente = b` y **(2)** `X.anterior = a` conectan el nodo **nuevo** mirando a sus futuros vecinos — esto no rompe nada, porque todavía nadie apunta a `X`. Recién entonces **(3)** `a.siguiente = X` y **(4)** `b.anterior = X` insertan a `X` en la cadena. Si hicieras (3) **antes** que (1), `a` dejaría de apuntar a `b`… y como `X` todavía no lo apunta, **perdés la referencia** al resto de la lista: es el error clásico de “la lista se corta”.

Eliminar es más simple: sólo **dos** re-enganches, porque el nodo que se va no necesita que lo desconecten “de su lado”:

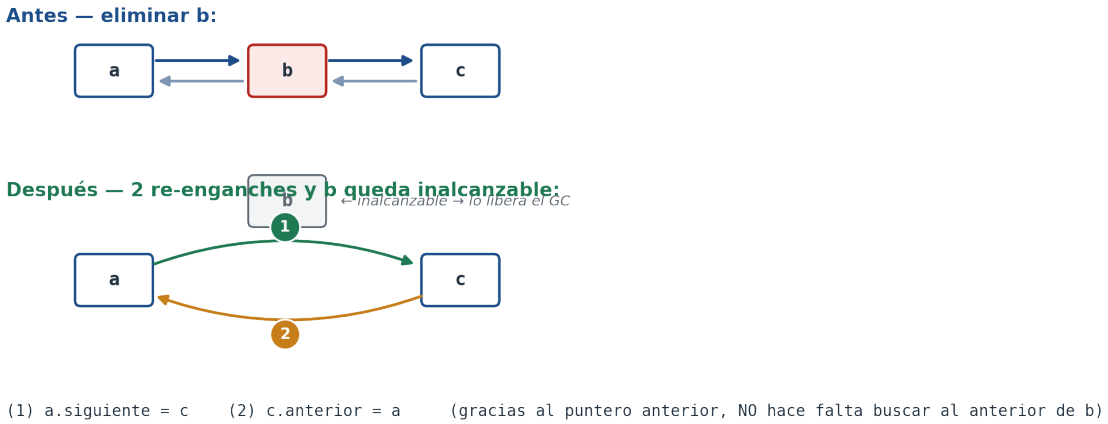


Figura 5.19: Eliminar en una lista doble: (1) `a.siguiente = c` y (2) `c.anterior = a`. El nodo `b` queda inalcanzable y lo libera el GC. Gracias al puntero `anterior`, no hace falta recorrer para encontrar al vecino izquierdo.

```
ListaDoblementeEnlazada<Integer> l = new ListaDoblementeEnlazada<>();
l.agregar(1); l.agregar(2); l.agregar(3);
l.insertar(1, 99);              // [1 <-> 99 <-> 2 <-> 3]
l.eliminar(0);                  // [99 <-> 2 <-> 3]
```

Idea clave

Ventaja clave. Además de recorrer en dos sentidos, para llegar a la posición i se puede empezar por el **extremo más cercano** (a lo sumo $n/2$ pasos). Es la que usa `LinkedList` de Java.

Error común

Costo extra. Cada nodo guarda **dos** referencias (anterior y siguiente) en vez de una: más memoria por elemento. Sigue sin tener acceso directo $O(1)$.

5.3.3 Lista circular

En una **lista circular**, el `siguiente` del último nodo vuelve a apuntar al **primero**, cerrando un anillo. Es ideal para recorrer **en ronda**.

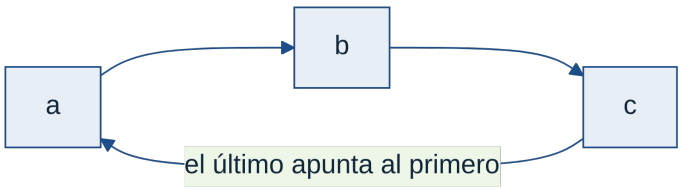


Figura 5.20: En la lista circular, el último nodo apunta de nuevo al primero.

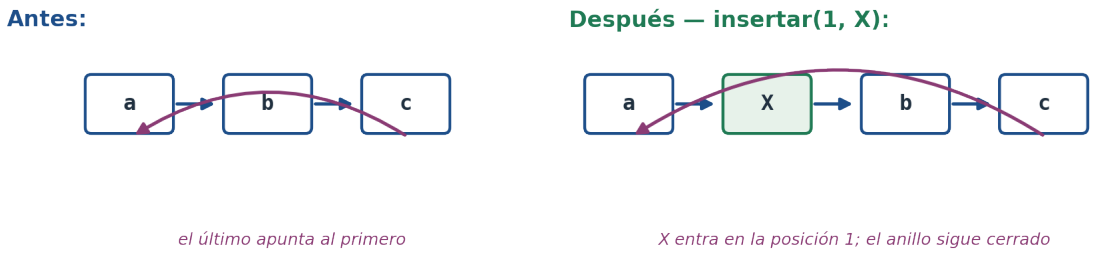


Figura 5.21: Insertar en la circular, antes y después: `X` entra en la posición 1 y el anillo sigue cerrado (el último siempre apunta al primero).

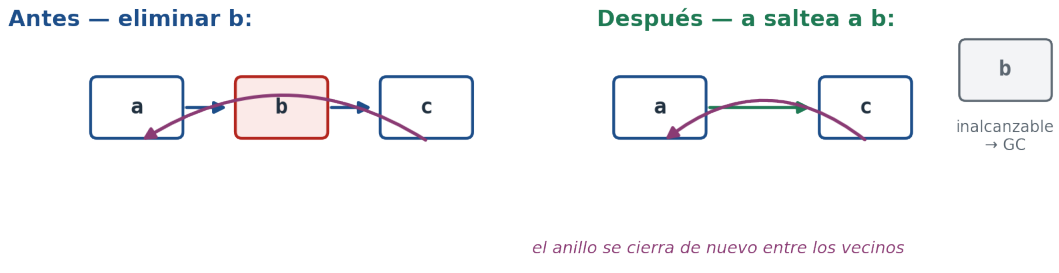


Figura 5.22: Eliminar en la circular, antes y después: `a` saltea a `b`, el anillo se cierra entre los vecinos, y `b` queda inalcanzable para el GC.

```
ListaCircular<String> c = new ListaCircular<>();
c.agregar("lun"); c.agregar("mar"); c.agregar("mie");
c.insertar(1, "feriado");      // (lun -> feriado -> mar -> mie -> lun…)
c.eliminar(2);                 // saca "mar"; el anillo se re-cierra
c.obtener(0);                  // lun
c.contiene("mie");             // true
```

Idea clave

Cuándo conviene. Para procesos que se repiten en **ronda**: turnos rotativos (round-robin), un reproductor en modo “repetir”, el problema de Josephus, un buffer circular.

Error común

Cuidado al recorrerla. Nunca se llega a `null`: si recorrés esperando el final, das **vueltas para siempre**. Hay que **contar** los elementos.

5.3.4 Lista circular doblemente enlazada

Combina las dos ideas: cada nodo conoce a su anterior y a su siguiente, y el anillo se cierra **en los dos sentidos** (`ultimo.siguiente == primero` y `primero.anterior == ultimo`). Permite avanzar y retroceder en ronda, y agregar/quitar en cualquiera de los dos extremos en $O(1)$.

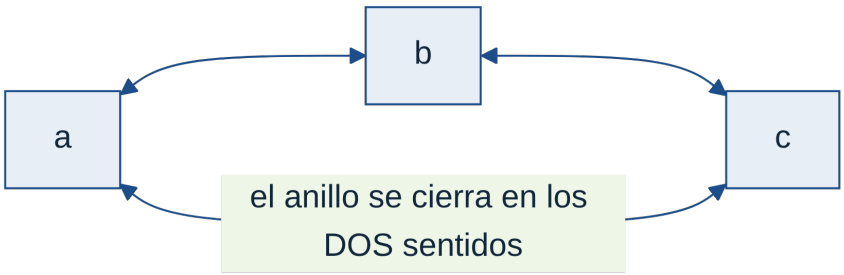


Figura 5.23: El anillo se cierra en ambos sentidos.

Sus operaciones combinan lo que ya sabés de la doble y de la circular: al insertar o eliminar se re-enganchan los punteros en **ambos** sentidos, y además hay que mantener cerrados los **dos** cierres del anillo (la flecha naranja, `anterior` del primero al último; y la violeta, `siguiente` del último al primero):

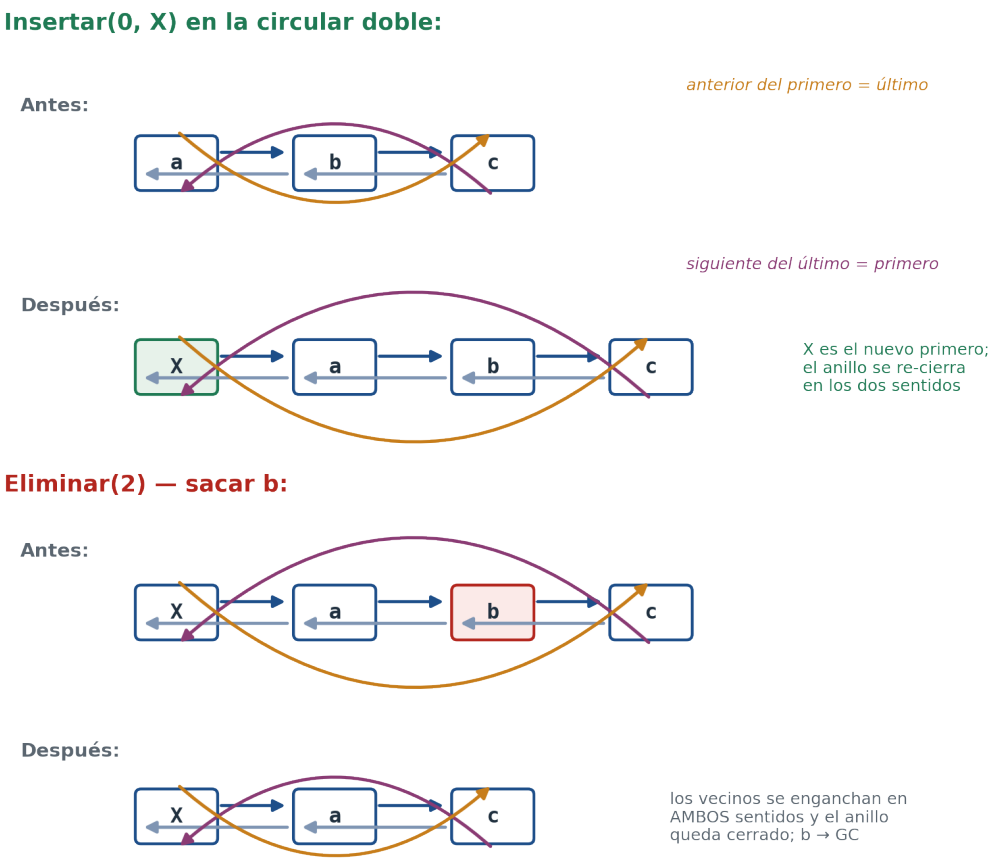


Figura 5.24: Insertar y eliminar en la circular doble, antes y después de cada operación. En todo momento se cumplen los invariantes `ultimo.siguiente == primero` y `primero.anterior == ultimo`.

```
ListaCircularDoble<String> c = new ListaCircularDoble<>();
c.agregar("a"); c.agregar("b"); c.agregar("c");
c.insertar(0, "X");            // (X <-> a <-> b <-> c <->…)
c.eliminar(2);                 // saca "b": vecinos re-enganchados en ambos sentidos
```

Error común

Limitaciones de la circular doble. Es la lista más “cara” de mantener: (1) cada nodo guarda **dos referencias** (más memoria por elemento); (2) cada inserción/borrado debe actualizar punteros en **ambos sentidos** y preservar los **dos invariantes** del anillo — más punteros que mantener consistentes, más lugares donde equivocarse; (3) igual que la circular simple, **nunca hay `null`**: recorrer esperando el final da vueltas infinitas (hay que contar); y (4) sigue sin acceso directo: `obtener(i)` es $O(n)$ (aunque puede arrancar por el extremo más cercano). Usala sólo cuando de verdad necesites **ronda + ambos sentidos**.

Idea clave

Cuándo conviene. Cuando necesitás recorrer una ronda en **los dos sentidos** (adelante y atrás) con inserción/borrado eficiente en los extremos: listas de reproducción con “anterior/siguiente” cíclico, la lista de ventanas abiertas (Alt+Tab), historiales circulares.

5.4 Recorrer listas enlazadas: iteradores

5.4.1 El problema: `obtener(i)` en un bucle es una trampa

Con un vector, recorrer con `for (i) { obtener(i) }` es perfecto: cada acceso es $O(1)$. Con una lista **enlazada**, ese mismo bucle es una trampa de rendimiento: cada `obtener(i)` arranca **desde el primero** y camina i nodos, así que el recorrido completo cuesta $1 + 2 + \cdots + n = O(n^2)$ — la suma aritmética de la Unidad 4, escondida en dos líneas inocentes:

```
// ¡TRAMPA! O(n^2) sobre una lista enlazada:
for (int i = 0; i < lista.tamanio(); i++) {
    procesar(lista.obtener(i));      // cada obtener recorre desde el principio
}
```

La forma correcta es **no soltar el nodo**: avanzar de nodo en nodo, una sola pasada, $O(n)$ total. Dentro de la clase (donde `Nodo` es visible) es el bucle clásico:

```
for (Nodo<T> n = primero; n != null; n = n.siguiente) {
    procesar(n.valor);
}
```

¿Y **desde afuera** de la clase, donde `Nodo` es privado? Para eso existe el **iterador**: un objeto-cursor que la propia lista ofrece y que recuerda por dónde va. Es la solución TDA al dilema "recorrer eficiente sin exponer los nodos".

5.4.2 `Iterator`: el cursor de ida

El contrato mínimo es la interfaz `Iterator<T>` de la API: `hasNext()` (¿queda algo?), `next()` (dame el próximo y avanzá) y `remove()` (borrá el último que me diste). Toda colección de Java sabe fabricar el suyo con `.iterator()`, y el for-each no es más que azúcar sintáctico sobre esto:

```
List<String> l = new LinkedList<>();
l.add("a"); l.add("b"); l.add("c");

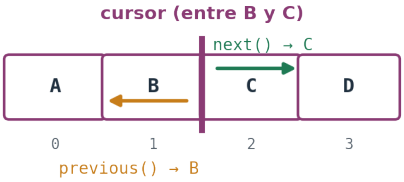
for (String s : l) {                      // por dentro usa un Iterator
    System.out.println(s);
}

Iterator<String> it = l.iterator();       // el mismo recorrido, explícito
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) {
        it.remove();                      // borra "b" SIN romper el recorrido
    }
}
```

Sobre una `LinkedList`, el iterador avanza de nodo en nodo ($O(1)$ por paso), y `it.remove()` re-engancha los punteros del nodo actual en $O(1)$: es **exactamente** el "saltear el nodo" de esta unidad, hecho por la lista misma.

5.4.3 `ListIterator`: el cursor de ida y vuelta

Las listas ofrecen además un iterador más poderoso, `ListIterator<T>`: un cursor que vive **entre** dos elementos y puede moverse en **ambos sentidos**, además de modificar la lista por donde pasa:



En una lista DOBLEMENTE enlazada, previous() es O(1): cada nodo conoce a su anterior.

Acciones donde está parado

`add(x)`	inserta en el cursor
`set(x)`	reemplaza el último
`remove()`	borra el último
`nextIndex()` / `previousIndex()`	

Figura 5.25: El `ListIterator` es un cursor que vive entre dos elementos: `next()` lo mueve a la derecha, `previous()` a la izquierda, y `add`/`set`/`remove` operan justo donde está parado.

```
ListIterator<String> lit = l.listIterator();
while (lit.hasNext()) {
    String s = lit.next();
    if (s.equals("a")) {
        lit.set("A");              // reemplaza el último devuelto
    }
    if (s.equals("c")) {
        lit.add("c2");             // inserta después, sin romper el recorrido
    }
}
while (lit.hasPrevious()) {        // y ahora... ¡marcha atrás!
    System.out.println(lit.previous());
}
```

La comparativa completa de qué permite cada cursor:

Acción	`Iterator`	`ListIterator`
Avanzar (`hasNext`/`next`)	sí	sí
Retroceder (`hasPrevious`/`previous`)	—	**sí**
Eliminar el actual (`remove`)	sí	sí
Reemplazar el actual (`set`)	—	**sí**
Insertar en la posición (`add`)	—	**sí**
Saber el índice (`nextIndex`/`previousIndex`)	—	**sí**
¿Quién lo ofrece?	toda colección	sólo las `List`

Ojo con `previous()` sobre una lista **simplemente** enlazada: los nodos no conocen a su anterior, así que retroceder obliga a re-recorrer desde el principio — carísimo. Por eso el `ListIterator` brilla en las **doblemente** enlazadas (y en los vectores, donde moverse es solo cambiar el índice).

5.4.4 Las listas de la API (y las que no están)

¿Cuáles de nuestras cuatro listas trae Java? Menos de las que pensás — y la que trae es la más completa. La declaración real de `LinkedList`:

```
public class LinkedList<E> extends AbstractSequentialList<E>
        implements List<E>, Deque<E>, Cloneable, java.io.Serializable {

    private static class Node<E> {
        E item;
        Node<E> next;
        Node<E> prev;      // ¡doblemente enlazada!
    }

    transient Node<E> first;
    transient Node<E> last;
    transient int size;
```

Es **nuestra lista doblemente enlazada**, nodo privado incluido — hasta los nombres coinciden (`first`/`last`/`size` = `primero`/`ultimo`/`tamanio`). Para las otras variantes:

Nuestra estructura	En la API	Si no está, ¿qué usar?
Lista doble	`LinkedList`	—
Lista simple	**no está**	`LinkedList` (paga una referencia extra y listo)
Lista circular	**no está** como clase	`ArrayDeque` (por dentro **es** un anillo, sobre arreglo)
Circular doble	**no está**	`LinkedList` + índice módulo `size()`, o una librería

Si de verdad necesitás semántica circular hecha y probada, la comunidad la tiene: **Apache Commons Collections** ofrece `CircularFifoQueue` (una cola circular de tamaño fijo que pisa lo más viejo) y **Guava** su equivalente `EvictingQueue`. Se agregan como dependencia en el `build.gradle`, igual que hicimos con Commons Lang en la Unidad 1:

```
implementation 'org.apache.commons:commons-collections4:4.5.0'
```

Idea clave

La moraleja profesional: la API cubre el 95% de los casos con `ArrayList`, `LinkedList` y `ArrayDeque`; para el resto, **primero buscá una librería madura** (Commons, Guava) antes de escribir tu estructura — y guardá el "a mano" para cuando el problema es de verdad especial (o para aprender, como acá).

5.5 Pila (LIFO)

Una **Pila** (stack) es una estructura lineal donde sólo se toca **un extremo**, el **tope**. Sigue la regla **LIFO** (*Last In, First Out*): el último que entra es el primero que sale, como una pila de platos. Sus operaciones son `apilar` (poner en el tope), `desapilar` (sacar del tope) y `tope` (mirar sin sacar).

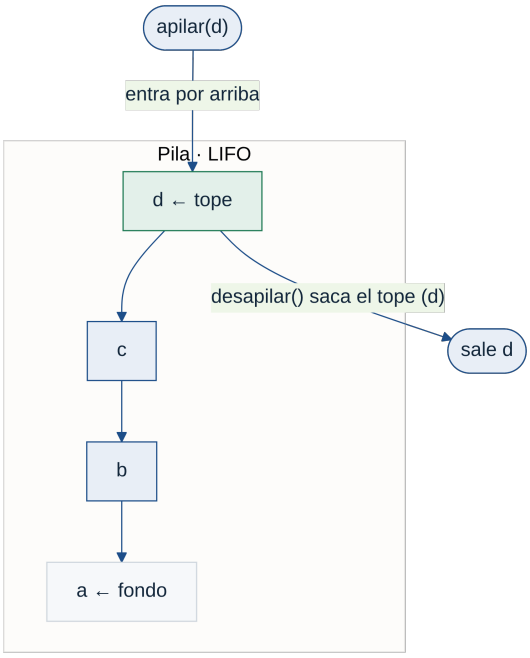


Figura 5.26: La pila crece y decrece por el tope: LIFO.

La construimos con **la técnica del vector**: un arreglo interno que se duplica al llenarse. Apilar y desapilar operan al final del arreglo, así que son $O(1)$ (amortizado). Su parte privada y sus tres operaciones caben en pocas líneas:

```java
public class Pila<T> {

    private T[] valores;      // el arreglo interno; el tope es valores[tamanio-1]
    private int tamanio;

    @SuppressWarnings("unchecked")
    public Pila() {
        this.valores = (T[]) new Object[4];
    }

    public void apilar(T x) {
        if (tamanio == valores.length) {
            valores = Arrays.copyOf(valores, valores.length * 2);
        }
        valores[tamanio++] = x;
    }

    public T desapilar() {
        T valor = valores[--tamanio];
        valores[tamanio] = null;   // no retener la referencia (Unidad 2)
        return valor;
    }

    public T tope() {
        return valores[tamanio - 1];
    }
}
```

Código de uso:

```java
Pila<String> p = new Pila<>();
p.apilar("a"); p.apilar("b"); p.apilar("c");   // tope = c
p.tope();          // c   (mira sin sacar)
p.desapilar();     // c   (LIFO: sale el último que entró)
p.desapilar();     // b
```

Idea clave

Casos de uso. Deshacer/rehacer (*undo*), el botón “atrás” del navegador, evaluar expresiones y **balancear paréntesis**, la **pila de llamadas** de la recursión (Unidad 4), y el recorrido **DFS** de un grafo (más adelante).

Error común

Limitaciones. Sólo se accede al **tope**: no hay acceso por índice ni recorrido arbitrario. Si necesitás mirar el elemento del medio, la pila no es la estructura.

En la API de Java: se recomienda `ArrayDeque` (con `push`/`pop`/`peek`); existe también `java.util.Stack`, pero es una clase **antigua** que conviene evitar.

5.6 Cola (FIFO)

Una **Cola** (queue) agrega por un extremo (el **fondo**) y saca por el otro (el **frente**). Sigue la regla **FIFO** (*First In, First Out*): el primero que entra es el primero que sale, como la fila del banco. Operaciones: `encolar`, `desencolar` y `frente`.

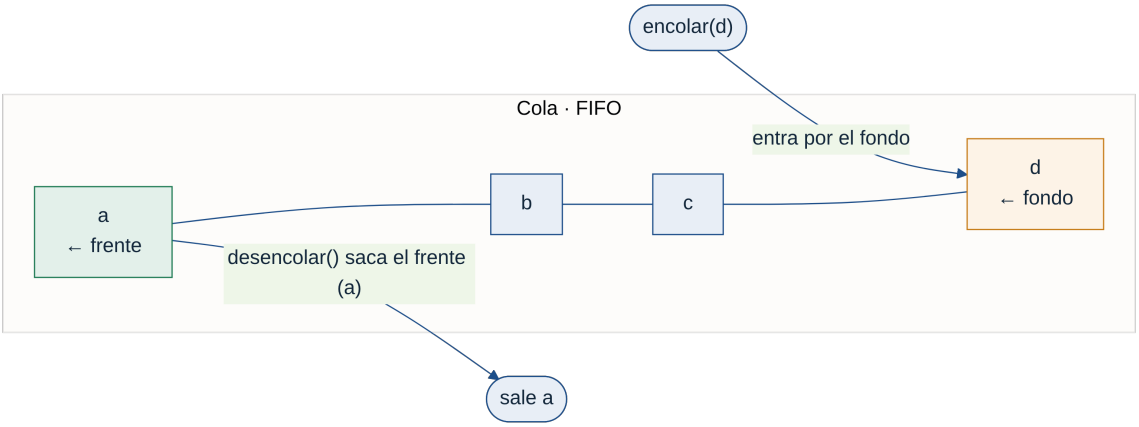


Figura 5.27: La cola entra por el fondo y sale por el frente: FIFO.

La construimos con **nodos enlazados** y punteros al `frente` y al `fondo`, para que encolar y desencolar sean $O(1)$. La parte privada es la de una lista enlazada mínima — el nodo privado y **dos** punteros, porque la cola trabaja por los dos extremos:

```
public class Cola<T> {

    private static final class Nodo<T> {
        T valor;
        Nodo<T> siguiente;
        Nodo(T valor) { this.valor = valor; }
    }

    private Nodo<T> frente;   // por acá SALEN (desencolar)
    private Nodo<T> fondo;    // por acá ENTRAN (encolar)
    private int tamanio;
}
```

Fijate por qué **no** la construimos sobre un arreglo como la pila: la pila opera en un solo extremo (el final del arreglo, barato), pero la cola saca **del frente** — y sacar del principio de un arreglo es $O(n)$ por el corrimiento. Con nodos y los dos punteros, ambos extremos son $O(1)$:

```
public void encolar(T x) {
    Nodo<T> n = new Nodo<>(x);
    if (fondo == null) {
        frente = fondo = n;
    } else { fondo.siguiente = n; fondo = n; }
    tamanio++;
}
public T desencolar() {
    T valor = frente.valor;
    frente = frente.siguiente;
    if (frente == null) {
        fondo = null;
    }
    tamanio--;
    return valor;
}
```

Código de uso:

```
Cola<String> c = new Cola<>();
c.encolar("a"); c.encolar("b"); c.encolar("c");   // frente = a
c.frente();        // a
c.desencolar();    // a   (FIFO: sale el primero que entró)
c.desencolar();    // b
```

Idea clave

Casos de uso. Atención por **orden de llegada** (una impresora, pedidos de un servidor), **buffers** de mensajes o streaming, el recorrido **BFS** de un grafo (más adelante), y la planificación *round-robin* de tareas.

Error común

Limitaciones. Sólo se ven el **frente** y el **fondo**: sin acceso por índice. Una variante, la **cola doble** (*deque*), permite agregar y quitar por **ambos** extremos.

En la API de Java: las interfaces son `Queue` y `Deque`; la implementación recomendada es `ArrayDeque` (también sirve `LinkedList`).

5.7 Comparación de complejidades

No hay una estructura "mejor": hay una mejor para cada uso.

Primero, las **secuencias** (las cuatro variantes de lista):

Operación	Vector	Lista simple	Lista doble	Circular (s/d)
Acceder por índice	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Agregar al final	$O(1)$ amort.	$O(1)$	$O(1)$	$O(1)$
Insertar/eliminar al principio	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Insertar/eliminar en el medio	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Buscar / `contiene`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Ordenar	$O(n \log n)$	$O(n \log n)$*	$O(n \log n)$*	—
Espacio extra	capacidad sin usar	1 ref/nodo	2 refs/nodo	1–2 refs/nodo

(*ordenar una lista **enlazada** se puede en $O(n \log n)$ con un Mergesort adaptado —lo vas a hacer en la guía de la Unidad 6—, pero con peores constantes que en el vector por la falta de acceso directo.)

Y los **TDAs de acceso restringido** (los que ganan quitando operaciones):

Operación	Conjunto (sobre lista)	Pila	Cola
Agregar / apilar / encolar	$O(n)$**	$O(1)$	$O(1)$
Quitar / desapilar / desencolar	$O(n)$	$O(1)$	$O(1)$
`contiene`	$O(n)$	—	—
Ver tope / frente	—	$O(1)$	$O(1)$
Unión / intersección / diferencia	$O(n \cdot m)$	—	—
Duplicados	no admite	admite	admite

(**el conjunto sobre lista paga $O(n)$ por chequear duplicados; con **hashing** —más adelante— agregar y `contiene` bajan a $O(1)$, y por eso `HashSet` es la implementación estrella. "—" = la operación no es parte de ese TDA.)

Idea clave

La regla de oro: **acceso por índice** → vector; muchas **inserciones/borrados en extremos** → lista enlazada; **pertenencia sin duplicados** → conjunto; recorridos **en ronda** → lista circular.

5.8 Cuadro resumen: fortalezas y debilidades

Las tablas de arriba dan los números; esta tabla da el **carácter** de cada estructura — sus puntos fuertes y sus debilidades, que es lo que de verdad se usa para elegir:

Estructura	Puntos fuertes	Desventajas
Vector / `ArrayList`	Acceso directo $O(1)$ por índice. Recorrido rapidísimo (memoria **contigua**, amiga de la caché del procesador). Se puede ordenar y hacer búsqueda binaria. Poca memoria extra por elemento.	Insertar/borrar al principio o en el medio corre todo ($O(n)$). Reserva **capacidad sin usar** (celdas vacías). La redimensión es una copia $O(n)$ puntual.
Lista enlazada	**No usa memoria contigua**: nunca redimensiona ni copia. **No ocupa memoria sin datos**: crece y decrece de a un nodo, justo lo necesario. Insertar/eliminar en los extremos (o donde ya estás parado) es $O(1)$.	**No tiene acceso directo**: llegar a la posición i es $O(n)$ — el **iterador** ayuda para un recorrido, pero no para saltar. El **recorrido inverso le cuesta** a la simple (la doble lo arregla pagando una referencia más por nodo). Memoria extra por las referencias, y peor uso de la caché (nodos dispersos).
Conjunto	Garantiza **sin duplicados** solo. Pertenencia como pregunta natural (`contiene`). Operaciones de conjuntos (unión, intersección, diferencia).	No tiene orden ni posición ("el elemento i" no existe). No sirve si hay que contar repetidos. Sobre lista es $O(n)$ por operación (el hashing de la Unidad 7 lo baja a $O(1)$).
Pila	`apilar`/`desapilar`/`tope` en $O(1)$. La disciplina LIFO **ordena el diseño**: imposible tocar lo que no es el tope.	Sólo se ve el tope: sin acceso por índice, sin recorrido arbitrario.
Cola	`encolar`/`desencolar` en $O(1)$. La disciplina FIFO garantiza **equidad**: se atiende por orden de llegada.	Sólo se ven frente y fondo. Sobre arreglo simple sería $O(n)$ (por eso se hace con nodos o con anillo).

Idea clave

Leé la tabla como un juego de **compromisos**: cada fortaleza se paga con una debilidad. Memoria contigua te da acceso directo pero te obliga a correr elementos; los nodos te liberan de la contigüidad pero te quitan el acceso directo. **No hay estructura sin costo: hay costos que tu problema tolera y costos que no.**

5.9 De lo nuestro a la API de Java

Todo lo que implementamos a mano, Java ya lo trae hecho y muy optimizado, en `java.util`. Entender el porqué —lo que hicimos acá— es lo que te permite **elegir bien**.

Lo que hicimos	En la API de Java	Interfaz
`Vector` propio	`ArrayList` (evitá `java.util.Vector`)	`List`
Listas enlazadas	`LinkedList`	`List` / `Deque`
`ConjuntoLista`	`HashSet`, `LinkedHashSet`, `TreeSet`	`Set`
`Pila`	`ArrayDeque` (o `Stack`, legada)	`Deque`
`Cola`	`ArrayDeque` (también `LinkedList`)	`Queue` / `Deque`

Idea clave

Importante para la cursada: en los **parciales, TPs y finales** se usan las **estructuras nativas de la API de Java**, no nuestras implementaciones. Las nuestras son para **entender el porqué**; las de Java son la herramienta profesional del día a día. Por eso el `Vector` propio vive **sólo en la guía** (difícil e01) — no está en `material`, para que no haya tentación —, y las demás implementaciones de referencia están en `material.i05_lineales`.

5.10 Ejemplo integrador: un sistema de turnos

Cerremos la unidad con un ejemplo que junta **las cuatro estructuras** —y, fiel a la regla de la materia, usando la **API nativa de Java**. Una mesa de ayuda atiende clientes: la **fila de espera** es una **Cola** (se atiende en orden de llegada, FIFO); los **DNI registrados** son un **Conjunto** (nadie saca dos turnos); el **historial de atendidos** es un **Vector** (orden de atención, acceso por índice); y las atenciones se pueden **deshacer** con una **Pila** (se deshace la última, LIFO).

```
private final Deque<Integer> filaDeEspera = new ArrayDeque<>();  // Cola  (FIFO)
private final Set<Integer>   registrados  = new HashSet<>();     // Conjunto
private final List<Integer>  atendidos    = new ArrayList<>();   // Vector
private final Deque<Integer> deshacer     = new ArrayDeque<>();  // Pila  (LIFO)

public boolean sacarTurno(int dni) {
    if (!registrados.add(dni)) {
        return false;   // el Conjunto rechaza duplicados
    }
    filaDeEspera.addLast(dni);               // encolar: al FONDO de la fila
    return true;
}

public int atender() {
    int dni = filaDeEspera.pollFirst();      // desencolar: del FRENTE (FIFO)
    atendidos.add(dni);                      // historial en orden
    deshacer.push(dni);                      // apilar para poder deshacer
    return dni;
}

public int deshacerUltimaAtencion() {
    int dni = deshacer.pop();                // el ÚLTIMO atendido (LIFO)
    atendidos.remove(atendidos.size() - 1);
    filaDeEspera.addFirst(dni);              // vuelve AL FRENTE de la fila
    return dni;
}
```

Tracemos una sesión y verifiquemos cada estructura en su papel:

```
mesa.sacarTurno(111);           // true
mesa.sacarTurno(222);           // true
mesa.sacarTurno(111);           // false: ya tenía turno (Conjunto)
mesa.sacarTurno(333);           // true
mesa.atender();                 // 111  (FIFO: el primero que llegó)
mesa.atender();                 // 222
mesa.historial();               // [111, 222]
mesa.deshacerUltimaAtencion();  // 222  (LIFO: el último atendido)
mesa.atender();                 // 222  (volvió al frente de la fila)
```

```
mesa.atender();                // 333
mesa.historial();              // [111, 222, 333]
```

Fijate el juego entre las dos disciplinas: la **Cola** decide *a quién* atender (el que más espera), la **Pila** decide *qué* deshacer (lo más reciente). El código completo, ejecutable, está en el repo: `material.i05_lineales`, clase `SistemaDeTurnos`.

5.11 El checklist del ejercicio de listas

Así como el diseño de un TDA se corrige contra el checklist de la Unidad 3, los ejercicios con estructuras (los de la guía de esta unidad, y los de parciales y TPs) se corrigen contra este checklist de **6 puntos**. Leelo antes de resolver, y repasalo antes de entregar.

1. Dividí y vencerás: partí el problema en subproblemas. Un enunciado casi nunca se resuelve con un solo método gigante. Preguntate: ¿qué subproblemas tiene esto? Cada subproblema se vuelve **un método con nombre propio**, y el método principal queda como un guion que los coordina. Si el enunciado dice "eliminar los alumnos con promedio menor a 4 y listar los restantes ordenados", ahí hay **tres** subproblemas: calcular un promedio, filtrar, ordenar — tres métodos, no uno.

2. Aplicá lo aprendido de TDA. Todo lo de la Unidad 3 sigue valiendo acá: encapsulá (los atributos privados), validá los parámetros, respetá el contrato de cada método (pre y post), null para "no hay valor" — y corregite contra el checklist de 20 puntos de la Unidad 3. Un ejercicio de listas es, casi siempre, un TDA que **contiene** una estructura: las reglas no cambian.

3. Cada método hace UNA sola cosa. Tres varas concretas para medirlo: **entra en una pantalla** (si hay que scrollear para leerlo, hace demasiado); su funcionalidad **se puede reutilizar** desde otro lado (si no se puede nombrar fácil qué hace, hace más de una cosa); y **resuelve orden n: no hay dos `for` anidados en un mismo método**. Si te aparecen dos `for` anidados, es la señal de que el `for` de adentro es un subproblema con nombre propio: extraelo a su propio método (que será $O(n)$), y el que llama queda legible —y el costo total queda igual, pero ahora **se ve**.

```
// MAL: dos fors anidados, el método hace dos cosas a la vez
public List<String> clientesConDeuda(List<Cliente> clientes, List<Factura> facturas) {
    List<String> resultado = new ArrayList<>();
    for (Cliente c : clientes) {
        for (Factura f : facturas) {              // <- subproblema escondido
            if (f.cliente().equals(c.nombre()) && !f.pagada()) {
                resultado.add(c.nombre());
                break;
            }
        }
    }
    return resultado;
}

// BIEN: el for interno era "tieneDeuda", un método con nombre, reutilizable
public List<String> clientesConDeuda(List<Cliente> clientes, List<Factura> facturas) {
    List<String> resultado = new ArrayList<>();
    for (Cliente c : clientes) {                        // O(n) acá...
        if (tieneDeuda(c, facturas)) {
            resultado.add(c.nombre());
        }
    }
    return resultado;
}

private boolean tieneDeuda(Cliente c, List<Factura> facturas) {
    for (Factura f : facturas) {                        // ...y O(m) acá
        if (f.cliente().equals(c.nombre()) && !f.pagada()) {
            return true;
        }
    }
```

```
    return false;
}
```

4. Usá los métodos de la estructura; no los reescribas. La API ya sabe hacer las operaciones básicas: usala. `contains` en vez de un `for` que compara; `indexOf` en vez de buscar a mano; `removeIf` en vez de recorrer borrando; `Collections.sort`/`list.sort` en vez de tu burbujeo; `isEmpty()` en vez de `size() == 0`; `addAll` en vez de un bucle de `add`. Reescribir lo que la estructura ya ofrece es más código, más bugs y menos claridad:

```
// MAL: reinventar contains
boolean esta = false;
for (String nombre : nombres) {
    if (nombre.equals(buscado)) {
        esta = true;
    }
}

// BIEN
boolean esta = nombres.contains(buscado);
```

5. Hacé una estrategia ANTES de escribir código. En papel o en un comentario: qué estructura uso y por qué (¿necesito índice? ¿pertenencia? ¿orden de llegada? — el cuadro de fortalezas y debilidades es para esto), qué métodos voy a tener, y qué hace cada uno en una frase. Codificar sin estrategia es como construir sin plano: se nota en el resultado. Si no podés contar tu estrategia en tres frases, todavía no entendiste el problema — volvé al punto 6.

6. Planteá los casos de uso y los resultados esperados. Antes de resolver, escribí ejemplos concretos de entrada → salida: el caso típico, la estructura **vacía**, un solo elemento, el elemento que no está, duplicados. Si no podés decir qué debería devolver el método para una entrada concreta, no entendiste el enunciado. Y estos casos no se tiran: son **exactamente tus tests JUnit** — el punto 6 hecho temprano es la mitad del test ya escrito.

Idea clave

El orden de uso en la práctica es: entender (6) → estrategia (5) → dividir (1) → codificar con TDA (2), métodos de una cosa (3) y la API (4). Los seis puntos se corrigen en los TPs y parciales igual que el checklist de TDA: un `for` anidado innecesario o un `contains` reescrito a mano cuestan puntos.

5.12 Errores comunes

Error común

- Confundir **tamaño** con **capacidad** en el vector: hay celdas libres que no son elementos.
- Olvidar poner en `null` la celda liberada al eliminar del vector (fuga de memoria, Unidad 2).
- Perder una referencia al re-enganchar una lista enlazada: la lista se "corta".
- Recorrer una lista **enlazada** con `for` + `obtener(i)`: es $O(n^2)$; usá el iterador o el for-each.
- Borrar de una lista **adentro de un for-each**: `ConcurrentModificationException`; la forma segura es `it.remove()` del `Iterator`.
- Usar `Collections.binarySearch` sobre una lista **desordenada**: devuelve cualquier cosa sin avisar.
- Usar `java.util.Vector` o `java.util.Stack` en código nuevo: son clases legadas; van `ArrayList` y `ArrayDeque`.
- Recorrer una lista **circular** esperando un `null`: hay que contar los elementos.
- Creer que la lista enlazada tiene acceso por índice $O(1)$: llegar a la posición i es $O(n)$.
- Usar un conjunto cuando necesitás contar **duplicados**, o una lista cuando necesitás **pertenencia** rápida.

- Usar nuestras implementaciones en los parciales/TP en vez de las de la API de Java.

5.13 Repaso

1. ¿Qué diferencia hay entre el **tamaño** y la **capacidad** de un vector? ¿Qué característica negativa implica?
2. ¿Por qué agregar al final de un vector es $O(1)$ **amortizado**?
3. ¿Cuál es la gran ventaja del vector (acceso directo) y cuál su gran limitación (insertar/borrar en el medio)?
4. En el `Vector<T>` propio, ¿por qué el constructor hace `(T[]) new Object[4]` en vez de `new T[4]`? ¿Por qué la capacidad no se guarda en un atributo aparte?
5. ¿En qué se parecen y en qué se diferencian nuestro `Vector`, `java.util.Vector` y `ArrayList`? ¿Cuál se usa hoy y por qué?
6. ¿Por qué `ArrayList`, `java.util.Vector` y `LinkedList` implementan la misma interfaz `List`? ¿Qué ventaja da declarar `List<String> l = new ArrayList<>()`?
7. Mostrá las tres formas de recorrer un `ArrayList`. ¿Cuál es la única forma segura de **borrar** mientras recorrés, y qué excepción aparece si lo hacés mal?
8. ¿Qué precondición tiene `Collections.binarySearch` y qué pasa si no se cumple?
9. ¿Qué caracteriza al TDA **Conjunto**? Definí unión, intersección y diferencia.
10. ¿Por qué una lista enlazada **no desperdicia espacio** como el vector, pero sí gasta memoria por nodo?
11. ¿Por qué recorrer una lista **enlazada** con `for` + `obtener(i)` es $O(n^2)$? ¿Qué lo resuelve?
12. ¿Qué puede hacer un `ListIterator` que un `Iterator` no? ¿Sobre qué lista conviene usar `previous()` y por qué?
13. ¿Qué ventaja aporta la lista **doblemente** enlazada? ¿Y qué costo extra? ¿Cuál de las listas de la API es doblemente enlazada?
14. ¿Para qué sirve una lista **circular** y una **circular doble**? Si la API no las trae, ¿qué alternativas hay?
15. Al insertar en una lista doble, ¿por qué hay que conectar **primero** el nodo nuevo y recién después re-apuntar los vecinos? ¿Qué se pierde si lo hacés al revés?
16. ¿Qué invariantes debe mantener una lista circular doble tras cada operación?
17. ¿Qué diferencia hay entre una **pila** (LIFO) y una **cola** (FIFO)? ¿Por qué la pila se construye bien sobre un arreglo y la cola no?
18. En el sistema de turnos del integrador, ¿por qué la fila es una Cola y el deshacer una Pila? ¿Qué estructura rechaza los turnos duplicados y con qué costo?
19. Elegí tres filas del cuadro de fortalezas y debilidades y explicá el **compromiso** que encierra cada una.
20. ¿Cuándo elegirías cada estructura? Asociá cada una con su clase de la API de Java.
21. Enunciá los 6 puntos del checklist del ejercicio de listas. ¿Qué señal da un método con dos `for` anidados y cómo se arregla?
22. ¿Por qué “plantear los casos de uso y resultados esperados” (punto 6) te deja los tests JUnit casi escritos?

Guía de ejercicios — Unidad 5

Objetivos de la unidad

Practicá las estructuras lineales. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i05_lineales` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. **Todo ejercicio se corrige contra el checklist del ejercicio de listas (6 puntos, al final de la unidad)**: casos de uso primero, estrategia antes de codificar, métodos de una sola cosa sin `for` anidados, y los métodos de la API en vez de reescribirlos. Las teóricas se responden en `guia/i05_lineales/i01_teorico`. Los niveles fácil y medio usan las **estructuras nativas de Java** (`ArrayList`, `LinkedList`, `HashSet`...); el nivel difícil las **implementa a mano** (arreglos y nodos), como en el material.

5.14 Preguntas teóricas

1. ¿Qué diferencia hay entre el tamaño y la capacidad de un vector dinámico?
2. ¿Por qué agregar al final de un vector es $O(1)$ amortizado? ¿Y si agrandáramos de a uno?
3. ¿Por qué insertar en el medio de un vector es $O(n)$ y en una lista enlazada (ubicado el lugar) es $O(1)$?
4. ¿Por qué acceder por índice es $O(1)$ en el vector y $O(n)$ en la lista enlazada?
5. ¿Qué ventaja aporta una lista doblemente enlazada frente a la simple?
6. ¿Para qué sirve una lista circular y qué cuidado hay que tener al recorrerla?
7. ¿Qué caracteriza al TDA Conjunto? ¿Por qué `agregar` es $O(n)$ en una implementación sobre lista?
8. Asociá cada estructura vista con su equivalente en la API de Java.
9. ¿Cuándo conviene un `ArrayList` y cuándo un `LinkedList`?
10. ¿Qué diferencia hay entre `HashSet`, `LinkedHashSet` y `TreeSet`?
11. ¿Por qué hoy se usa `ArrayList` y no `java.util.Vector`? ¿En qué se diferencian?
12. ¿Por qué recorrer una lista enlazada con `for` + `get(i)` es $O(n^2)$? ¿Cómo lo evita el iterador?
13. ¿Qué operaciones agrega `ListIterator` sobre `Iterator`? ¿Cuándo usarías cada uno?
14. ¿Cuál es la única forma segura de eliminar elementos mientras recorrés una lista? ¿Qué excepción aparece si lo hacés mal?

5.15 Nivel fácil (con la API de Java)

1. **Invertir.** `List<Integer> invertir(List<Integer> l)` que devuelva una nueva lista invertida. Test.
2. **Sin duplicados.** `List<Integer> sinDuplicados(List<Integer> l)` preservando el orden (usá `LinkedHashSet`). Test.
3. **Unión.** Unión de dos listas sin duplicados. Test.
4. **Palíndromo.** `boolean esPalindromo(List<Character> l)`. Test.
5. **Rotar.** Rotá una `List` `k` posiciones a la derecha (devolvé nueva). Test.
6. **Máximo.** `int maximo(List<Integer> l)`; excepción si está vacía. Test.
7. **Filtrar pares.** Devolvé una nueva lista con los pares. Test.
8. **Intersección.** Intersección de dos `Set<Integer>` (usá `retainAll` sobre una copia). Test.
9. **¿Contiene todos?** `boolean contieneTodos(List<Integer> l, List<Integer> req)`. Test.
10. **A arreglo y vuelta.** Convertí `List<Integer>` a `int[]` y de vuelta. Test.

5.16 Nivel medio (con la API de Java)

1. **Pila sobre ArrayList.** `Pila<T>` (`apilar`/`desapilar`/`tope`) usando `ArrayList` por dentro; test LIFO.

2. **Cola sobre LinkedList.** `Cola<T>` (`encolar`/`desencolar`/`frente`) usando `LinkedList`; test FIFO.
3. **Mezclar ordenadas.** Fusioná dos `List` ordenadas en una nueva ordenada. Test.
4. **Eliminar según condición.** Quitá de una `List` los múltiplos de 3 (`removeIf`). Test.
5. **Insertar ordenado.** Insertá un valor en una `List` ordenada manteniéndola ordenada. Test.
6. **¿Permutación?** `boolean esPermutacion(List<Integer> a, List<Integer> b)`. Test.
7. **Diferencia de conjuntos.** $A - B$ con `removeAll` sobre una copia. Test.
8. **¿Subconjunto?** `boolean esSubconjunto(Set<Integer> a, Set<Integer> b)`. Test.
9. **Mover el máximo al final.** Reordená una `List` dejando el máximo último. Test.
10. **Frecuencia de un elemento.** `int veces(List<Integer> l, int x)` (con `Collections.frequency` o a mano). Test.

5.17 Nivel difícil (implementadas a mano)

1. **TDA Vector propio.** `Vector<T>` con `private T[] valores` y redimensión por duplicación (`agregar`/`obtener`/`insertar`/`eliminar`/`contiene`/`indiceDe`/`ordenar`/`agregarTodos`/`tamanio`/`capacidad`); test que crece más allá de la capacidad inicial. Es la implementación de referencia del apunte, y vive acá **a propósito**: en la práctica se usa `ArrayList`.
2. **Lista simple.** `ListaSimplementeEnlazada<T>` (`agregar`/`insertar`/`eliminar`/`obtener`/`indiceDe`); test.
3. **Lista doble.** `ListaDoblementeEnlazada<T>` con `anterior`/`siguiente`; test (incluí eliminar del medio).
4. **Josephus.** Con una `ListaCircular<T>`, eliminá cada `k`-ésimo elemento hasta que quede uno; devolvé el sobreviviente. Test.
5. **Conjunto sobre lista.** `ConjuntoLista<T>` sin duplicados con `agregar`/`contiene`/`eliminar`/`union`; test.
6. **Pila enlazada.** `Pila<T>` sobre nodos (`Nodo` con `siguiente`); test LIFO.
7. **Cola enlazada.** `Cola<T>` sobre nodos con punteros a frente y fondo; test FIFO.
8. **Invertir enlazada.** Invertí una lista simplemente enlazada **in place** (re-enganchando); test.
9. **Nodo del medio.** Encontrá el nodo del medio de una lista enlazada con la técnica de los dos punteros (lento/rápido); test.
10. **Un TDA, dos implementaciones.** Interfaz `Lista<T>` con `VectorDinamico` y `ListaEnlazada`; un mismo test verifica que ambas se comportan igual.

Unidad 6

Recursividad y División y Conquista

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) escribir y razonar algoritmos **recursivos**, identificando el **caso base** y el **paso recursivo**, y clasificarlos por su tipo (**lineal, múltiple, indirecta y anidada**); (2) ver por qué el Fibonacci ingenuo es exponencial y cómo mejorarlo; (3) aplicar el esquema de **división y conquista** (dividir, resolver, combinar) y reconocerlo en **Mergesort** y **Quicksort**; (4) entender la multiplicación rápida de **Karatsuba**; y (5) ordenar **sin comparar** con **Counting**, **Radix** y **Bucket sort**, rompiendo el límite $O(n \log n)$.

En la Unidad 4 aprendimos a **medir** algoritmos recursivos con recurrencias y el Teorema Maestro. Ahora los vamos a **diseñar**: la recursión y la división y conquista son dos de las estrategias más potentes para resolver problemas, y están detrás de muchos de los algoritmos más importantes.

6.1 Un poco de historia

La recursión es una idea vieja de la matemática (las definiciones recursivas de Peano, por ejemplo). En algoritmos, **John von Neumann** describió el **Mergesort** en 1945. En 1959–1961, **Tony Hoare** inventó el **Quicksort**. Y en **1960**, un joven **Anatoly Karatsuba** sorprendió a todos: multiplicó dos números en menos de $O(n^2)$, **refutando** una conjetura de Kolmogorov que afirmaba que no se podía. Poco después, en 1969, **Volker Strassen** hizo lo mismo con la multiplicación de matrices. Estas ideas mostraron que **pensar distinto un problema** —partirlo y recombinarlo— puede cambiar radicalmente su costo.

6.2 Recursividad

Un algoritmo es **recursivo** cuando se resuelve en términos de sí mismo, sobre una entrada más chica. Todo algoritmo recursivo necesita dos partes:

- un **caso base**: una entrada tan simple que se resuelve sin recursión (y **corta** la cadena de llamadas);
- un **paso recursivo**: reduce el problema a uno más chico y se llama a sí mismo.

```java
public static long factorial(int n) {
    if (n <= 1) {
        return 1;   // caso base
    }
    return n * factorial(n - 1);      // paso recursivo
}
```

¿Y cómo ejecuta esto la máquina? Cada llamada recursiva se apila en la **pila de llamadas** (la *stack* de la Unidad 2). Sigamos `factorial(3)` paso a paso:

1. Llamo `factorial(3)`. Como $3 > 1$, necesita `factorial(2)`... pero para calcularlo, `factorial(3)` queda **esperando** en la pila.
2. Llamo `factorial(2)`. Necesita `factorial(1)`; también queda esperando.
3. Llamo `factorial(1)`. ¡**Caso base**! Devuelve `1` sin llamar a nadie. Acá se corta la cadena: la pila llegó a su punto más alto y empieza a **deshacerse**.
4. `factorial(2)` recibe ese `1`, calcula $2 \cdot 1 = 2$ y lo devuelve.
5. `factorial(3)` recibe el `2`, calcula $3 \cdot 2 = 6$ y devuelve el resultado final.

Fijate el patrón: primero se **abren** llamadas hasta tocar el caso base (bajada), y después los resultados se van **devolviendo** hacia atrás en orden inverso (subida). Ninguna multiplicación se hace “a la ida”: todas ocurren “a la vuelta”, cuando las llamadas se van cerrando.

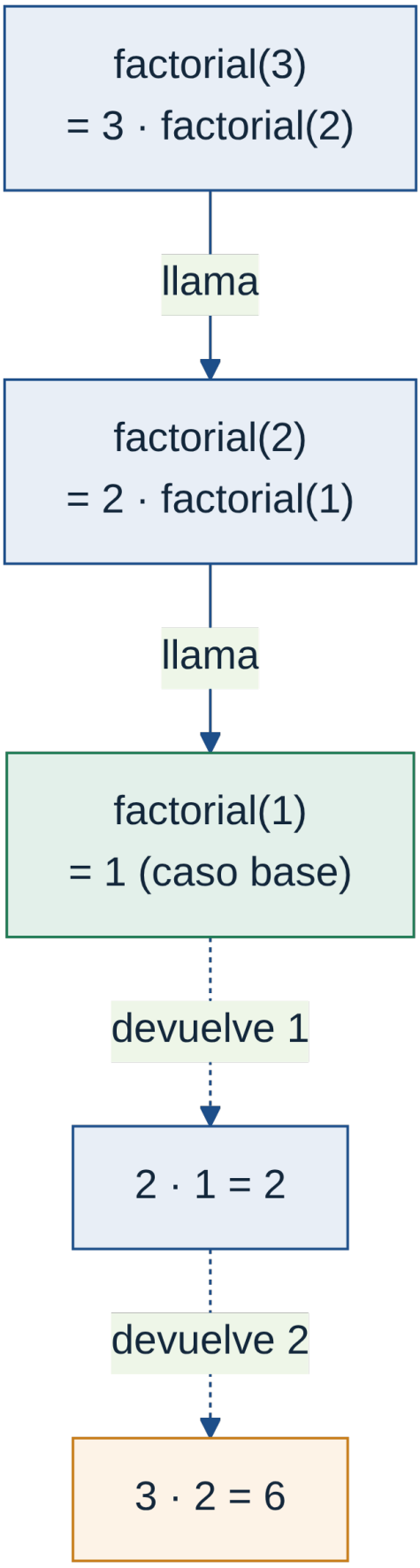


Figura 6.1: La pila de llamadas de `factorial(3)`: se abren llamadas hasta el caso base y luego se devuelven los resultados.

Error común

Cuidado con la recursión sin caso base (o que no achica el problema): las llamadas se apilan sin fin hasta agotar la pila y lanzar un `StackOverflowError`. Y ojo: a diferencia de otros lenguajes, la JVM **no** optimiza la recursión de cola (*tail call*), así que una recursión muy profunda puede desbordar la pila; cuando eso importa, conviene pasar a una versión **iterativa**.

6.3 Tipos de recursividad

No todas las recursiones tienen la misma forma. Según **cuántas** llamadas hace cada activación y **a quién** llama, se clasifican en cuatro tipos — y conviene reconocerlos, porque cada uno tiene su propia silueta en la pila y su propio costo. (El código de los tres nuevos está en el repo: `material.i06_estrategias.TiposDeRecursividad`.)

Tipo	Definición	Ejemplo
Lineal (o simple)	Cada activación hace **una sola** llamada recursiva.	`factorial(n)`
Múltiple	Cada activación hace **dos o más** llamadas recursivas.	`caminosEnGrilla`, Fibonacci, Mergesort
Indirecta (o mutua)	Un método **A llama a B**, y **B vuelve a llamar a A**: el ciclo recursivo pasa por dos (o más) métodos.	`esPar` / `esImpar`
Anidada	El **argumento** de la llamada recursiva es **otra llamada recursiva**: `f(f(...))`.	función 91 de McCarthy

Lineal es la que venimos usando: `factorial` hace exactamente una llamada por activación, así que la pila es una **fila india** de n marcos — por eso su costo es $T(n) = T(n-1) + O(1) = O(n)$ (Unidad 4).

Recursividad múltiple: dos o más llamadas. Cada activación abre **varias** ramas, así que las llamadas ya no forman una fila sino un **árbol**. Un ejemplo clásico: ¿de cuántas formas se puede ir de la esquina superior izquierda a la inferior derecha de una grilla, moviéndose sólo hacia la **derecha** o hacia **abajo**? A cada casilla se llega desde arriba **o** desde la izquierda, así que los caminos se **suman** — y eso es una llamada por cada opción:

```java
public static long caminosEnGrilla(int filas, int columnas) {
    if (filas == 1 || columnas == 1) {
        return 1;   // caso base: en un pasillo hay un solo camino
    }
    return caminosEnGrilla(filas - 1, columnas)     // vengo desde arriba
            + caminosEnGrilla(filas, columnas - 1); // o desde la izquierda
}
```

`caminosEnGrilla(3, 3)` devuelve 6. La silueta de la múltiple es el **árbol de llamadas**: con a llamadas por activación y n niveles, puede tener hasta a^n nodos ($O(2^n)$ acá) — es el Teorema Maestro por sustracción con $a > 1$ de la Unidad 4. La múltiple es la base de **divide y conquista** (Mergesort hace dos llamadas), con una diferencia crucial que vamos a ver enseguida: si las dos llamadas trabajan sobre **mitades distintas**, el árbol suma $O(n \log n)$; si trabajan sobre **casi lo mismo** (como acá, o como Fibonacci), los subproblemas se repiten y el árbol explota.

Recursividad indirecta (o mutua): el ciclo pasa por dos métodos. Ninguno de los dos se llama a sí mismo… pero `esPar` llama a `esImpar` y `esImpar` llama a `esPar`: la recursión existe igual, **rebotando** entre ambos. La idea: n es par si $n - 1$ es impar, y viceversa, hasta bajar al 0 (que es par por definición):

```java
public static boolean esPar(int n) {
    if (n == 0) {
        return true;    // caso base: el 0 es par
    }
    return esImpar(n - 1);   // n es par si n-1 es impar
}

public static boolean esImpar(int n) {
    if (n == 0) {
        return false;   // caso base: el 0 no es impar
    }
    return esPar(n - 1);     // n es impar si n-1 es par
}
```

`esPar(4)` llama a `esImpar(3)`, que llama a `esPar(2)`, que llama a `esImpar(1)`, que llama a `esPar(0)` → `true`. En la pila se **alternan** los dos métodos. Dos cuidados: el caso base puede estar en cualquiera de los dos (¡pero tiene que existir y ser alcanzable!), y para **leer** una recursión indirecta hay que mirar los métodos **en conjunto** — el ciclo A→B→A no se ve en ninguno por separado. (Este ejemplo es didáctico: en la práctica, la paridad es `n % 2 == 0`.)

Recursividad anidada: la llamada dentro de la llamada. El caso más vertiginoso: el **argumento** de la llamada recursiva es, a su vez, otra llamada recursiva. El ejemplo célebre es la **función 91 de McCarthy** (1970):

```java
public static int mcCarthy91(int n) {
    if (n > 100) {
        return n - 10;                      // caso base
    }
    return mcCarthy91(mcCarthy91(n + 11));   // ¡la llamada DENTRO de la llamada!
}
```

Para evaluar la línea recursiva hay que resolver **primero la llamada interna** (ella decide el argumento de la externa): con `n = 99`, la interna es `mcCarthy91(110)` → `100`, y recién entonces corre la externa, `mcCarthy91(100)`, que vuelve a anidar... hasta devolver `91`. Lo sorprendente —y por eso es famosa— es que devuelve **91 para todo** $n \leq 100$. Probalo en el debugger con un breakpoint: vas a ver la pila crecer y encogerse en oleadas, porque la profundidad de una anidada **no se ve a simple vista** en el código. La anidada más famosa es la **función de Ackermann**, $A(m, n)$, que crece tan brutalmente rápido que se usa para demostrar límites teóricos de la recursión ($A(4, 2)$ ya tiene 19.729 dígitos).

Idea clave

Para clasificar una recursión, hacete dos preguntas: **¿cuántas llamadas hace cada activación?** (una: lineal; varias: múltiple) y **¿quién llama a quién?** (a sí mismo: directa; a otro que me vuelve a llamar: indirecta; con la llamada como argumento de otra llamada: anidada). Y la pregunta de rendimiento de siempre: ¿los subproblemas son **distintos** o **se repiten**?

6.4 Cuidado con los subproblemas repetidos: Fibonacci

No toda recursión es eficiente. La sucesión de Fibonacci ($0, 1, 1, 2, 3, 5, 8, 13, \ldots$; cada término es la suma de los dos anteriores) tiene una definición recursiva tan natural que la traducción a código es directa:

```java
public static long fibIngenuo(int n) {
    if (n <= 1) {
        return n;   // casos base: fib(0)=0, fib(1)=1
    }
    return fibIngenuo(n - 1) + fibIngenuo(n - 2);  // dos llamadas recursivas
}
```

Elegante... y **desastrosa**. Mirá el árbol de llamadas de `fib(5)` con el valor que devuelve cada nodo:

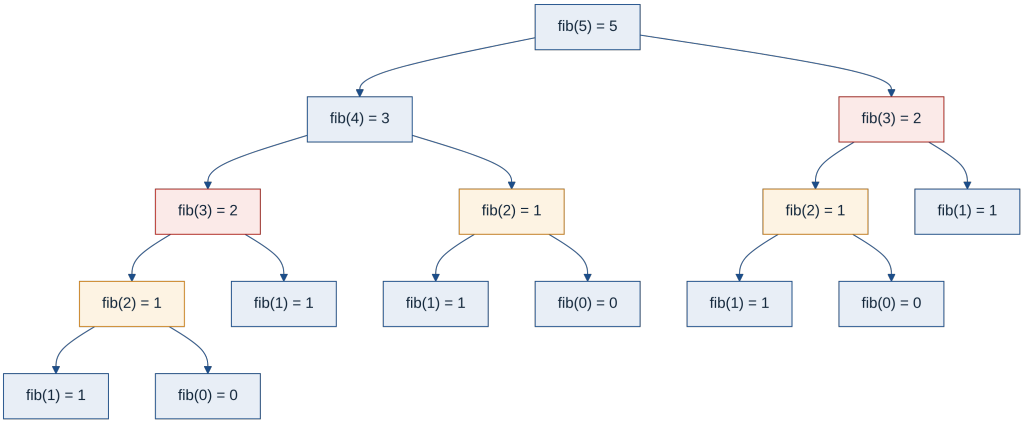


Figura 6.2: Árbol de llamadas de `fib(5)` con sus valores de retorno: `fib(3)` se calcula 2 veces (rojo) y `fib(2)` 3 veces (naranja). Cada subárbol repetido es trabajo hecho de nuevo.

`fib(3)` se calcula **dos veces** completas —con todo su subárbol— y `fib(2)`, **tres**. Y esto empeora al crecer n: para `fib(50)`, los subárboles repetidos se multiplican hasta hacer **miles de millones** de llamadas. La recurrencia $T(n) = T(n-1) + T(n-2) + O(1)$ crece como los propios números de Fibonacci: exponencial, del orden de $O(2^n)$. Impracticable.

La solución es **no repetir trabajo**. La versión iterativa arranca de los casos base y avanza hacia adelante, guardando sólo los dos últimos valores:

```java
public static long fib(int n) {
    if (n <= 1) {
        return n;
    }
    long anterior = 0, actual = 1;
    for (int i = 2; i <= n; i++) {
        long siguiente = anterior + actual;   // cada valor se calcula UNA vez
        anterior = actual;
        actual = siguiente;
    }
    return actual;
}
```

Cada valor se calcula **una sola vez**: $O(n)$ de tiempo y $O(1)$ de memoria. `fib(50)`, que a la versión ingenua le lleva minutos, sale al instante. Es el mismo problema que vimos en la Unidad 4; acá vemos gráficamente **por qué** pasa.

Idea clave

Recursión sí, pero con ojo. La recursión brilla cuando los subproblemas son **distintos** (como en Mergesort: cada mitad es diferente). Cuando los subproblemas se **repiten** (Fibonacci), la recursión ingenua explota, y conviene iterar o guardar los resultados ya calculados (*memoización*, que van a ver a fondo en programación dinámica).

6.5 División y conquista

La **división y conquista** (*divide and conquer*) es una forma estructurada de recursión, en **tres pasos**:

1. **Dividir** el problema en subproblemas más chicos del mismo tipo.
2. **Resolver** cada subproblema recursivamente (hasta un caso base).
3. **Combinar** las soluciones parciales en la solución del problema original.

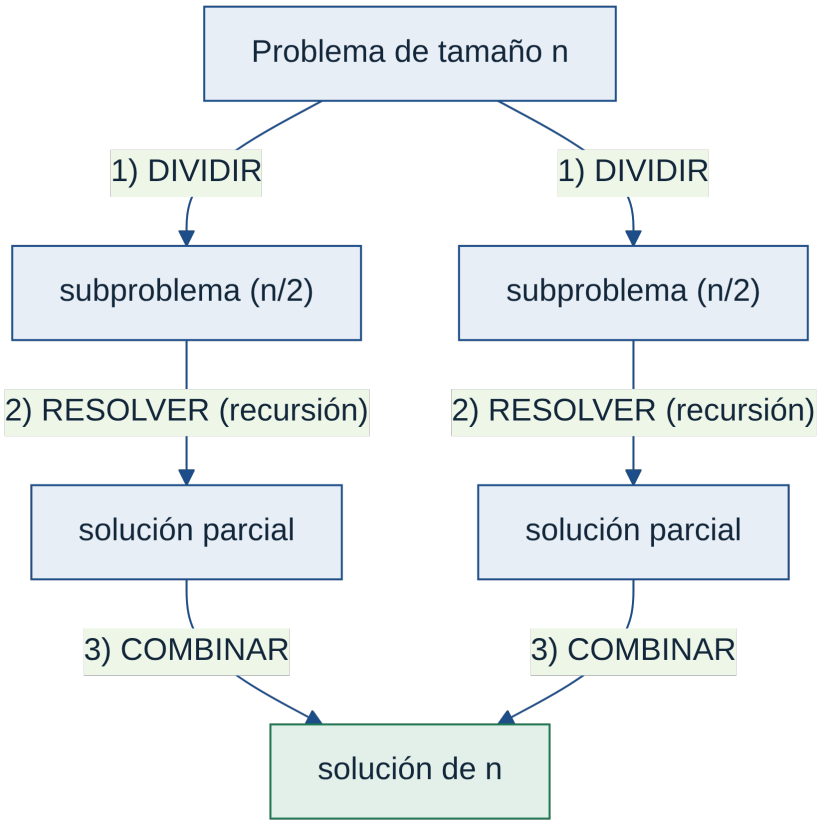


Figura 6.3: El esquema de división y conquista: dividir, resolver y combinar.

El costo de un algoritmo así se describe con una recurrencia $T(n) = a\,T(n/b) + f(n)$, que resolvemos con el **Teorema Maestro** (Unidad 4). Veamos los dos ejemplos más importantes.

6.6 Mergesort

Mergesort aplica la división y conquista al problema de ordenar. La idea, en una frase: *si sé ordenar dos mitades y sé fusionarlas, sé ordenar todo*. Los tres pasos son:

1. **Dividir**: partir el arreglo en dos mitades (izquierda y derecha).
2. **Resolver**: ordenar cada mitad **recursivamente**. El caso base es un arreglo de un solo elemento (o vacío), que ya está ordenado y corta la recursión.
3. **Combinar**: **fusionar** (*merge*) las dos mitades —que ya vienen ordenadas— en un único arreglo ordenado.

La siguiente figura muestra el esquema completo: se baja dividiendo hasta arreglos de un elemento, y se sube fusionando de a pares.

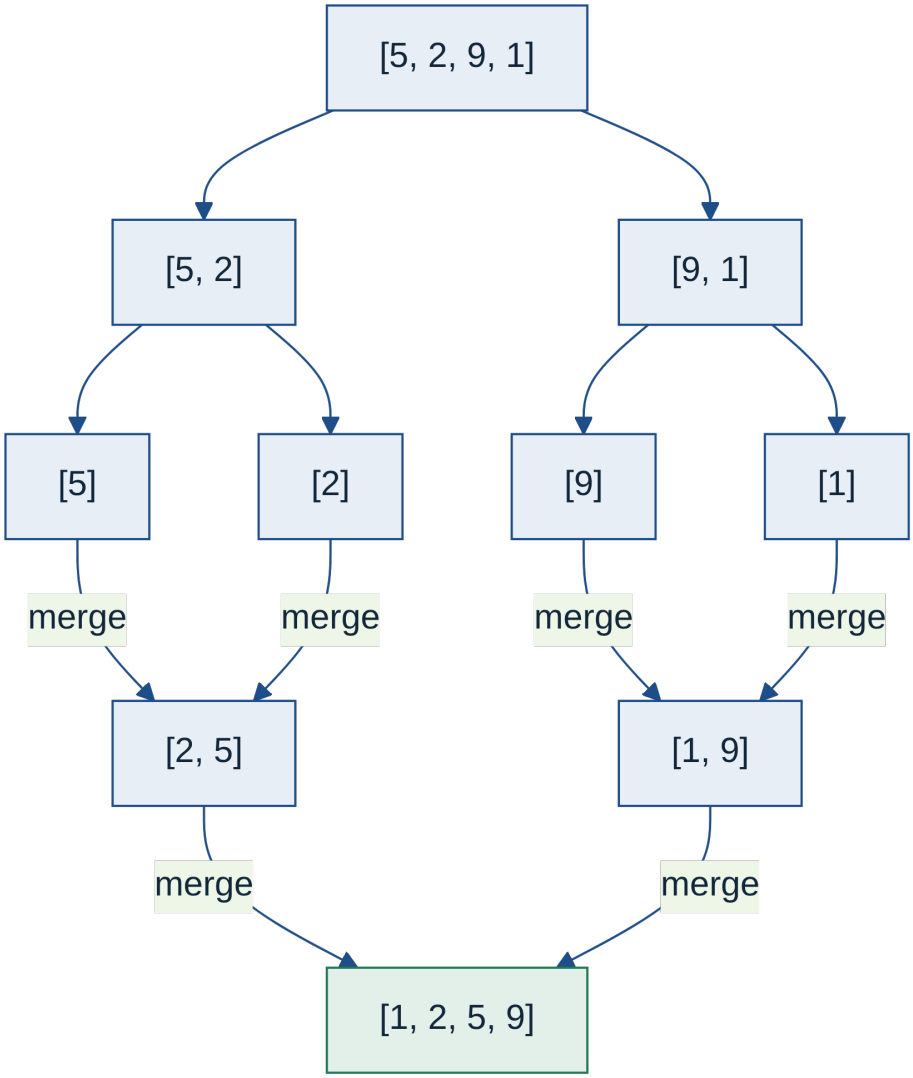


Figura 6.4: Mergesort: se divide hasta llegar a arreglos de un elemento y luego se fusiona hacia arriba.

6.6.1 La clave: fusionar dos mitades ordenadas

Todo el trabajo de Mergesort está en el paso de **fusión**. La pregunta es: si tengo dos arreglos **ya ordenados**, ¿cómo los combino en uno solo ordenado sin perder el orden? La respuesta es elegante: pongo un **puntero** al principio de cada mitad, comparo los dos elementos apuntados, **copio el menor** a la salida y avanzo **sólo** ese puntero. Repito hasta agotar una mitad; entonces copio lo que quede de la otra (que ya está ordenado). Como cada comparación copia un elemento, fusionar dos mitades que suman n elementos cuesta $O(n)$.

Veámoslo paso a paso fusionando la mitad izquierda $[3, 27, 38]$ con la derecha $[9, 10, 82]$:



Figura 6.5: Fusión paso a paso: en cada fila se comparan los elementos apuntados, se copia el menor a la salida y se avanza ese puntero. La celda resaltada es el puntero de cada mitad; en la salida se resalta el elemento recién agregado.

Fijate cómo la salida va quedando ordenada sola: nunca hace falta "reacomodar", porque en cada paso elegimos el menor de los dos candidatos. Cuando una mitad se vacía (acá, la izquierda tras copiar el 38), el resto de la otra se copia de un tirón.

6.6.2 Complejidad y propiedades

Como dividir es trivial y **fusionar** cuesta $O(n)$, y en cada nivel se parte por la mitad, la recurrencia es $T(n) = 2\,T(n/2) + O(n)$. Por el Teorema Maestro (Unidad 4, Caso 2) da $O(n \log n)$. Y es $O(n \log n)$ **siempre** —mejor, promedio y peor caso— porque la partición es siempre en dos mitades iguales y la fusión siempre cuesta lineal, sin importar cómo venían ordenados los datos. Su código completo está en el repositorio, en `material.i04_complejidad`.

Idea clave

Mergesort es **estable** (no altera el orden relativo de elementos iguales) y su peor caso es $O(n \log n)$ **garantizado**, pero necesita memoria extra $O(n)$ para fusionar. Es la base del ordenamiento de objetos en Java: `Arrays.sort` para objetos usa **TimSort**, una variante de Mergesort.

6.7 Quicksort

Quicksort también divide y vence, pero reparte el trabajo al revés que Mergesort. Mientras Mergesort divide fácil y **fusiona** con esfuerzo, Quicksort **divide con esfuerzo** (particionando) y combina gratis. Los pasos son:

1. **Elegir un pivote**: un elemento cualquiera del arreglo (por ejemplo, el último).
2. **Particionar**: recorrer el arreglo y separarlo en dos grupos, los **menores** que el pivote y los **mayores**. Al terminar, el pivote queda en su **posición final** (todos los menores a su izquierda, todos los mayores a su derecha).
3. **Resolver**: ordenar recursivamente el grupo de menores y el de mayores. La **combinación es trivial**: como el pivote ya está en su lugar y cada lado se ordena solo, no hay nada que fusionar.



Figura 6.6: Quicksort: se elige un pivote (acá el 5), se separan menores y mayores, el pivote queda en su lugar definitivo, y se ordena cada lado recursivamente.

6.7.1 ¿Por qué el peor caso es $O(n^2)$?

Todo depende de **cómo parta el pivote**. Si en promedio cae cerca del medio, cada partición divide el arreglo en dos mitades parejas: $T(n) = 2\,T(n/2) + O(n) = O(n \log n)$. Pero si el pivote cae siempre en un **extremo** (por ejemplo, elegir el último elemento sobre un arreglo **ya ordenado**), la partición queda muy desbalanceada —1 de un lado y $n-1$ del otro—: $T(n) = T(n-1) + O(n) = O(n^2)$. En la práctica se evita eligiendo el pivote con criterio (al azar, o la mediana de tres), lo que hace el peor caso extremadamente improbable.

Aun con ese peor caso teórico, Quicksort suele ser el **más rápido en la práctica** por sus constantes pequeñas y porque ordena *in place* (sin memoria extra). Su código está en `material.i04_complejidad`.

6.8 Multiplicación rápida: Karatsuba

Multiplicar dos números de n dígitos "a la escuela" —cada dígito de uno por cada dígito del otro— cuesta $O(n^2)$. Durante décadas se creyó que no se podía hacer mejor; Karatsuba demostró lo contrario con división y conquista.

La idea, paso a paso. Parto cada número en dos mitades. Con números de dos dígitos y base 10: $x = a \cdot 10 + b$ e $y = c \cdot 10 + d$. El producto se expande como:

$$x \cdot y = ac \cdot 100 + (ad + bc) \cdot 10 + bd$$

Esto pide, en principio, **cuatro** multiplicaciones de números de la mitad de tamaño: ac, ad, bc y bd. El truco de Karatsuba: el término del medio $ad + bc$ se puede obtener **sin** calcular ad ni bc por separado, usando la identidad

$$ad + bc = (a + b)(c + d) - ac - bd,$$

porque $(a+b)(c+d) = ac + ad + bc + bd$, y ac y bd **ya los tengo**. Quedan **tres** multiplicaciones: ac, bd y $(a+b)(c+d)$, más sumas y restas (baratas, lineales).

Ejemplo con 34×56. Parto $34 = 3 \cdot 10 + 4$ (o sea $a = 3$, $b = 4$) y $56 = 5 \cdot 10 + 6$ ($c = 5$, $d = 6$). Las tres multiplicaciones: $ac = 3 \cdot 5 = 15$; $bd = 4 \cdot 6 = 24$; y $(a+b)(c+d) = 7 \cdot 11 = 77$. El término del medio sale restando: $77 - 15 - 24 = 38$ (comprobalo: $ad + bc = 18 + 20 = 38$, ¡sin haberlos multiplicado!). Recombino:

$$34 \times 56 = 15 \cdot 100 + 38 \cdot 10 + 24 = 1500 + 380 + 24 = 1904. \checkmark$$

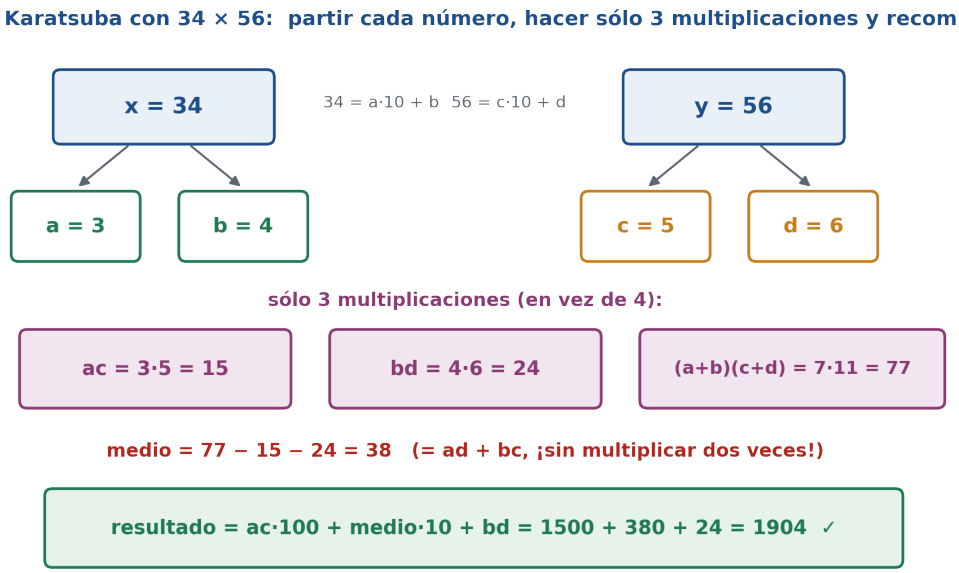


Figura 6.7: Karatsuba con 34 × 56: se parte cada número en dos mitades, se hacen sólo tres multiplicaciones chicas, y el término del medio se obtiene restando.

Con dos dígitos el ahorro parece menor (3 multiplicaciones en vez de 4), pero el esquema es **recursivo**: cada una de esas 3 multiplicaciones se resuelve a su vez con Karatsuba, y el ahorro se **multiplica en cada nivel**. Para números grandes se trabaja en binario ($x = a \cdot 2^m + b$) y la implementación usa `BigInteger`:

```
public static BigInteger multiplicar(BigInteger x, BigInteger y) {
    if (x.bitLength() <= 32 || y.bitLength() <= 32) {
        return x.multiply(y);                       // caso base: numeros chicos
    }
    int m = Math.max(x.bitLength(), y.bitLength()) / 2;
    BigInteger potencia = BigInteger.TWO.pow(m);
    BigInteger b = x.mod(potencia), a = x.divide(potencia);   // x = a·2^m + b
    BigInteger d = y.mod(potencia), c = y.divide(potencia);   // y = c·2^m + d

    BigInteger ac = multiplicar(a, c);
    BigInteger bd = multiplicar(b, d);
    BigInteger abcd = multiplicar(a.add(b), c.add(d));   // la 3.ª multiplicación
    BigInteger medio = abcd.subtract(ac).subtract(bd);   // = ad + bc

    // resultado = ac·2^(2m) + (ad+bc)·2^m + bd
    return ac.shiftLeft(2 * m).add(medio.shiftLeft(m)).add(bd);
}
```

La recurrencia baja de $4\,T(n/2) + O(n)$ a $T(n) = 3\,T(n/2) + O(n)$, que por el Teorema Maestro (Caso 1) da $O(n^{\log_2 3}) \approx O(n^{1.585})$: para $n = 1000$ dígitos, unas 60.000 operaciones contra el millón del método escolar. Con la misma idea, **Strassen** multiplica matrices con 7 (en vez de 8) multiplicaciones de bloques, logrando $O(n^{\log_2 7}) \approx O(n^{2.807})$ en lugar de $O(n^3)$. En el repo, `Karatsuba` compara su resultado con la multiplicación directa.

6.9 Ordenar sin comparar

Hasta acá, todos los ordenamientos que vimos —Mergesort, Quicksort, y también los de la Unidad 5— se basan en **comparar** elementos de a pares: preguntan "¿es $a < b$?" y deciden a partir de la respuesta. Hay un resultado sorprendente sobre todos ellos: **ningún ordenamiento por comparación puede bajar de** $O(n \log n)$ en el peor caso.

Definición

¿Por qué el límite $O(n \log n)$? Un algoritmo que sólo compara está recorriendo un **árbol de decisión**: cada comparación tiene dos resultados posibles y lleva a una rama distinta. Para ordenar n elementos hay que poder distinguir entre sus $n!$ ordenamientos posibles, así que el árbol necesita al menos $n!$ hojas. Un árbol binario con $n!$ hojas tiene altura mínima $\log_2(n!)$, y por la fórmula de Stirling $\log_2(n!) = \Theta(n \log n)$. Esa altura es la **cantidad de comparaciones en el peor caso**. Conclusión: comparando, no se puede hacer mejor que $n \log n$.

Entonces, ¿cómo ordenar más rápido? **No comparando.** Si en vez de comparar usamos el **valor** de cada elemento directamente —por ejemplo, como índice de un arreglo—, podemos esquivar el límite y ordenar en tiempo **lineal**. Ésa es la idea de los **ordenamientos no comparativos**: Counting, Radix y Bucket sort. La contra: dejan de ser algoritmos "para cualquier cosa" y pasan a exigir claves con estructura (enteros de rango acotado, dígitos, valores en un intervalo).

6.9.1 Counting sort

Counting sort ordena **contando**. La idea es tan simple como potente: si sé que todos los valores están en el rango $[0, k]$, uso un arreglo auxiliar `conteo` de tamaño $k + 1$ donde `conteo[v]` guarda **cuántas veces aparece el valor** `v`. Después recorro ese arreglo en orden y reconstruyo la salida. En ningún momento comparo dos elementos entre sí: uso el valor **como índice**.

Sigamos un ejemplo con la entrada $[4, 2, 2, 8, 3, 3, 1]$ (valores en $[0, 8]$):

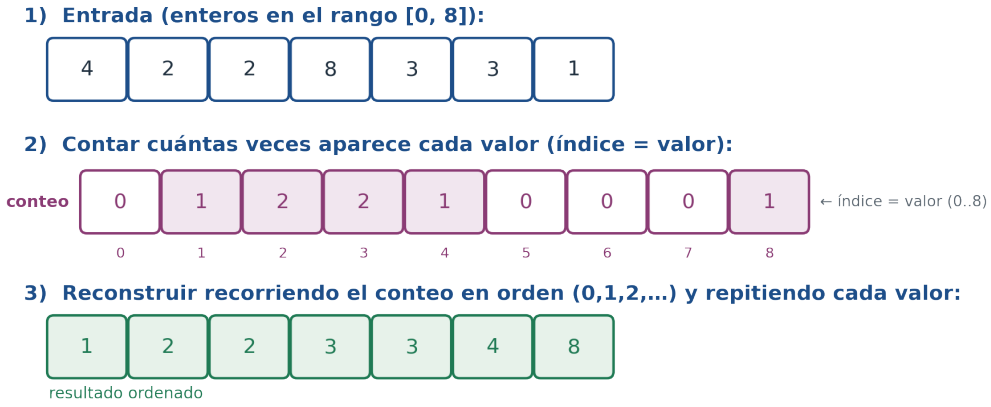


Figura 6.8: Counting sort paso a paso: (1) la entrada; (2) el arreglo de conteo, donde el índice es el valor y el contenido es la cantidad de apariciones; (3) la reconstrucción, recorriendo el conteo de 0 hacia arriba y escribiendo cada valor tantas veces como se contó.

Leámoslo: el 2 aparece dos veces, así que `conteo[2] = 2`; el 5, el 6 y el 7 no aparecen, así que su conteo es 0. Para reconstruir, recorro `conteo` de izquierda a derecha: escribo un `1`, dos `2`, dos `3`, un `4` y un `8`. El resultado sale ordenado sin una sola comparación.

```java
public static int[] ordenar(int[] a, int maximo) {
    int[] conteo = new int[maximo + 1];
    for (int x : a) {
        conteo[x]++;   // contar
    }
    int[] resultado = new int[a.length];
    int i = 0;
    for (int valor = 0; valor <= maximo; valor++)
        while (conteo[valor]-- > 0) {
            resultado[i++] = valor;   // reconstruir
        }
    return resultado;
```

```
}
```

Su costo es $O(n + k)$: $O(n)$ para contar y reconstruir, más $O(k)$ para recorrer el rango. Si k (el rango) no es mucho más grande que n, es **lineal** —más rápido que cualquier ordenamiento por comparación—. Su límite: sólo sirve para enteros de rango acotado, y gasta memoria proporcional a k (ordenar diez números que van de 0 a mil millones sería un disparate: reservaría mil millones de celdas).

6.9.2 Radix sort

¿Y si los números son grandes y su rango es enorme? Contar con un arreglo de tamaño k deja de servir. **Radix sort** resuelve esto ordenando por **dígitos** en vez de por el número entero. Procesa un dígito por pasada, **del menos significativo al más significativo** (unidades, después decenas, después centenas…), y en cada pasada usa un Counting sort **estable** —uno que, ante dos números con el mismo dígito, respeta el orden en que venían—.

La estabilidad es **la clave** de que funcione: cuando ordeno por las decenas, necesito que los que ya quedaron bien ordenados por las unidades **no se desordenen** entre los que empatan en la decena. Veamos las tres pasadas sobre $[170, 45, 75, 90, 2, 802, 24, 66]$:

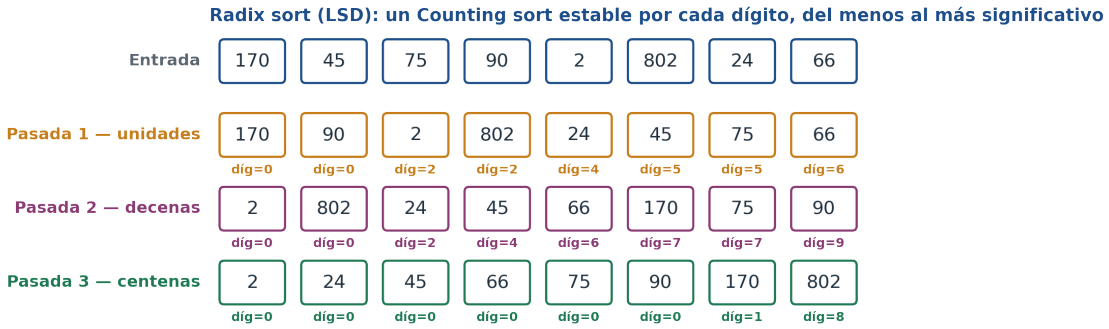


Figura 6.9: Radix sort: en cada pasada se ordena de forma estable por un dígito (primero unidades, luego decenas, luego centenas). Debajo de cada número se marca el dígito por el que se está ordenando en esa pasada. Tras la tercera pasada, el arreglo queda ordenado.

Mirá la Pasada 1: quedan adelante los que terminan en 0 (170, 90), después el que termina en 2 (2, 802)… ordenados por su **última** cifra. En la Pasada 2 se reordena por la decena, y en la Pasada 3 por la centena. Al procesar la cifra más significativa al final, ésta "manda" en el orden global, y las anteriores ya dejaron todo bien desempatado. Con d dígitos y base b, el costo es $O(d\,(n + b))$; para números de tamaño acotado, prácticamente $O(n)$.

6.9.3 Bucket sort

Bucket sort (ordenamiento por baldes) es ideal cuando los datos están **repartidos de forma pareja** en un intervalo conocido —el caso típico son reales en $[0, 1)$—. La idea: repartirlos en varios **baldes** (cada balde cubre un subrango del intervalo), ordenar cada balde por separado —como cada uno tendrá pocos elementos, ordenarlo es barato— y después **concatenar** los baldes en orden. Son **tres pasos**, y los vamos a seguir de punta a punta sobre un mismo ejemplo.

El vector inicial. Seis reales en $[0, 1)$, desordenados:

```
double[] a = {0.42, 0.26, 0.72, 0.17, 0.68, 0.09};    // n = 6
```

Como $n = 6$, usamos **6 baldes** (uno por elemento), y cada balde cubre un subrango de ancho $1/6 \approx 0.167$: el balde 0 recibe los valores de $[0, 0.167)$, el balde 1 los de $[0.167, 0.333)$, ... y el balde 5 los de $[0.833, 1)$.

Paso 1 — repartir. ¿A qué balde va cada valor? Al número $\lfloor n \cdot x \rfloor$: multiplicar por n "estira" el valor de $[0, 1)$ a $[0, n)$, y el piso lo convierte en un índice entero entre 0 y $n-1$. La cuenta completa, valor por valor:

Posición	Valor x	$6 \cdot x$	Balde $\lfloor 6x \rfloor$
`a[0]`	0.42	2.52	**2**
`a[1]`	0.26	1.56	**1**
`a[2]`	0.72	4.32	**4**
`a[3]`	0.17	1.02	**1**
`a[4]`	0.68	4.08	**4**
`a[5]`	0.09	0.54	**0**

Los valores chicos caen en los primeros baldes y los grandes en los últimos. Algunos baldes reciben dos valores (el 1 y el 4), otros quedan vacíos (el 3 y el 5) — está bien, es lo esperable.

Paso 2 — ordenar cada balde. Cada balde tiene a lo sumo un par de elementos, así que ordenarlo es barato (con inserción, o con el `sort` de la API). Acá sólo hay trabajo en dos: el balde 1 ($[0.26, 0.17] \to [0.17, 0.26]$) y el balde 4 ($[0.72, 0.68] \to [0.68, 0.72]$).

Paso 3 — concatenar. Se recorren los baldes **en orden** (0, 1, ..., 5) y se pegan sus contenidos uno tras otro. ¿Por qué queda ordenado? Por dos garantías que se suman: **entre** baldes, todo lo del balde i es menor que todo lo del balde $i + 1$ (lo garantiza el reparto por subrangos); y **dentro** de cada balde, los elementos quedaron ordenados en el Paso 2.

La película completa, con el ejemplo entero:

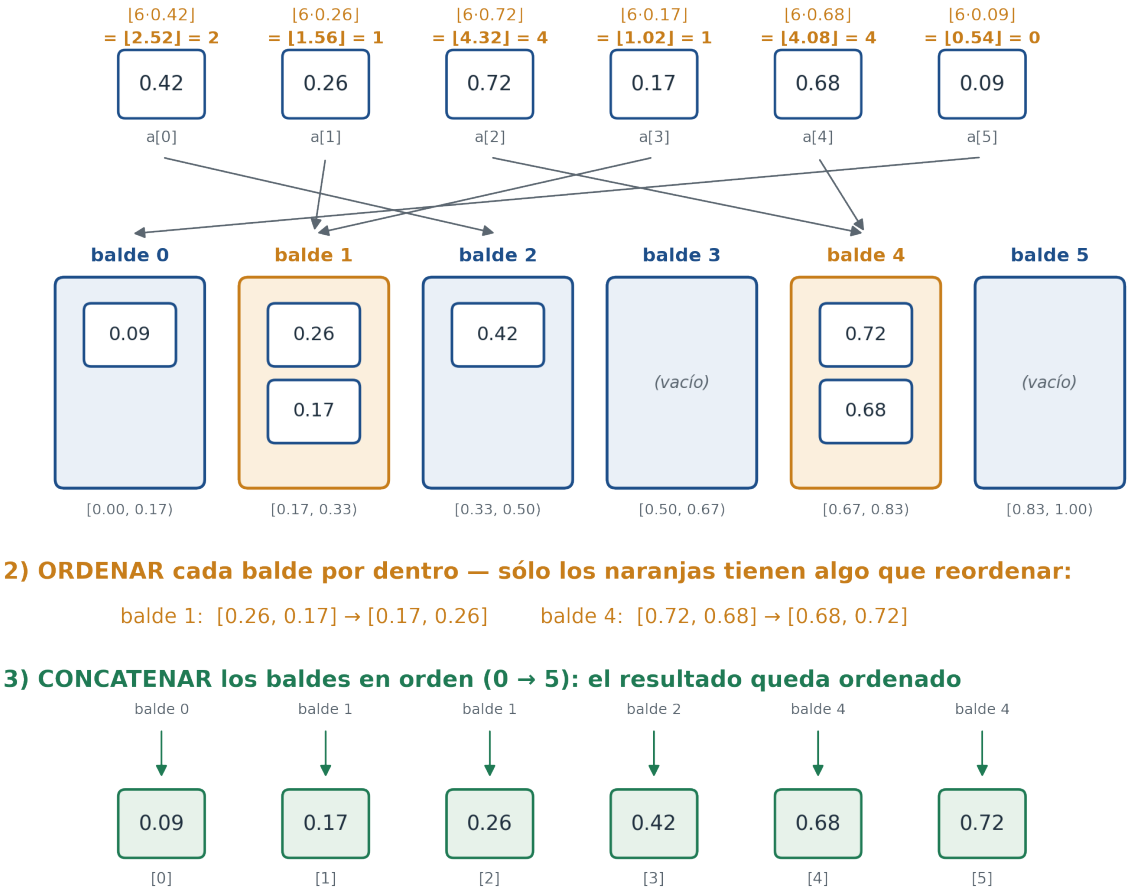


Figura 6.10: Bucket sort de punta a punta sobre `a = [0.42, 0.26, 0.72, 0.17, 0.68, 0.09]`: (1) cada valor va al balde ⌊6·x⌋ —la cuenta está arriba de cada celda—; (2) se ordenan por dentro los baldes que lo necesitan; (3) se concatenan los baldes en orden y el resultado queda ordenado.

Y el algoritmo completo en código, un bloque por paso (está en el repo, `material.i06_estrategias.BucketSort`):

```
public static double[] ordenar(double[] a) {
    int n = a.length;

    // Paso 1 — repartir: n baldes vacíos, y cada valor a su balde.
    List<List<Double>> baldes = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        baldes.add(new ArrayList<>());
    }
    for (double x : a) {
        int indice = (int) (n * x);     // el balde de x es ⌊n·x⌋
        baldes.get(indice).add(x);
    }

    // Paso 2 — ordenar cada balde por dentro.
    for (List<Double> balde : baldes) {
        Collections.sort(balde);
    }

    // Paso 3 — concatenar los baldes en orden.
    double[] resultado = new double[n];
    int k = 0;
    for (List<Double> balde : baldes) {
```

```
            for (double x : balde) {
                resultado[k++] = x;
            }
        }
        return resultado;
    }
```

(Los baldes son un `ArrayList` de `ArrayList` —Unidad 5— porque no sabemos de antemano cuántos valores recibe cada uno. El *cast* `(int) (n * x)` trunca el decimal: es el $\lfloor \cdot \rfloor$ de la fórmula, válido porque $n \cdot x$ nunca es negativo.)

¿Cuánto cuesta? Si los datos están **uniformemente distribuidos**, cada balde recibe en promedio un puñado constante de elementos, ordenar todos los baldes suma $O(n)$, y repartir y concatenar también son $O(n)$: costo promedio **lineal**, $O(n)$. El **peor caso** es $O(n^2)$ y aparece cuando la distribución es despareja y **todos los valores caen en el mismo balde** (ahí ordenar ese balde único cuesta lo mismo que ordenar todo). Por eso Bucket sort brilla con datos parejos y se degrada con datos apelotonados.

6.9.4 Comparativa de los algoritmos de ordenamiento

Reunimos en una tabla los cinco algoritmos de esta unidad. “Estable” indica si preserva el orden relativo de elementos iguales; “in place” indica si ordena sin memoria extra significativa.

Algoritmo	Mejor	Promedio	Peor	Memoria	Estable	¿Compara?
Mergesort	$n \log n$	$n \log n$	$n \log n$	$O(n)$	Sí	Sí
Quicksort	$n \log n$	$n \log n$	n^2	$O(\log n)$	No	Sí
Counting	$n + k$	$n + k$	$n + k$	$O(n + k)$	Sí	No
Radix (LSD)	$d(n + b)$	$d(n + b)$	$d(n + b)$	$O(n + b)$	Sí	No
Bucket	n	n	n^2	$O(n)$	Sí*	No**

* Estable si el orden interno de cada balde es estable. ** No compara para repartir, pero suele comparar **dentro** de cada balde.

Idea clave

¿Comparativo o no comparativo? Si tus datos son enteros o claves de rango acotado (o dígitos, o reales bien distribuidos), un ordenamiento no comparativo puede ser **lineal** y ganarle al límite $O(n \log n)$. Si son objetos cualesquiera que sólo sabés **comparar**, estás atado a ese límite y usás Mergesort (peor caso garantizado, estable) o Quicksort (el más rápido en la práctica). En Java, en los parciales, TPs y finales, se usa el ordenamiento **nativo de la API** (`Arrays.sort` / `Collections.sort`), que internamente es Quicksort dual-pivot para primitivos y TimSort para objetos.

6.10 Ejemplo integrador: las Torres de Hanói

Las Torres de Hanói reúnen el corazón de la unidad: recursión pura con un caso base y dos llamadas recursivas. El juego: tres postes (A, B, C) y n discos de tamaños distintos apilados en A, del más grande al más chico. Hay que mover toda la torre a C, moviendo **un disco por vez** y sin apoyar **nunca** un disco sobre otro más chico.

El razonamiento recursivo. ¿Cómo muevo n discos de A a C? Si supiera mover $n-1$… sería fácil: (1) muevo los $n-1$ discos de arriba al poste **auxiliar** B (para destrabar el grande); (2) muevo el disco n —el más grande— de A a C, de un solo movimiento; (3) muevo los $n-1$ discos de B a C, encima del grande. ¿Y cómo muevo esos $n-1$? **Con la misma receta.** El caso base: mover 0 discos no requiere nada.

```java
public static void resolver(int n, char origen, char destino, char auxiliar) {
    if (n == 0) {
        return;   // caso base
    }
    resolver(n - 1, origen, auxiliar, destino);         // 1) n-1 al auxiliar
    System.out.println("mover disco " + n + " de " + origen + " a " + destino);
    resolver(n - 1, auxiliar, destino, origen);         // 2) el grande al destino
                                                        // 3) n-1 al destino
}
```

Fijate el detalle clave: en cada llamada recursiva los postes **cambian de rol** (el auxiliar de hoy es el destino de mañana); por eso los tres parámetros se "rotan".

Traza completa con $n = 3$. La receta produce exactamente estos $2^3 - 1 = 7$ movimientos:

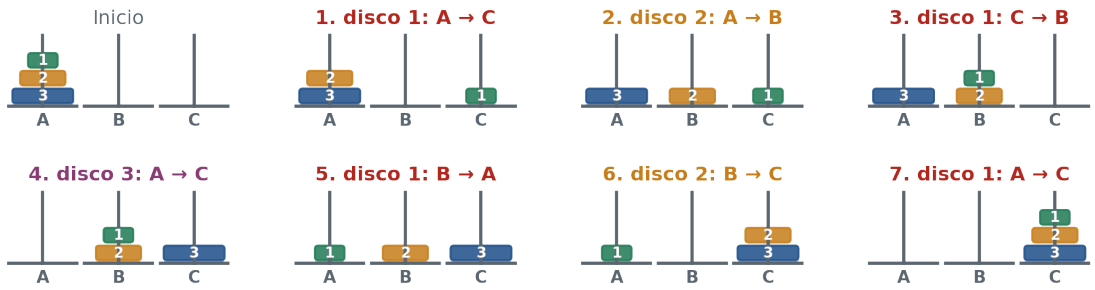


Figura 6.11: Torres de Hanói con 3 discos: los 7 movimientos, uno por panel. Movimientos 1–3: la torre de 2 discos se corre a B (destrabando el disco 3). Movimiento 4: el disco grande viaja a C. Movimientos 5–7: la torre de 2 se rearma encima.

Leé la figura con la receta en mente: los movimientos 1–3 son `resolver(2, A, B, C)` (la torre chica va al auxiliar), el movimiento 4 es el disco grande de A a C, y los movimientos 5–7 son `resolver(2, B, C, A)` (la torre chica lo sigue). Y dentro de cada mitad se repite el mismo patrón en miniatura: es recursión hasta el fondo.

¿Cuántos movimientos? Cada nivel hace 1 movimiento propio más dos veces lo que cueste el nivel anterior: $T(n) = 2\,T(n-1) + 1$, que da exactamente $2^n - 1$ movimientos $(1, 3, 7, 15, 31, \ldots)$. Es **exponencial**, pero acá no hay algoritmo mejor posible: mover la torre **requiere** esa cantidad de movimientos (el costo es inherente al problema, no un defecto de la recursión). Con 64 discos —la leyenda original— harían falta $2^{64} - 1$ movimientos: a un disco por segundo, unas 585 mil millones de años.

6.11 Errores comunes

Error común

- Olvidar el **caso base** o no achicar el problema: `StackOverflowError`.
- Repetir subproblemas (Fibonacci ingenuo): explosión exponencial; usá iteración o memoización.
- Plantear mal la recurrencia y aplicar el Teorema Maestro equivocado (Unidad 4).
- Creer que la JVM optimiza la recursión de cola: **no lo hace**.
- Usar un ordenamiento no comparativo con datos de **rango enorme** (Counting con k gigante) o con objetos no numéricos.
- Olvidar que Radix/Counting deben ser **estables** para funcionar por dígitos.

6.12 Repaso

1. ¿Qué dos partes tiene todo algoritmo recursivo? ¿Qué pasa si falta el caso base?
2. Definí los cuatro tipos de recursividad (lineal, múltiple, indirecta, anidada) y dá un ejemplo de cada uno.
3. En `esPar`/`esImpar`, ¿dónde está el ciclo recursivo? ¿Por qué no se ve mirando un solo método?
4. En `mcCarthy91(mcCarthy91(n + 11))`, ¿cuál llamada se evalúa primero y por qué?
5. ¿Por qué el Fibonacci recursivo ingenuo es $O(2^n)$? ¿Cómo se mejora? ¿Qué tipo de recursividad es?
6. Enumerá los tres pasos de la división y conquista.
7. Planteá la recurrencia de Mergesort y resolvela. ¿Por qué es siempre $O(n \log n)$?
8. ¿En qué se diferencian Mergesort y Quicksort? ¿Cuál es el peor caso de cada uno?
9. ¿Cómo logra Karatsuba multiplicar con **tres** multiplicaciones en vez de cuatro?
10. Explicá cómo se fusionan dos mitades ordenadas en Mergesort usando dos punteros. ¿Por qué cuesta $O(n)$?
11. En Quicksort, ¿qué significa “particionar”? ¿Por qué el peor caso aparece con un arreglo ya ordenado y pivote en el extremo?
12. ¿Por qué los ordenamientos por comparación no bajan de $O(n \log n)$? (pensá en el árbol de decisión).
13. Explicá Counting sort con un ejemplo. ¿Cuándo conviene y cuándo no?
14. ¿Por qué Radix sort necesita que su ordenamiento por dígito sea **estable**?
15. Enumerá los tres pasos de Bucket sort y hacé la tabla del reparto ($x \to \lfloor 6x \rfloor$) para `[0.42, 0.26, 0.72, 0.17, 0.68, 0.09]`. ¿Cuándo llega a su peor caso $O(n^2)$?
16. Aplicá Karatsuba a mano para calcular 47×82 (identificá a, b, c, d, las tres multiplicaciones y la resta).
17. En las Torres de Hanói, ¿qué hacen las dos llamadas recursivas y el movimiento del medio? ¿Por qué $2^n - 1$ movimientos es lo mejor posible?

Guía de ejercicios — Unidad 6

Objetivos de la unidad

Practicá **recursividad**, **división y conquista** y **ordenamiento no comparativo**. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i06_estrategias` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i06_estrategias/i01_teorico`. La consigna es **pensar en términos recursivos**: identificá siempre el **caso base** y el **paso recursivo**, y cuando corresponda, cómo **combinar** los resultados parciales.

6.13 Preguntas teóricas

1. ¿Qué dos partes tiene todo algoritmo recursivo y para qué sirve cada una?
2. ¿Qué es la **pila de llamadas** y qué error se produce si la recursión no termina?
3. ¿La JVM optimiza la recursión de cola (*tail call*)? ¿Qué implica esto para recursiones muy profundas?
4. ¿Por qué el Fibonacci recursivo ingenuo es $O(2^n)$? ¿Cómo se lo mejora a $O(n)$?
5. Enumerá y explicá los **tres pasos** de la división y conquista.
6. Planteá la recurrencia de Mergesort y resolvela con el Teorema Maestro. ¿Por qué es $O(n \log n)$ **siempre**?
7. ¿En qué se diferencian Mergesort y Quicksort? Indicá el peor caso y el uso de memoria de cada uno.
8. ¿Cómo logra Karatsuba multiplicar con **tres** multiplicaciones en vez de cuatro? ¿Qué complejidad alcanza?
9. ¿Por qué ningún ordenamiento **por comparación** puede bajar de $O(n \log n)$ en el peor caso?
10. Explicá Counting sort. ¿Cuál es su costo y en qué caso deja de convenir?

6.14 Nivel fácil (recursión directa)

1. **Factorial.** `long factorial(int n)` recursivo; excepción si `n < 0`. Test.
2. **Suma hasta n.** `long suma(int n)` que devuelva $1 + 2 + \cdots + n$ recursivamente. Test.
3. **Potencia.** `long potencia(int base, int exp)` recursiva (`exp >= 0`). Test.
4. **Suma de un arreglo.** `int sumar(int[] a)` recursivo (índice o subarreglo). Test.
5. **Longitud de un String.** `int largo(String s)` sin usar `length()`, recursivo. Test.
6. **Invertir un String.** `String invertir(String s)` recursivo. Test.
7. **Cuenta regresiva.** `List<Integer> cuentaRegresiva(int n)` de `n` a `0` recursivo. Test.
8. **¿Es par?** `boolean esPar(int n)` mutuamente recursivo con `esImpar` (sin `%`). Test.
9. **Máximo de un arreglo.** `int maximo(int[] a)` recursivo. Test.
10. **Contar dígitos.** `int cantidadDeDigitos(long n)` recursivo. Test.

6.15 Nivel medio (división y conquista y patrones recursivos)

1. **Búsqueda binaria recursiva.** `int buscar(int[] ordenado, int x)` que devuelva el índice o `-1`. Test (incluí ausentes).
2. **Mergesort.** Implementá `int[] ordenar(int[] a)` con Mergesort (dividir/fusionar). Test contra `Arrays.sort`.
3. **Quicksort.** Implementá `int[] ordenar(int[] a)` con Quicksort (pivote/partición). Test contra `Arrays.sort`.
4. **Potencia rápida.** `long potencia(long base, int exp)` en $O(\log exp)$ (exponenciación por cuadrados). Test.

5. **Máximo y mínimo D&C.** Devolvé el par (mín, máx) de un arreglo partiéndolo al medio. Test.
6. **Fibonacci eficiente.** `long fib(int n)` iterativo o con memoización, en $O(n)$. Test (incluí `fib(50)`).
7. **Torres de Hanói.** `List<String> resolver(int n)` que devuelva la lista de movimientos; verificá que son $2^n - 1$. Test.
8. **Contar inversiones.** Contá los pares desordenados de un arreglo usando una variante de Mergesort ($O(n \log n)$). Test.
9. **Suma de subarreglo máxima D&C.** Máxima suma contigua partiendo al medio (`maxCruzando`). Test.
10. **Producto de Karatsuba (long).** Multiplicá dos enteros con la idea de Karatsuba (podés usar `long`); compará con la multiplicación directa. Test.

6.16 Nivel difícil (recursión avanzada y ordenamiento no comparativo)

1. **Counting sort.** `int[] ordenar(int[] a, int maximo)` sin comparar; test contra `Arrays.sort`, incluí duplicados.
2. **Radix sort.** `int[] ordenar(int[] a)` LSD para enteros no negativos, con counting **estable** por dígito; test.
3. **Bucket sort.** `double[] ordenar(double[] a)` para reales en $[0, 1)$; test contra `Arrays.sort`.
4. **Subconjuntos.** Generá recursivamente todos los subconjuntos (*power set*) de una lista; verificá que son 2^n. Test.
5. **Permutaciones.** Generá recursivamente todas las permutaciones de una lista; verificá que son $n!$. Test.
6. **Combinaciones.** Generá las combinaciones de `k` elementos de una lista (backtracking); verificá la cantidad $\binom{n}{k}$. Test.
7. **Recorrer laberinto.** Dado un `boolean[][]` (transitables), decidí con recursión si hay camino de una esquina a la otra. Test.
8. **Select (k-ésimo menor).** Encontrá el `k`-ésimo menor con *quickselect* (partición al estilo Quicksort), en $O(n)$ promedio. Test.
9. **Mergesort sobre lista enlazada.** Ordená una `ListaSimplementeEnlazada<Integer>` con Mergesort (partiendo con dos punteros). Test.
10. **Comparativa de ordenamientos.** Un test que verifica que Mergesort, Quicksort, Counting y Radix producen **el mismo** resultado ordenado sobre varios arreglos enteros.

Unidad 7

Diccionarios y tablas de hash

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) definir el TDA **Diccionario** (mapa clave→valor) y reconocer cuándo usarlo; (2) explicar qué es una **función de hash** y qué la hace buena; (3) entender por qué las **colisiones** son inevitables y resolverlas por **encadenamiento** y por **direccionamiento abierto** (sondeo lineal, con lápidas); (4) razonar el **factor de carga** y el **rehashing**; (5) justificar por qué todas las operaciones son **O(1) promedio**; y (6) usar con soltura `HashMap`, `HashSet`, `LinkedHashMap` y `TreeMap` de la API de Java.

En la Unidad 5 dejamos una deuda: dijimos que `HashSet` respondía `contiene` en $O(1)$ "por hashing, que veremos más adelante". Llegó el momento. Esta unidad presenta la estructura más usada de toda la programación práctica: la **tabla de hash**, que convierte la búsqueda —que hasta ahora costaba $O(n)$ u $O(\log n)$— en una operación de **tiempo constante promedio**.

7.1 Un poco de historia

La idea del hashing nació dentro de IBM en los años 50: **Hans Peter Luhn** propuso en 1953 usar una función para "dispersar" claves en baldes (y resolver los choques encadenando), y **Arnold Dumey** publicó en 1956 la primera descripción abierta, incluyendo la idea de usar el **resto de una división** como función de dispersión. El análisis matemático fino llegó con **Donald Knuth** en los 60. Hoy el hashing está en todos lados: los diccionarios de Python, los objetos de JavaScript, las bases de datos, las caches, los sistemas de archivos y —como vamos a ver— el `HashMap` de Java.

7.2 El problema: buscar rápido

Repasemos cuánto cuesta **buscar** con lo que ya tenemos. En un arreglo o lista sin orden, la búsqueda lineal recorre todo: $O(n)$. Si mantenemos el arreglo **ordenado**, la búsqueda binaria baja a $O(\log n)$... pero insertar manteniendo el orden cuesta $O(n)$. Para un padrón de un millón de alumnos, cada búsqueda lineal son un millón de comparaciones.

¿Se puede mejor? Sí, y la idea ya la conocés: en el **Counting sort** (Unidad 6) esquivamos el límite de los ordenamientos usando el **valor como índice** de un arreglo. El hashing generaliza esa jugada: si pudiéramos calcular, **a partir de la clave**, el índice exacto donde está su valor, buscar costaría lo mismo que acceder a un arreglo: $O(1)$.

7.3 El TDA Diccionario

Lo que queremos construir es el TDA **Diccionario** (también llamado **mapa** o *tabla asociativa*): una colección de pares **clave → valor** donde cada clave es **única**, con estas operaciones:

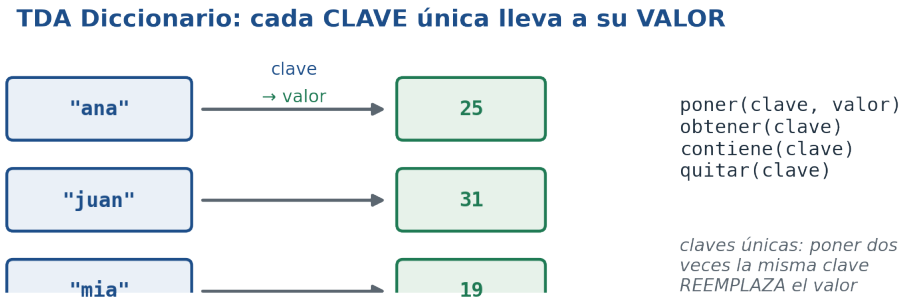


Figura 7.1: El TDA Diccionario: cada clave única lleva a su valor. Poner una clave repetida reemplaza el valor.

Es el TDA de "buscar por nombre": el stock por producto, la edad por DNI, el teléfono por contacto, la definición por palabra (de ahí "diccionario"). Fijate que el **Conjunto** de la Unidad 5 es un caso particular: un diccionario donde sólo importan las claves.

```
Diccionario<String, Integer> edades = new TablaHashEncadenamiento<>();
edades.poner("ana", 25);
edades.poner("juan", 31);
edades.obtener("ana");        // 25
edades.poner("ana", 26);      // la clave ya estaba: REEMPLAZA el valor
edades.contiene("pedro");     // false
edades.quitar("juan");        // 31
```

Como siempre, el contrato no dice **cómo**: en esta unidad lo implementamos con tablas de hash, y al final vemos las implementaciones de la API de Java.

7.4 La función de hash

El plan es guardar los pares en un **arreglo** de m celdas (los **baldes**). Para saber en qué celda va cada clave, necesitamos una **función de hash**: una función que convierte cualquier clave en un entero, que luego "aplastamos" al rango $[0, m)$ con el resto de la división. Éste es el viaje completo de una clave:

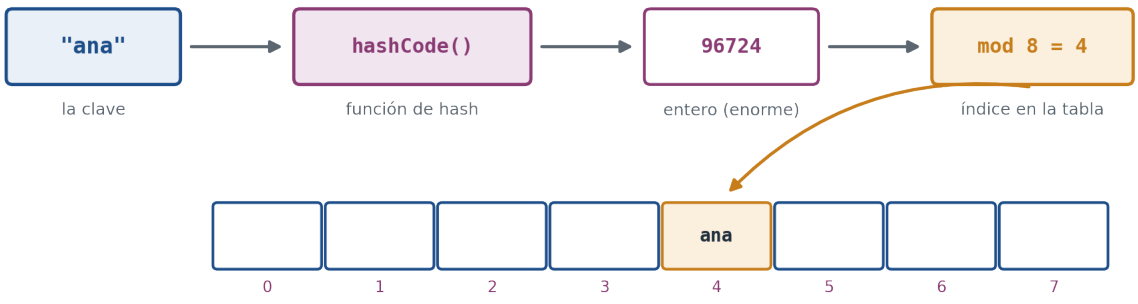


Figura 7.2: El camino de una clave hasta su celda: la función de hash la convierte en un entero (el `hashCode()` de Java), y el módulo lo lleva al rango de índices de la tabla. Para `"ana"`: hashCode 96724, y 96724 mod 8 = 4.

¿Qué hace **buena** a una función de hash? Tres cosas: (1) **determinista**: la misma clave da siempre el mismo número (si no, ¡no la encontraríamos nunca!); (2) **uniforme**: reparte las claves parejo entre las celdas, sin favoritismos; (3) **rápida**: se calcula en $O(1)$ (o $O(largo)$ para cadenas) — si calcular el hash fuera caro, perderíamos lo ganado.

7.4.1 El hash de las cadenas: polinomial

¿Y cómo se convierte un **texto** en número? Java usa un **hash polinomial**: recorre los caracteres acumulando `h = 31·h + carácter` (regla de Horner). Calculémoslo a mano para `"ana"` (los caracteres son sus códigos: `'a'` = 97, `'n'` = 110):

El hash polinomial de "ana" (h = 31·h + carácter, empezando en 0)

inicio	h = 0	
'a' (97)	h = 31·0 + 97 = 97	
'n' (110)	h = 31·97 + 110 = 3117	
'a' (97)	h = 31·3117 + 97 = 96724	**= "ana".hashCode() en Java**

la base 31 'mezcla' la posición de cada carácter: "amor" y "roma" dan hashes distintos

Figura 7.3: El hash polinomial de “ana”, paso a paso con la regla de Horner. El resultado, 96724, es exactamente lo que devuelve `"ana".hashCode()` en Java.

¿Por qué multiplicar por 31 en vez de simplemente **sumar** los caracteres? Porque la suma ignora el **orden**: `"amor"` y `"roma"` sumarían lo mismo y colisionarían siempre. La multiplicación “pesa” cada carácter según su posición, como los dígitos de un número. En el repo, `FuncionesDeHash.polinomial` reproduce exactamente el `String.hashCode()` de Java.

7.4.2 Otros métodos clásicos

Antes de que se impusieran el módulo y el polinomial, se usaron otros métodos que conviene conocer (aparecen en parciales y en bibliografía). Ojo con distinguir bien los dos primeros, que suelen confundirse:

- **Extracción**: tomar **algunos dígitos** de la clave tal cual está. Por ejemplo, de la clave `123456`, tomar el 3.º y 4.º dígito: `34`. Simple, pero malo si los dígitos elegidos no varían (los primeros dígitos de un año, por ejemplo).
- **Cuadrado medio**: **elevar la clave al cuadrado** y tomar los dígitos **del medio** del resultado. Para `123`: $123^2 = 15129$, y sus dos dígitos del medio dan `51`. La gracia: el cuadrado “mezcla” todos los dígitos de la clave, así que el medio depende de toda ella. (Notá la diferencia con extracción: acá **primero se eleva al cuadrado**; extraer dígitos directo de la clave es el otro método.)
- **Plegado** (*folding*): partir la clave en grupos de dígitos y **sumarlos**: `123456` → $12+34+56 = 102$. Útil para claves largas tipo números de tarjeta.

Los tres están implementados en `FuncionesDeHash` para experimentar. En la práctica moderna: **módulo** (idealmente con m primo), **multiplicativo** (Knuth) y **polinomial** para cadenas.

7.5 Las colisiones son inevitables

Idealmente, cada clave tendría su celda exclusiva. Pero los enteros de hash son 2^{32} posibles y las celdas son pocas: por el **principio del palomar**, si hay más claves posibles que celdas, **alguna celda recibirá dos claves distintas**. Y pasa mucho antes de lo que la intuición dice —es la *paradoja del cumpleaños*: con sólo 23 personas ya es más probable que no que dos cumplan años el mismo día. Miralo con claves reales de Java:

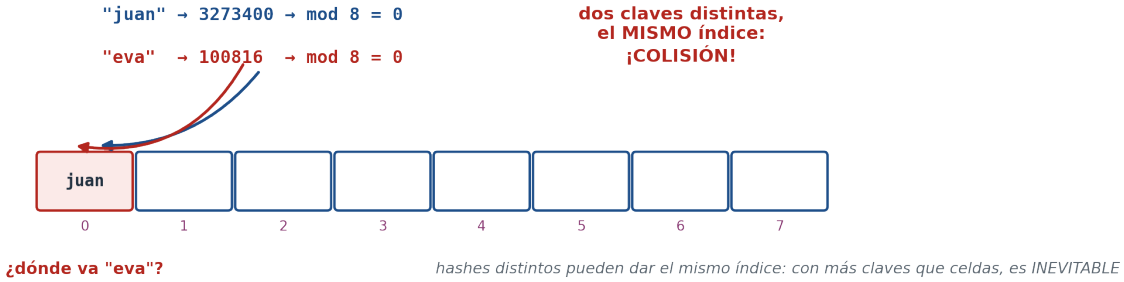


Figura 7.4: Una colisión real: `"juan"` y `"eva"` tienen hashCodes distintos (3273400 y 100816), pero ambos dan índice 0 en una tabla de 8 celdas. ¿Dónde va la segunda?

Toda tabla de hash necesita entonces una **estrategia de resolución de colisiones**. Hay dos grandes familias.

7.6 Resolución 1: encadenamiento

La estrategia más directa: cada celda no guarda un par, sino una **lista** de pares (su "cadena" o balde). Las claves que colisionan **conviven encadenadas** en la misma celda. Sigamos tres inserciones con los valores reales de Java:

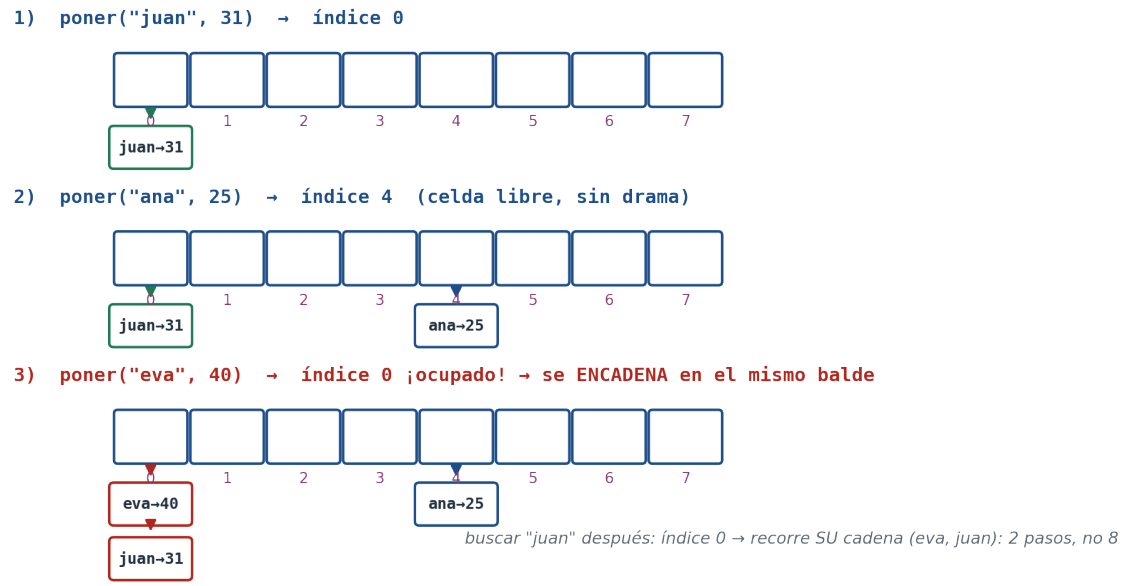


Figura 7.5: Encadenamiento paso a paso: `"juan"` entra en el balde 0 y `"ana"` en el 4 sin conflicto; cuando `"eva"` también da 0, simplemente se encadena en ese balde. Buscar recorre sólo la cadena de su balde.

El código es un arreglo de nodos encadenados (está completo en el repo, `TablaHashEncadenamiento`):

```
public void poner(K clave, V valor) {
    int i = indiceDe(clave, baldes.length);          // hash + modulo
    for (Par<K, V> p = baldes[i]; p != null; p = p.siguiente) {
        if (p.clave.equals(clave)) { p.valor = valor; return; }  // reemplazo
    }
    baldes[i] = new Par<>(clave, valor, baldes[i]);  // al frente de SU cadena
    cantidad++;
    if (factorDeCarga() > 0.75) {
```

```
        rehash();
    }
}

public V obtener(K clave) {
    int i = indiceDe(clave, baldes.length);
    for (Par<K, V> p = baldes[i]; p != null; p = p.siguiente) {
        if (p.clave.equals(clave)) {
            return p.valor;   // recorre SOLO su balde
        }
    }
    return null;
}
```

¿Cuánto cuesta? La clave del análisis es el **factor de carga** $\alpha = n/m$ (elementos sobre celdas): es el **largo promedio de cada cadena**. Buscar cuesta calcular el hash ($O(1)$) más recorrer una cadena de largo promedio α. Si mantenemos α acotado (por ejemplo $\alpha \leq 0.75$), las cadenas miden en promedio menos de un elemento, y todas las operaciones son $O(1)$ **promedio**.

7.7 Resolución 2: direccionamiento abierto

La otra familia no usa listas: **todos los pares viven en el arreglo mismo**. Si la celda que indica el hash está ocupada, se **sondea** (*probing*) la siguiente celda, y la siguiente, hasta encontrar una libre. Con **sondeo lineal**, se prueba $i, i+1, i+2, \ldots$ (con vuelta circular). Nuestros tres nombres con índice 0 lo muestran perfecto:

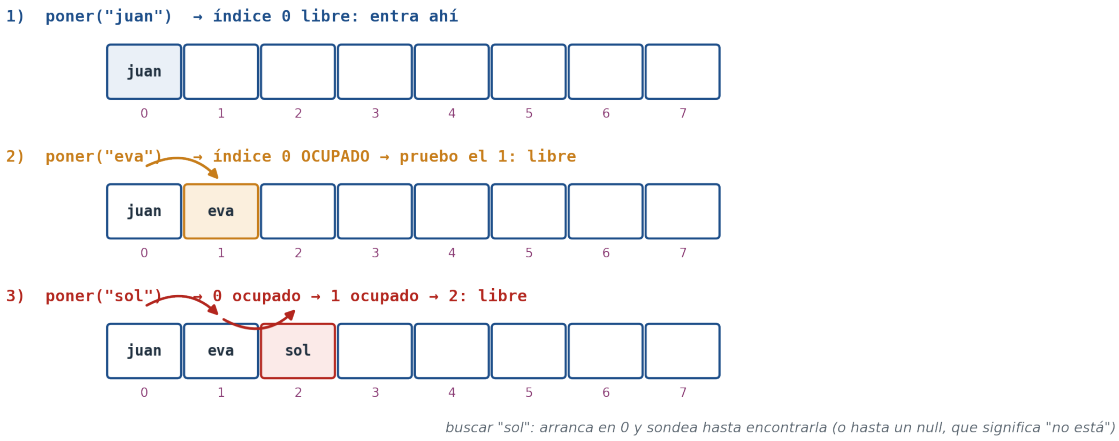


Figura 7.6: Sondeo lineal paso a paso: `"juan"` toma la celda 0; `"eva"` (también índice 0) la encuentra ocupada y prueba la 1; `"sol"` (índice 0 otra vez) sondea 0 → 1 → 2. Buscar repite el mismo recorrido.

Buscar una clave repite el sondeo desde su índice inicial: si aparece la clave, la encontró; si aparece una celda **vacía** (`null`), la clave **no está** (porque de estar, habría quedado en algún punto de ese recorrido).

El problema del clustering. El sondeo lineal tiende a formar **racimos** (*clusters*): corridas de celdas ocupadas consecutivas. Todo hash que caiga en cualquier celda del racimo termina agregándose **al final del racimo**, con lo que el racimo crece… y atrae aún más claves. Por eso el direccionamiento abierto exige factores de carga más bajos (típicamente $\alpha \leq 0.5$). Variantes que lo alivian: **sondeo cuadrático** (saltos $1, 4, 9, \ldots$) y **doble hashing** (el paso del sondeo lo da una segunda función de hash).

7.7.1 Borrar exige lápidas

Borrar en direccionamiento abierto tiene una trampa elegante. Si al quitar `"eva"` dejáramos su celda en `null`, la búsqueda de `"sol"` —que en su momento sondeó **pasando por** esa celda— se cortaría ahí y diría “no está”, ¡aunque está dos celdas más allá! La solución es dejar una **lápida** (*tombstone*): una marca que dice “acá hubo algo, seguí buscando”:

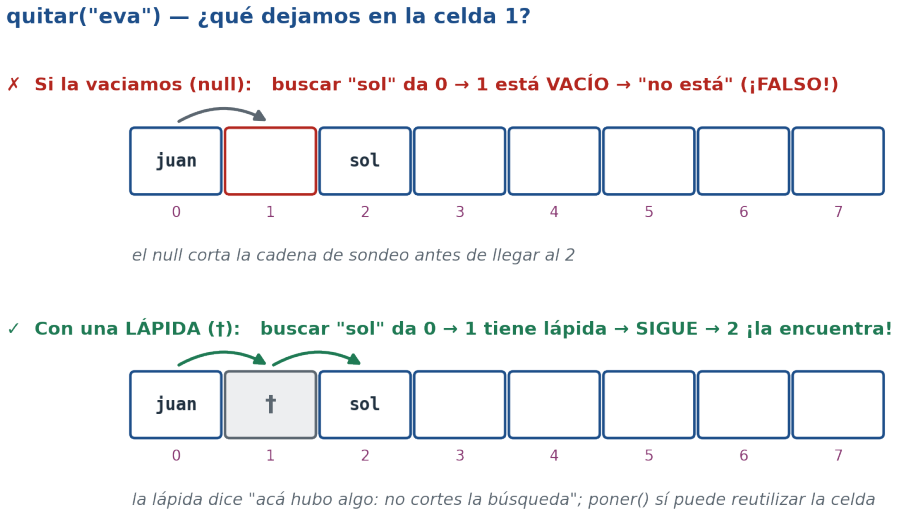


Figura 7.7: Borrar con lápida: si la celda quedara en null, la búsqueda de `"sol"` se cortaría antes de llegarle (panel de arriba). La lápida mantiene viva la cadena de sondeo (abajo); además, `poner` puede reutilizar esa celda.

El código completo, con lápidas y reutilización de celdas, está en el repo: `TablaHashAbierta`.

7.8 Factor de carga y rehashing

Las dos estrategias degradan cuando la tabla se llena: cadenas largas en un caso, racimos y sondeos eternos en el otro. La solución es la que ya conocés del vector dinámico (Unidad 4): cuando α supera un umbral, se **redimensiona**. Pero acá no alcanza con copiar: como el índice depende de `hash mod m`, al cambiar m **cambian los índices**. Hay que **re-ubicar cada par** con el módulo nuevo: eso es el **rehashing**.

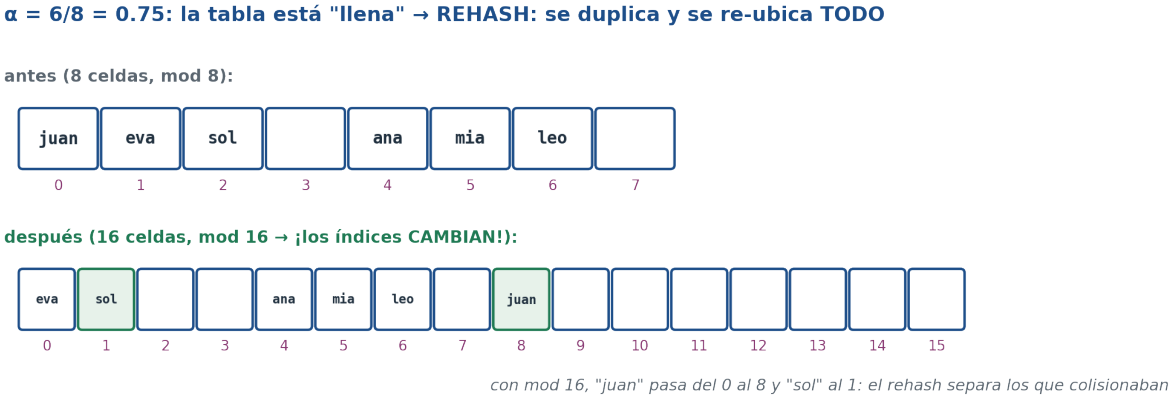


Figura 7.8: Rehash: al superar α = 0.75 la tabla se duplica, y cada clave se re-ubica con el módulo nuevo. Notá que `"juan"` salta del 0 al 8 y `"sol"` al 1: el rehash además deshace colisiones viejas.

El costo es $O(n)$ en el momento… pero **amortizado** $O(1)$ por inserción, por el mismo argumento de la suma geométrica de la Unidad 4. Y hay un bonus: como el módulo cambia, claves que colisionaban pueden separarse.

7.9 La complejidad, con precisión

Juntemos todo. Con una buena función de hash y factor de carga acotado (que el rehash garantiza):

Operación	Promedio	Peor caso
`poner`	$O(1)$*	$O(n)$
`obtener` / `contiene`	$O(1)$	$O(n)$
`quitar`	$O(1)$	$O(n)$

(*amortizado por el rehash.) El **peor caso** $O(n)$ ocurre si todas las claves caen en el mismo balde —función de hash pésima o mala suerte extrema—: la tabla degenera en una lista.

Error común

Corrección importante (un clásico de apuntes viejos, incluido el nuestro): la búsqueda en una tabla de hash es $O(1)$ **promedio — NO** $O(\log n)$. El $O(\log n)$ corresponde a los **árboles balanceados** (próxima unidad) y a la búsqueda binaria. Es exactamente el trade-off entre `HashMap` ($O(1)$, sin orden) y `TreeMap` ($O(\log n)$, con las claves ordenadas): rapidez pura contra orden.

7.10 Las claves y el contrato equals/hashCode

Todo este edificio se apoya en la regla que la Unidad 3 te pidió aceptar a crédito: **si dos claves son `equals`, deben tener el mismo `hashCode`**. Ahora tenés las herramientas para cobrar esa deuda. La tabla usa el `hashCode` para elegir el balde y el `equals` para confirmar dentro del balde — así que si el contrato está roto, la búsqueda va a un balde equivocado:

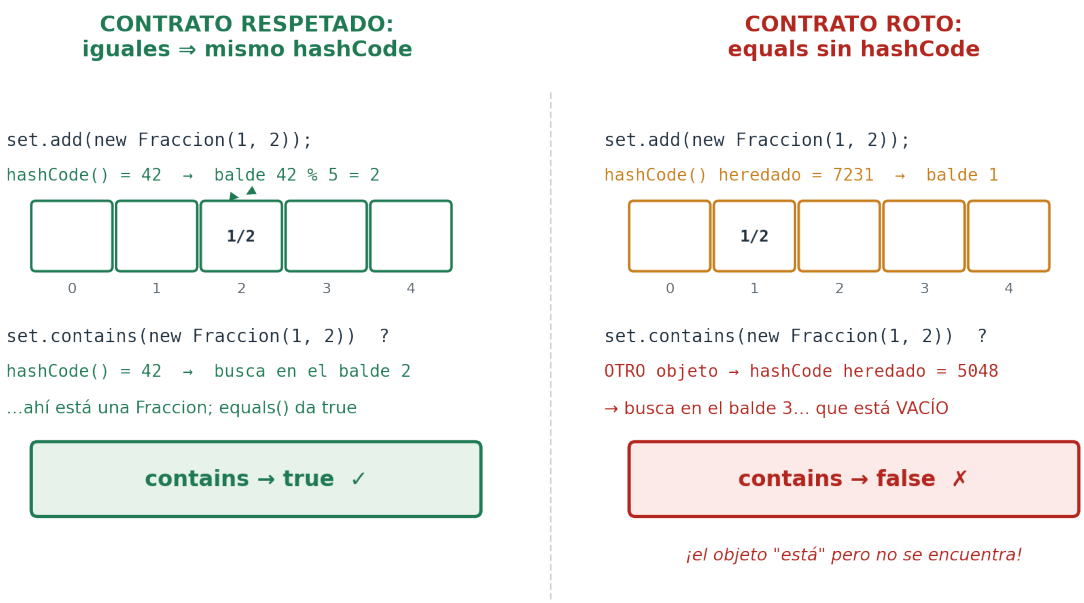


Figura 7.9: La deuda de la Unidad 3, saldada. A la izquierda, el contrato respetado: dos fracciones iguales comparten hashCode, la búsqueda va al mismo balde donde se guardó, y `contains` la encuentra. A la derecha, el contrato roto (`equals` redefinido SIN `hashCode`): el objeto nuevo hereda el hashCode de Object, la búsqueda va a un balde equivocado (vacío) y `contains` devuelve false aunque una fracción igual está guardada.

El panel derecho es desconcertante en la práctica: agregás `new Fraccion(1,2)` al conjunto, preguntás si contiene `new Fraccion(1,2)`… y te dice que **no**. El objeto "está", pero la búsqueda mira en el balde equivocado. Este bug no lanza excepciones ni avisa: simplemente la estructura "pierde" cosas. Reglas prácticas:

- Si tu clase de clave redefine `equals`, redefiní `hashCode` en consecuencia (generalos juntos con el IDE, o usá un `record`).
- Las claves deberían ser **inmutables**: si mutás una clave ya guardada, su `hashCode` cambia… pero el objeto quedó archivado en el balde del hash **viejo**: inhallable.

Error común

La trampa del `Integer ==` (otro clásico a corregir). Al usar claves `Integer`, compará siempre con `equals` — nunca con `==`:

```
Integer a = 1000, b = 1000;
a == b;         // ¡false! son dos objetos distintos (identidad, Unidad 2)
a.equals(b);    // true

Integer c = 100, d = 100;
c == d;         // true… ¡por casualidad! Java cachea los Integer de -128 a 127
```

Ese caché hace que el error **parezca funcionar** en pruebas con números chicos y explote en producción con números grandes: el peor tipo de bug. Los `HashMap` por dentro usan `equals`, así que ellos lo hacen bien; el riesgo es tu propio código.

7.11 De lo nuestro a la API de Java

Como siempre, lo que construimos a mano ya existe optimizado en `java.util`, y es lo que se usa en parciales, TPs y finales:

Lo que hicimos	En la API de Java	Costo	Orden de las claves
`TablaHashEncadenamiento`	`HashMap`	$O(1)$ prom.	ninguno
—	`LinkedHashMap`	$O(1)$ prom.	el de inserción
—	`TreeMap`	$O(\log n)$	**ordenadas**
Conjunto por hash	`HashSet`	$O(1)$ prom.	ninguno

(El `HashMap` de Java usa encadenamiento, con una mejora: si una cadena se hace muy larga, la convierte en un árbol — así ni el peor caso es tan malo.)

Código de uso de todas las operaciones:

```
Map<String, Integer> stock = new HashMap<>();
stock.put("tornillos", 120);              // poner
stock.put("tornillos", 150);              // clave repetida: reemplaza
stock.get("tornillos");                   // 150
stock.get("clavos");                      // null (no está)
stock.getOrDefault("clavos", 0);          // 0: con valor por defecto
stock.containsKey("tuercas");             // ¿está la clave?
stock.remove("tornillos");                // quitar
stock.size();

for (Map.Entry<String, Integer> e : stock.entrySet()) {   // recorrer los pares
    System.out.println(e.getKey() + " -> " + e.getValue());
}

Map<String, Integer> votos = new HashMap<>();
votos.merge("ana", 1, Integer::sum);      // el idioma del CONTEO: crea o suma
```

Idea clave

¿Cuál elijo? `HashMap` por defecto (lo más rápido). `LinkedHashMap` si al recorrer querés el orden en que insertaste. `TreeMap` sólo si necesitás las claves **ordenadas** (o rangos: "todas las claves entre A y B"), aceptando pagar $O(\log n)$. La misma lógica vale para `HashSet` / `LinkedHashSet` / `TreeSet`.

Casos de uso y características. A favor: la operación estrella es la búsqueda por clave exacta en $O(1)$ — índices, caches, contadores, agrupamientos, detección de duplicados. En contra: **no hay orden** (recorrer un `HashMap` da un orden arbitrario que además puede cambiar); no sirve para búsquedas por **rango** ("los DNI entre X e Y" pide un árbol); y gasta memoria extra (celdas libres + estructura), el precio de la velocidad.

7.12 Ejemplo integrador: contar palabras

El uso más clásico del diccionario: contar la frecuencia de cada palabra de un texto. Con `HashMap`, cada palabra se procesa en $O(1)$ promedio, así que contar n palabras cuesta $O(n)$ — con una lista de pares sería $O(n^2)$.

```java
public static Map<String, Integer> contar(String texto) {
    Map<String, Integer> frecuencia = new HashMap<>();
    for (String palabra : texto.toLowerCase().split("[^\\p{L}]+")) {
        if (palabra.isEmpty()) {
            continue;
        }
        frecuencia.merge(palabra, 1, Integer::sum);   // crea con 1, o suma 1
    }
    return frecuencia;
}
```

Tracemos `contar("el que lee mucho y anda mucho, ve mucho y sabe mucho")`: la primera vez que aparece `"mucho"`, `merge` no la encuentra y la crea con 1; las tres siguientes la encuentra y suma. Resultado: `mucho` → 4, `y` → 2, `el` → 1, etc. — 8 palabras distintas, un solo recorrido del texto. El código completo (con `masFrecuente`) está en el repo: `ContadorDePalabras`.

7.13 Errores comunes

Error común

- Decir que buscar en una tabla de hash es $O(\log n)$: es $O(1)$ **promedio** (el log es de los árboles).
- Confundir **extracción** (tomar dígitos de la clave) con **cuadrado medio** (elevar al cuadrado y tomar los del medio).
- Usar una clave que redefine `equals` sin `hashCode` (o viceversa): la tabla "pierde" objetos.
- Comparar claves `Integer` con `==`: funciona hasta 127 y explota después (caché de boxing).
- Mutar un objeto usado como clave: queda archivado bajo su hash viejo, inhallable.
- En direccionamiento abierto, borrar poniendo `null` en vez de **lápida**: corta las cadenas de sondeo.
- Esperar que un `HashMap`/`HashSet` mantenga algún orden al recorrerlo: no lo hace (usá `LinkedHashMap` o `TreeMap`).
- Olvidar el **rehash**: sin él, α crece y la tabla degenera a $O(n)$.

7.14 Repaso

1. ¿Qué operaciones definen al TDA Diccionario? ¿Qué pasa al poner una clave que ya existía?
2. ¿Qué tres propiedades necesita una buena función de hash?

3. Calculá a mano el hash polinomial de `"si"` (códigos: `'s'` = 115, `'i'` = 105) y su índice en una tabla de 8 celdas.
4. ¿Por qué sumar los caracteres de un string es peor que el hash polinomial? Dá un ejemplo de dos palabras que colisionarían siempre.
5. Explicá la diferencia entre **extracción** y **cuadrado medio** con la clave 123.
6. ¿Por qué las colisiones son inevitables? ¿Qué dice la paradoja del cumpleaños al respecto?
7. Mostrá cómo quedan las claves `"juan"`, `"eva"` y `"sol"` (todas índice 0) en una tabla con encadenamiento, y en una con sondeo lineal.
8. ¿Qué es el factor de carga α y qué mide en el encadenamiento? ¿Por qué el direccionamiento abierto exige α más bajo?
9. ¿Qué es el clustering primario y por qué se auto-alimenta?
10. ¿Para qué sirven las lápidas? ¿Qué fallaría sin ellas?
11. ¿Por qué el rehash debe **re-ubicar** todas las claves y no sólo copiar el arreglo? ¿Cuál es su costo amortizado?
12. ¿Cuál es la complejidad de buscar en una tabla de hash, en promedio y en el peor caso? ¿Y en un `TreeMap`?
13. ¿Por qué las claves de un `HashMap` deben ser inmutables y cumplir el contrato equals/hashCode?
14. ¿Qué imprime `Integer a = 200, b = 200; System.out.println(a == b);` y por qué?

Guía de ejercicios — Unidad 7

Objetivos de la unidad

Practicá **diccionarios y hashing**. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i07_hashing` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i07_hashing/i01_teorico`. Los niveles fácil y medio usan la **API nativa** (`HashMap`, `HashSet`…) — la herramienta de parciales y TPs —; el nivel difícil **implementa las tablas a mano**, como en el material.

7.15 Preguntas teóricas

1. ¿Qué es una función de hash y qué tres propiedades debe tener?
2. ¿Por qué las colisiones son inevitables aunque la función de hash sea perfecta?
3. Explicá el encadenamiento. ¿Qué representa el factor de carga α en esta estrategia?
4. Explicá el sondeo lineal. ¿Qué es el clustering primario?
5. ¿Por qué el borrado en direccionamiento abierto necesita lápidas?
6. ¿Qué es el rehashing y por qué no alcanza con copiar el arreglo?
7. ¿Cuál es la complejidad de `obtener` en una tabla de hash (promedio y peor caso)? ¿Por qué **no** es $O(\log n)$?
8. ¿Qué relación hay entre el contrato `equals`/`hashCode` y el funcionamiento de un `HashMap`?
9. ¿Cuándo conviene `TreeMap` en vez de `HashMap`? ¿Qué se gana y qué se paga?
10. ¿Por qué no hay que comparar claves `Integer` con `==`? ¿Qué es el caché de boxing?

7.16 Nivel fácil (con la API de Java)

1. **Frecuencias.** `Map<Character, Integer> frecuencias(String s)`: cuántas veces aparece cada carácter. Test.
2. **Primer duplicado.** `Integer primerDuplicado(int[] a)`: el primer valor que aparece por segunda vez, o `null` (usá `HashSet`). Test.
3. **Traductor.** Con un `Map<String, String>` de palabras ES→EN, traducí una frase palabra por palabra (las desconocidas quedan igual). Test.
4. **Inventario.** `Map<String, Integer>` con `agregarStock(prod, cant)` y `quitarStock(prod, cant)` (excepción si no alcanza). Test.
5. **¿Anagramas?** `boolean sonAnagramas(String a, String b)` comparando mapas de frecuencias. Test.
6. **Interseccion de arreglos.** Elementos comunes a dos `int[]`, sin duplicados (con `HashSet`). Test.
7. **Agenda.** `Map<String, String>` nombre→teléfono: alta, baja, consulta y "¿de quién es este número?" (búsqueda inversa recorriendo `entrySet`). Test.
8. **Padrones únicos.** ¿Cuántos padrones distintos hay en una lista con repetidos? (`HashSet.size`). Test.
9. **Conteo de votos.** `Map<String, Integer> escrutinio(List<String> votos)` con `merge`; devolvé también el ganador. Test.
10. **Última aparición.** `Map<String, Integer> ultimaPosicion(String[] palabras)`: índice de la última vez que aparece cada palabra. Test.

7.17 Nivel medio (con la API de Java)

1. **Agrupar por inicial.** `Map<Character, List<String>> agrupar(List<String>)` con `computeIfAbsent`. Test.

2. **Dos que suman.** `int[] dosQueSuman(int[] a, int objetivo)`: índices de dos elementos que suman el objetivo, en $O(n)$ con `HashMap` (valor→índice). Test.
3. **Índice invertido.** De un `Map<String, String>` documento→texto, construí `Map<String, Set<String>>` palabra→documentos que la contienen. Test.
4. **Caché con capacidad.** Un caché LRU simple sobre `LinkedHashMap` (redefiní `removeEldestEntry`). Test: al superar la capacidad, se va el menos usado.
5. **Primera no repetida.** `Character primeraNoRepetida(String s)` en dos pasadas con `LinkedHashMap`. Test.
6. **Ranking ordenado.** A partir de un mapa de puntajes, devolvé los nombres ordenados por puntaje descendente (y alfabético en empate). Test.
7. **Agrupar anagramas.** `List<List<String>> grupos(List<String>)`: palabras que son anagramas entre sí, agrupadas (clave: la palabra con sus letras ordenadas). Test.
8. **Diferencia de inventarios.** Dados dos `Map<String, Integer>` (stock ayer/hoy), devolvé qué productos cambiaron y en cuánto. Test.
9. **Números de línea.** `Map<String, List<Integer>> ocurrencias(List<String> lineas, Set<String> buscadas)`: en qué líneas aparece cada palabra buscada. Test.
10. **Sub-diccionario por rango.** Con un `TreeMap<Integer, String>` de legajos, devolvé los pares con legajo en `[desde, hasta]` (usá `subMap`). Test.

7.18 Nivel difícil (implementadas a mano)

1. **Tabla con encadenamiento.** `TablaHash<K, V>` con arreglo de nodos encadenados: `poner`/`obtener`/`quitar`/`tamanio` (sin rehash). Test con claves que colisionan.
2. **Rehash.** Agregale a la tabla del e01 el rehash al superar $\alpha = 0.75$; test que verifica que crece y que TODO sigue encontrándose.
3. **Sondeo lineal.** `TablaAbierta<K, V>` con sondeo lineal y **lápidas**; test que borra una clave "del medio" de una cadena de sondeo y verifica que las siguientes se siguen encontrando.
4. **Hash polinomial.** `int hashPolinomial(String s, int base, int modulo)` con Horner; test que con base 31 coincide con `String.hashCode()` (módulo aparte).
5. **Cuadrado medio y extracción.** Implementá ambos métodos y un test que muestre con la clave 123 que dan resultados distintos (51 vs. 12).
6. **Conjunto sobre tabla.** `ConjuntoHash<T>` (agregar/contiene/quitar $O(1)$ promedio) implementado sobre tu tabla del e01. Test.
7. **Contador sin API.** Reimplementá el contador de palabras del integrador usando TU tabla (no `HashMap`). Test con el texto del apunte.
8. **Detector de colisiones.** Dado un arreglo de claves `String` y una capacidad m, devolvé cuántas colisiones produce el hash de Java (pares distintos con el mismo índice, agrupando por índice). Test con "juan"/"eva"/"sol" y m = 8.
9. **Multi-mapa.** `MultiMapa<K, V>` donde una clave puede tener varios valores (`agregar(k, v)`, `valoresDe(k)`), sobre tu tabla con listas como valores. Test.
10. **Comparativa de estrategias.** Un test que inserta las mismas 20 claves en tu tabla con encadenamiento y en la de sondeo lineal y verifica que ambas responden igual (`obtener`, `contiene`, `quitar`, `tamanio`).

Unidad 8

Árboles

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) manejar la **terminología** de árboles (raíz, hoja, altura, subárbol); (2) explicar el invariante del **árbol binario de búsqueda** y ejecutar a mano **buscar**, **insertar** y **quitar** (con sus tres casos); (3) aplicar los cuatro **recorridos** y saber para qué sirve cada uno; (4) explicar por qué un ABB puede **degenerar** y cómo el **AVL** lo evita con factores de equilibrio y **rotaciones**; (5) describir la idea del árbol **rojo-negro** y de los árboles **B/B+**; y (6) usar `TreeMap` y `TreeSet` con sus consultas por rango.

Las estructuras que vimos hasta acá son **lineales**: cada elemento tiene un anterior y un siguiente. Los **árboles** rompen esa fila: cada elemento puede tener **varios** "siguientes". Esa forma jerárquica está en todos lados —el sistema de archivos, el DOM de una página web, un organigrama, este mismo apunte (capítulos → secciones → subsecciones)— y, bien aprovechada, resuelve el pendiente que nos dejó el hashing: buscar rápido **manteniendo el orden**.

8.1 Un poco de historia

Los árboles de búsqueda aparecen a fines de los 50 (Windley, Booth, Hibbard). El salto conceptual llegó en 1962 desde Moscú: **G. Adelson-Velsky y E. Landis** publicaron el primer árbol **auto-balanceado**, bautizado **AVL** por sus iniciales. En 1970 **Rudolf Bayer y Edward McCreight** inventaron los **árboles B** para los discos de Boeing, y en 1978 **Leonidas Guibas y Robert Sedgewick** dieron forma al **rojo-negro** — el que hoy vive dentro del `TreeMap` de Java, de los diccionarios ordenados de C++ y del kernel de Linux.

8.2 Terminología: el vocabulario de los árboles

Un **árbol** es un conjunto de **nodos** conectados jerárquicamente: un nodo **raíz** arriba de todo, y cada nodo con cero o más **hijos**. En un árbol **binario**, cada nodo tiene a lo sumo **dos** hijos (izquierdo y derecho).

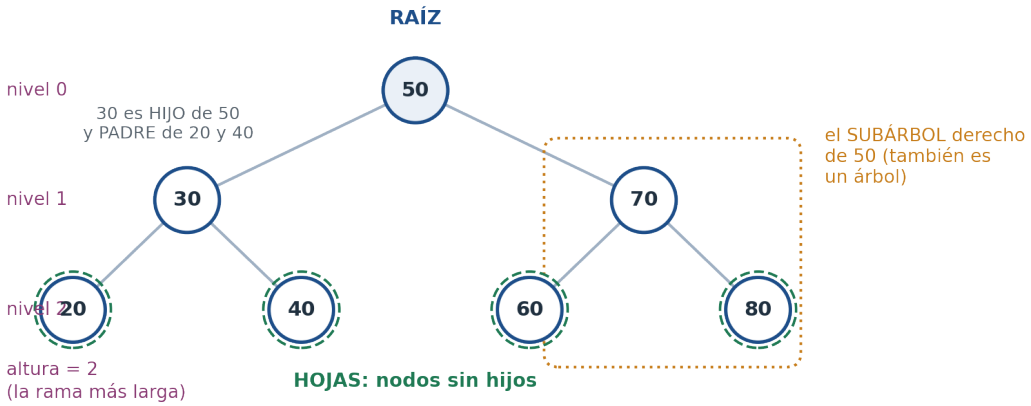


Figura 8.1: La terminología sobre un árbol concreto: raíz, hojas, padre/hijo, subárboles, niveles y altura.

El vocabulario esencial: la **raíz** es el único nodo sin padre; las **hojas** son los nodos sin hijos; cada nodo es raíz de su propio **subárbol** (la definición es recursiva, y por eso casi todo en árboles se programa recursivo); el **nivel** de un nodo es su distancia a la raíz; y la **altura** del árbol es la rama más larga. La altura es **el** número de esta unidad: todas las operaciones que siguen cuestan $O(h)$.

8.3 El árbol binario de búsqueda (ABB)

Un árbol binario se convierte en **árbol binario de búsqueda** cuando cumple, en **cada** nodo, el invariante que ya conocés de la Unidad 3 como contrato:

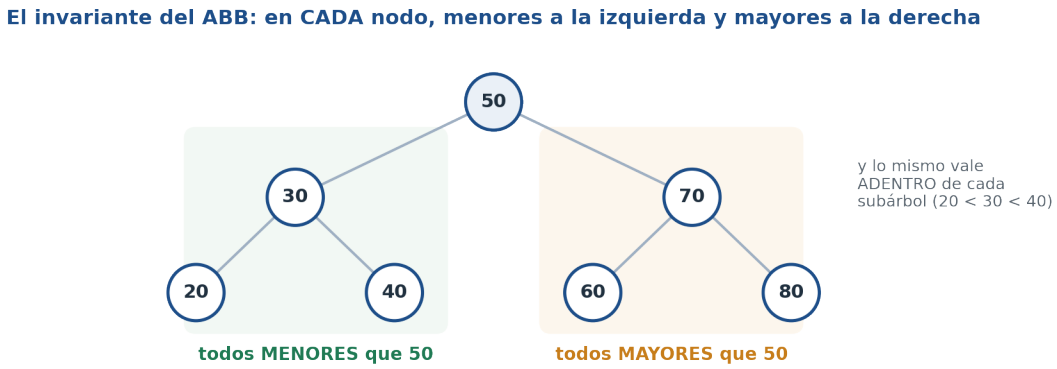


Figura 8.2: El invariante del ABB: en cada nodo, todo el subárbol izquierdo es menor y todo el derecho es mayor. Vale recursivamente en cada subárbol.

Ese invariante convierte cada comparación en una **decisión con descarte**: si lo que busco es menor que el nodo, está (si está) a la izquierda — el subárbol derecho **entero** queda descartado. Es la misma jugada de la búsqueda binaria (Unidad 4), hecha estructura.

8.3.1 Buscar

Busquemos el 60 paso a paso:

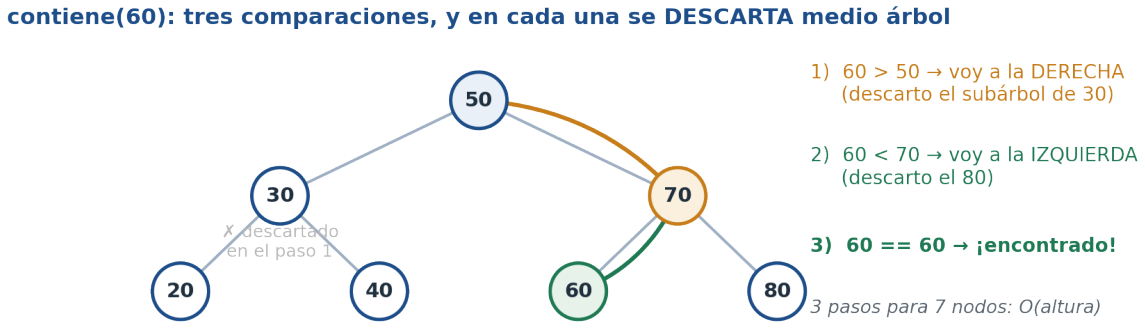


Figura 8.3: Buscar 60: tres comparaciones, cada una descarta medio árbol. El costo es O(altura).

```java
public boolean contiene(T valor) {
    Nodo<T> actual = raiz;
    while (actual != null) {
        int cmp = valor.compareTo(actual.valor);
        if (cmp == 0) {
            return true;
        }
```

```
        actual = (cmp < 0) ? actual.izquierdo : actual.derecho;   // desciende
    }
    return false;                       // me caí del árbol: no está
}
```

De paso, dos búsquedas especiales gratis: el **mínimo** es “todo a la izquierda” y el **máximo** “todo a la derecha” — imposibles de responder rápido en una tabla de hash.

8.3.2 Insertar

Insertar es **buscar hasta caerse del árbol**: el valor nuevo desciende comparando como una búsqueda y se engancha como **hoja** en el hueco exacto donde “debería haber estado”:

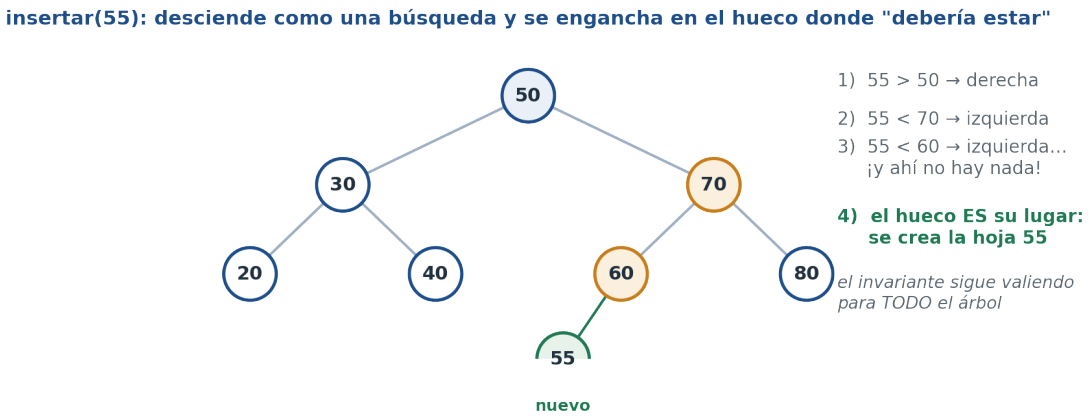


Figura 8.4: Insertar 55: desciende 50 → 70 → 60, y al no encontrar nada a la izquierda del 60, ahí mismo se engancha. El invariante se preserva solo.

```
private Nodo<T> insertar(Nodo<T> nodo, T valor) {
    if (nodo == null) { cantidad++; return new Nodo<>(valor); }   // el hueco es su lugar
    int cmp = valor.compareTo(nodo.valor);
    if (cmp < 0) {
        nodo.izquierdo = insertar(nodo.izquierdo, valor);
    } else if (cmp > 0) {
        nodo.derecho   = insertar(nodo.derecho, valor);
    }
    return nodo;                        // cmp == 0: ya estaba (sin duplicados)
}
```

Fijate el patrón recursivo `nodo.izquierdo = insertar(nodo.izquierdo, …)`: cada llamada devuelve la raíz (posiblemente nueva) de su subárbol y el padre la re-engancha. Este patrón vuelve en `quitar` y en el AVL.

8.3.3 Quitar: los tres casos

Quitar es la operación con casos, porque al sacar un nodo hay que decidir **quién ocupa su lugar** sin romper el invariante:

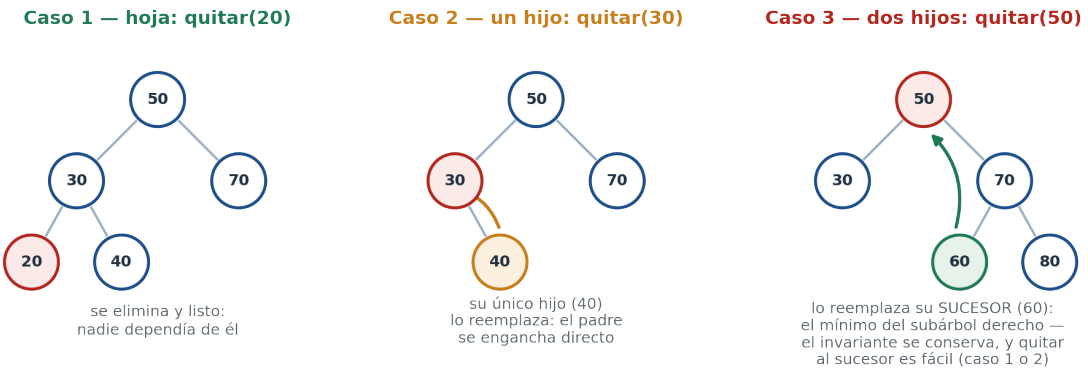


Figura 8.5: Los tres casos de quitar: la hoja se elimina y listo; el nodo con un hijo es reemplazado por ese hijo; el nodo con dos hijos es reemplazado por su sucesor (el mínimo del subárbol derecho).

El caso interesante es el tercero. ¿Por qué el **sucesor** (el mínimo del subárbol derecho)? Porque es el valor **inmediatamente siguiente** en el orden: ponerlo en el lugar del eliminado deja todo lo de la izquierda menor y todo lo de la derecha mayor — invariante intacto. Y como el sucesor es un mínimo, no tiene hijo izquierdo: quitarlo de su posición original es un caso 1 o 2, fácil. (El **predecesor** —máximo del subárbol izquierdo— funciona igual de bien.)

8.3.4 Los recorridos

¿Y si quiero visitar **todos** los nodos? Hay cuatro órdenes clásicos, y la diferencia es simplemente **cuándo** se visita el nodo respecto de sus hijos:

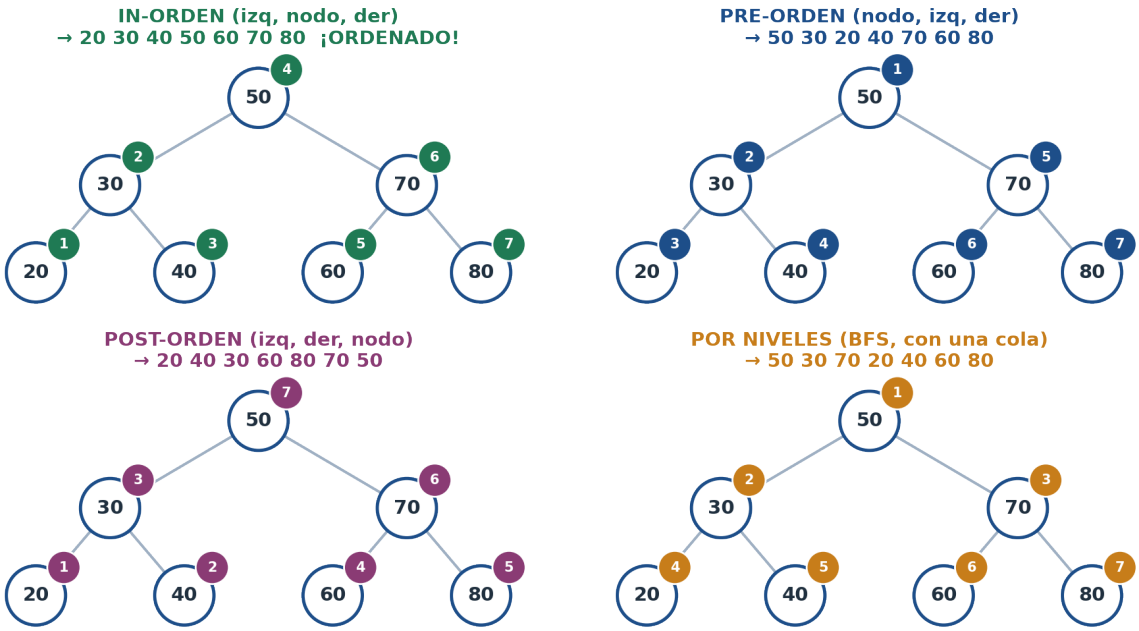


Figura 8.6: Los cuatro recorridos sobre el mismo árbol, con el orden de visita numerado en cada nodo. El in-orden de un ABB entrega los valores ordenados.

- **In-orden** (izquierda, nodo, derecha): en un ABB, entrega los valores **ordenados** — es "ordenar gratis", y la forma estándar de verificar que un árbol es realmente un ABB.

- **Pre-orden** (nodo, izquierda, derecha): visita el padre antes que los hijos; sirve para **copiar** o serializar un árbol (insertando en ese orden se reconstruye igual).
- **Post-orden** (izquierda, derecha, nodo): visita los hijos antes que el padre; sirve para **liberar** o para evaluar expresiones de abajo hacia arriba.
- **Por niveles** (BFS): visita por "pisos"; no es recursivo — usa una **cola** (Unidad 5): se desencola un nodo, se visita, y se encolan sus hijos.

8.3.5 El talón de Aquiles: el ABB degenerado

Todo lo anterior cuesta $O(h)$... y ahí está la trampa: **nadie garantiza que** h **sea chica**. El orden de inserción decide la forma del árbol, y el peor orden es el más común en la práctica: datos **ya ordenados**.

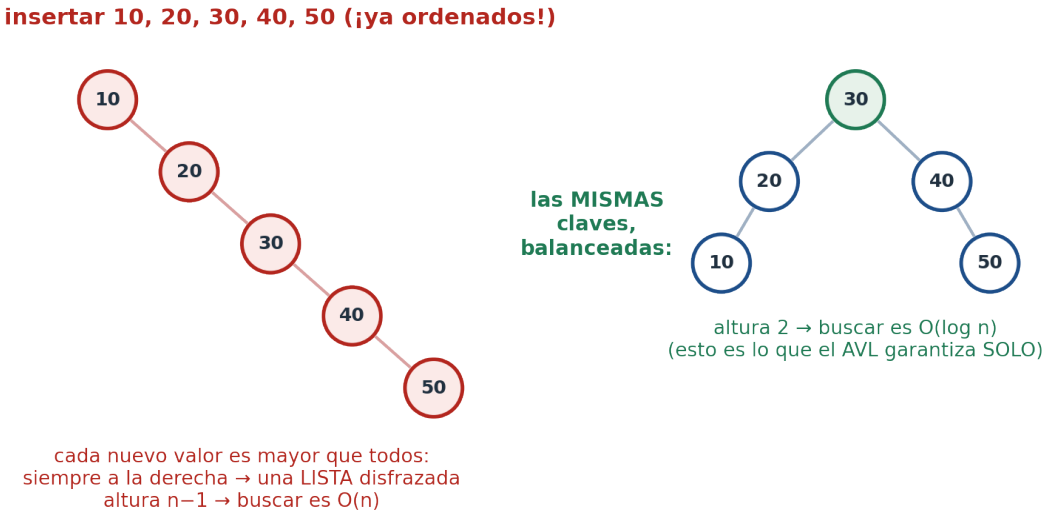


Figura 8.7: Insertar 10, 20, 30, 40, 50 en orden produce una "lista disfrazada" de altura n−1: buscar vuelve a ser O(n). Las mismas claves, balanceadas, tienen altura 2.

Un ABB construido con datos ordenados es una lista enlazada cara: $O(n)$ por operación. La solución no es cruzar los dedos: es que el árbol se **balancee solo**.

8.4 El árbol AVL: balanceo automático

El **AVL** es un ABB que, tras cada inserción o borrado, **verifica y repara** su equilibrio. La herramienta de diagnóstico es el **factor de equilibrio** de cada nodo: $FE = altura(izq) - altura(der)$.

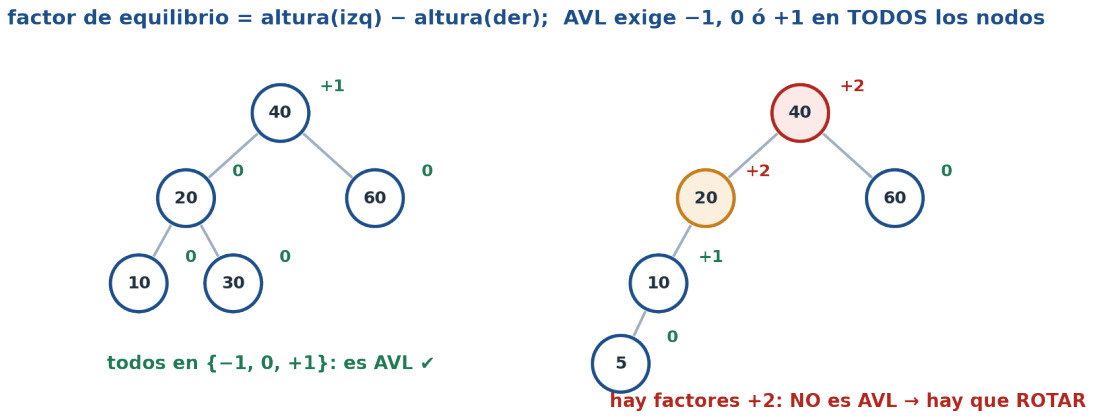


Figura 8.8: El factor de equilibrio en cada nodo. El AVL exige −1, 0 o +1 en todos; un +2 o −2 dispara una rotación.

La regla AVL: **todo nodo debe tener FE en** $\{-1, 0, +1\}$. Se permite una inclinación de un nivel (imposible pedir 0 exacto), pero no más. Se demuestra que con esa regla la altura es siempre $O(\log n)$ — como mucho ~1.44 veces el óptimo.

8.4.1 La reparación: rotaciones

Cuando una inserción deja un nodo con FE ± 2, se repara con una **rotación**: un re-enganche local de dos nodos y un subárbol que restaura el equilibrio **sin romper el orden**:

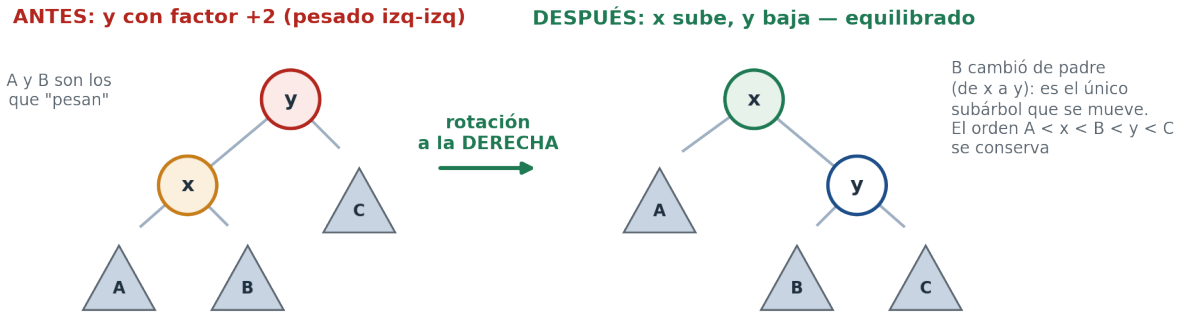


Figura 8.9: La rotación simple a la derecha: x sube, y baja, y el subárbol del medio (B) cambia de padre. El orden A < x < B < y < C se conserva; sólo cambian tres referencias.

Miralo con calma: es el re-enganche de punteros de la Unidad 5, con propósito. El único subárbol que se muda es **B** (el del medio): estaba a la derecha de x y pasa a la izquierda de y — y justamente $x < B < y$, así que queda bien en ambos lados. Tres referencias cambiadas, equilibrio restaurado, orden intacto: $O(1)$.

8.4.2 La rotación a la izquierda, paso a paso

La **rotación a la izquierda** es el espejo exacto: se usa cuando el nodo quedó **pesado hacia la derecha** (FE $= -2$) y el peso viene **derecha-derecha** (el hijo derecho tiene FE ≤ 0). El hijo derecho sube, el nodo baja a su izquierda:

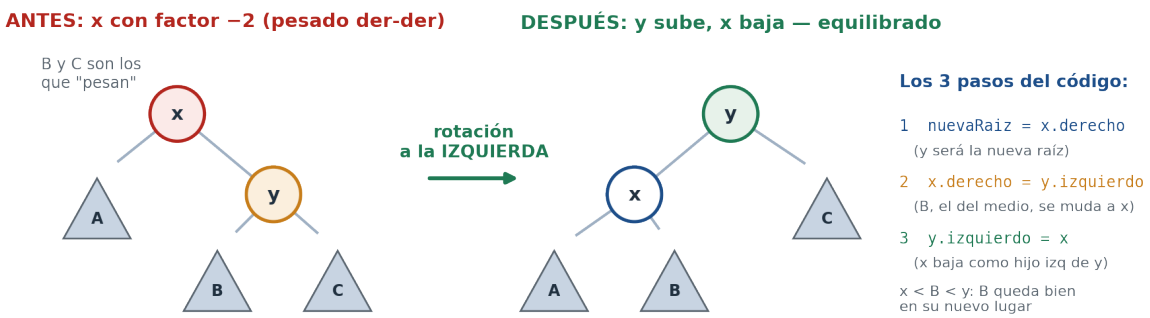


Figura 8.10: La rotación a la izquierda: y sube, x baja, y el subárbol del medio (B) se muda de y a x. A la derecha, los tres pasos del código en orden — cada uno re-engancha una referencia.

Y éste es el método completo del repo (`ArbolAVL`), con los tres pasos numerados de la figura:

```
private Nodo<T> rotarIzquierda(Nodo<T> nodo) {        // "nodo" es x
    Nodo<T> nuevaRaiz = nodo.derecho;                  // (1) y será la raíz
    nodo.derecho = nuevaRaiz.izquierdo;                // (2) B se muda: de y a x
    nuevaRaiz.izquierdo = nodo;                        // (3) x baja bajo y
    actualizarAltura(nodo);                            // primero el de abajo...
    actualizarAltura(nuevaRaiz);                       // ...después el de arriba
    return nuevaRaiz;                                  // y reemplaza a x
}
```

Leé el orden de los pasos con ojo de Unidad 5: **(1)** guarda la referencia a `y` antes de tocar nada (si no, se pierde); **(2)** el subárbol del medio (`y.izquierdo`, nuestro **B**) pasa a colgar de `x` — puede hacerlo porque $x < B < y$: lo que estaba a la izquierda de `y` está a la derecha de `x`; **(3)** recién entonces `x` baja como hijo izquierdo de `y`. Y las alturas se actualizan **de abajo hacia arriba**: primero `x` (que ya tiene sus hijos definitivos), después `y` (cuya altura depende de la de `x`). El `return` le entrega la nueva raíz del subárbol al padre — por eso en el código del AVL las rotaciones se llaman como `nodo = rotarIzquierda(nodo)`.

8.4.3 ¿Y si el desequilibrio viene “en zig-zag”? La doble izquierda-derecha

Cuando el peso no está en la punta sino en el hijo **del medio** —caso **izquierda-derecha** (o su espejo derecha-izquierda)— una sola rotación **no alcanza**. ¿Por qué? Probalo: si al `z` desequilibrado le aplicás directamente la rotación a la derecha, el subárbol pesado (y y los suyos) **se muda entero de lado**... y el árbol queda desequilibrado para el otro lado. La rotación simple sabe levantar el peso de la **punta**, no el del **medio**.

La solución es en **dos tiempos**: primero una rotación **al hijo** (`x`), que convierte el zig-zag en una **línea** — el caso izquierda-izquierda que ya sabemos arreglar —, y después la rotación conocida **al nodo** (`z`):

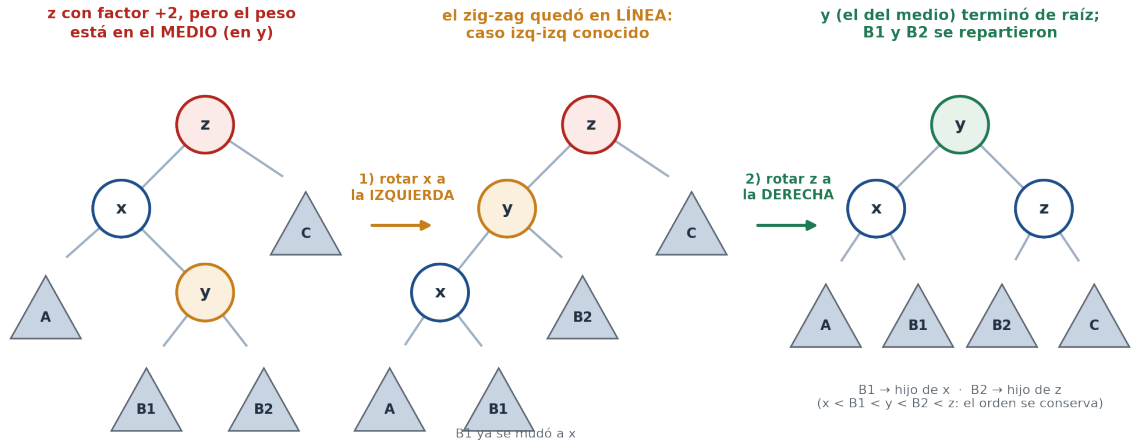


Figura 8.11: El caso izquierda-derecha en general, con todos los subárboles. Paso 1: rotar x a la izquierda endereza el zig-zag (B1 se muda a x). Paso 2: rotar z a la derecha sube a y. Los DOS subárboles de y se reparten: B1 queda bajo x, B2 bajo z — y el orden x < B1 < y < B2 < z se conserva.

Fijate el detalle que hace funcionar todo: y, el nodo del medio, tiene **dos** subárboles (B1 y B2), y la doble los **reparte**: B1 termina bajo x (vale, porque $x < B1 < y$) y B2 termina bajo z (vale, porque $y < B2 < z$). Por eso y puede subir a la raíz con las manos libres. En código no hay nada nuevo que escribir: son **las dos rotaciones simples, una tras otra** — `nodo.izquierdo = rotarIzquierda(nodo.izquierdo); return rotarDerecha(nodo);`.

Con números, el ejemplo mínimo — insertar 30, 10, 20:

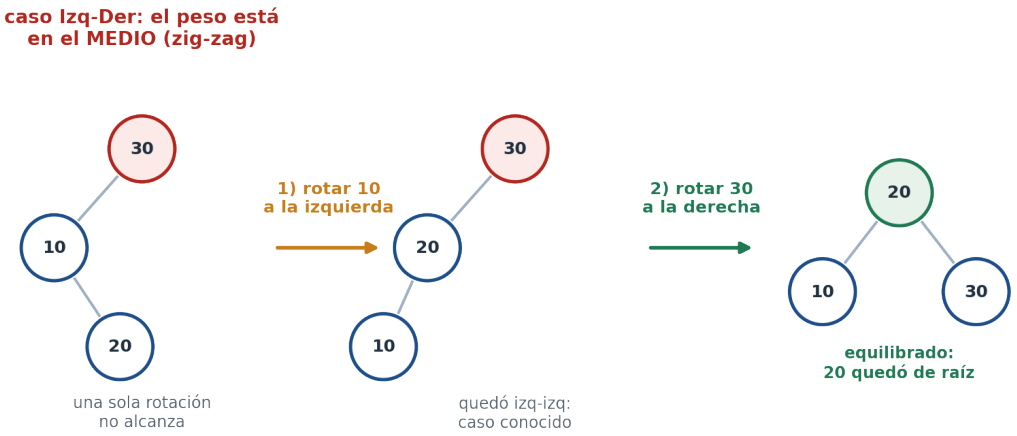


Figura 8.12: La rotación doble (caso izq-der) con el ejemplo mínimo 30, 10, 20: primero se rota 10 a la izquierda (queda izq-izq), después 30 a la derecha. El del medio (20) termina de raíz.

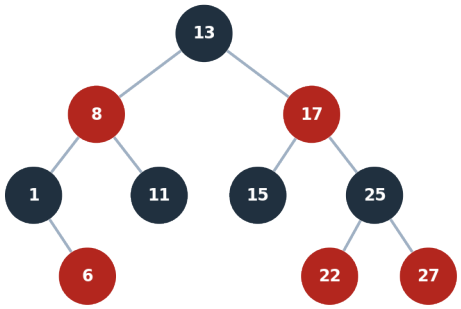
Regla mnemotécnica: en los cuatro casos (LL, RR, LR, RL), **el que termina arriba es el "del medio"** en valor. El código completo —con los cuatro casos en un solo método `balancear`— está en el repo, `ArbolAVL`; su `main` muestra que insertar 1..15 **en orden** deja altura 3 y raíz 8: el árbol perfecto que el ABB común jamás lograría con esa entrada.

```
private Nodo<T> balancear(Nodo<T> nodo) {
    actualizarAltura(nodo);
    int factor = factorDeEquilibrio(nodo);
    if (factor > 1) {                                  // pesado a la IZQUIERDA
        if (factorDeEquilibrio(nodo.izquierdo) < 0)    // zig-zag: caso Izq-Der
```

```
            nodo.izquierdo = rotarIzquierda(nodo.izquierdo);
        return rotarDerecha(nodo);                    // caso Izq-Izq
    }
    if (factor < -1) {                                // pesado a la DERECHA (simétrico)
        if (factorDeEquilibrio(nodo.derecho) > 0) {
            nodo.derecho = rotarDerecha(nodo.derecho);
        }
        return rotarIzquierda(nodo);
    }
    return nodo;
}
```

8.5 El árbol rojo-negro (la idea)

El AVL no es el único auto-balanceado. El **rojo-negro** relaja el criterio: en vez de vigilar alturas exactas, **pinta** cada nodo de rojo o negro y exige unas pocas reglas de color:



Las reglas del árbol rojo-negro:

1. cada nodo es rojo o negro
2. la raíz es negra
3. un rojo no tiene hijos rojos
4. todo camino raíz→hoja tiene la MISMA cantidad de negros

consecuencia: la rama más larga mide a lo sumo el DOBLE que la más corta → altura O(log n) garantizada

(es el árbol del TreeMap/TreeSet de Java)

Figura 8.13: Un árbol rojo-negro y sus reglas. La consecuencia de las reglas: ninguna rama puede medir más del doble que otra, así que la altura es O(log n).

El razonamiento: como un rojo no puede tener hijos rojos (regla 3) y todos los caminos tienen los mismos negros (regla 4), el camino más largo posible alterna rojo-negro y el más corto es todo negro: a lo sumo el **doble**. Altura $O(\log n)$ garantizada, con menos rotaciones que el AVL al modificar. Por eso es el elegido de las bibliotecas — **`TreeMap` y `TreeSet` son árboles rojo-negro**. El AVL queda algo más bajo (mejor para cargas de mucha lectura); el rojo-negro modifica más barato. Sus algoritmos de reparación exceden el curso: nos quedamos con las reglas, la garantía y saber que lo usás cada vez que tocás un `TreeMap`.

8.6 Árboles B y B+ (la idea)

¿Y si el árbol no entra en memoria? En disco, leer un nodo cuesta un acceso completo a un bloque (miles de veces más lento que la RAM): conviene que cada lectura traiga **mucha** información y que el árbol sea **bajo**. Ésa es la idea de los árboles **B**: nodos anchos, con **muchas claves y muchos hijos** por nodo.

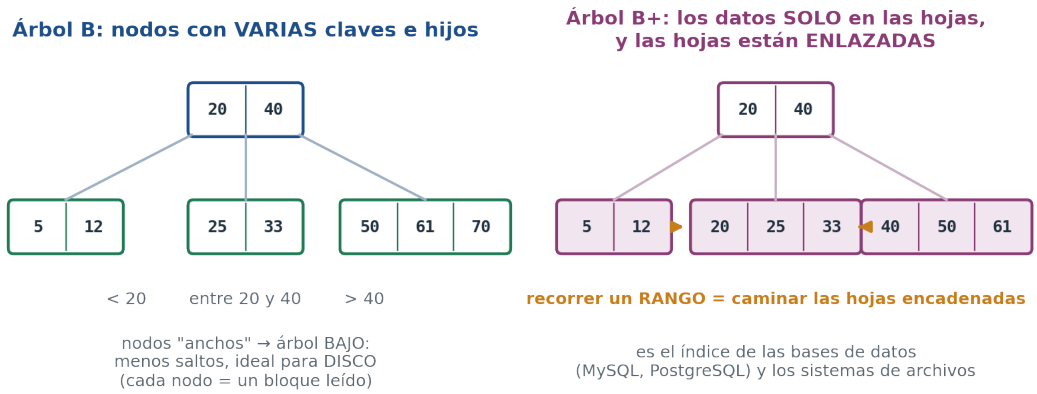


Figura 8.14: El árbol B usa nodos con varias claves (árbol bajo: ideal para disco). El B+ guarda los datos sólo en las hojas y las encadena: recorrer un rango es caminar las hojas.

La variante **B+** agrega dos refinamientos: los datos viven **sólo en las hojas** (los nodos internos son puro índice, así entran más claves por bloque), y las hojas están **enlazadas** entre sí — un rango ("todos los legajos entre 1000 y 2000") se responde bajando una vez y caminando las hojas, como una lista. Con miles de claves por nodo, un B+ de altura 3 indexa **miles de millones** de registros: por eso es el índice de MySQL y PostgreSQL, y de los sistemas de archivos. No lo implementamos: lo esencial es reconocer **por qué existe** (el costo del disco) y **dónde lo vas a encontrar** (todas las bases de datos).

8.7 De lo nuestro a la API de Java

Lo que hicimos	En la API de Java	Por dentro	Costo
`ArbolBinarioDeBusqueda`, `ArbolAVL`	`TreeMap` / `TreeSet`	rojo-negro	$O(\log n)$
— (comparación)	`HashMap` / `HashSet`	tabla de hash (U7)	$O(1)$ prom., sin orden

Código de uso — lo que un árbol sabe hacer y un hash no:

```
NavigableMap<Integer, String> porLegajo = new TreeMap<>();
porLegajo.put(1300, "ana");  porLegajo.put(1100, "juan");
porLegajo.put(1500, "mia");  porLegajo.put(1200, "leo");

porLegajo.keySet();                    // [1100, 1200, 1300, 1500]  SIEMPRE ordenado
porLegajo.firstKey();                  // 1100  (mínimo)
porLegajo.subMap(1150, true, 1350, true);   // {1200=leo, 1300=ana}  ¡rango!
porLegajo.floorKey(1250);              // 1200  (el mayor <= 1250)
porLegajo.ceilingKey(1250);            // 1300  (el menor >= 1250)

NavigableSet<String> nombres = new TreeSet<>(Set.of("mia", "ana", "leo"));
nombres.first();                       // ana
nombres.headSet("leo");                // [ana, juan...] los menores que "leo"
```

Idea clave

La decisión de diseño de la materia: ¿`HashMap` o `TreeMap`? Si sólo necesitás buscar por clave exacta → **hash,** $O(1)$. Si necesitás orden, rangos, mínimo/máximo o "el más cercano a X" → **árbol,** $O(\log n)$. Ninguno es mejor: responden preguntas distintas. Ésa es la respuesta completa a la comparación que abrimos en la Unidad 7.

Casos de uso del árbol ordenado: rankings y agendas (orden al recorrer), reservas y turnos ("el próximo horario libre desde las 15:00" = `ceilingKey`), facturación por período (`subMap`), diccionarios reales (palabras ordenadas), e índices de bases de datos (B+). **En contra**: $O(\log n)$ contra el $O(1)$ del hash, y exige claves comparables.

8.8 Ejemplo integrador: autocompletar

Un autocompletador de buscador: escribís `"arb"` y sugiere `arbol`, `arboleda`, `arbusto`. La observación clave: **las palabras con un prefijo dado forman un rango contiguo** en el orden alfabético — de `"arb"` (inclusive) hasta el primer string que ya no empieza así. Un `TreeSet` responde eso con `subSet`:

```
public List<String> sugerir(String prefijo, int limite) {
    String desde = prefijo;
    String hasta = prefijo + Character.MAX_VALUE;   // cota superior del rango
    List<String> sugerencias = new ArrayList<>();
    for (String p : palabras.subSet(desde, true, hasta, false)) {
        if (sugerencias.size() == limite) {
            break;
        }
        sugerencias.add(p);                          // ya salen ORDENADAS
    }
    return sugerencias;
}
```

Con un `HashSet` este problema no tiene solución razonable (habría que recorrer **todo** el conjunto); con el árbol, ubicar el inicio del rango cuesta $O(\log n)$ y cada sugerencia sale en orden. Probalo en el repo: `Autocompletar`.

8.9 Errores comunes

Error común

- Confundir árbol binario con ABB: el binario sólo limita a dos hijos; el ABB además **ordena**.
- Decir que buscar en un ABB "es $O(\log n)$" a secas: es $O(h)$, y **sin balanceo** h puede ser n (datos ordenados → lista).
- En quitar con dos hijos, reemplazar por un hijo directo en vez del **sucesor**: rompe el invariante.
- Olvidar que el in-orden del ABB da los valores **ordenados** (y usarlo al revés: si el in-orden no sale ordenado, el árbol no es un ABB).
- Creer que el AVL rota "de a muchos": cada rotación es $O(1)$ y re-engancha **tres** referencias.
- En el examen de factores: el FE se calcula con **alturas**, no con cantidades de nodos.
- Usar `TreeMap` cuando sólo se busca por clave exacta (pagás $\log n$ al pedo) — o `HashMap` cuando se necesitan rangos (no los puede responder).

8.10 Repaso

1. Definí raíz, hoja, nivel y altura. ¿Por qué la altura es "el" número de la unidad?
2. Enunciá el invariante del ABB. ¿Por qué habilita el descarte de medio árbol por comparación?
3. Ejecutá a mano `insertar(35)` en el árbol 50/30/70/20/40/60/80: ¿qué camino sigue y dónde queda?
4. Explicá los tres casos de `quitar`. En el caso de dos hijos, ¿por qué el sucesor mantiene el invariante?
5. Escribí los cuatro recorridos del árbol 50/30/70/20/40/60/80 y decí para qué sirve cada uno.
6. ¿Qué pasa si insertás 1, 2, 3, …, n en orden en un ABB común? ¿Qué altura queda y cuánto cuesta buscar?
7. ¿Qué es el factor de equilibrio? ¿Qué valores admite el AVL?

8. Insertá 10, 20, 30 en un AVL y mostrá la rotación que se dispara. ¿Quién queda de raíz?
9. ¿Cuándo hace falta rotación doble? ¿Quién termina arriba en los cuatro casos?
10. Escribí de memoria los tres pasos de `rotarIzquierda`. ¿Por qué el paso (1) va primero, y por qué las alturas se actualizan de abajo hacia arriba?
11. En la doble izquierda-derecha, ¿por qué la rotación simple sola no alcanza? ¿Qué pasa con los dos subárboles del nodo del medio?
12. Enunciá las reglas del rojo-negro y explicá por qué garantizan altura $O(\log n)$.
13. ¿Por qué los árboles B tienen nodos “anchos”? ¿Qué agrega el B+ y por qué es el índice de las bases de datos?
14. ¿`HashMap` o `TreeMap`? Dá dos casos de uso para cada uno y justificá con complejidades.
15. En el autocompletador, ¿por qué las palabras con un prefijo forman un rango? ¿Qué operación de `TreeSet` lo resuelve?

Guía de ejercicios — Unidad 8

Objetivos de la unidad

Practicá **árboles**. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i08_arboles` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i08_arboles/i01_teorico`. Fácil y medio usan la **API nativa** (`TreeMap`/`TreeSet`) o árboles chicos con nodos propios; el difícil implementa **ABB y AVL a mano**.

8.11 Preguntas teóricas

1. ¿Qué diferencia hay entre un árbol binario cualquiera y un ABB?
2. ¿Por qué todas las operaciones del ABB cuestan $O(h)$? ¿Cuándo h es $\log n$ y cuándo es n?
3. Explicá los tres casos de `quitar` en un ABB.
4. ¿Por qué el in-orden de un ABB entrega los valores ordenados?
5. ¿Qué recorrido usarías para copiar un árbol y cuál para liberarlo? ¿Por qué?
6. ¿Qué es el factor de equilibrio y qué exige el AVL?
7. ¿Qué hace una rotación simple y por qué preserva el orden? ¿Cuándo hace falta la doble?
8. Enunciá las reglas del árbol rojo-negro y su consecuencia sobre la altura.
9. ¿Por qué los árboles B/B+ son la estructura de los índices en disco?
10. ¿Cuándo conviene `TreeMap` sobre `HashMap`? Dá tres consultas que sólo el árbol responde bien.

8.12 El ejercicio de simulación: insertar, buscar y eliminar en cada árbol

Éste es **el ejercicio de entrenamiento para el parcial**, y es único por estudiante. En el repo está el generador: `material.i08_arboles.GeneradorDeEjercicios`. Ejecutalo pasándole tu **padrón** como argumento (en IntelliJ: *Run → Edit Configurations → Program arguments*):

```
java ... GeneradorDeEjercicios 104523
```

El programa **sortea un tipo de árbol** (ABB, AVL o Árbol B de orden 3) y genera **tu** secuencia: 40 números a insertar, 6 búsquedas y 8 eliminaciones. Con eso:

1. **Insertá** los 40 valores a mano, dibujando el árbol tras los primeros 10, tras 20, tras 30 y tras los 40. En el AVL, anotá **cada rotación** (tipo y nodo); en el B, **cada división** de nodo.
2. **Buscá** los 6 valores: para cada uno, decí si está y **cuántos nodos visitaste** (ésa es la altura trabajando a tu favor… o en contra).
3. **Eliminá** los 8 valores en orden, dibujando el árbol **después de cada eliminación** e indicando el caso (hoja / un hijo / dos hijos — y en el AVL, qué rotación disparó; en el B, qué fusión o préstamo).
4. **Verificá** cada etapa en el simulador del tipo que te tocó (la misma secuencia, en el mismo orden):

Árbol	Simulador para verificar (y para corregir)
ABB	https://cmps-people.ok.ubc.ca/ylucet/DS/BST.html
AVL	https://cmps-people.ok.ubc.ca/ylucet/DS/AVLtree.html
Árbol B (orden 3)	https://cmps-people.ok.ubc.ca/ylucet/DS/BTree.html (con *Max. Degree = 3*)

(Los tres son el mismo sitio de visualizaciones — Universidad de British Columbia Okanagan —, así

que la interfaz es idéntica; existe un espejo en https://www.cs.usfca.edu/~galles/visualization/Algorithms.html por si el primero no responde.)

Se entrega el **PDF con los dibujos** de los puntos de control (foto del papel o pizarra digital, da igual) con tu padrón visible. La cátedra corrige **regenerando tu ejercicio con tu padrón** y reproduciéndolo en el simulador: la semilla hace que tu secuencia sea exactamente reproducible.

8.13 Nivel fácil (con la API de Java)

1. **Ranking.** Con un `TreeMap<Integer, String>` puntaje→jugador, devolvé los 3 mejores en orden descendente (`descendingMap`). Test.
2. **Próximo turno.** Con un `TreeMap<LocalTime, String>` de turnos, `String proximoDesde(LocalTime hora)` con `ceilingEntry`. Test.
3. **Palabras ordenadas.** De una lista con repetidos, devolvé las palabras únicas en orden alfabético (`TreeSet`). Test.
4. **Rango de legajos.** Con un `TreeMap<Integer, String>`, devolvé los pares con legajo en `[desde, hasta]` (`subMap`). Test.
5. **Extremos.** `int[] minimoYMaximo(Collection<Integer>)` usando `TreeSet` (`first`/`last`). Test.
6. **El más cercano.** `Integer masCercano(TreeSet<Integer> valores, int x)` usando `floor` y `ceiling` (empate: el menor). Test.
7. **Agenda alfabética.** `TreeMap<String, String>` nombre→teléfono: alta, baja, y "los contactos cuyo nombre empieza con una letra dada" (`subMap`). Test.
8. **Frecuencias ordenadas.** Contá palabras con un `TreeMap<String, Integer>` y devolvé el listado en orden alfabético. Test.
9. **Mediana incremental (simple).** Con un `TreeSet<Integer>` de valores distintos, devolvé el k-ésimo menor recorriendo (k chico). Test.
10. **Anteriores y siguientes.** Para un valor x en un `TreeSet`, devolvé su anterior (`lower`) y su siguiente (`higher`), o null en los bordes. Test.

8.14 Nivel medio (nodos propios y API)

1. **Altura y tamaño.** Con una clase `Nodo` propia (valor, izq, der), `int altura(Nodo)` e `int tamanio(Nodo)` recursivos. Test.
2. **Recorridos.** `List<Integer> enOrden/preOrden/postOrden(Nodo)` recursivos. Test contra un árbol armado a mano.
3. **Por niveles.** `List<Integer> porNiveles(Nodo)` con una cola (`ArrayDeque`). Test.
4. **¿Es ABB?** `boolean esABB(Nodo)` verificando el invariante con cotas (mín/máx por rama). Test con un árbol que engaña (viola el invariante con un nieto).
5. **Contar hojas y niveles.** `int hojas(Nodo)` y `int nodosEnNivel(Nodo, int nivel)`. Test.
6. **Espejo.** `Nodo espejo(Nodo)`: el árbol con izquierda y derecha intercambiadas en todos los niveles. Test (el in-orden se invierte).
7. **Camino a un valor.** `List<Integer> caminoHasta(Nodo raiz, int valor)` en un ABB (la rama desde la raíz), vacía si no está. Test.
8. **Mínimo ancestro común.** `int ancestroComun(Nodo raizABB, int a, int b)` aprovechando el invariante del ABB (sin recorrer todo). Test.
9. **ABB desde ordenado.** `Nodo desdeOrdenado(int[] ordenado)`: construí un ABB balanceado tomando el medio como raíz (recursivo). Test: altura ≈ log n.
10. **Validar AVL.** `boolean esAVL(Nodo)`: chequeá factores de equilibrio de todos los nodos (con alturas recursivas). Test con uno que sí y uno que no.

8.15 Nivel difícil (implementados a mano)

1. **ABB completo.** `ArbolBusqueda<T extends Comparable<T>>`: insertar, contiene, tamanio, enOrden. Test (incluí el no-duplicado).

2. **Quitar en ABB.** Agregale `quitar` con los tres casos; test que quita hoja, nodo con un hijo y nodo con dos hijos, verificando el in-orden.
3. **Mínimo, máximo, piso y techo.** En tu ABB: `minimo()`, `maximo()`, `piso(x)` (mayor valor ≤ x) y `techo(x)` (menor ≥ x), iterativos. Test.
4. **Iterador in-orden.** Un iterador de tu ABB con una **pila explícita** (sin recursión ni lista auxiliar): `hasNext`/`next` entregan los valores en orden. Test.
5. **Rotaciones.** Sobre nodos con altura cacheada: `rotarDerecha` y `rotarIzquierda`; test que verifica raíces, hijos y alturas tras rotar un zig y un zag.
6. **AVL insertar.** `ArbolAVL<T>` con insertar + balancear (4 casos); test: insertar 1..15 en orden deja altura 3 y raíz 8; insertar 30,10,20 deja raíz 20.
7. **AVL quitar.** Agregale `quitar` re-balanceando; test que fuerza una rotación al borrar (armá un árbol donde borrar una hoja desbalancee).
8. **Serializar y reconstruir.** `String aPreOrden(arbol)` (valores separados por coma) y `reconstruir(String)` que inserta en ese orden; test: el árbol reconstruido tiene el mismo pre-orden e in-orden.
9. **K-ésimo menor.** En tu ABB con **tamaño de subárbol** cacheado en cada nodo, `T kEsimo(int k)` en $O(h)$. Test.
10. **Comparativa ABB vs AVL.** Un test que inserta 1..100 en orden en tu ABB (e02) y en tu AVL (e06) y verifica: mismo in-orden, pero altura 99 contra ≤ 7.

Unidad 9

Colas de prioridad y heaps

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) definir el TDA **Cola de Prioridad** y distinguirlo de la cola FIFO; (2) explicar el **heap binario**: árbol completo en un arreglo, su invariante y la aritmética de índices; (3) ejecutar a mano **encolar** (flotar) y **desencolar** (hundir); (4) construir un heap desde un arreglo con **heapify** en $O(n)$ — con su versión correcta; (5) ordenar con **Heapsort**; y (6) usar `PriorityQueue` con comparadores y desempates.

La cola de la Unidad 5 atiende "al que llegó primero". Pero muchas situaciones reales no funcionan así: en una guardia médica pasa primero **el más grave**; un sistema operativo ejecuta primero el proceso **más urgente**; un navegador descarga primero lo **más visible**. Eso pide otro TDA: una cola donde el turno lo decide la **prioridad** — y una estructura nueva, el **heap**, que lo resuelve con elegancia.

9.1 Un poco de historia

El heap nació dentro de un algoritmo de ordenamiento: **J. W. J. Williams** lo inventó en 1964 como el motor de su **Heapsort**, y ese mismo año **Robert Floyd** aportó la construcción en tiempo lineal (el heapify que vamos a ver). Con los años el heap se independizó del ordenamiento y se volvió la implementación estándar de la cola de prioridad — la pieza interna de algoritmos famosos que te cruzarás pronto, como **Dijkstra** (caminos mínimos, Unidad de Grafos) y la compresión de **Huffman**.

9.2 El TDA Cola de Prioridad

El contrato es corto: **encolar** un elemento, y **desencolar siempre el de mejor prioridad** (el mínimo o el máximo, según la variante). También: ver el primero sin sacarlo, tamaño y vacío. La diferencia con la cola FIFO está en **quién sale**: la FIFO mira la **llegada**; la de prioridad mira el **valor**.

¿Con qué la implementamos? Repasemos candidatos con lo que ya sabés. ¿Lista ordenada? Desencolar es $O(1)$, pero encolar manteniendo el orden es $O(n)$. ¿Lista desordenada? Encolar $O(1)$, pero desencolar busca el mínimo: $O(n)$. ¿ABB balanceado (U8)? Todo $O(\log n)$... pero mantiene **orden total**, mucho más del que necesitamos: sólo pedimos el **mínimo**. El heap logra lo mismo que el AVL para este contrato, con menos estructura, menos memoria y constantes mejores.

9.3 El heap binario: un árbol completo en un arreglo

Un **heap binario** es un árbol binario con dos condiciones. Primera, de **forma**: es **completo** — todos los niveles llenos salvo quizás el último, que se llena de izquierda a derecha. Esa regularidad tiene un premio enorme: el árbol se puede guardar en un **arreglo, sin nodos ni referencias**, porque la posición determina el parentesco:

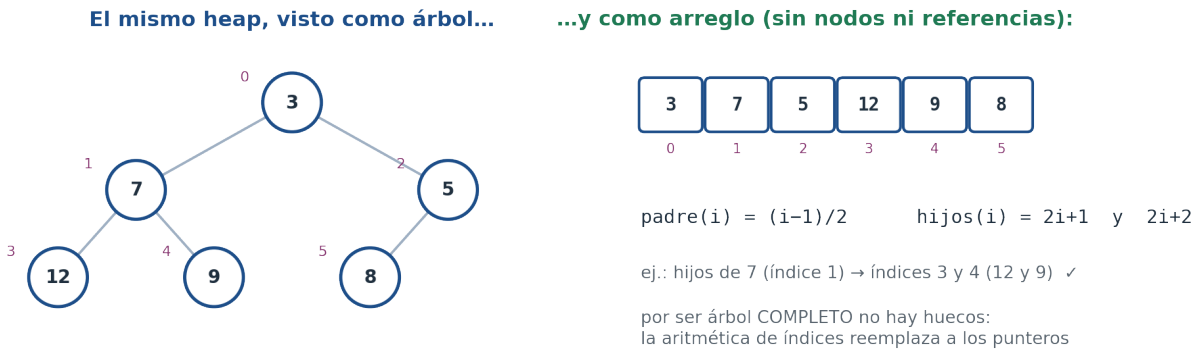


Figura 9.1: El mismo heap como árbol y como arreglo. Por ser completo no hay huecos: el padre de i está en (i−1)/2 y sus hijos en 2i+1 y 2i+2.

Segunda condición, de **contenido** — el invariante de heap (versión min-heap): **cada padre es menor o igual que sus hijos**. Consecuencia inmediata: el mínimo del conjunto está **siempre en la raíz** (índice 0), listo para leerse en $O(1)$.

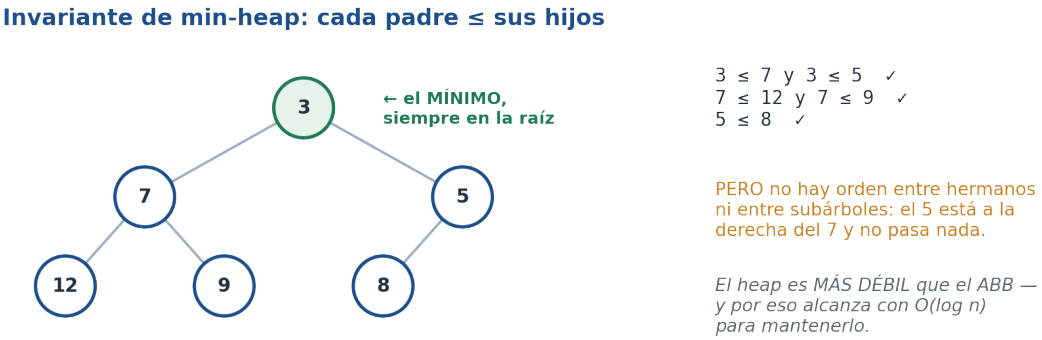


Figura 9.2: El invariante de min-heap: cada padre ≤ sus hijos. No hay ningún orden entre hermanos: es una condición más débil que la del ABB, y por eso es más barata de mantener.

Fijate lo que el invariante **no** pide: nada ordena a los hermanos entre sí (el 5 puede estar a la derecha del 7). El heap está "casi desordenado" — sólo lo justo para conocer al mínimo. Esa debilidad deliberada es su virtud: se mantiene con menos trabajo que un ABB.

9.4 Encolar: entrar al final y flotar

Para agregar un elemento hay que cuidar las dos condiciones a la vez. La forma se cuida sola: el nuevo entra en **la única posición que mantiene el árbol completo**: la primera libre del arreglo (la última hoja). Después se repara el invariante: si el nuevo es menor que su padre, se intercambian — y se repite hacia arriba. Es la operación **flotar** (*sift-up*):

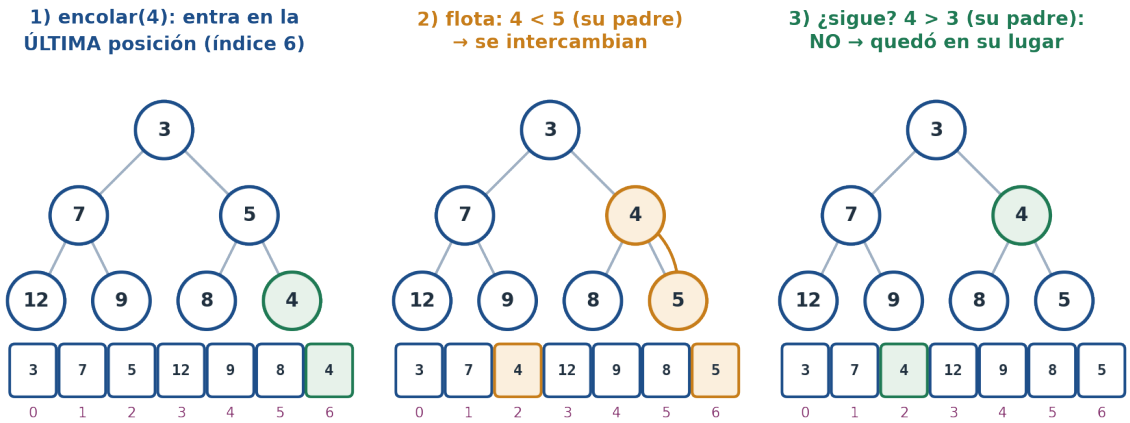


Figura 9.3: Encolar 4, paso a paso: entra en la última posición (índice 6), flota intercambiándose con su padre 5, y se detiene al encontrar un padre menor (3). Cada intercambio sube un nivel: O(log n).

```java
public void encolar(T valor) {
    datos[cantidad] = valor;         // la última hoja: única posición "completa"
    flotar(cantidad);
    cantidad++;
}

private void flotar(int i) {
    while (i > 0) {
        int padre = (i - 1) / 2;
        if (comparar(i, padre) >= 0) {
            break;   // ya no es menor: quedó bien
        }
        intercambiar(i, padre);
        i = padre;
    }
}
```

Como el árbol completo tiene altura $\lfloor \log_2 n \rfloor$, flotar hace a lo sumo $O(\log n)$ intercambios.

9.5 Desencolar el mínimo: la última hoja a la raíz, y hundir

Sacar el mínimo es sacar **la raíz** — pero un árbol sin raíz no es un árbol. El truco tiene dos tiempos: la **última hoja** (que es la única celda que puede desaparecer sin romper la forma) se muda a la raíz, y después se **hunde** (*sift-down*) hasta recuperar el invariante:

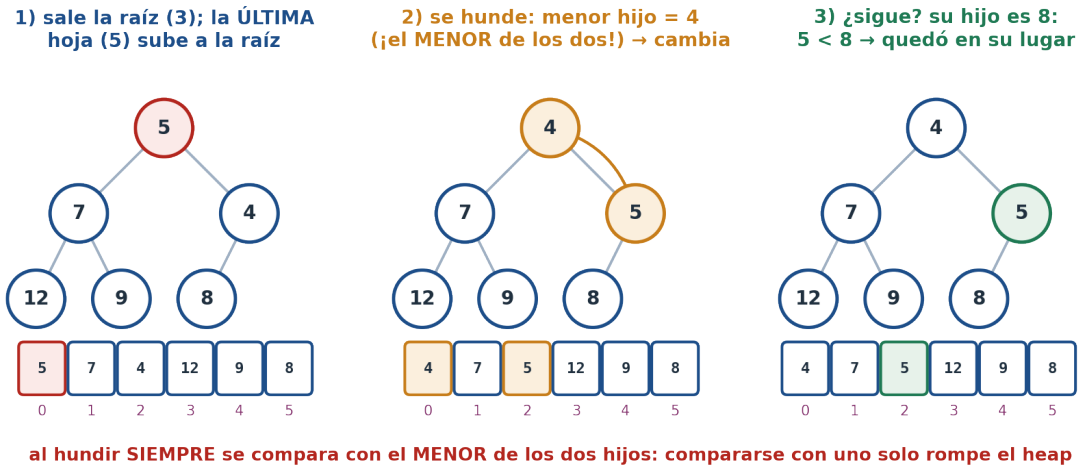


Figura 9.4: Desencolar el mínimo: sale el 3, la última hoja (5) sube a la raíz y se hunde intercambiándose con el MENOR de sus hijos hasta quedar bien ubicada.

```
private void hundir(int i) {
    while (true) {
        int izquierdo = 2 * i + 1, derecho = 2 * i + 2;
        int menor = i;
        if (izquierdo < cantidad && comparar(izquierdo, menor) < 0) {
            menor = izquierdo;
        }
        if (derecho < cantidad && comparar(derecho, menor) < 0) {
            menor = derecho;
        }
        if (menor == i) {
            break;   // ya es menor que ambos hijos
        }
        intercambiar(i, menor);
        i = menor;
    }
}
```

Error común

El error clásico de hundir (corrección de apuntes viejos, incluido el nuestro): al hundir hay que comparar el nodo con **el menor de sus dos hijos** y recién entonces intercambiar. Las versiones con bug comparan contra **un solo hijo** (o contra el izquierdo "por defecto"): el intercambio puede subir al hijo equivocado, dejando un padre mayor que su otro hijo — un heap **inválido** que no lanza ningún error, sólo entrega resultados mal ordenados de vez en cuando. En `MinHeap.hundir` del repo está la versión correcta; compará las tres líneas que eligen `menor`.

9.6 Heapify: construir un heap en O(n)

¿Y si ya tengo un arreglo entero y quiero volverlo heap? Encolar sus n elementos uno a uno cuesta $O(n \log n)$. Hay algo mejor — el **heapify** de Floyd: tratá al arreglo como un árbol tal cual está, y **hundí cada nodo interno, desde el último hacia la raíz**:

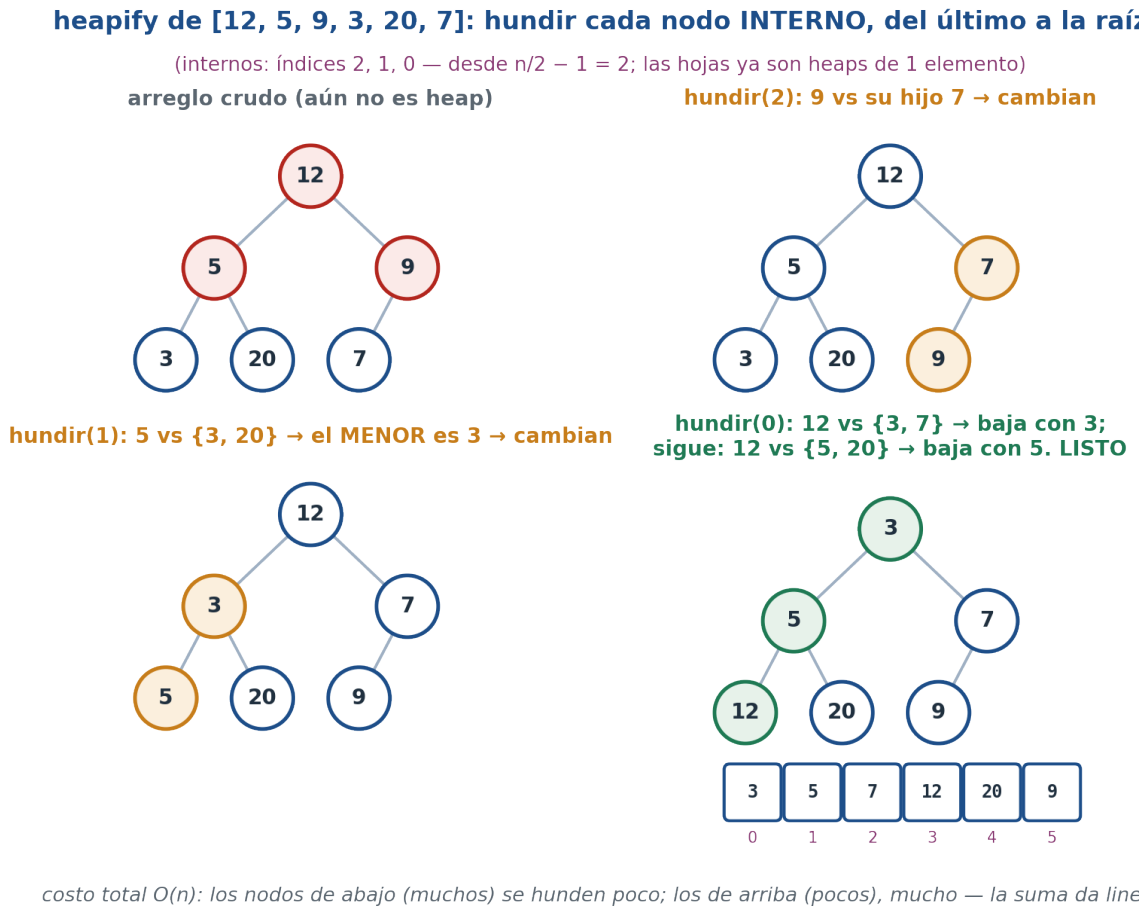


Figura 9.5: Heapify de [12, 5, 9, 3, 20, 7]: se hunden los nodos internos (índices 2, 1, 0, en ese orden). Al llegar a la raíz, todos los subárboles de abajo ya son heaps válidos.

¿Por qué de abajo hacia arriba? Porque hundir un nodo **presupone que sus dos subárboles ya son heaps** — y recorriendo desde el último interno hacia la raíz, esa condición siempre está garantizada al llegar a cada nodo. Las hojas (la mitad del arreglo) ni se tocan: ya son heaps de un elemento.

Error común

El otro error clásico (también de apuntes viejos): recorrer "desde la raíz hacia abajo" flotando o hundiendo en el orden 0, 1, 2, … **no** construye un heap correcto: al hundir un nodo temprano, sus subárboles todavía no son heaps, y el invariante queda roto más abajo. El orden correcto es **del último nodo interno ($n/2 - 1$) hacia el 0**.

¿Y por qué cuesta $O(n)$ y no $O(n \log n)$? El argumento es un primo del amortizado de la Unidad 4: la **mitad** de los nodos son hojas y se hunden 0 niveles; un cuarto puede hundirse 1 nivel; un octavo, 2… La suma $\sum \frac{n}{2^{k+1}} \cdot k$ converge a $O(n)$. Los que podrían hundirse mucho son poquísimos, y los muchos casi no se hunden.

9.7 Heapsort: ordenar con el heap

El heap nació para ordenar, y el algoritmo es un broche perfecto para el tema de ordenamientos (U6). Dos fases sobre el mismo arreglo, **sin memoria extra**: se hace heapify como **max-heap** ($O(n)$), y luego n veces: el máximo (raíz) se intercambia con la última posición del heap "vivo", el heap se achica en 1, y la nueva raíz se hunde ($O(\log n)$):

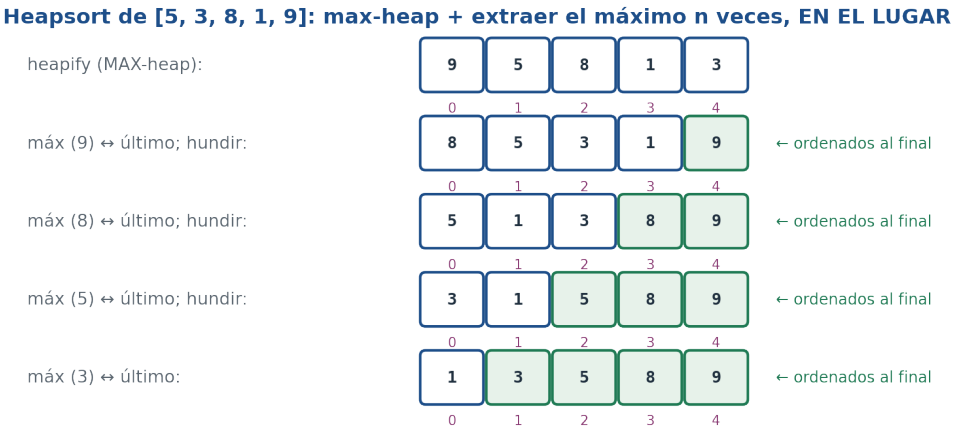


Figura 9.6: Heapsort de [5, 3, 8, 1, 9]: heapify como max-heap y n extracciones. En cada paso el máximo pasa al final (zona verde, ya ordenada) y el heap vivo (azul) se achica.

Total: $O(n \log n)$ **siempre** — sin el peor caso $O(n^2)$ de Quicksort y sin la memoria extra $O(n)$ de Mergesort. ¿El precio? No es estable y sus constantes son algo peores (salta por la memoria sin orden previsible). Completa la tabla comparativa de la Unidad 6; el código está en el repo (`Heapsort`).

9.8 La complejidad, junta

Operación	Heap binario	Lista ordenada	Lista desordenada	AVL
ver mínimo	$O(1)$	$O(1)$	$O(n)$	$O(\log n)$
encolar	$O(\log n)$	$O(n)$	$O(1)$	$O(\log n)$
desencolar mínimo	$O(\log n)$	$O(1)$	$O(n)$	$O(\log n)$
construir con n elementos	$O(n)$ heapify	$O(n \log n)$	$O(n)$	$O(n \log n)$

El heap gana el combinado: nada por debajo de él en ninguna fila que importe, y además vive en un arreglo compacto.

9.9 De lo nuestro a la API de Java

La cola de prioridad nativa es **`PriorityQueue`** — un min-heap binario, la que se usa en parciales, TPs y finales:

```
PriorityQueue<Integer> cola = new PriorityQueue<>();
cola.offer(7); cola.offer(3); cola.offer(9);      // encolar
cola.peek();                                       // 3: el mínimo, sin sacarlo
cola.poll();                                       // 3: lo saca
cola.size();

// max-heap: alcanza con invertir el comparador
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

// prioridad por un campo, con DESEMPATE por llegada
record Tarea(String nombre, int prioridad, long llegada) {}
PriorityQueue<Tarea> tareas = new PriorityQueue<>(
```

```
        Comparator.comparingInt(Tarea::prioridad)
                  .thenComparingLong(Tarea::llegada));
```

Error común

La trampa de recorrerla: si imprimís o iterás una `PriorityQueue`, los elementos **no** salen ordenados — el heap sólo garantiza la raíz; el resto del arreglo está "casi desordenado" a propósito. `[1, 2, 4, 5, 3]` es una salida perfectamente normal. Para consumir en orden: `poll()` en un bucle.

Casos de uso: planificadores de tareas (el SO, un simulador de eventos), triage y colas de atención, los **k mayores** de un stream gigante (min-heap de tamaño k), mezclar k listas ordenadas, y los algoritmos de grafos que vienen (Dijkstra saca siempre "el nodo más cercano": una cola de prioridad). **En contra**: no sirve para buscar un elemento arbitrario ($O(n)$: el heap no es un índice), ni para recorrer en orden sin vaciarla.

9.10 Ejemplo integrador: el triage de una guardia

Una guardia médica: los pacientes se atienden por **gravedad** (1 = crítico, 5 = leve), y a igual gravedad, por **orden de llegada**. Cola de prioridad con comparador compuesto — y un detalle profesional: el desempate FIFO hay que dárselo explícitamente, porque `PriorityQueue` **no** garantiza orden de llegada entre empatados:

```
public record Paciente(String nombre, int gravedad, long numeroDeLlegada) {}

private final PriorityQueue<Paciente> espera = new PriorityQueue<>(
        Comparator.comparingInt(Paciente::gravedad)              // más grave primero
                  .thenComparingLong(Paciente::numeroDeLlegada)); // empate: FIFO
```

Tracemos: ingresan ana (gravedad 3), juan (5), mia (1), leo (3). Se atiende: **mia** (crítica), **ana** (gravedad 3, llegó antes que leo), **leo**, **juan** (el leve espera a todos). Verificalo en el repo: `SalaDeEmergencias`.

9.11 Errores comunes

Error común

- Hundir comparando con **un solo hijo** en vez del **menor de los dos**: heap inválido silencioso (el bug clásico).
- Hacer heapify **de la raíz hacia abajo**: los subárboles aún no son heaps; el orden correcto es del último interno hacia la raíz.
- Esperar que iterar una `PriorityQueue` dé los elementos ordenados: sólo la raíz está garantizada.
- Confundir cola de prioridad con cola FIFO — o creer que los empatados salen por llegada: sin desempate explícito, el orden entre iguales es indefinido.
- Usar el heap para **buscar** un elemento: eso es $O(n)$; el heap sólo conoce a su mínimo.
- Olvidar que encolar los n elementos de a uno es $O(n \log n)$, mientras que heapify es $O(n)$.
- En Heapsort, olvidar que el heap "vivo" se **achica**: hundir con el tamaño total re-mezcla los ya ordenados.

9.12 Repaso

1. ¿Qué contrato tiene la Cola de Prioridad y en qué se diferencia de la cola FIFO?
2. ¿Por qué el heap puede vivir en un arreglo sin referencias? Escribí las fórmulas de padre e hijos.

3. Enunciá el invariante de min-heap. ¿Qué NO garantiza (a diferencia del ABB)?
4. Ejecutá a mano `encolar(2)` sobre el heap [3, 7, 5, 12, 9, 8]: mostrá cada intercambio.
5. Ejecutá a mano `desencolarMinimo()` sobre [3, 7, 4, 12, 9, 8, 5]: ¿quién sube a la raíz y cómo se hunde?
6. ¿Por qué al hundir hay que comparar con el MENOR de los dos hijos? ¿Qué pasa con la versión con bug?
7. Hacé el heapify de [10, 4, 8, 1, 6]: ¿qué índices se hunden, en qué orden, y cómo queda el arreglo?
8. ¿Por qué heapify se hace del último nodo interno hacia la raíz, y no al revés?
9. Justificá informalmente por qué heapify cuesta $O(n)$.
10. Describí las dos fases de Heapsort. ¿Qué ventajas tiene sobre Quicksort y Mergesort, y qué desventajas?
11. ¿Cómo se hace un max-heap con `PriorityQueue`? ¿Y una prioridad con desempate por llegada?
12. En el triage, ¿por qué hace falta el `numeroDeLlegada` en el comparador?

Guía de ejercicios — Unidad 9

Objetivos de la unidad

Practicá **colas de prioridad y heaps**. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i09_heaps` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i09_heaps/i01_teorico`. Fácil y medio usan **`PriorityQueue`**; el difícil implementa el **heap a mano** (¡con el hundir correcto!).

9.13 Preguntas teóricas

1. ¿Qué garantiza el invariante de min-heap y qué no garantiza?
2. ¿Por qué el heap se guarda en un arreglo? Escribí las fórmulas de padre e hijos y verificalas con un ejemplo.
3. Explicá flotar y hundir. ¿Cuál es el costo de cada uno y por qué?
4. ¿Cuál es el bug clásico de hundir y qué síntoma produce?
5. ¿Por qué heapify va del último nodo interno hacia la raíz? ¿Qué pasa al revés?
6. ¿Por qué heapify cuesta $O(n)$ y encolar de a uno $O(n \log n)$?
7. Describí las dos fases de Heapsort y su complejidad. ¿Es estable?
8. Compará el heap contra lista ordenada, lista desordenada y AVL para el contrato de cola de prioridad.
9. ¿Por qué iterar una `PriorityQueue` no da los elementos ordenados?
10. ¿Cómo se logra un max-heap con `PriorityQueue` y cómo un desempate FIFO?

9.14 Nivel fácil (con `PriorityQueue`)

1. **Mínimos sucesivos.** `List<Integer> enOrden(int[] a)`: encolá todo y desencolá hasta vaciar. Test: sale ordenado.
2. **Los k menores.** `List<Integer> kMenores(int[] a, int k)` con una `PriorityQueue`. Test.
3. **Máximos.** `int maximo(int[] a)` y `List<Integer> enOrdenDescendente(int[] a)` con `Comparator.reverseOrder()`. Test.
4. **Cola de impresión.** Trabajos con (nombre, páginas): imprime primero el de MENOS páginas. Test.
5. **El k-ésimo menor.** `int kEsimo(int[] a, int k)` desencolando k veces. Test.
6. **Turnos con prioridad.** Clientes (nombre, esJubilado): los jubilados pasan primero; empate por llegada (número incremental). Test.
7. **Palabras más largas.** Las 3 palabras más largas de una lista (heap por longitud descendente; empate alfabético). Test.
8. **Fusión simple.** Dadas dos `List<Integer>` ordenadas, fusionalas encolando todo y desencolando. Test.
9. **Tareas por vencimiento.** Tareas (descripcion, fechaLimite LocalDate): siempre sale la de fecha más próxima. Test.
10. **Cambiar de prioridad.** Simulá "subir la prioridad" de un elemento: quitalo (remove) y re-encolalo con otra prioridad. Test.

9.15 Nivel medio (patrones clásicos con `PriorityQueue`)

1. **Top-k de un stream.** `List<Integer> topK(Iterator<Integer> stream, int k)`: los k MAYORES vistos, con un MIN-heap de tamaño k (si llega algo mayor que la raíz, poll+offer). Test con 1000 valores.

2. **Mediana incremental.** `MedianaIncremental` con DOS heaps (max-heap de la mitad baja, min-heap de la alta), `agregar(x)` y `double mediana()` en O(log n). Test.
3. **Mezclar k listas ordenadas.** `List<Integer> mezclar(List<List<Integer>> listas)` con un heap de (valor, lista, índice). Test.
4. **Planificador de CPU.** Procesos (pid, prioridad, llegada): ejecutar devuelve el de mayor prioridad (número menor), empate por llegada; `ejecutarTodos` da la secuencia. Test.
5. **Huffman (frecuencias).** Dado un mapa char→frecuencia, armá el árbol de Huffman con un heap y devolvé el costo total (suma de frecuencia × profundidad). Test con un caso conocido.
6. **k más cercanos.** Los k puntos (x, y) más cercanos al origen, con un MAX-heap de tamaño k. Test.
7. **Reuniones y salas.** Dadas reuniones (inicio, fin), calculá la cantidad mínima de salas (heap de horarios de fin). Test.
8. **Suavizado de carga.** n tareas con duración: asignalas a m trabajadores, siempre al MENOS cargado (min-heap de cargas); devolvé la carga máxima final. Test.
9. **Cola con capacidad.** `ColaAcotada<T>`: capacidad k; al llenarse, encolar un elemento MEJOR que el peor lo desplaza (guardá el peor accesible con heap invertido). Test.
10. **Eventos de simulación.** Un mini simulador: eventos (tiempo, nombre) se procesan en orden de tiempo, y un evento puede agendar otros futuros. Test con una cadena de 3 eventos.

9.16 Nivel difícil (heap implementado a mano)

1. **MinHeap básico.** `MonticuloMinimo<T extends Comparable<T>>` sobre arreglo: encolar (flotar), verMinimo, tamanio. Test: el mínimo siempre en la raíz.
2. **Desencolar.** Agregá `desencolarMinimo` (última hoja a la raíz + hundir con el MENOR de los hijos). Test: vaciar entrega ordenado; incluí un caso que con el "bug del un solo hijo" fallaría (documentalo en el test).
3. **Heapify.** `static MonticuloMinimo desde(T[] valores)` hundiendo desde n/2−1 hasta 0. Test: [12,5,9,3,20,7] queda [3,5,7,12,20,9] y desencolar todo da ordenado.
4. **Heapsort.** `void ordenar(int[] a)` en el lugar con max-heap (dos fases). Test contra Arrays.sort, incluí arreglo con repetidos y ya ordenado.
5. **MaxHeap genérico con Comparator.** `Monticulo<T>` que recibe un `Comparator<T>` (sirve de min o max según el comparador). Test con ambos sentidos.
6. **Validador de heap.** `static boolean esMinHeap(int[] a)`: verifica el invariante recorriendo padres e hijos. Test: detectá un arreglo "casi heap" roto en una sola posición.
7. **K-ésimo con heap propio.** `int kEsimoMenor(int[] a, int k)` usando TU heapify + k extracciones; test y comparación de costo teórico con ordenar todo.
8. **Actualizar prioridad.** Heap con `disminuirClave(indiceInterno, nuevoValor)` que re-flota (guardá un mapa valor→posición para ubicarlo; claves únicas). Test.
9. **Heap d-ario.** `MonticuloDario` con d hijos por nodo (fórmulas: hijo j de i = d·i+j+1; padre = (i−1)/d). Test con d=3: sigue entregando ordenado.
10. **Cola de prioridad estable.** `ColaEstable<T>`: ante prioridades iguales respeta el orden de llegada (número de secuencia interno + tu heap). Test: empates salen FIFO — y documentá por qué el heap "pelado" no lo garantiza.

Unidad 10

Técnicas de diseño de algoritmos

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) atacar un problema nuevo eligiendo una **estrategia** en vez de improvisar; (2) diseñar soluciones por **backtracking** (explorar con poda y vuelta atrás) con su esquema elegir–avanzar–deshacer; (3) reconocer cuándo un algoritmo **goloso** (greedy) es correcto y cuándo es una trampa — con contraejemplo incluido; (4) aplicar **programación dinámica** en sus dos formas (memoización y tabulación), construyendo y leyendo tablas de subproblemas; y (5) decidir **qué técnica usar** ante un problema desconocido.

Hasta acá el curso te dio estructuras (dónde guardar) y análisis (cuánto cuesta). Esta unidad completa el tridente con lo más difícil de enseñar: **cómo se diseña** un algoritmo cuando el problema es nuevo. Ya conocés una técnica —la **división y conquista** de la Unidad 6—; ahora se suman las otras tres grandes: **backtracking**, **greedy** y **programación dinámica**. Y como hilo conductor, un mismo problema (dar el vuelto) resuelto con las tres, para ver en carne propia qué gana y qué pierde cada una.

10.1 Un poco de historia

El término *backtrack* lo acuñó el matemático **D. H. Lehmer** en los años 50, formalizando la búsqueda exhaustiva con vuelta atrás que ya usaban los ajedrecistas de papel. La **programación dinámica** nació en 1953 de **Richard Bellman** — y su nombre es una anécdota célebre: Bellman lo eligió porque sonaba importante e inofensivo para los burócratas que financiaban su investigación (“dynamic” no podía usarse peyorativamente, contó en su autobiografía). Los algoritmos **golosos** no tienen padre único: cristalizaron en los 50–70 con Kruskal, Prim y Huffman — que ya conociste en la Unidad 9 y volverás a ver en Grafos.

10.2 Backtracking: explorar con vuelta atrás

10.2.1 La idea desde cero

Hay problemas donde no se conoce ninguna fórmula ni atajo: hay que **probar**. Pero probar todo a lo bruto explota — y ahí entra el backtracking: recorrer el **árbol de decisiones** del problema en profundidad, con dos reglas de oro: **podar** apenas una rama viola una restricción (ni mirar su subárbol), y **volver atrás** cuando un camino se agota, deshaciendo la última decisión para probar la siguiente.

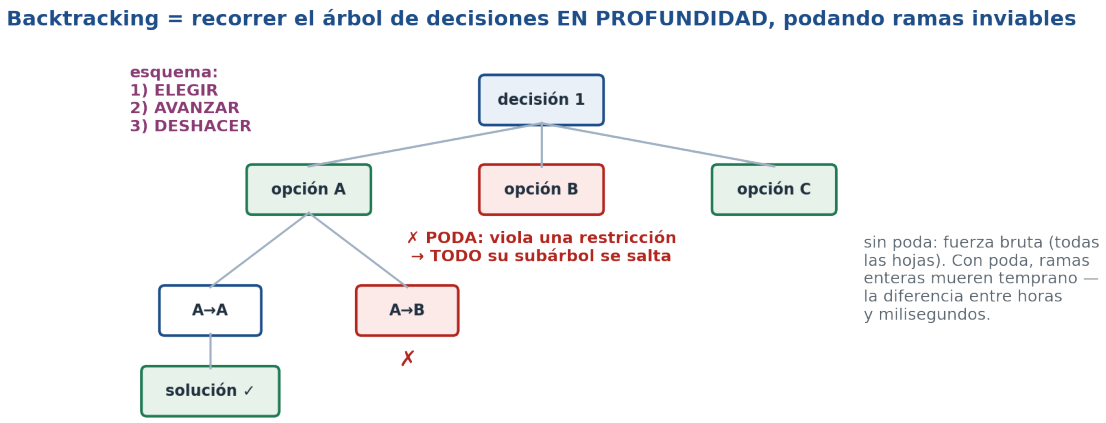


Figura 10.1: El backtracking recorre el árbol de decisiones en profundidad. La poda descarta subárboles enteros apenas se viola una restricción — la diferencia entre horas y milisegundos.

El esquema es siempre el mismo, y vale la pena aprenderlo como plantilla:

```
void resolver(estadoParcial) {
    if (esSolucionCompleta(estadoParcial)) { registrar(); return; }
    for (cada candidato : candidatosPosibles(estadoParcial)) {
        if (esValido(candidato)) {          // PODA: si no es válido, ni entrar
            aplicar(candidato);             // 1) ELEGIR
            resolver(estadoSiguiente);      // 2) AVANZAR (recursión)
            deshacer(candidato);            // 3) DESHACER (la vuelta atrás)
        }
    }
}
```

10.2.2 Ejemplo trabajado: las 4 reinas

El clásico: ubicar N reinas en un tablero de N×N sin que se ataquen (misma fila, columna o diagonal). La modelización ya poda sola: como no puede haber dos reinas en una fila, ubicamos **una reina por fila**, y la decisión de cada nivel es **qué columna** darle. Sigamos el algoritmo con N = 4:

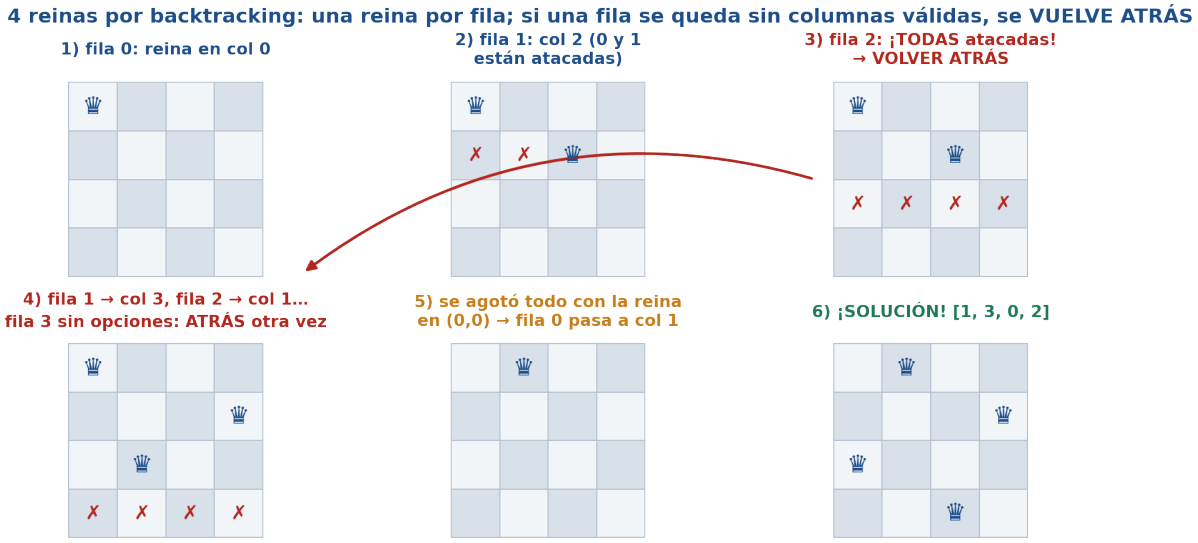


Figura 10.2: Las 4 reinas paso a paso: se avanza fila por fila; cuando la fila 2 se queda sin columnas válidas, se vuelve atrás y se mueve la reina anterior. Tras agotar todo lo que empezaba en (0,0), la primera reina se corre — y aparece la solución [1, 3, 0, 2].

Mirá el vaivén: el algoritmo se mete hasta el fondo por un camino, choca, retrocede **lo mínimo necesario** y sigue por la siguiente opción. Nada se explora dos veces y nada inválido se explora en absoluto. El código (`NReinas` en el repo) confirma: 4×4 tiene exactamente **2 soluciones**, y 8×8 las famosas **92** — halladas en milisegundos gracias a la poda, contra los $8^8 \approx 16$ millones de combinaciones de la fuerza bruta.

Idea clave

Backtracking = fuerza bruta + inteligencia. La recursión garantiza que se consideran todas las posibilidades; la **poda** garantiza que las inviables mueren temprano. La calidad de tu solución depende casi siempre de cuán temprano podás podar.

Su costo sigue siendo **exponencial** en el peor caso — es la técnica de "no hay nada mejor": puzzles, sudokus, generación de combinaciones/permutaciones (¡las hiciste en la guía de la U6!), satisfacción de restricciones.

10.3 Greedy: el atajo goloso

10.3.1 La idea desde cero

El extremo opuesto del backtracking: en vez de explorar todo, tomar en cada paso **la mejor decisión local** y **no volver atrás jamás**. Sin recursión, sin árbol, sin tabla: un bucle. Cuando funciona, es imbatible: simple y rápido. La pregunta del millón es *cuándo* funciona.

10.3.2 Cuando funciona… y cuando es una trampa

Probemos con el problema del vuelto: dar $180 con billetes de {100, 50, 20, 10, 1}, usando la menor cantidad. La estrategia golosa natural: **el billete más grande que entre**, repetir. Da 100 + 50 + 20 + 10 = 4 billetes — y es óptimo. ¿Siempre anda? No:

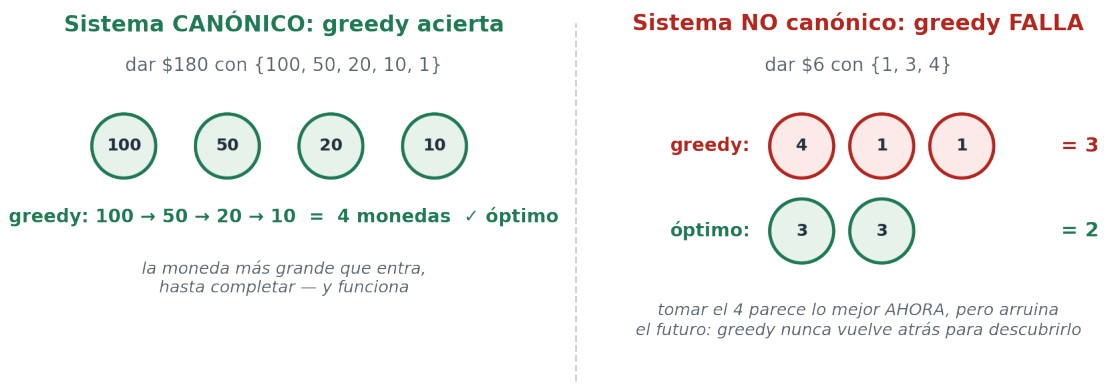


Figura 10.3: Greedy con el vuelto: con un sistema canónico (como los billetes argentinos) acierta siempre; con el sistema {1, 3, 4} y monto 6 elige 4+1+1 (3 monedas) cuando el óptimo es 3+3 (2). Tomar lo mejor AHORA arruinó el futuro.

El fracaso con {1, 3, 4} es la lección central: la decisión **localmente** mejor (agarrar el 4) dejó un resto ($2) que sale caro, y greedy **no tiene mecanismo para enterarse** — nunca revisa. Los sistemas monetarios reales están diseñados ("canónicos") para que greedy funcione; el problema general necesita otra técnica (spoiler: programación dinámica, más abajo).

Error común

Un greedy no se declara: se demuestra. Que la estrategia golosa "suene razonable" no alcanza — {1, 3, 4} sonaba igual de razonable. Para confiar en un greedy hay que dar un argumento de por qué la elección local nunca arruina el óptimo global (el argumento típico es "de intercambio": toda solución óptima se puede transformar en la golosa sin empeorar). Sin ese argumento, tratalo como una heurística.

10.3.3 Ejemplo trabajado donde SÍ es óptimo: selección de actividades

Tenés un aula y seis actividades que la piden, cada una con inicio y fin. ¿Cuántas podés aceptar, sin solapamientos? La elección golosa correcta no es la más corta ni la que empieza antes: es **la que termina antes** — porque terminar temprano deja el máximo margen para todo lo que sigue.

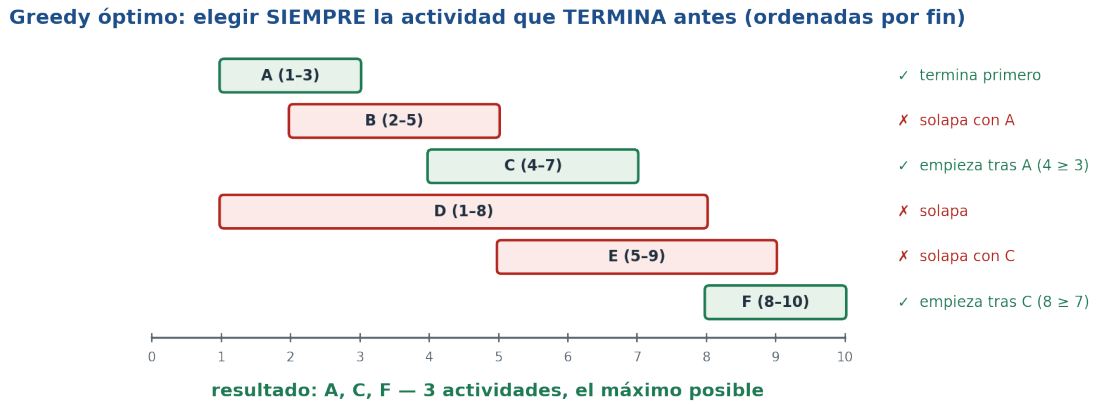


Figura 10.4: Selección de actividades: ordenadas por hora de fin, se acepta cada una que empiece después de la última aceptada. Resultado: A, C, F — el máximo posible (3).

El algoritmo completo son tres líneas de idea: ordenar por fin, recorrer, aceptar la que no solape. $O(n \log n)$ por el ordenamiento (U6), y **demostrable**: si una solución óptima no empieza con la

actividad que termina primero, se puede intercambiar su primera actividad por ésa sin perder nada — repitiendo el argumento, la solución golosa es tan buena como cualquiera. Código: `SeleccionDeActividades`.

Greedy correcto ya te cruzaste varios: **Huffman** (U9, guía), y en Grafos vienen **Dijkstra**, **Prim** y **Kruskal** — todos golosos con demostración.

10.4 Programación dinámica: no calcular nada dos veces

10.4.1 La idea desde cero

Volvamos al Fibonacci de la Unidad 6: la recursión ingenua explotaba porque **recalculaba** los mismos subproblemas una y otra vez. La solución que vimos —guardar lo ya calculado— tiene nombre y apellido: **programación dinámica** (DP). Aplica cuando el problema tiene dos propiedades:

1. **Subestructura óptima**: la solución óptima se arma con soluciones óptimas de subproblemas ("el mejor camino hasta acá pasa por el mejor camino hasta el paso anterior").
2. **Subproblemas superpuestos**: los mismos subproblemas aparecen muchas veces (si no se repiten, alcanza división y conquista, U6).

Y tiene dos formas de implementarse, dos caras de la misma moneda:

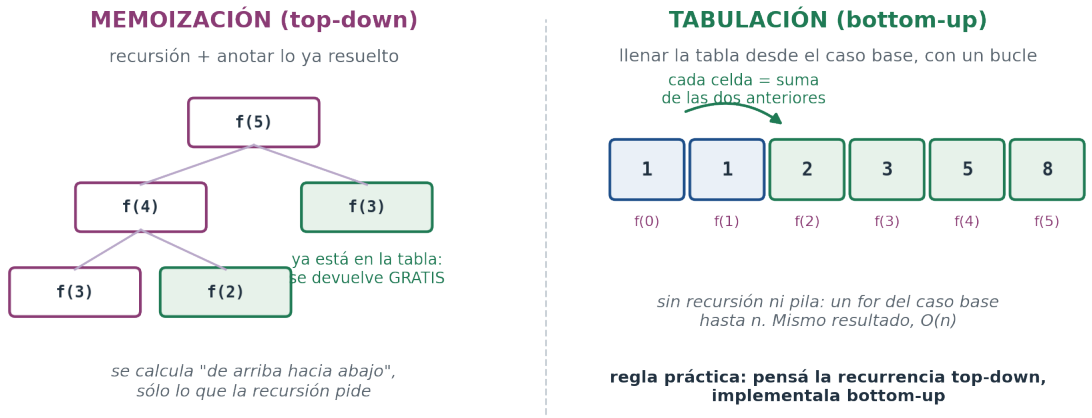


Figura 10.5: Memoización (top-down): la recursión natural más una tabla que evita recalcular — los subproblemas repetidos se devuelven gratis. Tabulación (bottom-up): un bucle llena la tabla desde el caso base. Regla práctica: pensá la recurrencia top-down, implementala bottom-up.

10.4.2 Ejemplo de entrada: las escaleras

¿De cuántas formas se sube una escalera de n escalones con pasos de 1 o de 2? El razonamiento DP empieza siempre igual: **¿cómo se relaciona el problema con sus subproblemas?** Para pisar el escalón n vine del $n-1$ (paso simple) o del $n-2$ (paso doble), y esos dos caminos no se mezclan:

$$formas(n) = formas(n-1) + formas(n-2), \qquad formas(0) = formas(1) = 1$$

¡Fibonacci disfrazado! La tabulación es un bucle que llena la tabla desde el caso base: para $n = 5$: $[1, 1, 2, 3, 5, 8]$ → **8 formas**. La versión ingenua es $O(2^n)$; ambas versiones DP son $O(n)$ (`Escaleras` en el repo, con memoización y tabulación lado a lado).

10.4.3 El vuelto, ahora bien: DP donde greedy fallaba

Retomemos el sistema {1, 3, 4} con monto 6, donde greedy daba 3 monedas. El razonamiento DP: para formar el monto m, la **última** moneda fue de 1, de 3 o de 4 — y lo anterior debió formarse **óptimamente** (subestructura óptima):

$$minimo[m] = 1 + \min(minimo[m-1],\ minimo[m-3],\ minimo[m-4])$$

Llenemos la tabla de 0 a 6, cada celda mirando celdas **ya resueltas**:

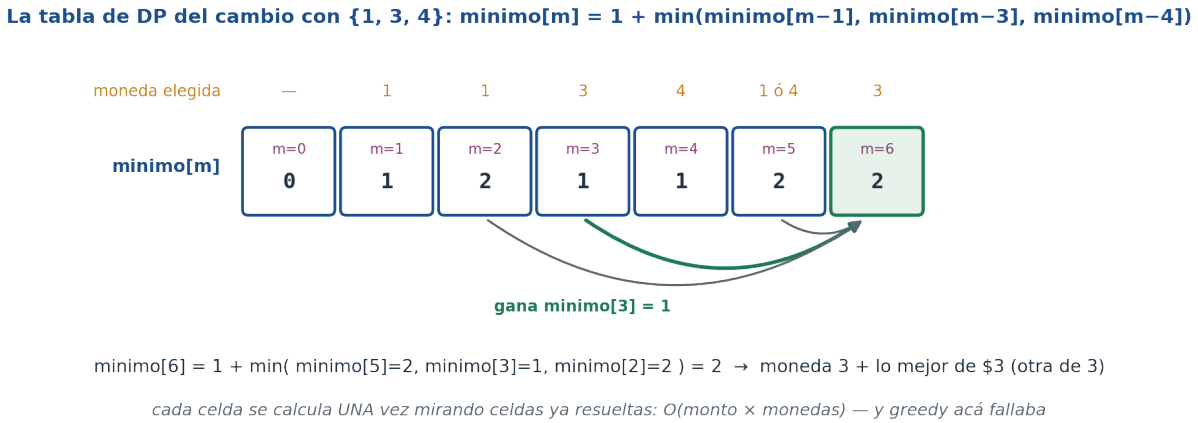


Figura 10.6: La tabla de DP del vuelto con {1, 3, 4}: cada celda toma 1 + el mínimo de las celdas alcanzables con una moneda. minimo[6] = 1 + minimo[3] = 2: la solución 3+3 que greedy nunca vio.

`minimo[6] = 2`: la respuesta correcta, en $O(monto \times monedas)$. Y guardando **qué moneda** eligió cada celda, el camino se puede **reconstruir** hacia atrás: $6 \xrightarrow{3} 3 \xrightarrow{3} 0$, o sea [3, 3]. Ésa es la otra mitad del oficio DP: la tabla da el valor óptimo; la reconstrucción, la solución en sí (`CambioDeMonedas.dpConMonedas`).

10.4.4 El ejemplo mayor: la mochila 0/1

El problema estrella de DP, con **dos** dimensiones. Una mochila de capacidad 5 y tres objetos: A (peso 2, valor 3), B (peso 3, valor 4), C (peso 4, valor 5). Cada objeto se lleva **entero o no se lleva** (por eso "0/1"): maximizar el valor. Greedy por valor o por densidad falla en general; backtracking explora 2^n subconjuntos. DP define `mejor[i][c]` = mejor valor usando los primeros i objetos con capacidad c, y la decisión binaria de cada objeto se vuelve un `max`:

$$mejor[i][c] = \max(\underbrace{mejor[i-1][c]}_{\text{no llevarlo}},\ \underbrace{valor_i + mejor[i-1][c-peso_i]}_{\text{llevarlo (si entra)}})$$

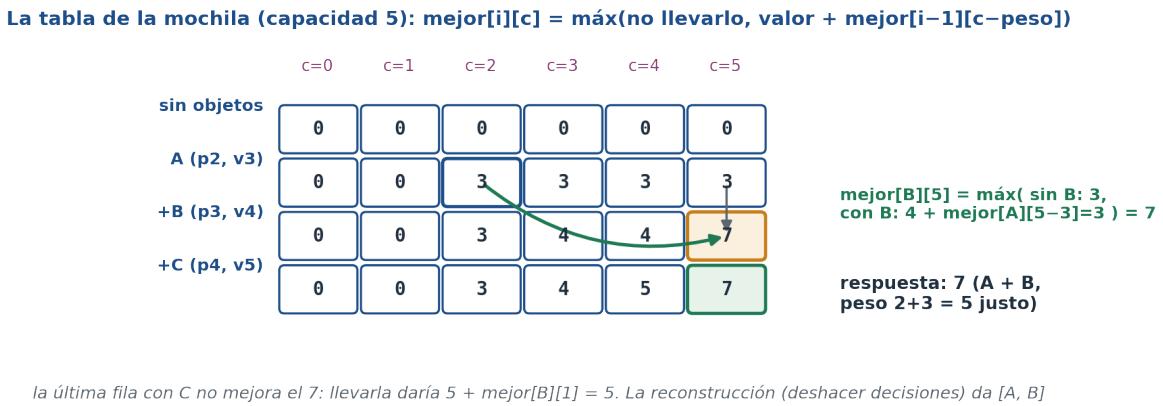


Figura 10.7: La tabla de la mochila, fila por fila. La celda destacada decide sobre B: no llevarlo vale 3; llevarlo vale 4 + mejor[A][2] = 7 — gana llevarlo. El óptimo final es 7 (A + B), y la reconstrucción lo confirma.

Leé la tabla como una historia: la fila "sin objetos" es todo ceros (caso base); cada fila siguiente copia la de arriba ("no llevarlo") salvo donde llevarse el objeto mejora. El resultado, $O(n \times capacidad)$ — contra el 2^n de probar todo. El código con reconstrucción está en `Mochila`.

Idea clave

La receta DP en 4 pasos: (1) definí QUÉ significa una celda (en palabras: "mejor[i][c] = el máximo valor usando…"); (2) escribí la **recurrencia** (cómo se arma una celda con celdas menores) y los **casos base**; (3) elegí memoización o tabulación; (4) si hace falta la solución además del valor, guardá las decisiones y **reconstruí** hacia atrás. El paso (1) es el difícil: si la definición es correcta, el resto fluye.

10.5 ¿Qué técnica uso? La guía de decisión

Frente a un problema nuevo, las preguntas en orden:

Pregunta	Si la respuesta es sí →
¿Se parte en subproblemas **independientes** (no se repiten)?	**División y conquista** (U6): Mergesort, Quicksort, búsqueda binaria
¿Los subproblemas se **repiten** y hay subestructura óptima?	**Programación dinámica**: vuelto, mochila, caminos, comparación de secuencias
¿Hay una elección local que **se puede demostrar** segura?	**Greedy**: actividades, Huffman, Dijkstra/Prim/Kruskal
¿Nada de lo anterior, pero el espacio de soluciones es explorable con **poda**?	**Backtracking**: N reinas, sudoku, combinaciones con restricciones

Dos consejos de práctica: probá greedy primero (si encontrás la demostración, ganaste el algoritmo más simple), y si sospechás DP pero no ves la tabla, escribí primero la **recursión ingenua** — sus parámetros son las dimensiones de la tabla.

10.6 Ejemplo integrador: el vuelto por triplicado

El cierre es el experimento completo, en `CambioDeMonedas`: el mismo problema con las tres técnicas.

```
int[] raras = {1, 3, 4};
CambioDeMonedas.greedy(raras, 6);         // 3   rapidísimo… y MAL (4+1+1)
CambioDeMonedas.backtracking(raras, 6);   // 2   correcto, pero exponencial
CambioDeMonedas.dp(raras, 6);             // 2   correcto Y eficiente
CambioDeMonedas.dpConMonedas(raras, 6);   // [3, 3]  con reconstrucción
```

La moraleja condensa la unidad: **greedy** es el más rápido pero exige demostración (acá no la hay: falla); **backtracking** siempre acierta pero paga precio exponencial (mitigable con poda — el del repo poda cuando ya no puede mejorar); **DP** logra lo mejor de ambos cuando el problema tiene la estructura justa: correcto y polinomial. Elegir técnica **es** diseñar el algoritmo.

10.7 Errores comunes

Error común

- Olvidar el **deshacer** en backtracking: el estado queda sucio y las ramas siguientes exploran mundos inválidos.
- Podar de menos (fuerza bruta encubierta)… o podar de más (cortar ramas que contenían la solución).
- Confiar en un greedy **sin demostración**: “suena bien” no es un argumento — {1, 3, 4} suena bien.
- En DP, definir mal qué significa la celda: si la definición es ambigua, la recurrencia sale mal seguro.
- Olvidar los **casos base** de la tabla, o llenar la tabla en un orden donde las celdas necesarias aún no están resueltas.
- Usar memoización con parámetros que no identifican el subproblema (dos estados distintos caen en la misma celda).
- Aplicar DP a un problema **sin** subestructura óptima: la tabla da un número, pero no significa nada.
- Devolver el valor óptimo cuando pedían la **solución**: falta la reconstrucción.

10.8 Repaso

1. Enunciá el esquema elegir–avanzar–deshacer del backtracking. ¿Qué papel juega la poda?
2. En las 4 reinas, ¿por qué “una reina por fila” ya es una poda? Reproducí a mano los primeros retrocesos.
3. ¿Qué diferencia hay entre backtracking y fuerza bruta pura?
4. ¿Qué hace a un algoritmo “goloso”? ¿Por qué su gran riesgo es no volver atrás?
5. Mostrá con {1, 3, 4} y monto 6 que el greedy del vuelto no es óptimo. ¿Por qué con billetes argentinos sí funciona?
6. En selección de actividades, ¿por qué la elección correcta es “la que termina antes” y no “la más corta”? Esbozá el argumento de intercambio.
7. ¿Qué dos propiedades debe tener un problema para que aplique programación dinámica?
8. Explicá la diferencia entre memoización y tabulación. ¿Cuándo preferirías cada una?
9. Escribí la recurrencia del vuelto con {1, 3, 4} y llenala a mano hasta minimo[6].
10. En la mochila, ¿qué significa la celda mejor[i][c]? Escribí su recurrencia y explicá el max.
11. ¿Para qué sirve guardar las decisiones en una tabla de DP? Mostralo reconstruyendo [3, 3] del vuelto.
12. Para cada problema, decí qué técnica usarías y por qué: (a) sudoku; (b) dar vuelto en pesos argentinos; (c) el camino más barato en una grilla con costos; (d) ordenar un millón de números.

Guía de ejercicios — Unidad 10

Objetivos de la unidad

Practicá las **técnicas de diseño**. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i10_tecnicas` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i10_tecnicas/i01_teorico`. En cada ejercicio, identificá PRIMERO la técnica y justificala en el javadoc — ése es el músculo que esta guía entrena.

10.9 Preguntas teóricas

1. Describí el esquema general del backtracking y el rol de cada paso (elegir, avanzar, deshacer).
2. ¿Qué es la poda y por qué puede cambiar el costo en órdenes de magnitud?
3. ¿Por qué un algoritmo goloso necesita demostración? ¿Qué es el argumento de intercambio?
4. Dá un ejemplo de greedy correcto y uno incorrecto para el mismo tipo de problema.
5. ¿Qué son la subestructura óptima y los subproblemas superpuestos? ¿Por qué DP necesita ambas?
6. Compará memoización y tabulación: cómo funcionan, ventajas de cada una.
7. ¿Cómo se reconstruye la solución (no sólo el valor) en un problema de DP?
8. ¿Qué relación hay entre la recursión ingenua de un problema y las dimensiones de su tabla de DP?
9. ¿Cuándo alcanza división y conquista y cuándo hace falta DP?
10. Ordená de menor a mayor costo típico: DP, backtracking con poda, greedy. ¿Qué se sacrifica en cada escalón?

10.10 Nivel fácil

1. **Escaleras con memoización.** `long formas(int n)` (pasos de 1 o 2) con memoización; test contra la versión tabulada.
2. **Escaleras generalizadas.** Pasos de 1, 2 o 3: `long formas3(int n)` por tabulación. Test (formas3(4) = 7).
3. **Greedy del vuelto.** `int monedasGreedy(int[] sistema, int monto)`; test con sistema canónico (acierta) y con {1,3,4} (test documenta que da 3, no el óptimo 2).
4. **Suma máxima sin adyacentes.** `int maximo(int[] a)`: mayor suma de elementos no consecutivos, DP 1D. Test ([2,7,9,3,1] → 12).
5. **Subconjuntos por backtracking.** `List<List<Integer>> subconjuntos(int[] a)` con el esquema elegir/avanzar/deshacer explícito (incluir o no cada elemento). Test: 2^n subconjuntos.
6. **Mochila greedy (fraccionaria).** Con objetos divisibles, greedy por valor/peso ES óptimo: `double valorMaximo(pesos, valores, capacidad)`. Test.
7. **Actividades.** Implementá la selección de actividades por fin más temprano. Test con el ejemplo del apunte (A, C, F).
8. **Pasos mínimos a 1.** Desde n, operaciones: −1, /2 (si es par), /3 (si es divisible): `int pasosMinimos(int n)` por DP. Test (10 → 3).
9. **Tribonacci.** `long tribonacci(int n)` (suma de los 3 anteriores) por tabulación; test incluye n=25 (instantáneo).
10. **Conteo de caminos en grilla.** `long caminos(int filas, int columnas)` moviéndose sólo derecha/abajo, DP 2D. Test (3×3 → 6).

10.11 Nivel medio

1. **Vuelto por DP con reconstrucción.** `List<Integer> monedasOptimas(int[] sistema, int monto)`; test: {1,3,4}/6 → [3,3] (en cualquier orden) y un monto imposible → lista vacía.
2. **Mochila 0/1.** `int valorMaximo(int[] pesos, int[] valores, int capacidad)` + reconstrucción de índices elegidos. Test con el ejemplo del apunte (7, [A,B]).
3. **Subsecuencia común más larga (LCS).** `int lcs(String a, String b)` DP 2D. Test ("AGGTAB", "GXTXAYB" → 4).
4. **Subsecuencia creciente más larga.** `int lis(int[] a)` DP $O(n^2)$. Test ([10,9,2,5,3,7,101,18] → 4).
5. **Suma de subconjunto.** `boolean existeSubconjuntoQueSuma(int[] a, int objetivo)` por DP (tabla booleana). Test.
6. **Distancia de edición.** `int distancia(String a, String b)` (insertar/borrar/reemplazar), DP 2D. Test ("casa", "calle" → 3).
7. **N reinas contadas.** Backtracking que cuenta soluciones de N reinas; test: n=4 → 2, n=6 → 4, n=8 → 92.
8. **Sudoku 4×4.** Resolvé un mini-sudoku 4×4 por backtracking (celdas 0 = vacías). Test con un tablero resoluble y uno imposible.
9. **Mochila con backtracking podado.** La mochila 0/1 por backtracking con poda por cota (si el valor restante posible no alcanza, cortar); test que coincide con la DP del e02.
10. **Camino de costo mínimo en grilla.** `int costoMinimo(int[][] grilla)` (derecha/abajo) por DP + reconstrucción del camino. Test.

10.12 Nivel difícil

1. **Generador de expresiones.** Con los dígitos 1..9 en orden y operadores {+, −, nada (concatenar)}, encontrá por backtracking todas las expresiones que dan 100. Test: la cantidad de soluciones es 11.
2. **Particiones de un entero.** `int particiones(int n)`: de cuántas formas se escribe n como suma de enteros positivos (sin importar orden), DP. Test (particiones(5) = 7).
3. **Coloreo de intervalos.** Asigná la mínima cantidad de "aulas" a un conjunto de intervalos y devolvé la asignación (greedy por inicio + heap de la U9). Test.
4. **Corte de varilla.** `int mejorCorte(int[] precioPorLargo, int largo)`: maximizar el precio cortando una varilla, DP + reconstrucción de los cortes. Test clásico (precios [1,5,8,9], largo 4 → 10 con cortes [2,2]).
5. **Caminos con obstáculos.** Conteo de caminos en grilla con celdas bloqueadas, DP; test incluye grilla con la salida bloqueada (0 caminos).
6. **Palíndromo más largo.** `String subcadenaPalindromicaMasLarga(String s)` por DP (tabla esPalindromo[i][j]). Test ("babad" → "bab" o "aba").
7. **Suma objetivo con signos.** ¿De cuántas formas se puede asignar + o − a cada elemento para que la suma dé un objetivo? Backtracking Y DP; test que ambas coinciden.
8. **Salto mínimo.** `int saltosMinimos(int[] alcance)`: mínimo de saltos para llegar al final (desde i podés avanzar hasta alcance[i]); resolvelo GREEDY O(n) y demostrá en el javadoc por qué es correcto; test contra una DP $O(n^2)$.
9. **Empaquetado de canciones.** Con duraciones de canciones y un CD de capacidad fija, ¿cuál es la máxima duración total alcanzable? (subset-sum con reconstrucción). Test.
10. **Torneo de técnicas.** Para el vuelto con sistemas {1,3,4}, {1,5,10,25} y montos varios, un test que ejecuta greedy, backtracking podado y DP, verifica que backtracking == DP siempre, y detecta automáticamente para qué casos greedy se equivoca (assert de los casos conocidos).

Unidad 11

Grafos

Objetivos de la unidad

Al terminar esta unidad vas a poder: (1) reconocer un **grafo** escondido en un problema real (mapas, redes, correlatividades, dependencias); (2) elegir entre **matriz** y **lista de adyacencia** con argumentos de memoria y costo; (3) recorrer un grafo con **BFS** y **DFS** y saber qué pregunta responde cada uno; (4) ordenar tareas con dependencias mediante **orden topológico**; (5) calcular **caminos mínimos** con Dijkstra; y (6) conectar todo al menor costo con un **árbol de tendido mínimo** (Prim y Kruskal). Todo con la cola de la U5, el heap de la U9 y el análisis goloso de la U10 — esta unidad es el gran punto de encuentro del curso.

Mirá el mapa del subte, el plan de correlatividades de tu carrera, la lista de seguidores de una red social, las dependencias entre módulos de un proyecto de software, la red eléctrica de tu ciudad. Son cosas distintas con la MISMA forma: **cosas** conectadas por **relaciones**. Esa forma se llama *grafo*, y es la estructura de datos más general que vas a ver en el curso: una lista es un grafo en fila, un árbol es un grafo sin ciclos. Cuando el problema es "¿cómo llego?", "¿qué depende de qué?", "¿están conectados?" o "¿cuál es el camino más barato?", el problema es de grafos — y esta unidad te da las cuatro o cinco herramientas con las que se responde casi todo.

11.1 Un poco de historia

Los grafos tienen partida de nacimiento: **1736**, cuando **Leonhard Euler** resolvió el problema de los **siete puentes de Königsberg** (¿se puede pasear cruzando cada puente exactamente una vez?). Euler entendió que lo único que importaba era qué orillas conectaba cada puente — el primer grafo de la historia — y demostró que el paseo era imposible. Doscientos años después los grafos se volvieron computación: **Edsger Dijkstra** contó que inventó su algoritmo de caminos mínimos en 1956, en veinte minutos, tomando un café en una terraza de Ámsterdam, y que lo publicó recién en 1959 en tres páginas. Los algoritmos de tendido mínimo son de la misma época: **Joseph Kruskal** (1956) y **Robert Prim** (1957) — aunque el de Prim ya lo había descubierto el checo **Vojtěch Jarník** en 1930. Son los mismos "golosos que funcionan" que anticipamos en la U10: acá vas a ver por fin sus demostraciones en acción.

11.2 Qué es un grafo

11.2.1 La idea desde cero

Un **grafo** es un conjunto de **vértices** (o nodos: las "cosas") unidos por **aristas** (las "relaciones"). Nada más. La potencia está en lo que NO fija: no hay raíz, no hay orden, no hay jerarquía; cualquier vértice puede conectarse con cualquier otro.

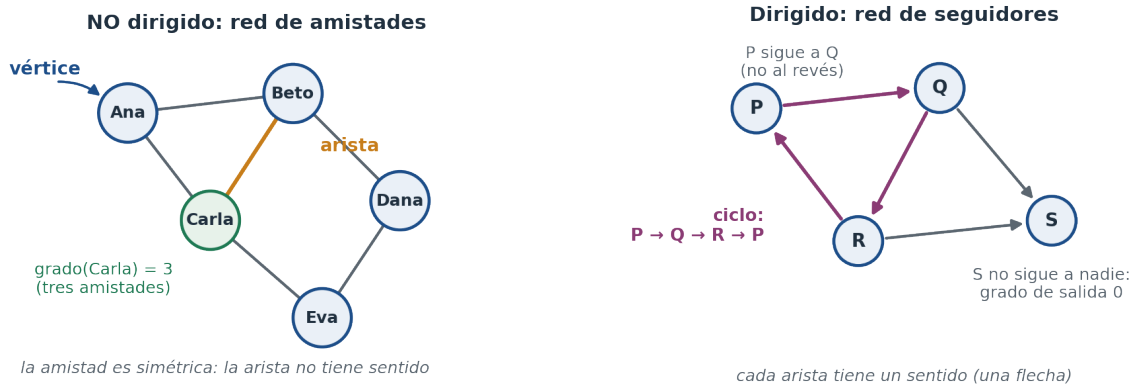


Figura 11.1: A la izquierda, un grafo NO dirigido: la amistad va en ambos sentidos y las aristas son simples líneas. A la derecha, uno dirigido: "P sigue a Q" no implica que Q siga a P, y las aristas son flechas. El ciclo P → Q → R → P está resaltado.

Definición

- **Vértice** (o nodo): cada elemento del grafo. Los llamamos $0, 1, \ldots, n-1$ (o A, B, C… en los dibujos).
- **Arista**: una conexión entre dos vértices. Puede tener un **peso** (costo, distancia, tiempo).
- **Grafo dirigido**: las aristas tienen sentido (flechas). **No dirigido**: van en ambos sentidos.
- **Grado** de un vértice: cuántas aristas lo tocan (en dirigidos se separa en grado de *entrada* y de *salida*).
- **Camino**: una secuencia de vértices conectados por aristas. Su **largo** es la cantidad de aristas (o la suma de pesos, si los hay).
- **Ciclo**: un camino que vuelve al vértice de partida.
- **Conexo** (no dirigido): hay camino entre todo par de vértices.
- Notación de costos: n = cantidad de vértices, m = cantidad de aristas.

Idea clave

Ya conocés dos grafos disfrazados: una **lista enlazada** es un grafo dirigido en fila (cada nodo con una arista al siguiente) y un **árbol** es un grafo conexo **sin ciclos** con un vértice distinguido (la raíz). Al quitar esas dos restricciones — puede haber ciclos, no hay raíz — aparece el grafo general. Por eso los recorridos de esta unidad se parecen a los de la U8… con una novedad: como puede haber ciclos, ahora hay que **marcar los visitados** o el recorrido gira en círculos para siempre.

Para toda la unidad usamos un mismo grafo de ejemplo, con 6 vértices A–F y 7 aristas:

```
A-B   A-C   B-D   C-D   C-E   D-F   E-F
```

11.3 Cómo se representa un grafo

En la memoria no existen las flechas: hay que decidir en qué estructura guardar "quién se conecta con quién". Hay dos representaciones clásicas, y elegir bien entre ellas es la primera decisión de diseño de todo problema de grafos.

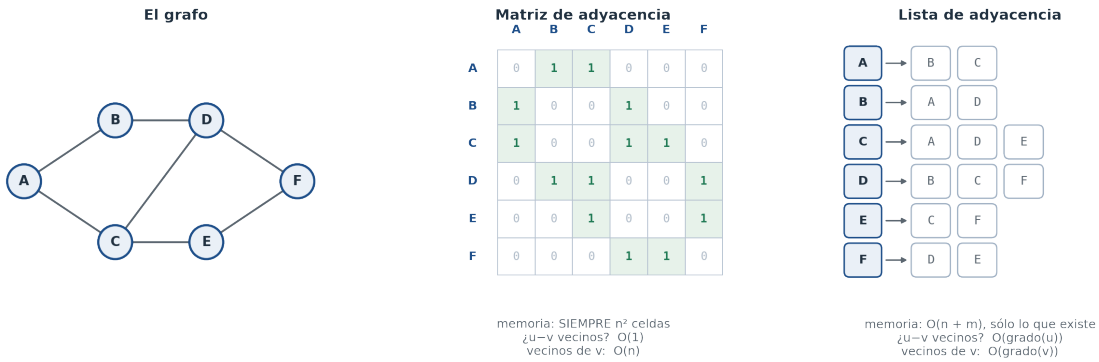


Figura 11.2: El mismo grafo tres veces: el dibujo, la matriz de adyacencia (una tabla n×n con 1 donde hay arista) y la lista de adyacencia (cada vértice con la lista de sus vecinos). La matriz gasta n² celdas aunque casi todas tengan 0; la lista guarda sólo lo que existe.

11.3.1 Matriz de adyacencia

Una tabla de $n \times n$: la celda `[u][v]` vale 1 (o el peso) si existe la arista u→v. En no dirigidos la matriz es simétrica.

```
public class GrafoMatriz {
    public static final int SIN_ARISTA = -1;
    private final int[][] peso;                     // la tabla n×n

    public void agregarArista(int origen, int destino, int pesoDeLaArista) {
        peso[origen][destino] = pesoDeLaArista;
        if (!dirigido) {
            peso[destino][origen] = pesoDeLaArista;  // simétrica en no dirigidos
        }
    }

    /** O(1): una sola consulta a la tabla. */
    public boolean existeArista(int origen, int destino) {
        return peso[origen][destino] != SIN_ARISTA;
    }
}
```

11.3.2 Lista de adyacencia

Cada vértice guarda la lista de sus aristas salientes. Es un `ArrayList` de `ArrayList` (¡la U2 y la U5 trabajando juntas!).

```
public class Grafo {
    public record Arista(int destino, int peso) {}
    private final List<List<Arista>> adyacentes;     // adyacentes.get(v) = vecinos de v

    public void agregarArista(int origen, int destino, int peso) {
        adyacentes.get(origen).add(new Arista(destino, peso));
        if (!dirigido) {                             // no dirigido: también al revés
            adyacentes.get(destino).add(new Arista(origen, peso));
        }
    }

    /** Los vecinos de v: la operación estrella, O(grado(v)). */
    public List<Arista> vecinos(int vertice) {
        return List.copyOf(adyacentes.get(vertice));
    }
}
```

11.3.3 ¿Cuál conviene?

	Matriz	Lista de adyacencia
Memoria	$O(n^2)$ SIEMPRE	$O(n+m)$: sólo lo que existe
¿u−v son vecinos?	$O(1)$	$O(\text{grado}(u))$
Recorrer vecinos de v	$O(n)$: toda la fila	$O(\text{grado}(v))$
Agregar arista	$O(1)$	$O(1)$

Error común

La matriz parece inocente pero su memoria es **cuadrática**: para un grafo con un millón de vértices (una red social chica) son 10^{12} celdas — imposible. En la práctica casi todos los grafos reales son **ralos** (*sparse*: m mucho menor que n^2) y la lista de adyacencia es la elección por defecto. La matriz queda para grafos chicos y **densos**, o cuando la pregunta dominante es "¿estos dos son vecinos?" repetida millones de veces.

En IntelliJ

Java no trae un TDA `Grafo` en `java.util` — y eso es una lección de diseño: con las piezas que ya conocés se arma en 30 líneas. En el mundo profesional vas a ver grafos representados como `Map<String, List<String>>` (vértices con nombre, sin numerar) o con bibliotecas como JGraphT. En el repo, `Grafo` (lista de adyacencia) y `GrafoMatriz` están en `material.i11_grafos`, con `main` ejecutables — corré ambos y comparalos.

11.4 Recorrer un grafo: BFS y DFS

"Recorrer" es visitar todos los vértices alcanzables desde un origen, una sola vez cada uno. Hay dos maneras canónicas de hacerlo, y la ÚNICA diferencia entre ellas es la estructura de la U5 que guarda los pendientes: una **cola** (BFS) o una **pila** (DFS). Todo lo demás es idéntico.

11.4.1 BFS: a lo ancho, por niveles

BFS (*Breadth-First Search*) explora "en ondas": primero el origen, después todos sus vecinos, después los vecinos de los vecinos. Los pendientes esperan en una **cola** (FIFO): el primero en ser descubierto es el primero en ser procesado.

```java
public static List<Integer> bfs(Grafo grafo, int origen) {
    List<Integer> orden = new ArrayList<>();
    boolean[] visitado = new boolean[grafo.cantidadDeVertices()];
    Deque<Integer> cola = new ArrayDeque<>();

    visitado[origen] = true;                    // se marca al ENCOLAR
    cola.addLast(origen);
    while (!cola.isEmpty()) {
        int vertice = cola.removeFirst();       // el más viejo de la cola
        orden.add(vertice);
        for (Grafo.Arista arista : grafo.vecinos(vertice)) {
            if (!visitado[arista.destino()]) {
                visitado[arista.destino()] = true;
                cola.addLast(arista.destino());    // a la fila del fondo
            }
        }
    }
    return orden;
}
```

Sigamos la ejecución completa en nuestro grafo, arrancando en A:

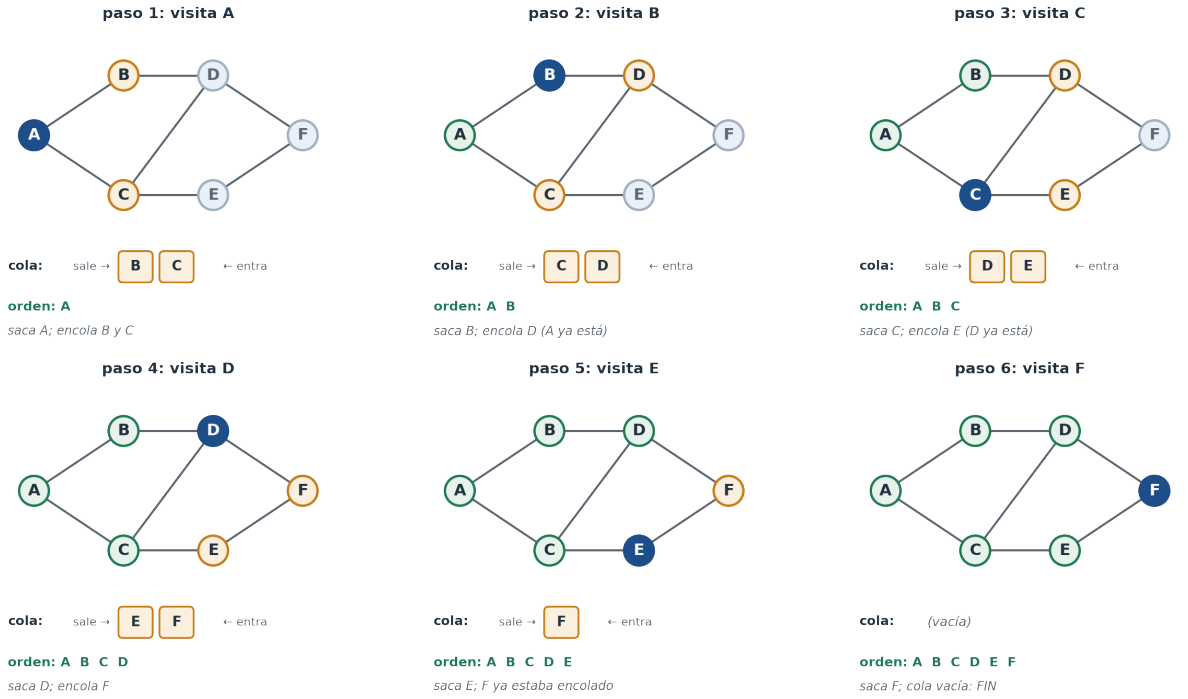


Figura 11.3: BFS paso a paso desde A. En cada paso: azul = el que se está visitando (recién salió de la cola), naranja = descubierto y esperando en la cola, verde = ya visitado. El orden final es A B C D E F.

¿Notás el patrón? BFS agota TODO el nivel 1 (B y C, los vecinos directos de A) antes de tocar el nivel 2 (D y E), y termina en el nivel 3 (F). Esa es su propiedad estrella:

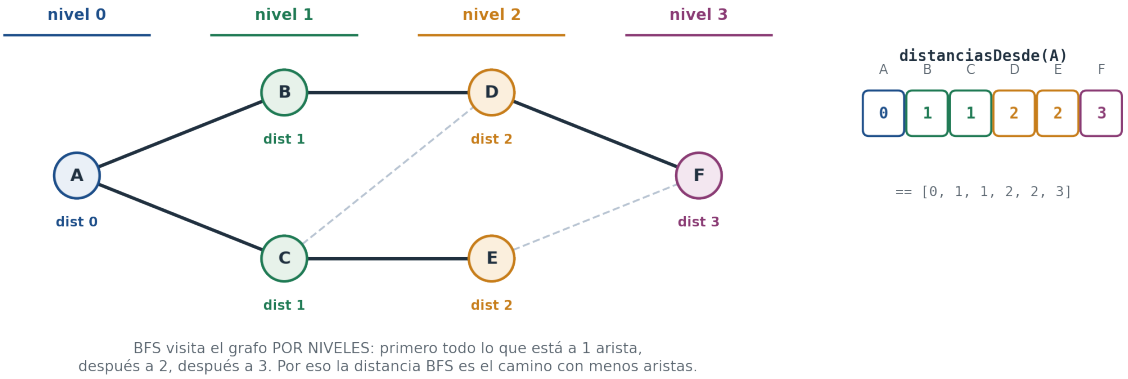


Figura 11.4: El mismo grafo estirado por niveles. Las aristas oscuras son las que BFS usó para descubrir cada vértice; las punteadas existen pero no descubrieron a nadie. La distancia de cada vértice es su nivel — el arreglo que devuelve distanciasDesde(A).

Idea clave

BFS encuentra el camino con MENOS ARISTAS. Como visita por niveles, la primera vez que descubre un vértice lo hace por el camino más corto posible (en cantidad de aristas). Por eso `distanciasDesde` es el algoritmo de caminos mínimos cuando todas las aristas "cuestan lo mismo": la menor cantidad de transbordos, de saltos, de clics. Guardando además *quién descubrió a quién* (un arreglo `anterior[]`) se reconstruye el camino — exactamente igual que en Dijkstra, que viene enseguida.

```java
public static int[] distanciasDesde(Grafo grafo, int origen) {
    int[] distancia = new int[grafo.cantidadDeVertices()];
    Arrays.fill(distancia, -1);                    // -1 = todavía inalcanzable
    Deque<Integer> cola = new ArrayDeque<>();
    distancia[origen] = 0;
    cola.addLast(origen);
    while (!cola.isEmpty()) {
        int vertice = cola.removeFirst();
        for (Grafo.Arista arista : grafo.vecinos(vertice)) {
            if (distancia[arista.destino()] == -1) {
                distancia[arista.destino()] = distancia[vertice] + 1;   // un nivel más
                cola.addLast(arista.destino());
            }
        }
    }
    return distancia;                              // [0, 1, 1, 2, 2, 3] desde A
}
```

11.4.2 DFS: en profundidad, hasta el fondo

DFS (*Depth-First Search*) hace lo contrario: apenas descubre un vecino nuevo, **se mete** — y sigue metiéndose hasta tocar fondo; recién entonces vuelve un paso atrás y prueba el siguiente. ¿Te suena? Es el **backtracking** de la U10 aplicado a recorrer: elegir, avanzar, deshacer. La versión natural es recursiva — la pila de llamadas de la U4 hace de “estructura de pendientes” gratis.

```java
private static void dfsDesde(Grafo grafo, int vertice, boolean[] visitado,
                             List<Integer> orden) {
    visitado[vertice] = true;
    orden.add(vertice);
    for (Grafo.Arista arista : grafo.vecinos(vertice)) {
        if (!visitado[arista.destino()]) {
            dfsDesde(grafo, arista.destino(), visitado, orden);  // se mete a fondo
        }
    }
}
```

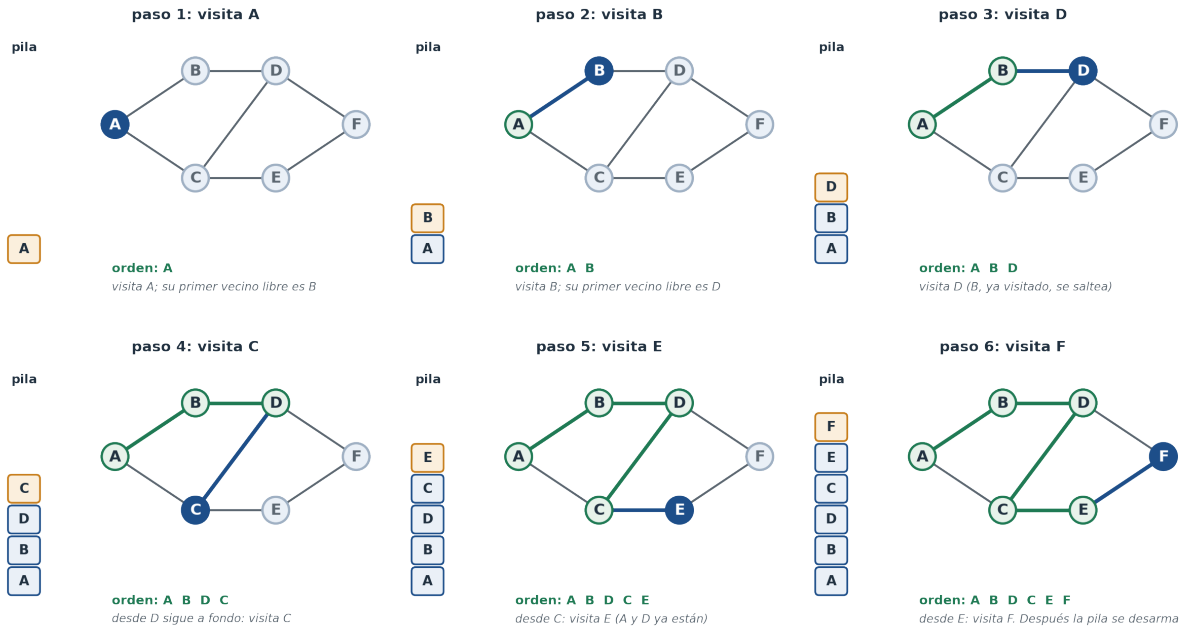


Figura 11.5: DFS paso a paso desde A. La pila de llamadas crece a la izquierda de cada panel: A llama a B, B llama a D, D llama a C… El recorrido “serpentea” hasta el fondo: A B D C E F. Azul = arista por la que acaba de meterse; verde = camino ya recorrido.

Comparemos con BFS sobre el MISMO grafo desde el MISMO origen:

```
System.out.println(bfs(grafo, 0));           // [0, 1, 2, 3, 4, 5] = A B C D E F
System.out.println(dfs(grafo, 0));           // [0, 1, 3, 2, 4, 5] = A B D C E F
System.out.println(dfsIterativo(grafo, 0));  // [0, 2, 4, 5, 3, 1] = A C E F D B
```

Idea clave

La estructura de pendientes ES el recorrido. Cola FIFO → BFS, por niveles, caminos mínimos. Pila LIFO → DFS, a fondo, detección de estructura (ciclos, componentes, orden de dependencias). En el repo está `dfsIterativo`, el mismo DFS con `Deque` explícito en vez de recursión: da un orden distinto (procesa los vecinos apilados al revés) pero igual de válido. Es la prueba viviente de la dualidad recursión ↔ pila de la U5.

Ambos recorridos visitan cada vértice una vez y miran cada arista una cantidad constante de veces: $O(n+m)$, el costo de "leer" el grafo entero. No se puede recorrer más barato: hay que mirar todo al menos una vez.

Error común

El error clásico #1 de grafos: olvidar `visitado[]`. En un árbol (U8) los recorridos no lo necesitan porque no hay ciclos; en un grafo, sin la marca, el recorrido entra al ciclo A→B→D→C→A y no sale **nunca**. El error clásico #2 (en BFS): marcar al *desencolar* en vez de al *encolar* — el algoritmo termina, pero mete el mismo vértice varias veces en la cola y el costo explota. Marcá SIEMPRE en el momento de encolar.

11.5 Orden topológico: el plan de correlatividades

Problema real que sufriste: tenés materias con correlativas y querés un orden para cursarlas sin violar ningún requisito. En términos de grafos: dado un grafo **dirigido** cuyas aristas significan "esto va antes que aquello", encontrar un orden de los vértices que respete todas las flechas. Eso es un **orden topológico**, y el algoritmo clásico para calcularlo (Kahn, 1962) es de una elegancia brutal.

La idea: el **grado de entrada** de un vértice cuenta cuántos requisitos le faltan. Los de grado 0 están "listos" (se pueden cursar ya). Sacás uno, y al sacarlo descontás 1 a cada materia que lo esperaba — alguna puede quedar en 0 y sumarse a los listos. Repetir hasta vaciar.

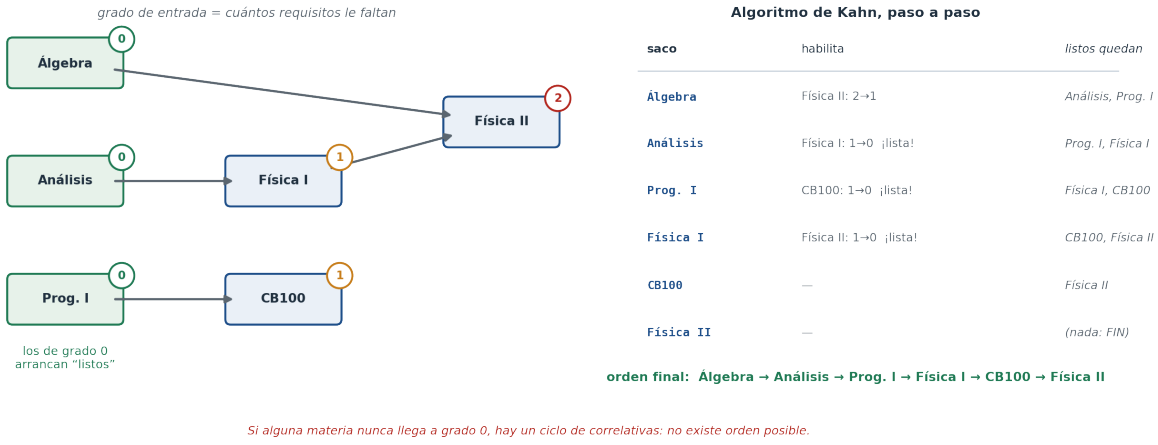


Figura 11.6: A la izquierda, el grafo de correlatividades con el grado de entrada de cada materia en su esquina. A la derecha, la traza completa de Kahn: qué materia sale en cada paso, a quién habilita, y cómo queda la fila de "listas". El orden final respeta todas las flechas.

```java
public static List<Integer> ordenar(Grafo grafo) {
    int n = grafo.cantidadDeVertices();
    int[] gradoDeEntrada = new int[n];                // cuántas aristas LLEGAN a cada uno
    for (int vertice = 0; vertice < n; vertice++) {
        for (Grafo.Arista arista : grafo.vecinos(vertice)) {
            gradoDeEntrada[arista.destino()]++;
        }
    }
    Deque<Integer> listos = new ArrayDeque<>();       // los que ya no esperan a nadie
    for (int vertice = 0; vertice < n; vertice++) {
        if (gradoDeEntrada[vertice] == 0) {
            listos.addLast(vertice);
        }
    }
    List<Integer> orden = new ArrayList<>();
    while (!listos.isEmpty()) {
        int vertice = listos.removeFirst();
        orden.add(vertice);                           // "cursamos" esta materia
        for (Grafo.Arista arista : grafo.vecinos(vertice)) {
            gradoDeEntrada[arista.destino()]--;       // un requisito menos
            if (gradoDeEntrada[arista.destino()] == 0) {
                listos.addLast(arista.destino());     // quedó habilitada
            }
        }
    }
    if (orden.size() < n) {                           // alguien nunca llegó a 0...
        throw new IllegalStateException("el grafo tiene un ciclo: no hay orden posible");
    }
    return orden;
}
```

Idea clave

El `if` final es un **detector de ciclos gratis**: si el grafo tiene un ciclo (A requiere B, B requiere C, C requiere A), ninguno de los tres llega jamás a grado 0 y el orden sale incompleto. Kahn responde entonces DOS preguntas con el mismo $O(n+m)$: "¿en qué orden?" y "¿es posible algún orden?". Esto se usa a diario: Gradle ordena así las tareas de compilación (`build` depende de `test`, que depende de `compileJava`…) y te grita "circular dependency" cuando detecta el ciclo. Un orden topológico puede no ser único — cualquier orden que respete las flechas vale.

11.6 Caminos mínimos con pesos: Dijkstra

BFS minimiza la cantidad de aristas — pero en el mundo real las aristas no cuestan lo mismo: 300 km no es lo mismo que 1150 km, 2 minutos no es lo mismo que 45. Con **pesos**, el camino de menos tramos puede ser carísimo. Hace falta otro algoritmo, y es EL algoritmo famoso de los grafos: **Dijkstra**.

11.6.1 La idea desde cero

Dijkstra mantiene, para cada vértice, la **mejor distancia conocida hasta ahora** desde el origen (al principio: 0 para el origen, ∞ para el resto). Y repite un paso goloso:

1. De todos los vértices **pendientes**, tomá el de MENOR distancia conocida y **confirmalo**: esa distancia es definitiva.
2. **Relajá** sus aristas: para cada vecino, si llegar "pasando por acá" es más barato que lo que el vecino tenía anotado, actualizale la anotación.

¿Por qué es correcto confirmar al más cercano? Porque los pesos **no son negativos**: cualquier otro camino hasta él tendría que pasar por algún vértice pendiente más lejano… y de ahí sólo se puede empeorar. (Ese es el argumento de intercambio de la U10 — el goloso con demostración.)

Y ¿quién entrega eficientemente "el pendiente de menor distancia"? La **cola de prioridad** de la U9.

Dijkstra = BFS con el heap en el lugar de la cola.

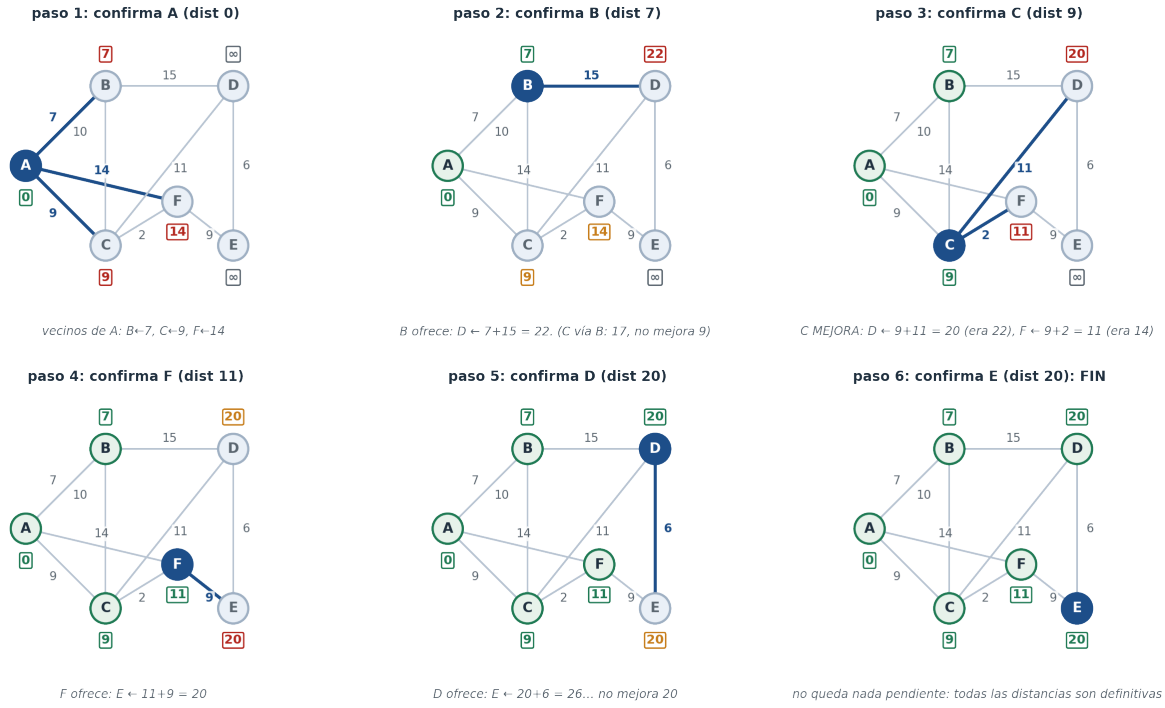


Figura 11.7: Dijkstra paso a paso desde A. Cada panel confirma un vértice (azul), que pasa a verde con su distancia definitiva. Los recuadros muestran la mejor distancia conocida: rojo = recién actualizada, naranja = pendiente, verde = confirmada. Mirá el paso 3: C mejora a D (22 → 20) y a F (14 → 11) — el camino directo A–F nunca fue la respuesta.

La misma ejecución en formato tabla — así conviene hacerlo a mano en el parcial:

	A	B	C	D	E	F	
inicio	0*	∞	∞	∞	∞	∞	*A entra con 0*
confirma A		7*	9	∞	∞	14	*B, C y F reciben ofertas*
confirma B (7)			9*	22	∞	14	*D ← 22 vía B*
confirma C (9)				20*	∞	11	*¡C mejora D (20) y F (11)!*
confirma F (11)				20*	20		*E ← 20 vía F*
confirma D (20)					20*		*E vía D: 26, no mejora*
confirma E (20)							*FIN*

n*: el pendiente MÁS CHICO — se confirma en el paso siguiente

distancia ya confirmada (definitiva): no se toca nunca más

camino a E: se lee al revés con anterior[]:
E ← F ← C ← A ⇒ A → C → F → E (20)

Figura 11.8: La traza de Dijkstra como tabla: una fila por confirmación. En cada fila, el pendiente más chico (naranja) es el que se confirma a continuación (verde). Abajo: el camino a E se reconstruye con anterior[] leyendo al revés.

11.6.2 El código

```java
public static Resultado caminosMinimos(Grafo grafo, int origen) {
    int[] distancia = new int[n];      // mejor distancia conocida
    int[] anterior = new int[n];       // de dónde vino esa mejor distancia
    boolean[] confirmado = new boolean[n];
    Arrays.fill(distancia, INFINITO);
    Arrays.fill(anterior, -1);

    PriorityQueue<Pendiente> cola =            // ¡el heap de la U9!
            new PriorityQueue<>(Comparator.comparingInt(Pendiente::distancia));
    distancia[origen] = 0;
    cola.add(new Pendiente(origen, 0));

    while (!cola.isEmpty()) {
        Pendiente pendiente = cola.poll();     // el más cercano al origen
        int vertice = pendiente.vertice();
        if (confirmado[vertice]) {
            continue;                          // copia vieja: ya lo confirmamos mejor
        }
        confirmado[vertice] = true;            // su distancia es DEFINITIVA

        for (Grafo.Arista arista : grafo.vecinos(vertice)) {
            int nuevaDistancia = distancia[vertice] + arista.peso();
            if (nuevaDistancia < distancia[arista.destino()]) {
                distancia[arista.destino()] = nuevaDistancia;   // relajar la arista
                anterior[arista.destino()] = vertice;
                cola.add(new Pendiente(arista.destino(), nuevaDistancia));
            }
        }
    }
    return new Resultado(distancia, anterior);
}
```

Dos detalles finos del código, que son pregunta de final:

- **Copias viejas en el heap.** Cuando un vértice mejora su distancia, no lo buscamos dentro del heap para actualizarlo (sería $O(n)$): simplemente agregamos OTRA entrada con la distancia nueva. La vieja queda adentro, pero cuando salga, `confirmado[]` la descarta con el `continue`. Se llama *eliminación perezosa* — el mismo truco vale para cualquier `PriorityQueue`.
- **Reconstrucción.** `distancia[]` da el CUÁNTO; `anterior[]` da el POR DÓNDE. El camino a E se lee al revés: `anterior[E]=F`, `anterior[F]=C`, `anterior[C]=A` → A, C, F, E. Igual que la tabla de decisiones de la DP (U10).

Con lista de adyacencia y heap, el costo es $O((n + m) \log n)$: cada arista puede provocar una inserción al heap ($\log n$) y cada extracción cuesta lo mismo. Para el grafo del dibujo es instantáneo; para un mapa vial con millones de nodos, sigue siendo práctico — por eso tu GPS responde en milisegundos.

```java
Resultado resultado = caminosMinimos(mapa, 0);
System.out.println(Arrays.toString(resultado.distancia()));  // [0, 7, 9, 20, 20, 11]
System.out.println(resultado.caminoHasta(4));                // [0, 2, 5, 4] = A C F E
```

Error común

Dijkstra exige pesos no negativos. Con una arista negativa, “confirmar al más cercano” deja de ser válido: un camino más largo podría abaratarse después con el tramo negativo, y Dijkstra ya cerró la puerta. Si hay pesos negativos se usa otro algoritmo (Bellman–Ford, que verás en materias siguientes). En tus modelos, si aparece la tentación de un peso negativo (“me PAGAN por pasar por acá”), pará y replanteá el modelo.

11.7 Conectar todo al menor costo: árboles de tendido mínimo

Cambio de pregunta. Ya no "¿cómo llego de A a B?" sino: **"¿cómo conecto TODOS los vértices gastando lo menos posible?"** — tender fibra óptica entre ciudades, cablear un edificio, unir estaciones de subte. La respuesta siempre tiene forma de **árbol**: si tuviera un ciclo, sobra una arista (sacala y sigue todo conectado, más barato). El árbol que conecta todo con costo total mínimo se llama **árbol de tendido mínimo** (MST, *minimum spanning tree*), tiene exactamente $n - 1$ aristas, y hay dos algoritmos golosos famosos para encontrarlo. Los dos funcionan; se diferencian en CÓMO miran el grafo.

11.7.1 Kruskal: las aristas más baratas primero

Kruskal es el goloso más directo imaginable: **ordená todas las aristas de menor a mayor peso** y aceptalas en ese orden, salvo que la arista forme un **ciclo** (una las dos puntas de algo ya conectado). Parás al juntar $n - 1$.

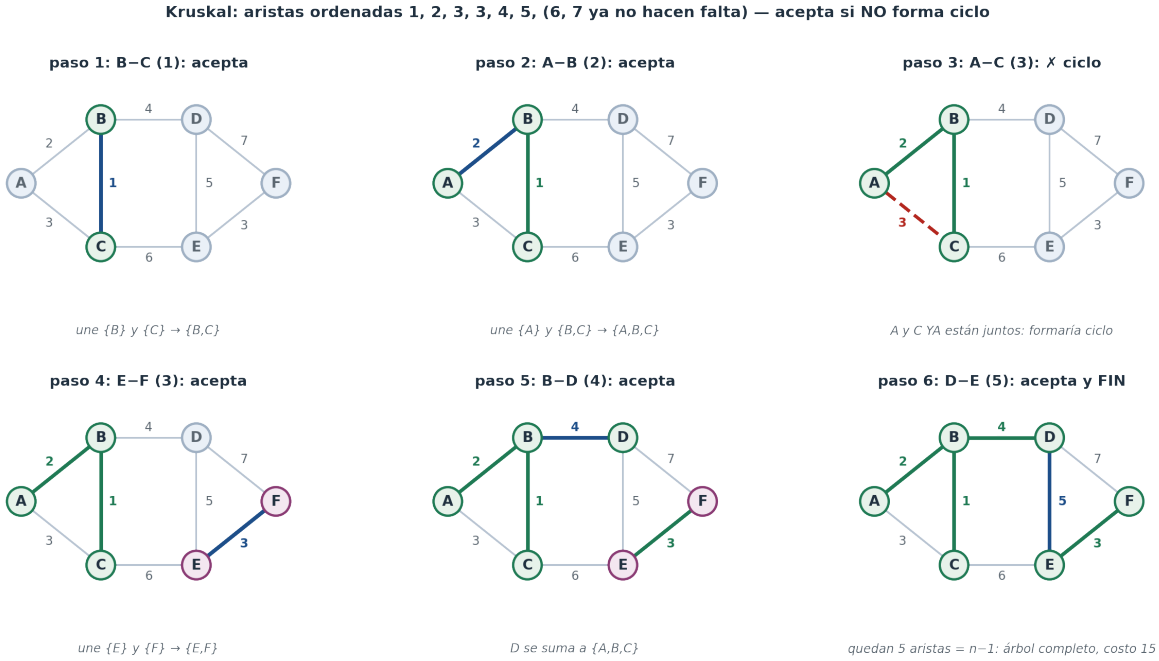


Figura 11.9: Kruskal sobre la red de ejemplo. Las aristas se procesan ordenadas por peso: 1, 2, 3, 3, 4, 5. El color de cada vértice indica su grupo. La única rechazada es A−C (3): A y C ya estaban en el mismo grupo verde y cerrarían un ciclo. Con 5 aristas aceptadas (n−1) el árbol está listo: costo 15.

El único problema técnico: ¿cómo saber RÁPIDO si dos vértices ya están conectados? Recorrer con BFS antes de cada arista costaría $O(m)$ cada vez. Para eso existe una mini-estructura auxiliar preciosa: **union-find**.

11.7.2 Union-find: la estructura de los grupos

Union-find mantiene n elementos repartidos en grupos, con dos operaciones: `encontrar(x)` (¿a qué grupo pertenece x?) y `unir(x, y)` (fusionar dos grupos). Cada grupo es un arbolito donde cada elemento apunta a su "padre"; el representante del grupo es la raíz.

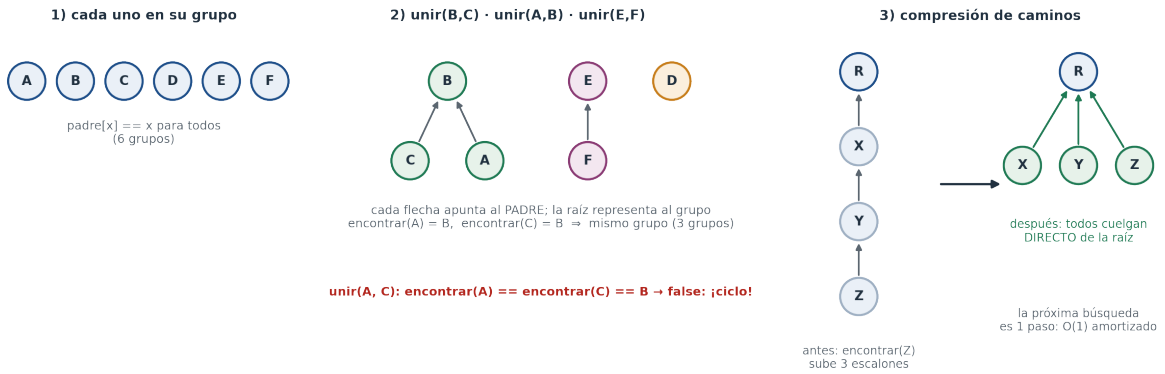


Figura 11.10: Union-find en tres tiempos. (1) Cada elemento arranca solo. (2) Tras unir(B,C), unir(A,B) y unir(E,F): tres arbolitos; encontrar sube hasta la raíz, y dos elementos están conectados si llegan a la MISMA raíz. (3) La compresión de caminos: después de una búsqueda, todos los visitados quedan colgados directo de la raíz — la próxima vez es un paso.

```java
public int encontrar(int elemento) {
    if (padre[elemento] != elemento) {
        padre[elemento] = encontrar(padre[elemento]);   // compresión de caminos
    }
    return padre[elemento];
}

public boolean unir(int unElemento, int otroElemento) {
    int unaRaiz = encontrar(unElemento);
    int otraRaiz = encontrar(otroElemento);
    if (unaRaiz == otraRaiz) {
        return false;              // ya estaban juntos: unirlos formaría un ciclo
    }
    // unión por rango: el arbolito más bajo se cuelga del más alto
    ...
    return true;
}
```

Con las dos optimizaciones (compresión de caminos + unión por rango), cada operación cuesta en la práctica $O(1)$ **amortizado** — el análisis formal da la famosa inversa de Ackermann, una función que crece tan lento que nunca pasa de 5. Amortizado: la palabra de la U2, cerrando el círculo.

Con union-find, Kruskal queda en tres líneas conceptuales — y su costo es el del ordenamiento: $O(m \log m)$.

```java
aristas.sort(Comparator.comparingInt(Tramo::peso));       // el paso goloso
UnionFind grupos = new UnionFind(n);
for (Tramo tramo : aristas) {
    if (grupos.unir(tramo.origen(), tramo.destino())) {   // ¿NO forma ciclo?
        arbol.add(tramo);
        if (arbol.size() == n - 1) {
            break;   // árbol completo
        }
    }
}
```

11.7.3 Prim: el árbol que crece

Prim mira el mismo problema desde el otro lado: en vez de aristas sueltas que se van agrupando, un ÚNICO árbol que **crece** desde un vértice inicial. En cada paso, de todas las aristas que salen del árbol hacia afuera (la “frontera”), agrega la más barata. ¿Quién entrega la más barata? Otra vez la

cola de prioridad de la U9 — Prim es a Kruskal lo que Dijkstra es a BFS: el mismo esqueleto con un heap adentro.

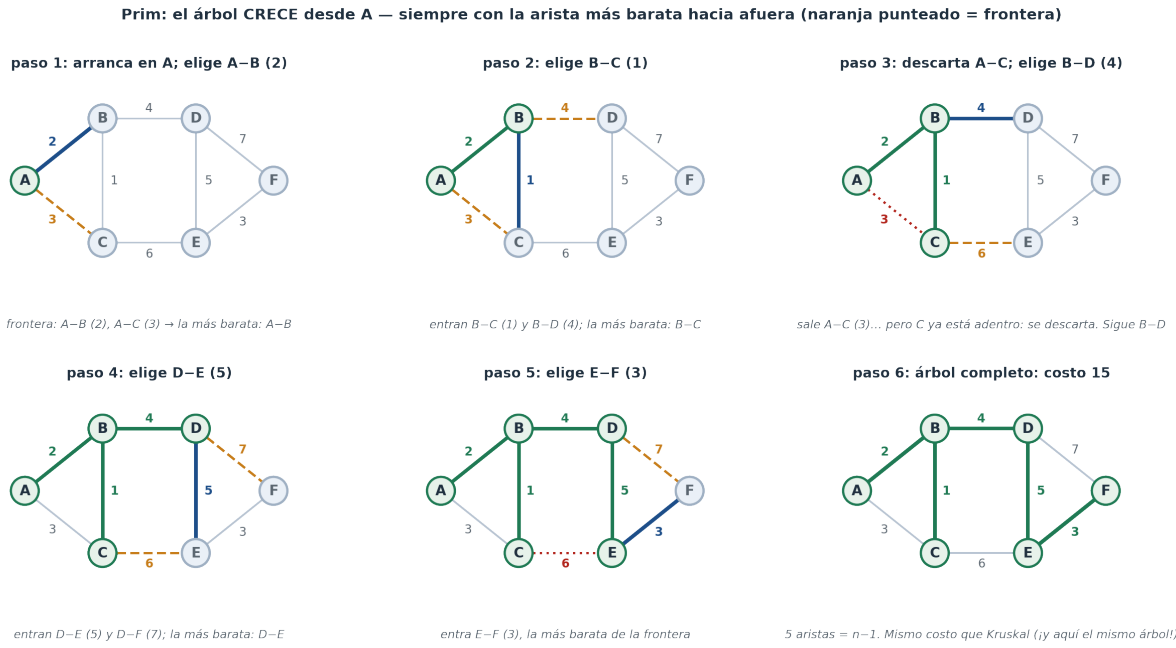


Figura 11.11: Prim desde A. El árbol (verde) crece un vértice por paso; las aristas punteadas naranjas son la frontera (candidatas en el heap); la azul es la elegida del paso. En el paso 3 sale del heap A−C (3), pero C ya está adentro: se descarta y sigue B−D (4). Mismo costo final que Kruskal: 15.

```
agregar(grafo, inicial, enElArbol, frontera);       // el árbol arranca con 1 vértice
while (!frontera.isEmpty() && arbol.size() < n - 1) {
    Tramo tramo = frontera.poll();                  // la arista más barata hacia afuera
    if (enElArbol[tramo.destino()]) {
        continue;                                   // ya entró por otro lado: descartada
    }
    arbol.add(tramo);                               // ¡nuevo tramo del árbol!
    agregar(grafo, tramo.destino(), enElArbol, frontera);
}
```

```
System.out.println(Kruskal.arbolDeTendidoMinimo(red));      // B-C, A-B, E-F, B-D, D-E
System.out.println(Prim.arbolDeTendidoMinimo(red, 0));      // A-B, B-C, B-D, D-E, E-F
// distinto ORDEN de descubrimiento... y el mismo árbol: costo 15 en ambos
```

Idea clave

¿Por qué estos golosos SÍ funcionan, cuando el del vuelto con {1, 3, 4} fallaba? Por el **argumento de intercambio** (U10): si un supuesto árbol óptimo no usara la arista más barata que cruza entre dos grupos de vértices, se le puede *intercambiar* una arista más cara de ese cruce por la barata — sigue conectando todo y cuesta menos o igual. La elección golosa nunca se paga caro. Kruskal y Prim son el final feliz de la historia greedy: elección local, óptimo global, demostración incluida.

Con Git / Gradle

Momento de repasar tu flujo: `git switch -c unidad-11`, implementá los ejercicios de la guía, un commit por ejercicio con mensaje imperativo (“Agrega BFS sobre la red de vuelos”), y al final `git switch main` + `git merge unidad-11`. En la unidad final vas a ver cómo este mismo flujo se convierte, con GitHub y pull requests, en la forma en que trabajan los equipos reales.

11.8 Ejemplo integrador: el planificador de viajes

Una red de rutas argentinas: 7 ciudades, 7 rutas con sus distancias. Sobre el MISMO grafo, tres preguntas distintas — y cada una la responde un algoritmo distinto de esta unidad.

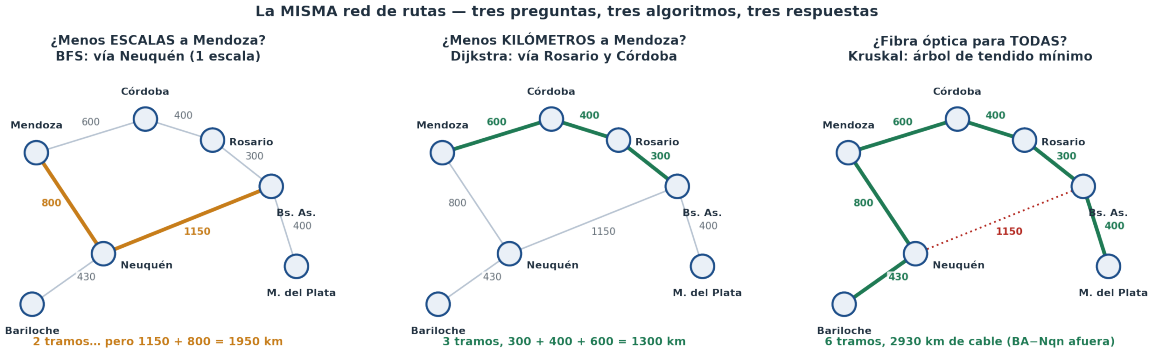


Figura 11.12: La misma red, tres preguntas. Izquierda: BFS minimiza tramos — a Mendoza por Neuquén, 1 escala, pero 1950 km. Centro: Dijkstra minimiza kilómetros — por Rosario y Córdoba, 3 tramos pero 1300 km. Derecha: Kruskal conecta las 7 ciudades con 2930 km de fibra, descartando la ruta más redundante (BA−Neuquén).

```java
public final class PlanificadorDeViajes {
    private final Grafo rutas;          // no dirigido, pesos en km
    private final String[] nombres;

    /** ¿Con cuántas escalas llego? → BFS: menos ARISTAS. */
    public int escalasMinimas(int origen, int destino) {
        int tramos = Recorridos.distanciasDesde(rutas, origen)[destino];
        if (tramos == -1) {
            throw new IllegalArgumentException("no hay ruta");
        }
        return Math.max(0, tramos - 1);             // n tramos = n−1 escalas
    }

    /** ¿El viaje más corto en km? → Dijkstra: menos PESO. */
    public String viajeMasCorto(int origen, int destino) {
        Dijkstra.Resultado resultado = Dijkstra.caminosMinimos(rutas, origen);
        List<Integer> camino = resultado.caminoHasta(destino);
        // ... arma "Buenos Aires -> Rosario -> Córdoba -> Mendoza (1300 km)"
    }

    /** ¿Fibra óptica para todas al menor costo? → Kruskal: MST. */
    public String tendidoDeFibraMinimo() {
        List<Tramo> arbol = Kruskal.arbolDeTendidoMinimo(rutas);
        // ... arma la lista de tramos + " (2930 km de cable)"
    }
}
```

La salida completa (corré `PlanificadorDeViajes` en el repo):

```
1
Buenos Aires -> Rosario -> Córdoba -> Mendoza (1300 km)
Buenos Aires -> Neuquén -> Bariloche (1580 km)
Buenos Aires-Rosario, Buenos Aires-Mar del Plata, Rosario-Córdoba,
Neuquén-Bariloche, Córdoba-Mendoza, Mendoza-Neuquén (2930 km de cable)
```

Idea clave

"El camino mínimo a Mendoza" no significa NADA hasta decir **qué se minimiza**: ¿tramos (BFS), kilómetros (Dijkstra), o costo total de infraestructura (MST)? Modelar el problema —

decidir qué es un vértice, qué es una arista, qué significa el peso y qué pregunta se hace — es el 80 % de resolver un problema de grafos. El algoritmo después sale de esta tabla:

La pregunta	El algoritmo	Costo
¿Están conectados? ¿Quiénes?	BFS o DFS	$O(n+m)$
Menos aristas de u a v	BFS + `anterior[]`	$O(n+m)$
¿Orden compatible con dependencias? ¿Hay ciclo?	Kahn	$O(n+m)$
Menor peso de u a v (pesos ≥ 0)	Dijkstra	$O((n+m)\log n)$
Conectar todo al menor costo	Kruskal o Prim	$O(m \log m)$

Con IA (plugin de Claude)

Los grafos son ideales para trabajar con el plugin de Claude en IntelliJ, con reglas claras: (1) pedile que genere **casos de prueba** (“un grafo de 50 vértices con 3 componentes conexas”, “un dirigido con exactamente un ciclo”) y verificá tu código contra ellos; (2) pedile el dibujo de un grafo que no entiendas (“dibujame este grafo en ASCII por niveles”); (3) NUNCA le pidas que te escriba Dijkstra — escribilo vos y pedile después una revisión crítica (“¿qué caso borde me falta testear?”). En el parcial no hay IA: el músculo de trazar BFS, Dijkstra y Kruskal a mano se entrena a mano.

11.9 Errores comunes

Error común

- Olvidar `visitado[]` en un grafo con ciclos: recorrido infinito (en árboles zafabas; acá no).
- En BFS, marcar al desencolar en vez de al encolar: el mismo vértice entra varias veces a la cola.
- Usar BFS para caminos mínimos **con pesos**: BFS cuenta aristas, no suma pesos. Menos tramos ≠ más corto.
- Usar Dijkstra con pesos **negativos**: el resultado puede ser directamente incorrecto, sin aviso.
- Pedir orden topológico de un grafo **no dirigido** o con ciclos: no existe (el código debe detectarlo, no colgarse).
- En Kruskal, detectar ciclos con un recorrido por arista ($O(m^2)$) en vez de union-find.
- En no dirigidos con lista de adyacencia, olvidar la arista espejo al agregar — o contarla dos veces al listar aristas (en el repo: `if (vertice < arista.destino())`).
- Elegir matriz de adyacencia “porque es más fácil” en un grafo grande y ralo: n^2 de memoria por pereza.
- Confundir las preguntas: MST no sirve para caminos mínimos (el camino A→B dentro del MST puede ser mucho más largo que el camino mínimo real).

11.10 Repaso

1. Definí grafo dirigido, no dirigido y ponderado, con un ejemplo real de cada uno que no esté en el apunte.
2. Para un grafo de $n = 10^6$ vértices y $m = 10^7$ aristas: ¿cuánta memoria gasta cada representación? ¿Cuál elegís y por qué?
3. ¿Qué operación es $O(1)$ en la matriz y cara en la lista? ¿Y al revés?
4. Trazá BFS y DFS desde B en el grafo A–F del apunte (a mano, con cola/pila dibujadas). ¿En qué paso divergen?
5. ¿Por qué BFS encuentra el camino con menos aristas? ¿Qué estructura hay que agregar para reconstruir ese camino?
6. ¿Por qué en grafos hace falta `visitado[]` y en árboles (U8) no hacía falta?

7. Explicá el algoritmo de Kahn con la metáfora de las correlatividades. ¿Cómo detecta ciclos sin buscarlos?
8. Trazá Dijkstra desde B en el grafo de la unidad (tabla completa). ¿Cambia el camino mínimo entre A y E?
9. ¿Por qué Dijkstra necesita pesos no negativos? Construí un grafo de 3 vértices con una arista negativa donde falle.
10. ¿Qué hace `unir` cuando devuelve `false`, y por qué eso es exactamente lo que Kruskal necesita saber?
11. Kruskal y Prim son golosos. ¿Cuál es la elección golosa de cada uno, y qué argumento (U10) garantiza que no se equivocan?
12. Tu GPS te ofrece “menos cuadras” y “menos minutos”. ¿Qué algoritmo hay detrás de cada opción y qué cambia en el grafo?

Guía de ejercicios — Unidad 11

Objetivos de la unidad

Practicá **grafos**: modelar, representar, recorrer y optimizar. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i11_grafos` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i11_grafos/i01_teorico`. Podés (y conviene) reutilizar las clases de `material.i11_grafos` (`Grafo`, `Recorridos`, `Dijkstra`, `UnionFind`…) — en los difíciles vas a implementar las tuyas. En cada ejercicio, arrancá por el MODELO: ¿qué es un vértice, qué es una arista, qué significa el peso?

11.11 Preguntas teóricas

1. ¿Qué diferencia a un grafo de un árbol y de una lista enlazada? ¿Por qué se dice que es la estructura más general?
2. Compará matriz y lista de adyacencia: memoria y costo de las dos consultas típicas. ¿Cuándo gana cada una?
3. ¿Qué es un grafo ralo (*sparse*) y por qué domina en los problemas reales?
4. ¿Por qué BFS usa una cola y DFS una pila? ¿Qué propiedad del recorrido produce cada estructura?
5. ¿Qué garantiza que BFS encuentre el camino con menos aristas? ¿Por qué esa garantía se rompe con pesos?
6. Explicá el grado de entrada y su papel en el algoritmo de Kahn. ¿Cómo detecta ciclos?
7. Explicá el paso de “relajación” de Dijkstra y el rol de la cola de prioridad.
8. ¿Qué es la eliminación perezosa en Dijkstra y por qué evita actualizar el heap?
9. ¿Qué es un árbol de tendido mínimo? ¿Por qué tiene exactamente $n-1$ aristas?
10. Compará Kruskal y Prim: qué mira cada uno, qué estructura auxiliar necesita cada uno y qué costo tienen.

11.12 Nivel fácil

1. **Red de amistades.** Modelá con `Grafo` la red de la figura 150 (Ana, Beto, Carla, Dana, Eva) y respondé por código: grados de todos, ¿son amigos Ana y Dana?, ¿cuántas aristas hay? Test.
2. **Grados en dirigidos.** Para un `Grafo` dirigido, calculá grado de salida y de entrada de cada vértice: `int[] gradosDeEntrada(Grafo g)`. Test con la red de seguidores de la figura 150.
3. **BFS a mano y por código.** Trazá BFS desde C en el grafo A–F del apunte (comentario en el código) y verificá con `Recorridos.bfs`. Test del orden completo.
4. **¿Están conectados?** `boolean hayCamino(Grafo g, int u, int v)` usando `distanciasDesde`. Test con un grafo de dos componentes.
5. **Escalas.** Con la red de ciudades del integrador, respondé con BFS: ¿cuántas escalas hay de Bariloche a Mar del Plata? Test.
6. **Matriz desde lista.** `int[][] aMatriz(Grafo g)`: convertí un `Grafo` (lista) a matriz de adyacencia. Test comparando contra `GrafoMatriz`.
7. **Vecinos comunes.** `List<Integer> vecinosComunes(Grafo g, int u, int v)` (amigos en común). Test.
8. **Correlativas propias.** Modelá 6 materias de TU plan con sus correlativas y calculá un orden topológico con `OrdenTopologico.ordenar`. Test de que el orden respeta cada arista.
9. **Dijkstra de lectura.** Sobre el grafo de la figura 156, obtené `distancia[]` desde A con `Dijkstra.caminosMinimos` y verificá contra la tabla del apunte ([0,7,9,20,20,11]). Test.

10. **Costo de un tendido.** Dada la red de fibra del apunte, calculá el MST con `Kruskal.arbolDeTendidoMinimo` y verificá costo 15 y que tiene n−1 tramos. Test.

11.13 Nivel medio

1. **Componentes conexas.** `int cantidadDeComponentes(Grafo g)` y `int[] componenteDe(Grafo g)` con BFS repetido. Test con 1, 2 y 3 componentes.
2. **Camino BFS reconstruido.** `List<Integer> caminoConMenosAristas(Grafo g, int u, int v)` guardando `anterior[]`. Test contra el grafo A–F (camino A→F = A,B,D,F o A,C,D,F o A,C,E,F: verificá largo 3).
3. **Detección de ciclos en no dirigidos.** `boolean tieneCiclo(Grafo g)` con DFS (ojo: la arista al padre no cuenta). Test con un árbol (false) y con el grafo A–F (true).
4. **¿Es bipartito?** `boolean esBipartito(Grafo g)` coloreando con BFS en dos colores. Test con un ciclo par (true) y uno impar (false).
5. **Todos los caminos mínimos.** Corré Dijkstra desde cada vértice del grafo de la figura 156 y armá la matriz de distancias `int[][]`. Verificá que es simétrica (grafo no dirigido). Test.
6. **Red de vuelos con escalas máximas.** Con BFS, `List<Integer> destinosConHasta(Grafo g, int origen, int escalas)`: aeropuertos alcanzables con a lo sumo k escalas. Test.
7. **Kahn con desempate.** Modificá el orden topológico para que, entre varios "listos", salga siempre el de menor número (usá una `PriorityQueue` en vez de la cola). Test: el orden es único y predecible.
8. **Dijkstra con camino.** Para la red de ciudades del integrador, `String rutaMasCorta(String origen, String destino)` con nombres. Test: BA→Bariloche = "Buenos Aires -> Neuquén -> Bariloche (1580 km)".
9. **Prim vs Kruskal.** Generá 20 grafos conexos aleatorios (semilla fija) y verificá que Prim y Kruskal dan SIEMPRE el mismo costo total. Test.
10. **Aristas críticas.** Una arista del MST es "crítica" si al quitarla del grafo el costo del nuevo MST empeora (o el grafo se desconecta). Encontrá las críticas de la red de fibra del apunte por fuerza bruta (quitar y recalcular). Test.

11.14 Nivel difícil

1. **Grafo genérico con Map.** Implementá `GrafoConNombres<T>` sobre `Map<T, List<T>>` (sin números): agregar, vecinos, BFS y DFS. Test con vértices `String`.
2. **Laberinto.** Dado un `char[][]` con paredes `#`, entrada `E` y salida `S`, modelalo como grafo implícito (cada celda libre = vértice, vecinos = casillas adyacentes) y encontrá el camino más corto con BFS. Test con laberinto sin salida.
3. **Orden topológico por DFS.** Implementá la variante de orden topológico basada en DFS (apilar cada vértice al TERMINAR su exploración y leer la pila al revés) con detección de ciclos por marcas gris/negro. Test comparando validez contra Kahn.
4. **Dijkstra sin heap.** Implementá Dijkstra con búsqueda lineal del pendiente mínimo, $O(n^2)$, y comparalo con la versión con heap: mismos resultados en 30 grafos aleatorios, y medí tiempos para n = 2000 denso. Test.
5. **Bellman–Ford.** Investigá e implementá Bellman–Ford (soporta pesos negativos, detecta ciclos negativos). Test: un grafo donde Dijkstra se equivoca y Bellman–Ford no.
6. **Union-find desde cero.** Implementá tu propio `UnionFind` con compresión de caminos y unión por rango, con contadores de "escalones subidos" para MEDIR el efecto de la compresión (con y sin). Test.
7. **Segunda mejor ruta.** Para la red de ciudades, encontrá el segundo camino más corto entre dos ciudades (puede repetir vértices del primero pero no ser idéntico). Pista: quitá una a una las aristas del camino óptimo y recalculá. Test.
8. **Cuello de botella.** El "camino minimax": entre u y v, el camino que minimiza la arista MÁS PESADA (útil para camiones con peso máximo por puente). Pista: se resuelve con una variante de Prim/Kruskal. Test.

9. **Kruskal máximo.** Adaptá Kruskal para árbol de tendido MÁXIMO y usalo para resolver: ¿cuál es la arista más barata que sí o sí conviene mantener en toda red? Test.
10. **Torneo de recorridos.** Un test propiedad: para 50 grafos conexos aleatorios (semilla fija), verificá que (a) BFS y DFS visitan el mismo CONJUNTO de vértices, (b) `distanciasDesde` nunca da distancias mayores que las de Dijkstra con pesos 1, (c) el costo de Prim == el de Kruskal.

Unidad 12

Colecciones, Streams y el trabajo profesional

Objetivos de la unidad

La unidad de cierre. Al terminarla vas a poder: (1) ver TODO el curso en un solo mapa — cada estructura que estudiaste es una clase de `java.util` — y elegir la correcta en segundos; (2) escribir código **declarativo** con **Streams** (filter, map, reduce, collect) como se escribe hoy en la industria; (3) entender el **flujo de trabajo profesional** completo: issues, ramas, pull requests, code review, integración continua y testing; y (4) usar la **IA** como multiplicador dentro de ese flujo, sin regalarle lo que no se delega. Esta unidad es el puente entre la materia y tu primer día de trabajo.

Once unidades atrás escribiste tu primer `System.out.println`. En el camino implementaste a mano pilas, colas, tablas de hash, árboles balanceados, heaps y grafos — y ésa era la idea: entender las tripas. Pero en el trabajo NO vas a reimplementar un `HashMap`: vas a elegirlo, combinarlo y justificar la elección con los costos que ahora conocés. Esta unidad da vuelta la mirada: del "cómo se construye" al "cómo se usa profesionalmente" — las colecciones como caja de herramientas, los streams como estilo de trabajo, y el taller completo donde todo eso sucede: git, PRs, CI y tu criterio como última línea de defensa.

12.1 Un poco de historia

El **Java Collections Framework** apareció con Java 2 (1998), diseñado por **Joshua Bloch** — el mismo del libro *Effective Java* que vas a cruzar en cualquier empresa seria. Los **Streams** llegaron con Java 8 (2014), la mayor revolución del lenguaje: junto con las *lambdas* trajeron a Java el estilo funcional que hoy domina el código de negocio. Del otro lado del taller, **Linus Torvalds** escribió **git** en 2005 (en dos semanas, para gestionar el código de Linux), GitHub lo volvió social en 2008, y la **integración continua** — la práctica de compilar y testear TODO en cada cambio — pasó de herejía de los "extremistas" del *Extreme Programming* a estándar universal. El último invitado llegó en esta década: los asistentes de IA, que hoy escriben un porcentaje enorme del código nuevo del mundo… supervisados (cuando el equipo es serio) por gente que entiende lo que firma. Como vos, a partir de acá.

12.2 El mapa completo de las colecciones

Cada estructura que construiste a mano en este curso vive en `java.util`, probada y optimizada por veinticinco años de uso industrial. Éste es el mapa:

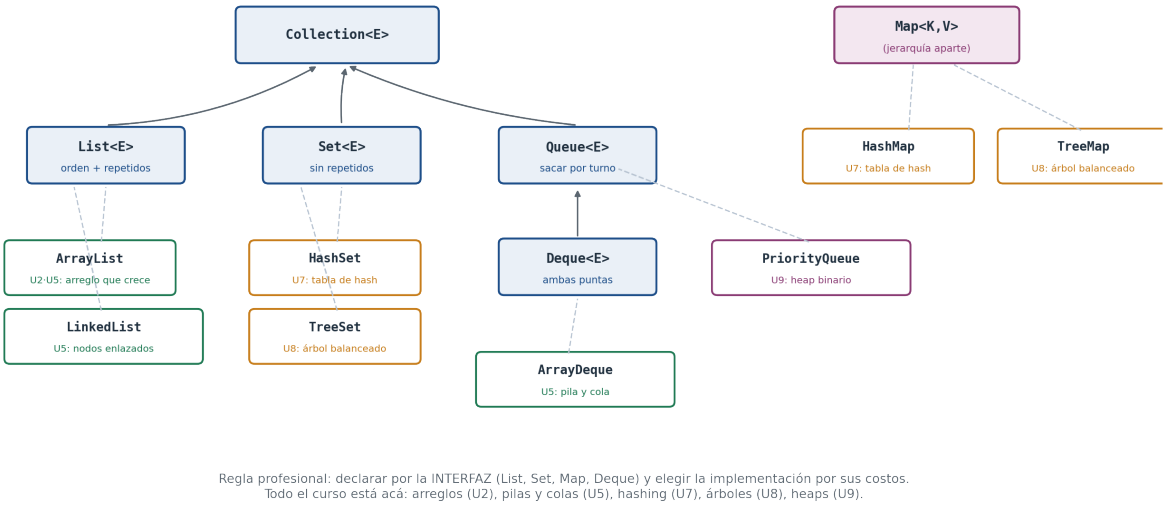


Figura 12.1: El Java Collections Framework con nombres propios: las interfaces (List, Set, Queue/Deque, y Map en su jerarquía aparte) y, debajo de cada una, las implementaciones que construiste durante el curso, con su unidad de origen.

Idea clave

Declarás por la interfaz, elegís la implementación por los costos. `List<Turno> turnos = new ArrayList<>()` — el tipo de la variable dice QUÉ necesitás (una secuencia con posiciones); el `new` dice CÓMO se implementa (un arreglo que crece, U2). Si mañana hace falta cambiar la implementación, el resto del programa ni se entera: eso es la abstracción de la U3 aplicada a la biblioteca estándar. En una entrevista de trabajo, "¿por qué usaste `HashMap` y no `TreeMap`?" se responde con una palabra de este curso: costos.

12.2.1 ¿Cuál uso? El árbol de decisión

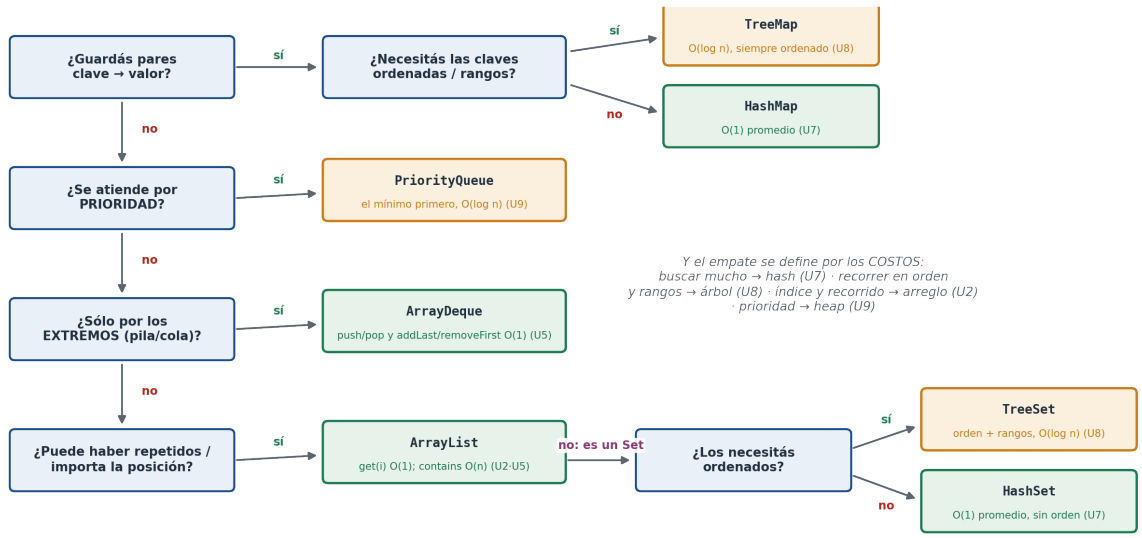


Figura 12.2: Cuatro preguntas bastan: ¿clave→valor? ¿prioridad? ¿sólo extremos? ¿repetidos? — y el desempate final entre hash (O(1), sin orden) y árbol (O(log n), con orden).

Y la tabla que resume once unidades — la que vale la pena saber de memoria:

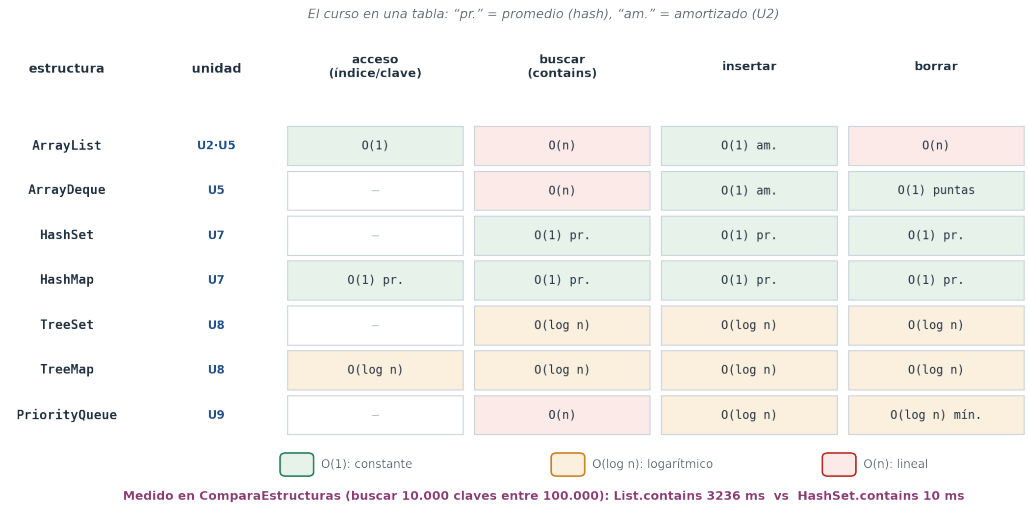


Figura 12.3: La tabla de costos de todas las estructuras del curso. Verde = constante, naranja = logarítmico, rojo = lineal. Abajo, la medición real de ComparaEstructuras: la diferencia entre O(n) y O(1) no es teoría — es 3236 ms contra 10 ms.

```
// El experimento de ComparaEstructuras (corrélo en el repo): los MISMOS datos,
// las MISMAS búsquedas — sólo cambia la estructura.
List<String> lista = new ArrayList<>();      // ... 100.000 elementos
Set<String> conjunto = new HashSet<>(lista); // los mismos, en una tabla de hash

lista.contains("fantasma-99");      // O(n): recorre TODO   → 10.000 búsquedas: 3236 ms
conjunto.contains("fantasma-99");   // O(1) promedio (U7)   → 10.000 búsquedas:   10 ms
```

Error común

Tres trampas de colecciones que aparecen en todo código real:

- **`List.of(...)` es inmutable**: `add` sobre ella lanza `UnsupportedOperationException`. Para una lista modificable a partir de otra: `new ArrayList<>(otra)`.
- **Modificar mientras recorrés**: borrar dentro de un `for-each` lanza `ConcurrentModificationException`. Usá `removeIf(condicion)` o un `Iterator` explícito.
- **Objetos mutables como claves**: si un objeto cambia después de entrar a un `HashSet`/`HashMap`, su `hashCode` cambia y el objeto "se pierde" adentro (U7). Claves inmutables — `String`, `record` — siempre.

12.3 Streams: de decir CÓMO a decir QUÉ

Mirá dos versiones del mismo problema — "los nombres de los aprobados, ordenados por nota descendente" — tal como están en `DemoStreams`:

```
// IMPERATIVA: describís CÓMO, paso a paso, con estado intermedio
List<Alumno> aprobados = new ArrayList<>();
for (Alumno alumno : CURSO) {
    if (alumno.nota() >= 6) {
        aprobados.add(alumno);
    }
}
aprobados.sort(Comparator.comparingInt(Alumno::nota).reversed());
List<String> nombres = new ArrayList<>();
for (Alumno alumno : aprobados) {
    nombres.add(alumno.nombre());
}
```

```java
// DECLARATIVA: describís QUÉ querés, como una tubería
List<String> nombres = CURSO.stream()
        .filter(alumno -> alumno.nota() >= 6)
        .sorted(Comparator.comparingInt(Alumno::nota).reversed())
        .map(Alumno::nombre)
        .toList();
```

Las dos devuelven `[Dana, Ana, Carla, Fede]`. La segunda no es "más corta y ya": eliminó las dos listas intermedias, los dos `for`, y sobre todo eliminó la posibilidad de mezclarlos mal. Se lee como la consigna.

12.3.1 La anatomía de un pipeline

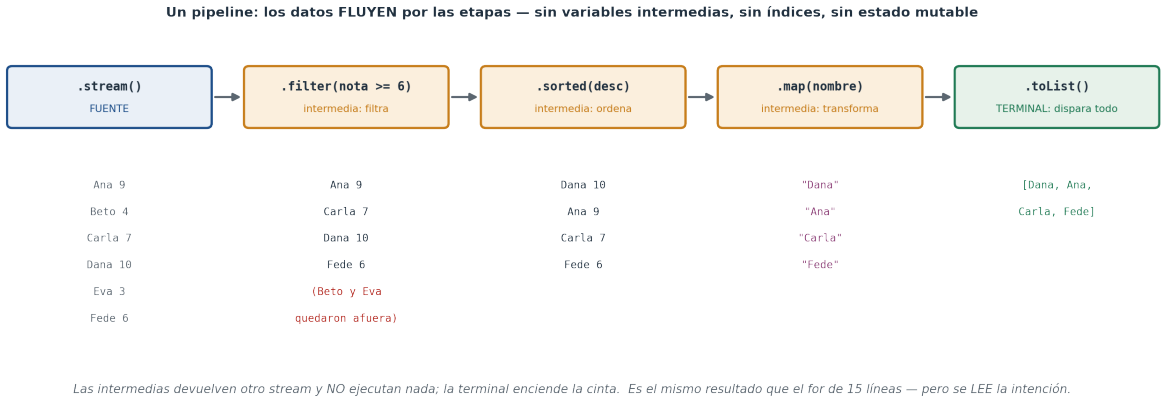


Figura 12.4: Un pipeline de stream: una FUENTE, operaciones INTERMEDIAS que describen transformaciones (y devuelven otro stream), y UNA operación TERMINAL que dispara la ejecución. Abajo, los datos del curso fluyendo etapa por etapa.

Definición

- **Fuente**: `coleccion.stream()`, `Stream.of(...)`, `Arrays.stream(arreglo)`.
- **Intermedias** (devuelven `Stream`, se encadenan): `filter` (quedarse con los que cumplen), `map` (transformar cada uno), `sorted`, `distinct`, `limit`, `skip`.
- **Terminales** (una sola, dispara todo): `toList()`, `count()`, `sum()`, `average()`, `max/min`, `findFirst()`, `anyMatch`, `forEach`, `collect(...)`.
- **Lambda**: una función anónima — `alumno -> alumno.nota() >= 6`.
- **Referencia a método**: la lambda que sólo llama a un método, abreviada: `Alumno::nombre` equivale a `a -> a.nombre()`.
- **`Optional<T>`**: el resultado que puede no existir (¿máximo de un stream vacío?). Se abre con `orElse`, `orElseThrow`, `map` — y se acabaron los `null` sorpresa.

Las herramientas de reducción — comprimir el stream a UN resultado:

```java
int suma = CURSO.stream().mapToInt(Alumno::nota).sum();              // 39
double promedio = CURSO.stream().mapToInt(Alumno::nota)
        .average().orElse(0.0);                                     // 6.5
Optional<Alumno> mejor = CURSO.stream()
        .max(Comparator.comparingInt(Alumno::nota));                // Dana
String lista = CURSO.stream().filter(a -> a.nota() >= 6)
        .map(Alumno::nombre)
        .collect(Collectors.joining(", "));       // "Ana, Carla, Dana, Fede"
```

12.3.2 La pereza: el pipeline no corre hasta que hace falta

Las operaciones intermedias no ejecutan NADA por sí solas: arman la descripción. Recién la terminal enciende la cinta — y entonces los elementos fluyen **de a uno** por todo el pipeline, con la posibilidad de cortar antes.

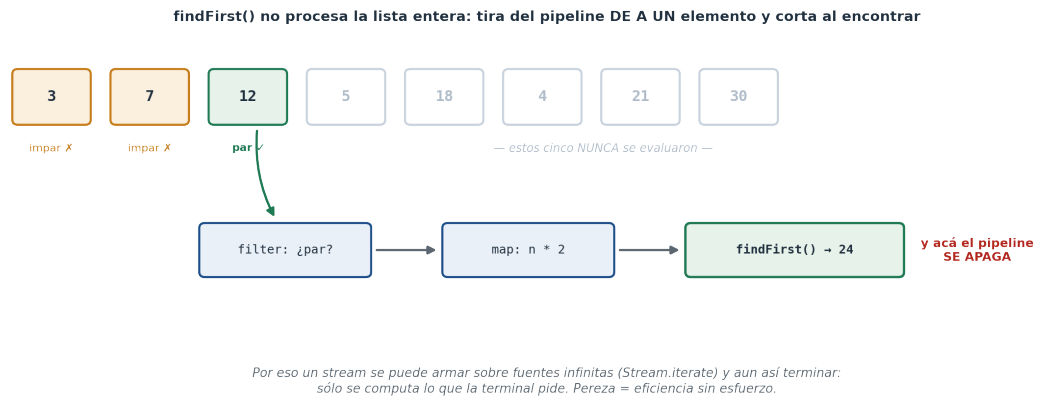


Figura 12.5: PipelinePerezoso, visible: buscando el primer par, sólo 3, 7 y 12 se evalúan. El 12 pasa el filtro, se mapea, findFirst lo captura — y los otros cinco números jamás se miran.

Corré `PipelinePerezoso` en el repo: los `println` metidos dentro del `filter` y el `map` te muestran exactamente qué se evalúa (y qué no).

Error común

Streams, con criterio:

- **No reemplazan entender los costos**: `lista.stream().filter(...)` sigue recorriendo TODA la lista — O(n). Si buscás por clave todo el tiempo, la respuesta sigue siendo un `Map`, no un stream más lindo.
- **Un stream se consume una vez**: reutilizarlo lanza `IllegalStateException`. Se arma otro (es barato).
- **Sin efectos colaterales**: una lambda que modifica variables de afuera es un bug esperando su momento. Todo lo que produce el pipeline debe salir POR el pipeline.
- **Un `for` simple sigue siendo mejor** cuando hay que cortar con lógica compleja, modificar estructuras existentes, o el pipeline quedaría ilegible. Declarativo no es religión: es legibilidad.

12.4 Cómo se trabaja hoy en una empresa

Sabés programar. Falta el contexto donde eso ocurre: un **equipo** tocando el mismo código a la vez, sin pisarse y sin romper lo que anda. Todo el ecosistema profesional — git, PRs, review, CI — existe para sostener esas dos promesas. La mejor forma de contarlo es seguir un cambio real de punta a punta:

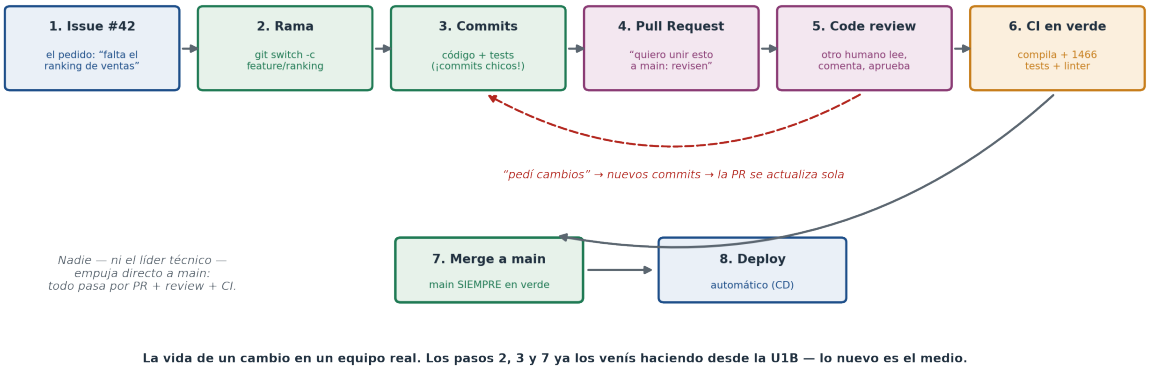


Figura 12.6: La vida de un feature: nace como issue, vive en una rama, crece en commits, se propone en una pull request, lo revisa otro humano, lo verifica el robot del CI, y recién entonces se une a main y se despliega.

12.4.1 Ramas y pull requests

Cada cambio vive en su **rama** (`git switch -c feature/ranking`): un universo paralelo donde podés equivocarte sin molestar a nadie — lo venís haciendo desde la U1B. La novedad profesional es el final: la rama NO se mergea a mano. Se abre una **pull request** (PR): "quiero unir esto a `main` — revisen". La PR muestra el diff completo, la conversación, y el estado del CI.

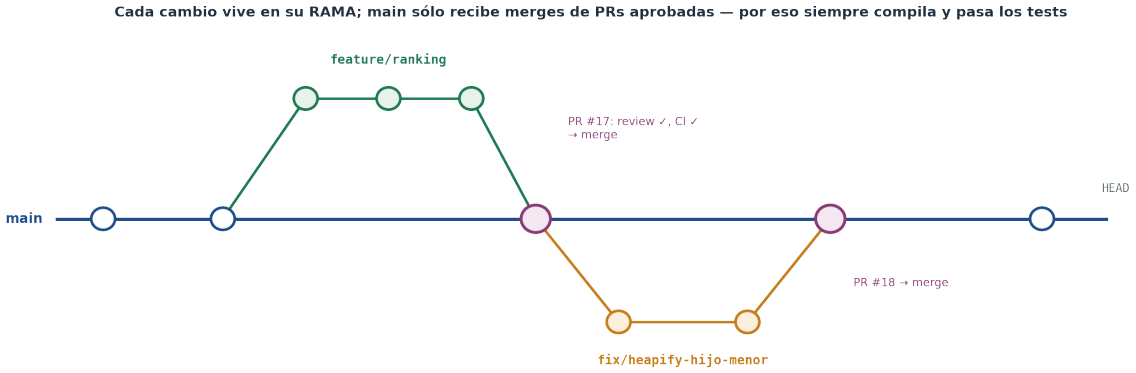


Figura 12.7: El grafo de git en un equipo: main es la línea sagrada — sólo recibe merges de PRs aprobadas — y cada feature o fix vive en su rama hasta estar listo.

12.4.2 Code review: otro humano lee tu código

Antes del merge, un compañero lee tu diff y comenta: un posible bug, un nombre confuso, un caso sin test, una estructura con el costo equivocado ("esto es $O(n^2)$, con un `HashSet` es $O(n)$" — ¿te suena?). No es un examen: es la herramienta de calidad más efectiva que existe, y además es cómo los equipos esparcen conocimiento. Dos reglas para cada lado:

- **Como autor**: PRs chicas (una idea por PR), descripción clara de qué y por qué, y los comentarios no son ataques — "tenés razón, lo cambio" es una respuesta de profesional, no de derrotado.
- **Como revisor**: comentá el código, no la persona; preguntá antes de afirmar ("¿qué pasa si la lista está vacía?"); y aprobá cuando está BIEN, no cuando está perfecto-según-vos.

12.4.3 CI: el robot que no se cansa

La **integración continua** es un robot que, en cada push, compila TODO y corre TODOS los tests. Si algo falla, la PR queda bloqueada — sin discusión, sin "en mi máquina andaba".

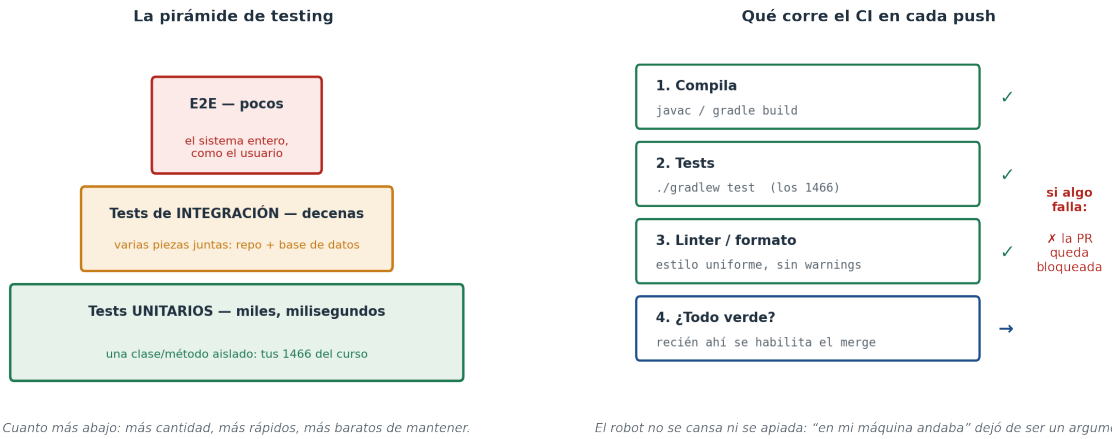


Figura 12.8: Izquierda: la pirámide de testing — muchos tests unitarios rápidos como base (los 1466 de este curso son exactamente eso), integración en el medio, pocos E2E arriba. Derecha: lo que el CI ejecuta en cada push.

Un CI mínimo para el proyecto de la materia es un archivo de 15 líneas en el repo (`.github/workflows/ci.yml` en GitHub Actions):

```yaml
name: CI
on: [push, pull_request]        # se dispara en cada push y cada PR
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '25' }
      - run: ./gradlew test     # los 1466 tests, en cada cambio, siempre
```

Idea clave

¿Ves cómo cierra el curso? Los **tests JUnit** que escribiste en cada guía son la base de la pirámide; el **`./gradlew test`** que corrés local es lo mismo que ejecuta el robot; el **git** que usás desde la U1B es el idioma del equipo; y el criterio de costos de las U4–U11 es lo que el reviewer espera leer en tu PR. No aprendiste "temas sueltos": aprendiste el oficio completo en miniatura.

12.4.4 ¿Y dónde entra la IA?

En todos lados — y por eso importa tanto saber QUÉ delegarle. Hoy un programador profesional usa IA para autocompletar, generar borradores y tests, explicar código ajeno y hacer una primera pasada de review. Lo que NO cambió: la responsabilidad. El código que se mergea con tu nombre es tuyo, lo haya tipeado quien lo haya tipeado.

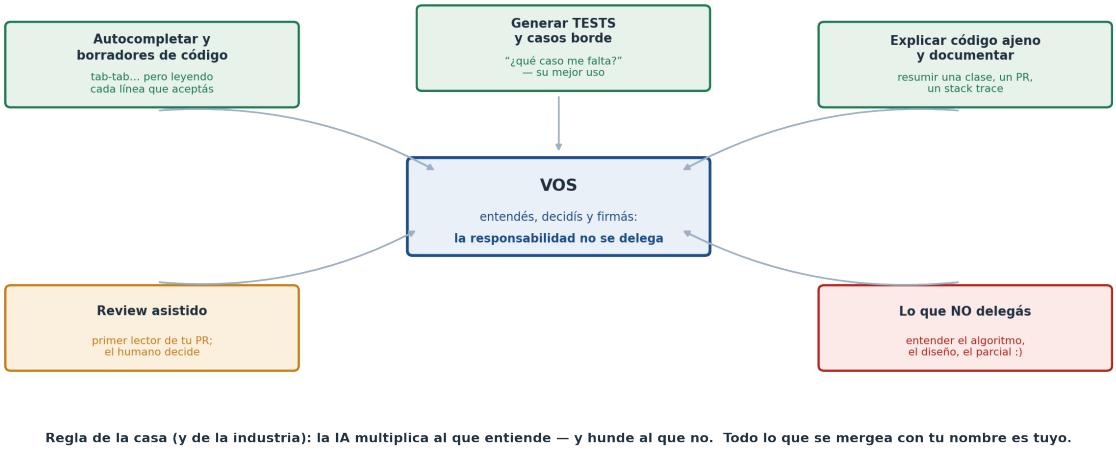


Figura 12.9: El mapa honesto de la IA en el flujo profesional: ayuda en todo el perímetro — autocompletar, tests, explicación, review — pero el centro no se delega: entender, decidir y firmar.

Con IA (plugin de Claude)

La regla de esta cátedra es la misma que la de la industria seria: **la IA multiplica al que entiende y hunde al que no**. Usala para: generar casos de prueba retorcidos, explicarte un stack trace, proponerte nombres mejores, revisar tu PR antes que tu compañero. No la uses para: producir código que no podés explicar línea por línea. El test es simple: si en la defensa del TP (o en la entrevista de trabajo) no podés justificar cada decisión, no era tu código — y se nota en tres preguntas.

12.5 Ejemplo integrador: el informe de ventas

El encargo típico de la primera semana de trabajo: "tenemos las ventas, armá el reporte". Datos inmutables en un `record`, cada pregunta un pipeline, y los agrupados con `Collectors.groupingBy` — el `HashMap` de la U7 que ahora se arma solo:

```
public record Venta(String vendedor, String producto, int monto) {}

/** ¿Cuánto vendió cada uno? groupingBy = el Map armado por el stream. */
public Map<String, Integer> totalPorVendedor() {
    return ventas.stream()
            .collect(Collectors.groupingBy(
                    Venta::vendedor,                    // la clave del mapa
                    TreeMap::new,                       // ordenado por nombre (U8)
                    Collectors.summingInt(Venta::monto))); // qué acumular por grupo
}

/** Ranking: los vendedores de mayor a menor facturación. */
public List<String> rankingDeVendedores() {
    return totalPorVendedor().entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
            .map(Map.Entry::getKey)
            .toList();
}

/** El producto con más operaciones: groupingBy + counting + max. */
public String productoMasVendido() {
    return ventas.stream()
            .collect(Collectors.groupingBy(Venta::producto, Collectors.counting()))
            .entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
```

```
            .orElseThrow(() -> new IllegalStateException("no hay ventas"));
}
```

La salida completa con las 10 ventas del repo (verificala corriendo `InformeDeVentas`):

```
84800                                  <- totalRecaudado()
{Ana=29700, Beto=13600, Carla=41500}   <- totalPorVendedor()  (TreeMap: ordenado)
[Carla, Ana, Beto]                     <- rankingDeVendedores()
cuaderno                               <- productoMasVendido() (3 operaciones)
8480.0                                 <- promedioPorVenta()
```

Con Git / Gradle

El broche del curso, como ejercicio real: subí `cb100-codigo` a un repositorio en GitHub, agregá el `ci.yml` de arriba, abrí una rama con tu solución de la guía 12, y hacete tu primera **pull request** — aunque el reviewer seas vos mismo desde otra pestaña. Ver el CI correr tus 1466 tests en una máquina ajena y ponerse verde es la mejor demostración de que todo lo que hiciste este cuatrimestre es, literalmente, lo que se hace en la industria.

12.6 Errores comunes

Error común

- Declarar `ArrayList<String> lista = new ArrayList<>()` en vez de `List<String>`: casarse con la implementación sin necesidad.
- Elegir la colección por costumbre (“siempre uso ArrayList”) en vez de por costos: el `contains` en lista es la causa #1 de programas lentos.
- `List.of` / `Map.of` son inmutables: modificar lanza excepción — no es un bug de Java, es el contrato.
- Borrar dentro de un for-each → `ConcurrentModificationException`; usar `removeIf`.
- Streams con efectos colaterales (lambdas que modifican variables externas): funciona… hasta que no.
- `get()` sobre un `Optional` sin chequear: es el `null` con sombrero. `orElse`/`orElseThrow`.
- PRs gigantes (“todo el TP en un commit”): imposibles de revisar bien; el review se vuelve teatro.
- Mergear con el CI en rojo “porque es un flake”: el día que era un bug de verdad, main quedó roto para todo el equipo.
- Aceptar código de la IA que no podés explicar: funciona hoy, te hunde en la defensa (y en la entrevista).

12.7 Repaso

1. Dibujá de memoria el mapa de colecciones: interfaces, implementaciones y de qué unidad del curso viene cada una.
2. Para cada necesidad, elegí colección y justificá por costos: (a) historial de navegación con “volver”; (b) padrón de DNI únicos con búsqueda; (c) agenda ordenada por fecha con “próximos 7 días”; (d) cola de impresión con prioridades; (e) stock por código de producto.
3. ¿Por qué se declara `List<X>` y se instancia `new ArrayList<>()`? ¿Qué principio de la U3 es?
4. Explicá con la medición de `ComparaEstructuras` por qué “andaba bien en la demo y explotó en producción” suele ser un `contains` sobre lista.
5. ¿Qué diferencia hay entre una operación intermedia y una terminal? ¿Qué pasa si un pipeline no tiene terminal?
6. Escribí (en papel) el pipeline que, dada una lista de `Venta`, devuelve los 3 productos más caros vendidos por “Ana”, sin repetir producto.
7. ¿Qué es la evaluación perezosa? ¿Por qué `findFirst` sobre un stream de un millón de elementos puede evaluar sólo tres?

8. ¿Qué problemas resuelve `Optional` y cuáles son sus dos formas seguras de apertura?
9. Contá la vida completa de un feature desde el issue hasta el deploy. ¿En qué dos puntos puede "rebotar" y por qué eso es bueno?
10. ¿Qué corre el CI en cada push y por qué "en mi máquina andaba" dejó de ser un argumento?
11. ¿Qué hace buena a una pull request? ¿Y a un code review?
12. Enunciá la regla de la IA de esta unidad y defendela con un ejemplo concreto de uso bueno y uno de uso malo.

12.8 Cierre del curso

Hagamos el inventario honesto de lo que te llevás. Sabés **modelar**: elegir si algo es una lista, un mapa, un árbol o un grafo — y eso es el 80 % de resolver cualquier problema. Sabés **medir**: big-O dejó de ser una letra griega y es tu forma de discutir diseño. Sabés **construir**: TDAs con invariantes, encapsulamiento y tests que los defienden. Sabés **diseñar algoritmos**: dividir y conquistar, backtracking, greedy (con demostración) y programación dinámica. Y sabés **trabajar**: git, ramas, tests, PRs, CI — el taller completo, no sólo las herramientas.

El apunte termina acá; la materia no. Cada estructura que usaste tiene una versión más profunda esperándote (grafos con flujo, árboles B en bases de datos, hashing distribuido en sistemas grandes), y cada empresa donde trabajes tendrá su versión del flujo que viste en esta unidad. Cuando te cruces con ellas, vas a notar algo: ya conocés el idioma.

Fue un gusto. Ahora andá y hacete tu primera PR. ✓

Guía de ejercicios — Unidad 12

Objetivos de la unidad

La guía final: colecciones bien elegidas, streams y el flujo profesional. Cada ejercicio va en su propio paquete dentro de `ar.uba.fi.cb100.guia.i12_profesional` (niveles `i02_facil`, `i03_medio`, `i04_dificil`; un paquete por ejercicio), con su **test JUnit**. Las teóricas se responden en `guia/i12_profesional/i01_teorico`. En esta guía el enunciado NO te dice qué colección usar — elegirla (y justificarla en el javadoc por costos) es parte del ejercicio.

12.9 Preguntas teóricas

1. ¿Por qué conviene declarar variables por la interfaz (`List`, `Map`) y no por la implementación (`ArrayList`, `HashMap`)?
2. Armá la tabla de costos de ArrayList, ArrayDeque, HashSet/HashMap, TreeSet/TreeMap y PriorityQueue para buscar, insertar y borrar. ¿De qué unidad sale cada costo?
3. ¿Cuándo elegís TreeMap sobre HashMap, pagando O(log n) en vez de O(1)?
4. ¿Qué son las operaciones intermedias y la terminal de un stream? ¿Por qué se dice que las intermedias son perezosas?
5. ¿Qué es una lambda y qué una referencia a método? Dá tres ejemplos equivalentes lambda ↔ referencia.
6. ¿Qué problema resuelve `Optional`? Compará `orElse`, `orElseThrow` y `map` sobre un Optional.
7. ¿Qué hace `Collectors.groupingBy`? ¿Qué estructura del curso construye por dentro y cómo se personaliza el "qué acumular"?
8. Describí la vida completa de un feature: issue → rama → commits → PR → review → CI → merge → deploy. ¿Qué garantiza cada etapa?
9. ¿Qué es la integración continua y por qué "main siempre en verde" es la regla de oro de un equipo?
10. Enunciá tres usos buenos y tres usos malos de la IA en el flujo profesional, y la regla que los separa.

12.10 Nivel fácil

1. **Elegí la colección.** Implementá `RegistroDeAsistencia`: registrar alumnos presentes (sin repetidos) y responder `estuvo(nombre)` rápido. Justificá la colección en el javadoc. Test.
2. **Historial del navegador.** `HistorialDeNavegacion` con `visitar(url)`, `volver()` y `paginaActual()`. Elegí la colección correcta. Test.
3. **Contador de palabras, edición streams.** Dado un `List<String>` de palabras, armá el `Map<String, Long>` de frecuencias con `groupingBy` + `counting`. Test.
4. **Primeros streams.** Sobre `List<Integer>`: la suma de los pares, la lista de los impares al cuadrado ordenada descendente, y el máximo como `Optional`. Test.
5. **De imperativo a declarativo.** Convertí a stream un método imperativo dado (filtrar mails válidos —contienen "@"—, pasarlos a minúsculas, sin duplicados, ordenados). El test verifica que ambas versiones coinciden.
6. **Agenda ordenada.** `Agenda` de eventos (fecha como `String` "2026-08-11", título) que siempre lista en orden cronológico y responde `proximos(desde, hasta)`. Elegí la colección (¡rangos!). Test.
7. **Ranking de notas.** Con el `record Alumno(nombre, nota)` del apunte: promedio del curso, mejores 3 nombres, y `Map` de nota → cantidad de alumnos. Todo con streams. Test.
8. **Cola de atención.** `MesaDeEntradas`: trámites con número creciente, se atienden en orden de llegada, pero los urgentes pasan primero (entre urgentes, por llegada). Elegí la(s)

colección(es). Test.

9. **Optional sin sustos.** `Optional<Alumno> buscarPorNombre(String)`: devolvé el alumno o `Optional.empty()`; el que llama decide con `orElse`/`orElseThrow`. Test de ambos caminos.
10. **joining.** Generá el texto "Aprobados: Ana (9), Carla (7), Dana (10)" desde la lista de alumnos con un solo pipeline (`filter` + `map` + `Collectors.joining`). Test del formato exacto.

12.11 Nivel medio

1. **Informe de ventas extendido.** Partiendo del `InformeDeVentas` del apunte, agregá: total por producto, la venta más grande de cada vendedor (`Map<String, Venta>` con `Collectors.toMap` y merge), y los vendedores con total ≥ un umbral. Test contra los datos del apunte.
2. **groupingBy anidado.** `Map<String, Map<String, Integer>>`: vendedor → (producto → total). Test: el total de Ana en "lapicera" es 1200.
3. **Índice invertido.** De un `List<String>` de documentos, armá `Map<String, Set<Integer>>`: palabra → en qué documentos aparece. Elegí las colecciones y justificá. Test.
4. **Deduplicar preservando orden.** `List<T> sinDuplicados(List<T>)` en O(n) manteniendo la primera aparición (¿qué colección auxiliar?). Compará en el javadoc con la alternativa $O(n^2)$. Test.
5. **Top-K eficiente.** Los K productos más vendidos de un stream de millones de ventas SIN ordenar todo: usá una `PriorityQueue` de tamaño K (U9). Test que coincide con la versión "ordenar todo y cortar".
6. **Fusión de agendas.** Dadas varias agendas (`Map<String, List<Evento>>` por persona), armá la agenda global ordenada y detectá conflictos (dos eventos a la misma hora). Streams + la colección correcta. Test.
7. **Estadísticas en una pasada.** Con `IntSummaryStatistics` (o tu propio collector), obtené min, max, promedio y cantidad de una lista de montos en UNA sola pasada. Test.
8. **Simulador de PR.** Modelá `PullRequest` (autor, aprobaciones `Set<String>`, checks `Map<String, Boolean>`) con `puedeMergearse()`: ≥ 2 aprobaciones de NO-autores y todos los checks en verde. Es el flujo de la unidad hecho código. Test de los cuatro casos.
9. **Pipeline de CI en miniatura.** `PipelineDeCi` con etapas (`List<Etapa>`, cada una con nombre y `Supplier<Boolean>`): corre en orden, corta en la primera falla y devuelve el reporte. Test con pipelines que pasan y fallan.
10. **Historial con undo/redo.** `EditorConHistorial` para texto: `escribir`, `deshacer`, `rehacer` — dos pilas (U5), pero ahora elegidas de `java.util` y con tests de las secuencias clásicas.

12.12 Nivel difícil

1. **Collector propio.** Implementá un `Collector<Venta, ?, Map<String, Double>>` que calcule el porcentaje de facturación de cada vendedor sobre el total, en una sola pasada. Test contra el cálculo en dos pasadas.
2. **Stream de Fibonacci.** Con `Stream.iterate` sobre un par `long[]`, generá los primeros n Fibonacci y verificá contra la versión de la U4. ¿Por qué la pereza hace esto posible? Test incluye n=90 (¡long!).
3. **flatMap.** Dado `Map<String, List<String>>` (materia → alumnos), invertilo a `Map<String, List<String>>` (alumno → materias) usando `flatMap`. Test con alumnos en varias materias.
4. **LRU cache.** Implementá `CacheLru<K,V>` con capacidad fija: al superarla, expulsa el menos recientemente usado. Pista: `LinkedHashMap` con `accessOrder=true` y `removeEldestEntry` — investigá y explicá en el javadoc por qué es O(1). Test.
5. **El robot de CI.** `VerificadorDeRepo`: dado un directorio, verifica reglas (todo archivo `.java` de `main` tiene su `*Test.java` en `test`; ningún archivo contiene "TODO"; nombres de paquete en minúsculas) recorriendo con `Files.walk` + streams. Corrélo de verdad sobre `guia/i12_profesional`. Test con un directorio temporal.
6. **Órdenes de compra concurrentes (conceptual).** Simulá dos "cajeros" (hilos) sumando sobre un `HashMap` compartido de stock y mostrá con un test que el resultado a veces pierde actualizaciones; después arreglalo con `ConcurrentHashMap.merge`. Documentá en el javadoc por

qué el HashMap de la U7 no promete nada bajo concurrencia. (Test determinista: muchas iteraciones, asserts sobre la versión correcta.)

7. **Mini motor de consultas.** `Consulta<T>` componible: `donde(predicado)`, `ordenadoPor(comparador)`, `tomar(n)`, `agrupadoPor(clasificador)` — una fachada (U3) sobre streams que se encadena estilo SQL. Test con las ventas del apunte.
8. **Detector de estructura equivocada.** Escribí `AnalisisDeCostos`: recibe un "escenario de uso" (cantidades de operaciones get/contains/insert) y una estructura elegida, y devuelve la estructura óptima con la estimación de operaciones de cada una. Usalo para demostrar con números los 3236 ms vs 10 ms del apunte. Test.
9. **Streams paralelos, con cuidado.** Compará `stream()` vs `parallelStream()` para sumar 50 millones de enteros: verificá que el resultado es idéntico y documentá cuándo conviene (y los peligros con operaciones con estado). El test verifica igualdad de resultados, no tiempos.
10. **El TP integrador del curso.** `SistemaDeInscripciones`: materias con cupo y correlativas (U11: orden topológico para validar el plan), alumnos con prioridad por promedio (U9: PriorityQueue), inscripción con lista de espera (U5: cola), búsquedas por nombre (U7: mapa) y reporte final con streams (U12). Cada estructura justificada en el javadoc. Tests de todo el flujo — el examen final autoadministrado del curso.

Unidad 13

Programación agéntica

Objetivos de la unidad

Este capítulo cierra la materia mirando hacia adelante: cómo cambia el trabajo del programador cuando la IA deja de ser un autocompletador y pasa a ser un **equipo de agentes** que dirigís. Al terminarlo vas a poder: (1) distinguir un **chat** de un **agente**, y explicar el **bucle agéntico** que los separa; (2) conocer las **herramientas** que un agente usa y cómo se conectan (tools, MCP, sandbox); (3) entender la **nueva forma de trabajo** —de escribir código a dirigir, revisar y responder por el código de otros—; (4) manejar la **nomenclatura** completa del área (el glosario de este capítulo); y (5) formarte una opinión propia, con los pies en la tierra, sobre **lo bueno y lo malo** de esta era y sobre dónde va a estar el programador en diez años.

Programación agéntica (del inglés *agentic coding*) y *programación con agentes* nombran lo mismo: programar **dirigiendo agentes de IA** que leen el repositorio, editan archivos, corren los tests y vuelven a intentar, en lugar de escribir cada línea a mano. Vos ya la practicaste todo el cuatrimestre con el plugin de Claude en IntelliJ: este capítulo le pone nombre, estructura y perspectiva a lo que venís haciendo.

13.1 Un poco de historia (muy reciente)

Todo esto es nuevo — tan nuevo que la mitad de los términos del glosario de este capítulo no existían cuando empezaste la secundaria. Los **LLM** (modelos grandes de lenguaje) se hicieron públicos masivamente a fines de 2022; los primeros asistentes fueron **copilotos** que sugerían la próxima línea de código; los **chats** les siguieron, respondiendo preguntas y generando funciones sueltas que había que copiar y pegar. El salto a **agentes** — modelos que usan herramientas, ejecutan lo que escriben y corrigen según el resultado — se consolidó entre 2024 y 2025. Hoy un agente puede tomar un *issue*, explorar un repositorio que nunca vio, escribir el fix, correr la suite de tests y dejar el commit listo para revisión. La velocidad del cambio es parte del tema: lo que este capítulo describe va a seguir moviéndose, y esa inestabilidad **también** es una característica de la era, no un defecto del capítulo.

13.2 Chat vs. agente: la diferencia que importa

Usar un chat para programar y usar un agente parecen lo mismo (“le hablo a una IA”) pero son dos formas de trabajo distintas. En un **chat**, el modelo sólo produce **texto**: te sugiere código, pero el que lo aplica, lo compila, lo prueba y vuelve a preguntar sos **vos** — el bucle lo hacés vos a mano. En un **agente**, el modelo tiene **herramientas y un objetivo**: él mismo edita los archivos, corre el compilador y los tests, **observa** el resultado y decide el próximo paso, hasta terminar o hasta necesitarte.

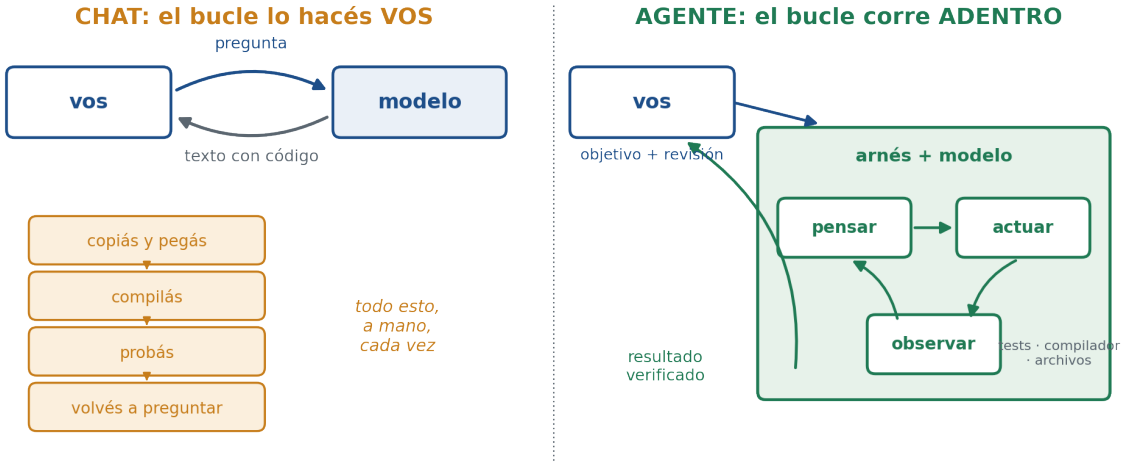


Figura 13.1: Chat vs. agente: en el chat, el bucle (aplicar, compilar, probar, volver a preguntar) lo hacés vos; en el agente, el bucle corre adentro, con herramientas, y vos entrás al principio (objetivo) y al final (revisión).

	Chat	Agente
Produce	texto (que vos aplicás)	**acciones**: edita, ejecuta, prueba
¿Quién itera?	vos, copiando y pegando	el agente, en su propio bucle
Contexto	lo que vos le pegás	lo que él mismo lee del repo
Verificación	la hacés vos	corre los tests y se corrige
Tu rol	preguntar bien	**definir el objetivo y revisar**
Riesgo típico	código suelto que no compila	cambios que no pediste

La frontera práctica: si el sistema puede **equivocarse, darse cuenta solo y reintentar**, es un agente. Esa capacidad de auto-corrección sale de un ciclo muy concreto, que es el corazón técnico de todo esto.

13.3 El bucle agéntico

Un agente es, en esencia, un **bucle** alrededor de un modelo. En cada vuelta el modelo **piensa** (decide el próximo paso), **actúa** (llama a una herramienta: editar un archivo, correr `./gradlew test`, buscar en la web), **observa** el resultado (la salida del comando, el test que falló), y con esa información **vuelve a pensar** — hasta que la tarea está cumplida o hace falta un humano. Al patrón clásico que intercala razonamiento y acción se lo conoce como **ReAct** (*Reason* + *Act*).

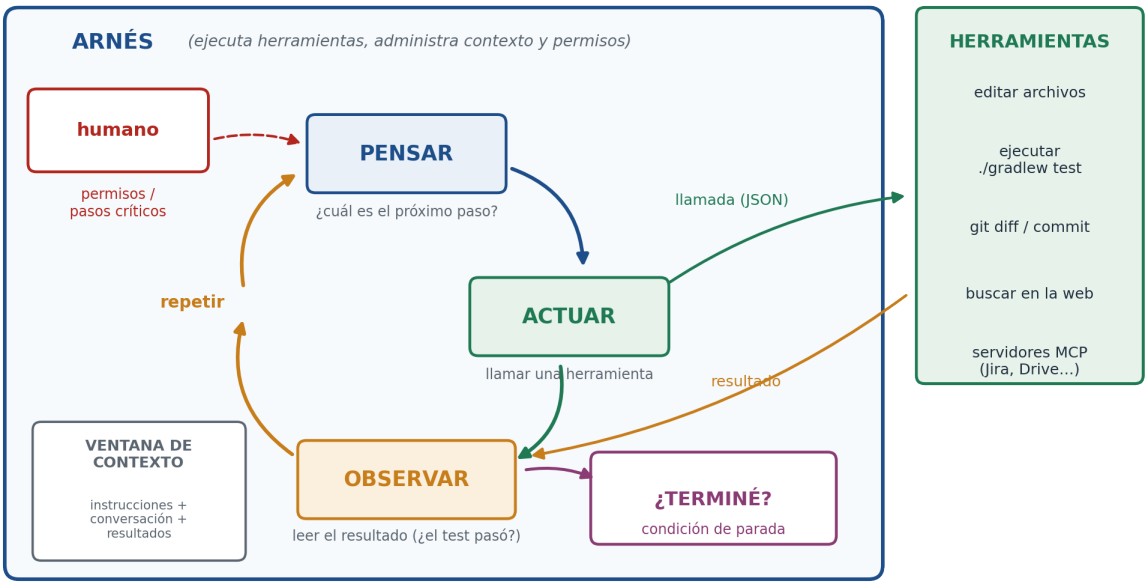


Figura 13.2: El bucle agéntico: pensar → actuar (herramienta) → observar (resultado) → repetir, hasta la condición de parada. Todo corre dentro del arnés, que ejecuta las herramientas, administra la ventana de contexto y aplica los permisos — con el humano entrando en los pasos críticos.

Alrededor del bucle hay piezas con nombre propio, que conviene reconocer en el gráfico:

- El **arnés** (*harness*) es el software que envuelve al modelo y lo conecta con el mundo: ejecuta las herramientas, le devuelve los resultados, maneja el contexto y los permisos. El plugin de Claude en IntelliJ y Claude Code son arneses; el modelo es el "cerebro", el arnés es el "cuerpo".
- La **ventana de contexto** es la memoria de trabajo del modelo: todo lo que "ve" en cada inferencia (las instrucciones, la conversación, los resultados de las herramientas). Es **finita** y se administra: qué entra, qué se resume, qué se descarta — a eso se dedica la *ingeniería de contexto*.
- Los **permisos** y el **humano en el bucle** (*human-in-the-loop*) marcan qué puede hacer el agente sin preguntar y en qué pasos críticos una persona aprueba o corrige. Más autonomía = menos fricción pero más riesgo: es un dial, no un interruptor.
- La **condición de parada** evita el bucle infinito — tarea completa, límite de pasos, o error que requiere ayuda. Sí: hasta el agente necesita su "caso base", como toda recursión de la Unidad 6.

¿Te suena la silueta? Es un `while` con estado, decisiones y corte — el mismo patrón que analizaste todo el cuatrimestre, con un LLM adentro.

13.4 Las herramientas: las manos del agente

Un modelo solo **no puede** tocar nada: sólo genera texto. Lo que convierte texto en acción son las **herramientas** (*tools*): funciones que el arnés ofrece y el modelo invoca. El mecanismo se llama **tool use** (o *function calling*): en vez de responder prosa, el modelo devuelve una llamada estructurada — el nombre de la herramienta y sus parámetros en JSON, validados contra un **esquema** (JSON Schema). El arnés la ejecuta y le devuelve el **resultado de herramienta**, que entra al contexto para la próxima vuelta del bucle.

Las herramientas típicas de un agente de programación te van a resultar conocidas, porque son tu propio día de trabajo: **leer y editar archivos**, **ejecutar comandos** (compilar, `./gradlew test`, `git diff`), **buscar** en el repo y en la web. Para no ejecutar código dudoso sobre tu máquina real, los arneses usan un **sandbox**: un entorno aislado donde romper algo no rompe nada.

Y para conectar al agente con el resto del mundo —Jira, Google Drive, Slack, una base de datos— existe **MCP** (*Model Context Protocol*): un protocolo estándar, abierto, para que cualquier servicio exponga herramientas y datos que cualquier agente pueda usar. Un **conector** es eso: una integración MCP ya armada. La idea es la misma que el contrato de un TDA (Unidad 3): una **interfaz estándar** separa al que usa (el agente) del que implementa (el servicio), y entonces las piezas se combinan sin conocerse.

En IntelliJ

En el plugin de Claude podés ver el bucle y las herramientas en vivo: cada paso muestra qué herramienta llamó el agente (leer tal archivo, correr tal comando) y qué recibió. Mirarlo trabajar un rato es la mejor clase de programación agéntica que existe — fijate cómo ante un test rojo **lee el mensaje de error**, igual que te enseñamos en la Unidad 1.

13.5 La nueva forma de trabajo: de escribir a dirigir

Acá está el cambio de fondo, el que le da sentido a este capítulo. Hasta hace poco, un programador **escribía** código: el teclado era el cuello de botella. Con agentes, el mismo programador **dirige**: define el objetivo, divide el trabajo, lanza agentes —uno o varios, cada **subagente** con su propia subtarea y su propio contexto, a veces coordinados por un **orquestador**—, revisa lo que producen, corrige el rumbo e integra el resultado. El título del capítulo podría ser: **cada programador, jefe de un equipo de diez agentes.**

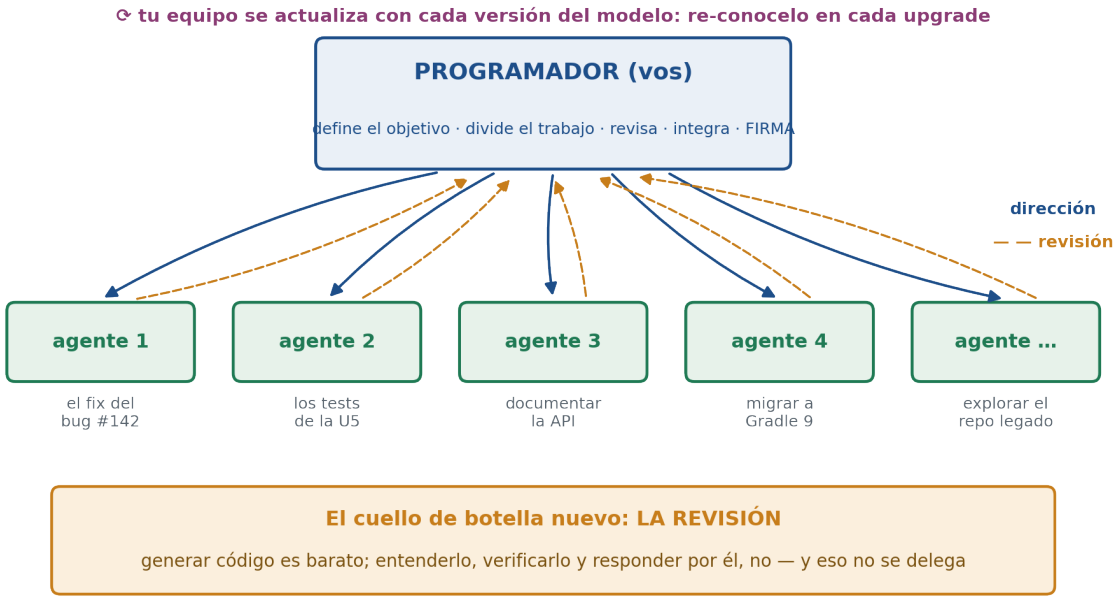


Figura 13.3: La nueva forma de trabajo: el programador define, divide y revisa; los agentes ejecutan en paralelo (el fix, los tests, la documentación, la migración). La revisión es el cuello de botella nuevo — y tu responsabilidad no se delega.

¿Y cuánto multiplica eso el resultado? Depende — y desconfiá de quien te dé un número redondo sin contexto. En tareas que calzan bien con los agentes (código repetitivo, tests, migraciones mecánicas, prototipos, explorar una base de código ajena) el multiplicador se siente enorme: **5x, 10x, a veces más**. En trabajo de diseño fino, código crítico o problemas mal definidos, el multiplicador es mucho menor — y a veces, si aceptás sin revisar código que no entendés, es **menor que 1**: vas más rápido hoy y mucho más lento en tres meses. La productividad real de la era agéntica no viene de "el agente tipea rápido": viene de **paralelizar** (varios agentes a la vez), de **no gastar cerebro en lo mecánico**, y de que la máquina **nunca se aburre** de correr los tests una vez más.

Hay otra novedad sin precedentes: **tu equipo cambia cuando se actualiza el modelo.** Un jefe humano conoce a su gente durante años; vos vas a tener un equipo que, con cada versión nueva de Claude (o del modelo que uses), se vuelve más capaz, con otras fortalezas y otros límites — de un día para el otro. Parte del oficio nuevo es **re-conocer a tu equipo** en cada actualización: qué le podés delegar ahora que antes no, qué supervisión ya no hace falta, cuál sí. Lo que no cambia nunca: **el responsable sos vos**. El agente no firma el commit, no rinde el parcial y no responde ante el cliente. Vos sí.

Importante

La regla de oro de la era agéntica: no aceptes código que no podés explicar. El agente escribe; vos firmás. Todo lo que esta materia te dio —el checklist de TDA, el análisis de complejidad, los tests, el checklist de listas— es exactamente el instrumental para revisar con criterio lo que un agente produce. La materia no te entrenó para competir contra el agente tipeando: te entrenó para ser **su jefe técnico**.

13.6 Lo bueno y lo “malo” de esta era

Seamos honestos con las dos columnas — adaptarse empieza por ver claro:

Lo bueno	Lo “malo” (los riesgos)
Alcance: una persona puede construir lo que antes pedía un equipo.	**Atrofia**: si el agente hace todo, podés no aprender los fundamentos… y entonces no podés revisarlo.
Velocidad: lo mecánico (boilerplate, tests, migraciones) sale solo.	**Deuda de revisión**: generar código es barato; entenderlo y mantenerlo, no. El cuello de botella se movió a la revisión.
Aprendizaje: tenés un explicador incansable 24/7 (usá la IA para *entender*, la regla de la materia desde la Unidad 1).	**Alucinaciones**: el modelo puede inventar APIs, datos o justificaciones con total confianza. Verificar es parte del trabajo.
Menos fricción: explorar un repo ajeno, entender código legado, probar una idea — minutos en vez de días.	**Seguridad nueva**: *prompt injection* (un texto externo que le da órdenes a tu agente), permisos mal puestos, código ejecutado sin sandbox.
Oportunidad: las reglas se están escribiendo ahora; un estudiante de hoy puede estar un paso adelante de un senior que no se adaptó.	**Inestabilidad**: herramientas, precios y capacidades cambian cada pocos meses. Lo que domina hoy puede ser obsoleto mañana.

Fijate que casi todo lo “malo” tiene el mismo antídoto: **fundamentos + criterio + verificación** — o sea, el contenido de las unidades 1 a 12. No es casualidad: cuanto más barato se vuelve generar código, más valioso se vuelve **saber si ese código está bien**.

13.7 El espectro de autonomía

No todo es blanco o negro entre “lo escribo yo” y “lo hace un agente”. En la práctica te vas a mover por un espectro, y elegir el punto correcto **para cada tarea** es una habilidad en sí misma:

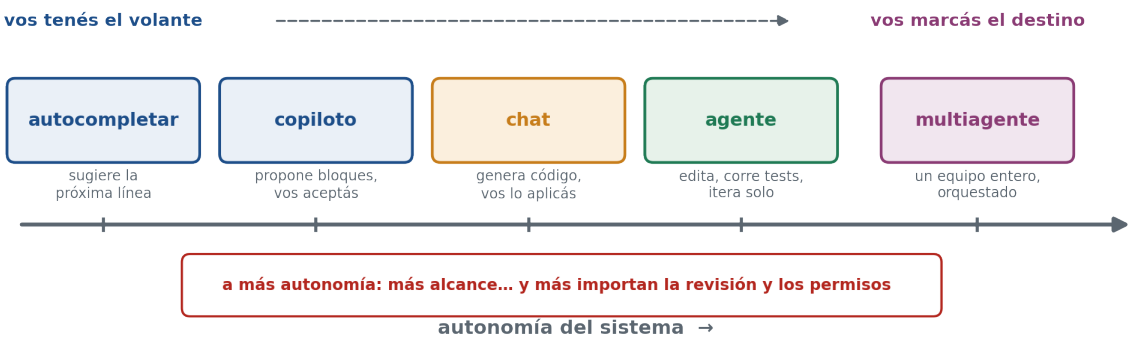


Figura 13.4: El espectro de autonomía: del autocompletado al sistema multiagente. A más autonomía, más alcance por unidad de tu tiempo — y más importancia de la revisión y los permisos. No hay un punto "correcto": hay uno adecuado para cada tarea y su riesgo.

La pregunta guía para elegir el punto: **¿qué tan caro es un error acá, y qué tan fácil es detectarlo?** Para un script descartable, autonomía total. Para la lógica de facturación en producción, el agente propone y vos aprobás cada paso. Es el mismo razonamiento de *human-in-the-loop* que ya usás cuando el plugin te pide confirmar un comando.

13.8 Nomenclatura: el idioma de la era agéntica

Como toda disciplina nueva, ésta ya tiene su idioma. Esta sección es tu **glosario de referencia** — no para memorizar de corrido, sino para volver cada vez que un término aparezca en una documentación, una oferta de trabajo o una charla técnica. Está organizado en siete grupos, de adentro hacia afuera: el modelo, el agente, sus herramientas, su memoria, la organización del trabajo, la operación seria, y la jerga de todos los días.

13.8.1 Fundamentos del modelo

Término	Qué es
LLM (*Large Language Model*)	Modelo de lenguaje entrenado con enormes cantidades de texto; el "cerebro" que razona y genera respuestas (Claude, GPT, Gemini, Llama…).
Token	Unidad mínima de texto que procesa el modelo (un pedazo de palabra, una palabra corta o un signo). Con tokens se miden costo, velocidad y límites.
Ventana de contexto (*context window*)	Cantidad máxima de tokens que el modelo puede "ver" a la vez: instrucciones + conversación + resultados de herramientas.
Prompt	El texto de entrada que recibe el modelo.
System prompt	Instrucciones de base que definen el rol, las reglas y el comportamiento del agente.
Inferencia	Cada ejecución del modelo para generar una respuesta.
Temperatura	Parámetro que controla qué tan aleatoria o determinista es la salida.
Alucinación	Cuando el modelo inventa datos con apariencia de verdad.
Razonamiento extendido / thinking	El modelo "piensa" en pasos intermedios antes de responder.

13.8.2 El agente y su funcionamiento

Término	Qué es
Agente	Sistema donde un LLM decide por sí mismo qué pasos dar, usa herramientas y observa resultados hasta cumplir un objetivo.
Arnés (*harness*)	El software que envuelve al modelo y lo conecta con el mundo: maneja el bucle, ejecuta herramientas, administra el contexto y los permisos (Claude Code, por ejemplo, es un arnés).
Bucle agéntico (*agentic loop*)	Ciclo de pensar → actuar (llamar herramienta) → observar resultado → repetir hasta terminar.
ReAct (*Reason + Act*)	Patrón clásico de bucle que intercala razonamiento y acciones.
Turno	Una vuelta de intercambio: mensaje del usuario + respuesta del agente (que puede incluir varias llamadas a herramientas).
Autonomía	Grado en que el agente actúa sin pedir confirmación.
Human-in-the-loop	Diseño donde una persona aprueba o corrige los pasos críticos.
Condición de parada	Criterio para que el bucle termine: tarea completa, límite de pasos, error.

13.8.3 Herramientas e integraciones

Término	Qué es
Herramientas (*tools*)	Funciones que el agente puede invocar: leer archivos, ejecutar comandos, buscar en la web, llamar APIs.
Tool use / function calling	Mecanismo por el cual el modelo devuelve una llamada estructurada (nombre + parámetros JSON) en vez de texto.
Esquema (*schema*)	Definición en JSON Schema de los parámetros que acepta cada herramienta.
Resultado de herramienta (*tool result*)	La salida que el arnés le devuelve al modelo tras ejecutar la herramienta.
MCP (*Model Context Protocol*)	Protocolo estándar para conectar agentes con herramientas y datos externos (servidores MCP).
Conector	Integración ya armada con un servicio (Jira, Drive, Slack…), generalmente vía MCP.
Sandbox	Entorno aislado donde el agente ejecuta código sin riesgo para el sistema real.

13.8.4 Contexto y memoria

Término	Qué es
Ingeniería de contexto (*context engineering*)	Decidir qué información entra en la ventana de contexto, y cómo.
Prompt engineering	Diseñar las instrucciones para obtener mejores resultados.
Memoria de corto plazo	Lo que está en el contexto actual.
Memoria de largo plazo / persistente	Información guardada fuera del modelo (archivos, bases) que se recupera entre sesiones.
RAG (*Retrieval-Augmented Generation*)	Buscar documentos relevantes e inyectarlos en el contexto antes de responder.
Embeddings	Representación numérica de un texto que permite buscar por similitud de significado.
Base de datos vectorial	Almacena embeddings para búsquedas semánticas (pgvector, Qdrant, Pinecone).
Compactación / resumen de contexto	Comprimir la conversación cuando se acerca al límite de tokens.
Prompt caching	Reutilizar partes del prompt ya procesadas para ahorrar costo y latencia.

13.8.5 Organización del trabajo

Término	Qué es
Planificación	El agente descompone una tarea grande en pasos antes de ejecutar.
Subagente	Agente secundario al que el principal delega una subtarea, con su propio contexto.
Orquestador	Agente (o código) que coordina a varios subagentes.
Sistema multiagente	Varios agentes especializados colaborando.
Skill	Paquete de instrucciones (y a veces scripts) que el agente carga para hacer bien un tipo de tarea.
Workflow	Secuencia de pasos predefinida por código — a diferencia del agente, que decide el camino solo.
Hooks	Scripts que el arnés ejecuta automáticamente ante eventos (antes/después de usar una herramienta, al terminar, etc.).
Slash command	Atajo tipo `/comando` que dispara una acción o skill predefinida.

13.8.6 Calidad, seguridad y operación

Término	Qué es
Evals (evaluaciones)	Pruebas sistemáticas para medir qué tan bien funciona un agente — los “tests JUnit” del comportamiento del agente.
Guardrails	Límites y validaciones que impiden acciones peligrosas o salidas indebidas.

Término	Qué es
Permisos	Qué puede hacer el agente sin preguntar: leer, escribir, ejecutar, acceder a red.
Prompt injection	Ataque donde un texto externo (una web, un archivo, un mail) intenta darle órdenes al agente.
Observabilidad / trazas (*tracing*)	Registro detallado de cada paso, llamada y token, para depurar.
Latencia	Tiempo de respuesta.
Rate limit	Límite de peticiones o tokens por minuto que impone el proveedor.
Grounding	Anclar las respuestas en fuentes verificables para reducir alucinaciones.

13.8.7 Términos de la jerga cotidiana

Término	Qué es
Vibe coding	Programar describiendo lo que querés y dejando que el agente escriba casi todo el código.
Agentic coding	Programación donde el agente lee el repo, edita archivos, corre tests e itera solo — el tema de este capítulo.
Copiloto	Asistente que sugiere código mientras escribís, con menos autonomía que un agente.
Fine-tuning	Reentrenar un modelo con datos propios para especializarlo.
Modelo open-weights / local	Modelo cuyos pesos se pueden descargar y correr en hardware propio (vía Ollama, vLLM, etc.).

13.9 ¿Y el programador en diez años?

Nadie lo sabe — y cualquiera que te lo asegure está vendiendo algo. Pero las tendencias ya visibles permiten dibujar escenarios serios. Lo más probable es una continuación de lo que este capítulo describe, más profundo:

Lo que casi seguro cambia. Escribir código a mano va a ser cada vez más raro, como hoy es raro escribir *assembly*: se va a hacer, pero en los bordes donde importa el control fino. La unidad de trabajo va a ser cada vez menos "la función que escribo" y cada vez más **la especificación que redacto y los tests que la verifican** — definir *qué* es correcto, para que un equipo de agentes resuelva el *cómo*. Equipos chicos van a operar sistemas que hoy piden decenas de personas, y "programar" se va a parecer cada vez más a **dirigir ingeniería**: objetivos, división del trabajo, revisión, criterio.

Lo que casi seguro NO cambia. Alguien tiene que **decidir qué construir y responder por lo construido** — eso es un humano, por razones técnicas y legales. Los fundamentos tampoco caducan: los agentes del futuro seguirán eligiendo entre un arreglo y un diccionario, entre $O(n^2)$ y $O(n \log n)$; quien no entienda esas decisiones no va a poder dirigirlas ni auditarlas. Y la confianza —saber de quién es el error cuando algo falla a las 3 de la mañana— va a seguir siendo el activo más caro de la profesión.

La incógnita real es la velocidad y el techo: si los agentes se vuelven confiables a escalas cada vez mayores, el rol se corre más rápido hacia la dirección pura; si se estancan, el programador-que-

revisa sigue siendo el centro por décadas. En cualquiera de los dos escenarios, la jugada ganadora es la misma: **fundamentos sólidos + dominio de las herramientas del momento + disposición a re-aprender cada año**. Adaptarse no es una molestia de esta era: es la *skill* principal. Los que la tengan van a estar un paso adelante en cada actualización del mundo — y esa, al final del día, es la ventaja.

Idea clave

La paradoja final de la materia: cuanto más poderosos los agentes, más valen los fundamentos. Generar código ya casi no cuesta; **juzgarlo** —con TDA, complejidad, tests y criterio— es el trabajo. Esta materia te enseñó a programar para que puedas hacer algo más grande que programar: **dirigir**.

13.10 Errores comunes

Error común

- Aceptar código del agente que no podés explicar: hoy ganás una hora, en el parcial (o en producción) la perdés multiplicada.
- Confundir chat con agente: pegarle código a un chat y copiarlo de vuelta no es programación agéntica, es un copy-paste con pasos extra.
- Darle a un agente permisos totales “para que no moleste”: la autonomía es un dial que se ajusta al riesgo de cada tarea.
- Creer que el multiplicador es automático: sin dividir bien el trabajo y sin revisar, diez agentes producen diez veces más… problemas.
- Pensar que los fundamentos ya no hacen falta “porque la IA lo hace”: la IA lo hace, y alguien que no los tiene no puede saber si lo hizo bien.
- Ignorar la seguridad: un agente que lee una página web o un mail puede estar leyendo una *prompt injection* dirigida a él.

13.11 Repaso

1. ¿Qué diferencia a un **agente** de un **chat**? Nombrá las tres cosas que el agente hace solo y el chat no.
2. Dibujá el **bucle agéntico** y ubicá: arnés, herramientas, ventana de contexto, permisos y condición de parada.
3. ¿Qué es el **arnés** y qué responsabilidades tiene? Dá un ejemplo concreto.
4. ¿Cómo funciona **tool use**? ¿Qué papel juegan el esquema JSON y el resultado de herramienta?
5. ¿Qué problema resuelve **MCP** y qué tiene en común con el contrato de un TDA (Unidad 3)?
6. ¿Por qué decimos que el multiplicador de productividad “depende”? Dá un ejemplo de tarea de multiplicador alto y uno de multiplicador bajo (o negativo).
7. ¿Qué significa que “tu equipo cambia al actualizar el modelo”? ¿Qué parte del trabajo NO se delega nunca?
8. Elegí tres riesgos de la tabla de “lo malo” y explicá su antídoto.
9. ¿Qué es el espectro de autonomía? ¿Con qué pregunta elegís el punto correcto para una tarea?
10. Definí con tus palabras: token, ventana de contexto, alucinación, subagente, orquestador, guardrails, prompt injection, RAG.
11. ¿Qué es *vibe coding* y en qué se diferencia de *agentic coding*? ¿Cuál practicaste en esta materia?
12. Escribí tu propia respuesta, en diez renglones, a la pregunta del capítulo: ¿cómo te imaginás tu trabajo en diez años, y qué vas a hacer este año para estar un paso adelante?